

---

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ  
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

---

Дата генерации: 15.09.2026, 16:32:14  
Файлов исходного кода: 111  
Суммарный объём кода: 1.97 MB  
Глав/билетов в пособии: 36

---

---

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

---

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. 6. Графы. Представление графов, DFS, BFS [id: graph-basics]
7. 7. Графы. Планарные. Покраска [id: graph-planarity]
8. 7. Графы. Планарные. Покраска [id: planarity-euler]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 11. Графы. Мосты [id: bridges-code]
14. 12. Графы. Точки сочленения [id: graph-articulation-points]
15. 13. Графы. Эйлеров цикл [id: graph-euler]
16. 13. Графы. Эйлеров цикл [id: euler-path-vs-cycle]
17. Билет 14. Дейкстра [id: dijkstra]
18. Билет 15. Форд-Беллман [id: bellman-ford]
19. Билет 16. Флойд-Уоршелл [id: floyd]
20. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
21. 17. Графы. Поиск кратчайшего пути. Ускорения [id: shortest-path-accelerations]
22. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
23. 18–20. Графы. Остовное дерево: Краскал, Прима, Борувка
24. 19. Графы. Остовное дерево. Прима [id: mst-prim]
25. 20. Графы. Остовное дерево. Борувка [id: mst-kruskal]

- 18. src/data/content\_temp.ts
- 19. src/data/content.ts
- 20. src/data/debugRunner.ts
- 21. src/data/glossary.ts
- 22. src/data/graphContent.ts
- 23. src/data/quizzes.ts
- 24. src/data/vizStepBus.ts
- 25. src/data/vizSync.ts

# Билеты (учебный контент)

- 26. src/data/tickets/ticket1\_segtree.ts
- 27. src/data/tickets/ticket14\_20.ts
- 28. src/data/tickets/ticket18\_20\_mst.ts
- 29. src/data/tickets/ticket21\_24.ts
- 30. src/data/tickets/ticket6\_13.ts

# Интерактивные компоненты и симуляторы

- 31. src/components/ArticulationPointsViz.tsx
- 32. src/components/BellmanFordViz.tsx
- 33. src/components/BfsViz.tsx
- 34. src/components/ChapterImage.tsx
- 35. src/components/ChapterNav.tsx
- 36. src/components/ChapterQuiz.tsx
- 37. src/components/ChapterView.tsx
- 38. src/components/ComplexityViz.tsx
- 39. src/components/DfsBridgesSimulator.tsx
- 40. src/components/DijkstraViz.tsx
- 41. src/components/DPVisualizer.tsx
- 42. src/components/EulerSimulator.tsx
- 43. src/components/FloydViz.tsx
- 44. src/components/GraphComponentsViz.tsx
- 45. src/components/GraphTraversalViz.tsx
- 46. src/components/HeapViz.tsx
- 47. src/components/hints/demoKit.tsx
- 48. src/components/hints/demosExtra.tsx
- 49. src/components/hints/demosGraph.tsx
- 50. src/components/hints/demosStruct.tsx
- 51. src/components/hints/MiniDemo.tsx

86. src/layout/header.html

# Утилиты

87. src/utils/cn.ts

88. src/utils/exportHtml.ts

89. src/utils/ticketLinks.ts

# Скрипты сборки и экспорта

90. scripts/audit-coverage.mjs

91. scripts/audit-terms.mjs

92. scripts/export/build-guide-pdf.mjs

93. scripts/export/build-pdf.mjs

94. scripts/export/capture-viz-shots.mjs

95. scripts/export/collect-code.mjs

96. scripts/export/common.mjs

97. scripts/export/html-to-blocks.mjs

98. scripts/export/render-pages.mjs

99. scripts/verify/browser-checks.tsx

100. scripts/verify/check-links.mjs

101. scripts/verify/check-reference-code.mjs

102. scripts/verify/dom-env.mjs

103. scripts/verify/live-matrix.test.mjs

104. scripts/verify/probe-compiler.tsx

105. scripts/verify/README.md

106. scripts/verify/run-browser-checks.mjs

107. scripts/verify/run-probe.mjs

# CI / автоматизация

108. .github/workflows/deploy-pages.yml

109. .github/workflows/rebuild-pdf.yml

110. .github/workflows/verify.yml

# Прочее

111. .ai\_skills/universal\_guide\_authoring/SKILL.md

```
contents: read
pages: write
id-token: write
```

```
concurrency:
  group: pages
  cancel-in-progress: true
```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist
```

```
+ group: pages
+ cancel-in-progress: true
+
+ jobs:
+   build:
+     name: Build static site
+     runs-on: ubuntu-latest
+     steps:
+       - name: Checkout
+         uses: actions/checkout@v6
+
+       - name: Setup Node.js
+         uses: actions/setup-node@v7
+         with:
+           node-version: 24
+           cache: npm
+
+       - name: Install dependencies
+         run: npm ci --no-audit --no-fund
+
+       - name: Build for the repository subpath
+         env:
+           VITE_BASE: /algv0/
+         run: npm run build
+
+       - name: Make GitHub Pages fallback
+         run: cp dist/index.html dist/404.html
+
+       - name: Upload Pages artifact
+         uses: actions/upload-pages-artifact@v5
+         with:
+           path: ./dist
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
```

```
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-`${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot: ')"

    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommends
          # На Ubuntu политика ImageMagick иногда запрещает
          # разрешаем то, что нужно пайплайну (text -> pdf)
          for policy in /etc/ImageMagick-6/policy.xml /etc/ImageMagick-6/policy.xml; do
            if [ -f "$policy" ]; then
```

```
      name: full-code-export
      path: |
        public/export/full_code_all_pages.pdf
        public/export/full_code_all_pages.txt
      if-no-files-found: error
      retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action@users.noreply.github.com"
    git add -- public/export/full_code_all_pages.*
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full-code-export"
    git push origin "HEAD:${GITHUB_REF_NAME}"
  ....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

      steps:
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

```
````html
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height" />
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/public/favicon.svg" />
    <title> Дискретка для тупых – Интерактивный гид по дискреткам
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen" >
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
````
```

### Patch относительно `main`

```
````diff
diff --git a/index.html b/index.html
index cdea4c7..a44c135 100644
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
   <head>
     <meta charset="UTF-8" />
     <meta name="viewport" content="width=device-width, height=device-height" />
+    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/public/favicon.svg" />
     <title> Дискретка для тупых – Интерактивный гид по дискреткам
   </head>
   <body class="bg-slate-950 text-slate-100 min-h-screen" >
````
```

## `public/favicon.svg`

### Полное содержимое после изменений



```

-----
import { SimulatorHub } from "../components/SimulatorHub
import { PythonCompiler } from "../components/PythonComp

export default function App() {
  const [activeTab, setActiveTab] = useState<"guide" |
  const [activeChapterId, setActiveChapterId] = useStat
  const [modalVizId, setModalVizId] = useState<string |

  const index = Math.max(
    0,
    chapters.findIndex((c) => c.id === activeChapterId)
  );
  const activeChapter = chapters[index] ?? chapters[0];
  const prev = index > 0 ? chapters[index - 1] : undefin
  const next = index < chapters.length - 1 ? chapters[in

  const goTo = useCallback((id: string) => {
    setActiveChapterId(id);
    window.scrollTo({ top: 0, behavior: "smooth" });
  }, []);

  // Стрелки ← → листают темы (как на обучающих сайтах)
  useEffect(() => {
    const onKey = (e: KeyboardEvent) => {
      const target = e.target as HTMLElement | null;
      if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
      if (e.metaKey || e.ctrlKey || e.altKey) return;
      if (e.key === "ArrowRight" && next) goTo(next.id)
      if (e.key === "ArrowLeft" && prev) goTo(prev.id);
    };
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown",
  }, [goTo, next, prev]);

  const progress = useMemo(() => ((index + 1) / chapter

  return (
    <div className="min-h-screen bg-slate-950 text-slate

```

```

        <ChapterView
            key={activeChapter.id}
            chapter={activeChapter}
            prev={prev}
            next={next}
            index={index}
            total={chapters.length}
            onGo={goTo}
            onOpenSimulator={setModalVizId}
        />
    </main>
</div>
</>
) : activeTab === "simulator" ? (
    <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
        <SimulatorHub
            onOpen={setModalVizId}
            onGoChapter={({id}) => {
                setActiveTab("guide");
                goTo(id);
            }}
        />
    </main>
) : (
    <PythonCompiler />
)
}

<SimulatorModal vizId={modalVizId} onClose={() => {
    setModalVizId(null);
}} />

<footer className="p-8 text-center text-slate-600" >
    © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>
);
}
....

```

```
## `src/components/Navbar.tsx`
```

```
### Полное содержимое после изменений
```

```
```tsx
```

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, C
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
  activeTab: 'guide' | 'simulator' | 'compiler';
  setActiveTab: (tab: 'guide' | 'simulator' | 'compiler
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeT
  const [busy, setBusy] = useState(false);
```

```
  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };
};
```

```
  return (
    <header className="bg-slate-900 border-b border-sla
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg
        ap-3 lg:gap-0">
          <div className="flex items-center gap-3 w-full
            <div className="bg-gradient-to-tr from-indigo
              k-0">
                <Sparkles className="w-6 h-6" />
              </div>
            <div className="min-w-0 flex-1">
```

```

        className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
            activeTab === 'compiler'
                ? 'bg-emerald-600 text-white shadow-lg
                : 'text-slate-400 hover:text-white'
        }}
    >
    <Terminal className="w-4 h-4" /> Python
</button>
<a
    id="download-pdf-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
    title="Скачать полный PDF сборник всех стра
    >
    <FileDown className="w-4 h-4" /> Скачать PD
</a>
<a
    id="download-txt-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.txt"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
    title="Скачать полный TXT файл со всем кодом
    >
    <FileText className="w-4 h-4" /> TXT
</a>
<button
    id="download-html-btn"
    onClick={saveBook}
    disabled={busy}
    className="flex items-center justify-center

```



```
export function PythonCompiler() {
  const [code, setCode] = useState(STARTER_CODE);
  const [stdin, setStdin] = useState("");
  const [output, setOutput] = useState("Нажмите «Запустить»");
  const [running, setRunning] = useState(false);
  const [runtimeReady, setRuntimeReady] = useState(false);
  const [runtimeMessage, setRuntimeMessage] = useState("");

  const runCode = useCallback(async () => {
    if (running) return;

    setRunning(true);
    setOutput("");
    setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово: программа ничего не вывела");

    const stdout: string[] = [];
    const stderr: string[] = [];
    const inputLines = stdin.split(/\r?\n/);

    try {
      const runtime = await getPythonRuntime();
      setRuntimeReady(true);
      setRuntimeMessage("Python готов — код выполняется");

      runtime.setStdout({ batched: (message) => stdout.push(message) });
      runtime.setStderr({ batched: (message) => stderr.push(message) });
      runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
      await runtime.runPythonAsync(code);

      const text = [...stdout, ...stderr].join("");
      setOutput(text || "Готово: программа ничего не вывела");
    } catch (error) {
      const captured = [...stdout, ...stderr].join("");
      setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
      setRuntimeMessage("Не удалось выполнить код — подробности в консоли");
    } finally {
      setRunning(false);
    }
  }, [code, stdin]);
}
```

```

        <span className="text-[11px] text-slate
</div>
<div className="flex items-center gap-2">
  <button
    type="button"
    onClick={reset}
    className="inline-flex items-center gap-2 text-slate-800 transition-colors"
  >
    <RotateCcw className="w-3.5 h-3.5" />
  </button>
  <button
    type="button"
    onClick={() => void runCode()}
    disabled={running}
    className="inline-flex items-center gap-2 text-white text-xs font-bold transition-colors"
  >
    {running ? <Loader2 className="w-3.5 h-3.5" /> : "Запустить"}
  </button>
</div>
</div>

<textarea
  value={code}
  onChange={(event) => setCode(event.target.value)}
  onKeyDown={(event) => {
    if ((event.ctrlKey || event.metaKey) && event.defaultPrevented()) {
      void runCode();
    }
  }}
  spellCheck={false}
  aria-label="Код Python"
  className="block w-full min-h-[360px] resize-none focus:ring-2 focus:ring-inset focus:ring-slate-500"
  placeholder="Напишите Python-код..."

```

```
-----
    <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
      <Info className="w-4 h-4 shrink-0 text-indigo-600" >
        <span>Ctrl или Cmd + Enter запускает код. Введите код в поле ввода.
      </span>
    </div>
    <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
      <Info className="w-4 h-4 shrink-0 text-amber-600" >
        <span>Первый запуск скачивает примерно несколько килобайт кода.
      </span>
    </div>
    <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
      <Info className="w-4 h-4 shrink-0 text-emerald-600" >
        <span>Код выполняется в браузере и не отправляется на сервер.
      </span>
    </div>
  </div>
</div>
</main>
);
}
....
```

### Patch относительно `main`

```
````diff
diff --git a/src/components/PythonCompiler.tsx b/src/components/PythonCompiler.tsx
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python и библиотеки
+name = "алгоритмы"
+
+def main():
+    print("Привет, мир!")
+    return "Ваш код выполнен успешно!"
+
+main()
+`;
```



```

+
+ const runCode = useCallback(async () => {
+   if (running) return;
+
+   setRunning(true);
+   setOutput("");
+   setRuntimeMessage(runtimeReady ? "Выполняю код..." :
+
+   const stdout: string[] = [];
+   const stderr: string[] = [];
+   const inputLines = stdin.split(/\r?\n/);
+
+   try {
+     const runtime = await getPythonRuntime();
+     setRuntimeReady(true);
+     setRuntimeMessage("Python готов – код выполняется");
+
+     runtime.setStdout({ batched: (message) => stdout
+     runtime.setStderr({ batched: (message) => stderr
+     runtime.setStdin({ stdin: () => (inputLines.length
+     await runtime.runPythonAsync(code);
+
+     const text = [...stdout, ...stderr].join("");
+     setOutput(text || "Готово: программа ничего не выв
+   } catch (error) {
+     const captured = [...stdout, ...stderr].join("")
+     setOutput(`${captured}${captured && !captured.en
+     setRuntimeMessage("Не удалось выполнить код – пр
+   } finally {
+     setRunning(false);
+   }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+   setCode(STARTER_CODE);
+   setStdin("");
+   setOutput("Нажмите «Запустить», чтобы увидеть резу.
+   };

```

```

late-800 transition-colors"
+
+       >
+       <RotateCcw className="w-3.5 h-3.5" />
+       </button>
+       <button
+       type="button"
+       onClick={() => void runCode()}
+       disabled={running}
+       className="inline-flex items-center
-60 text-white text-xs font-bold transition-colors"
+       >
+       {running ? <Loader2 className="w-3.5
+       {running ? "Запускаю..." : "Запустить"}
+       </button>
+       </div>
+     </div>
+
+     <textarea
+     value={code}
+     onChange={(event) => setCode(event.target
+     onKeyDown={(event) => {
+       if ((event.ctrlKey || event.metaKey) &
+       event.preventDefault();
+       void runCode();
+     }
+     }}
+     spellCheck={false}
+     aria-label="Код Python"
+     className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+     placeholder="Напишите Python-код..."
+     />
+
+     <div className="border-t border-slate-800
+     <label className="block px-4 pt-3 text-[
+       Ввод для input() • по одной строке на
+     </label>
+     <textarea

```

```
-----
+         </div>
+         <div className="flex gap-2 rounded-xl border
+             <Info className="w-4 h-4 shrink-0 text-eme
+             <span>Код выполняется в браузере и не отпра
+         </div>
+     </div>
+ </div>
+ </main>
+ );
+}
+...
```

## `vite.config.ts`

### Полное содержимое после изменений

```
````typescript
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  // На GitHub Pages приложение живёт в /algv0/, локальн
  // VITE_BASE можно переопределить, если репозиторий б
  base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
},
```

Универсальное пособие • полный исходный код

-----  
КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 7  
-----

# AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

**\*\*CRITICAL DIRECTIVE\*\***:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md``

This file contains the mandatory rich HTML templates, strictly required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML tags.

-----  
ФАЙЛ 3/111: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2  
-----

# Gemini App Canvas Instructions

Ты — AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas).  
При запросе пользователя создать или сгенерировать новую страницу, ты **\*\*строго\*\*** придерживаешься готовых шаблонов разметки, описанных ниже.

## Правила для малых страниц (Билетов / Глав)

- \*\*Единый стиль:\*\*** Всегда используй Tailwind CSS и HTML-теги.
- \*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна быть в формате `{ "id": "...", "content": ... }` для вставки в ``src/data/content.json``.
- \*\*Строгая структура контента:\*\***
  - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
  - Определение/правило в центре (``border-blue-500`` или ``border-purple-500``).
  - Обязательная сетка аналогий (2 колонки: неформальный язык и формальный язык).

---

ФАЙЛ 6/111: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 54 | РАЗМЕР: 2

---

```
{
  "name": "algv0-exam-guide",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "audit:content": "node scripts/audit-coverage.mjs",
    "audit:terms": "node scripts/audit-terms.mjs",
    "export:txt": "node scripts/export/collect-code.mjs",
    "export:png": "node scripts/export/render-pages.mjs",
    "export:pdf": "node scripts/export/build-pdf.mjs",
    "export": "npm run export:txt && npm run export:png",
    "export:guide": "node scripts/export/build-guide-pdf.mjs",
    "export:shots": "node scripts/export/capture-viz-shots.mjs",
    "verify:unit": "node scripts/verify/live-matrix.test.mjs",
    "verify:code": "node scripts/verify/check-reference.mjs",
    "verify:links": "node scripts/verify/check-links.mjs",
    "verify:dom": "node scripts/verify/run-browser-check.mjs",
    "verify:probe": "node scripts/verify/run-probe.mjs",
    "verify": "npm run verify:unit && npm run verify:links",
  },
  "dependencies": {
    "clsx": "2.1.1",
    "lucide-react": "^1.21.0",
    "motion": "^12.40.0",
    "pdfkit": "^0.20.1",
    "react": "19.2.6",
    "react-dom": "19.2.6",
  }
}
```

Универсальное пособие • полный исходный код

```
-----  
npm run dev          # предпросмотр на http://0.0.0.0:5  
npm run build        # сборка в dist/ (single-file)  
````
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **\*\*весь\*\*** исходный код репозитория в о

| Команда              | Шаг       | Результат                               |
|----------------------|-----------|-----------------------------------------|
| `npm run export:txt` | код → txt | `public/export/full_code_all_pages.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/`                     |
| `npm run export:pdf` | png → pdf | `public/export/full_code_all_pages.pdf` |
| `npm run export`     | всё сразу | txt + png + pdf                         |

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash  
sudo apt-get install -y imagemagick fonts-dejavu-core  
````
```

Настройки через переменные окружения:

- \* `EXPORT\_DENSITY=200` – DPI растеризации (по умолчанию 300)
- \* `EXPORT\_GRAYSCALE=1` – оттенки серого вместо 1-битного

### Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается **\*\*на каж**  
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док  
и коммитит обновлённые `public/export/full\_code\_all\_pages`  
(с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code`

## Аудит контента

```
```bash
```

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `<h2>` и
- \* Подзаголовки: Используют тег `<h3>`.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах `<p>`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика параграфы `<p>`.

## ## Математические формулы

В приложении используется MathJax.

- \* В самом исходном коде страницы (и внутри тегов) формулы.
- \* Таким образом, при извлечении текстового содержимого математически остаются в валидном Markdown-формате латеха (и

## ## Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторы

- \* Они помещены внутрь оберток с классом `<div class="img">`
- \* Все отрисовки находятся внутри векторного тега `<svg>`
- \* Подпись к картинке, если она есть, идет сразу после `<img>`

## ## Списки

- \* Стандартные списки оформлены через `<ul class="list-n">`

## ## Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`
2. Элементы `.analogy-badge` получают выделение маркера
3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `<li>` получают маркер `-`.
5. Результат извлекается в виде текста и чистится от лишнего

Универсальное пособие • полный исходный код

```
-----
* `stack-dfs` → `StackViz.tsx` — **Стек и DFS**
* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **Очередь и BFS**
* `heap-beam-search` → `HeapViz.tsx` — **Куча и лучевой поиск**
* `segment-trees` → `SegmentTreeVisualizer.tsx` — **Деревья сегментов**
* `dynamic-programming` → `DPVisualizer.tsx` — **Динамическое программирование**
---
```

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухари»)

| Глава                                      | id                    | Комментарий             |
|--------------------------------------------|-----------------------|-------------------------|
| 15. Планарность: K5, K3,3 и формула Эйлера | `planarity-euler`     | т «на пальцах».         |
| 16. Флойд (Флойд–Уоршелл)                  | `floyd`               | Есть матричный алгоритм |
| 17. Алгоритм Джонсона                      | `johnson-algo`        | Перевзвешивание         |
| Обзор: Кратчайшие пути и Графы             | `intro`               | Вводная страница        |
| Эйлер: Путь ≠ Цикл                         | `euler-path-vs-cycle` | Ключевая концепция      |

## Уровень 4. Есть аналогия, но НЕТ 2D-визуализации («сухари»)

> Замечание: «5. Двумерная матрица префиксных сумм» пока закрыто

| Глава                                | id                   | Чего не хватает                     |
|--------------------------------------|----------------------|-------------------------------------|
| 3. Декартово дерево (Treap)          | `treap`              | Нет ни визуализации, ни интерактива |
| 8. Графы. Компоненты связности       | `graph-components`   | Нет визуализации                    |
| 24. Классы сложности, сведение задач | `complexity-classes` | Нет визуализации                    |

## Сводная таблица по всем главам

| #   | Глава | Аналогий | 2D-виз. | SVG | Картинка |
|-----|-------|----------|---------|-----|----------|
| --- | ---   | ---      | ---     | --- | ---      |



ФАЙЛ 9/111: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

---

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "types": ["node"],

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",

    /* Path mapping */
    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"]
    },

    /* Linting */
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src", "vite.config.ts"]
}
```

```
    },  
  });  
});
```

---

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

---

ФАЙЛ 11/111: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 218 | РАЗМЕР: 11.3 КБ

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";  
import { PanelLeftClose, PanelLeftOpen } from "lucide-react";  
import { chapters } from "../data/content";  
import { Navbar } from "../components/Navbar";  
import { Sidebar } from "../components/Sidebar";  
import { MobileToc } from "../components/MobileToc";  
import { ChapterView } from "../components/ChapterView";  
import { ChapterNav } from "../components/ChapterNav";  
import { SimulatorModal } from "../components/hints/SimulatorModal";  
import { LiteGuide } from "../components/LiteGuide";  
import { PythonCompiler } from "../components/PythonCompiler";
```

```
/**
```

```
 * «Лёгкий» режим для печати и внешних парсеров: `?lite`
```

```
 * или `?lite=1&topic=mst` (одна тема). Без навигации, без поиска.
```

```
 */
```

```
function readLiteMode(): { lite: boolean; topic?: string } {  
  if (typeof window === "undefined") return { lite: false };  
  const q = new URLSearchParams(window.location.search);  
  const lite = q.has("lite") || q.get("view") === "lite";  
  return { lite, topic: q.get("topic") ?? undefined };  
}
```

```
export default function App() {  
  const liteMode = useMemo(readLiteMode, []);
```

```

    setSidebarMode((m) => {
      const next = m === "wide" ? "compact" : m === "co
      window.localStorage.setItem("sidebarMode", next);
      return next;
    });
  }, []);
const sidebarCollapsed = sidebarMode === "hidden";

// Десктопный отступ контента = ширине панели компиля
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
  const apply = () => {
    const el = contentRef.current;
    if (!el) return;
    el.style.paddingRight = compilerOpen && window.inn
  };
  apply();
  window.addEventListener("resize", apply);
  return () => window.removeEventListener("resize", a
}, [compilerOpen, compilerWidth]);

const index = Math.max(
  0,
  chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefi
const next = index < chapters.length - 1 ? chapters[in

const goTo = useCallback((id: string) => {
  setActiveChapterId(id);
  window.scrollTo({ top: 0, behavior: "smooth" });
}, []);

// Открыть компилятор и подогнать визуализацию вверх:
const openCompilerWithViz = useCallback(() => {
  setCompilerOpen(true);
  requestAnimationFrame(() => {

```

```

<div className="flex flex-col lg:flex-row max-w-
  {/* Сайдбар только на десктопе — на мобильном
  <div
    id="sidebar-root"
    className={`hidden lg:block shrink-0 border
      -all duration-200 print:hidden ${sidebarCol
    " : "lg:w-56"}` }
  >
    {!sidebarCollapsed && (
      <Sidebar selectedChapterId={activeChapterId}
    )}
  </div>
  {/* Вкладка-переключатель содержания у левого
  <button
    type="button"
    onClick={toggleSidebar}
    className="hidden lg:flex items-center gap-1
    border-slate-700 bg-slate-900/90 px-1 py-3
    aria-label={sidebarMode === "wide" ? "Сжать
    ание"}
    title={sidebarMode === "wide" ? "Сжать содер
    }
  >
    {sidebarCollapsed ? <PanelLeftOpen className="
    <span className="text-[10px] font-bold [wri
  </button>

  <main id="main-content" className="flex-1 min
    {/* Шапка главы с быстрыми переходами */}
    <div className="flex items-center justify-b
      <div className="min-w-0">
        <div className="text-[10px] uppercase t
          {activeChapter.category || "Универсальн
        </div>
        <h2 className="text-base sm:text-xl font
          {activeChapter.title}
        </h2>
      </div>
    </div>

```

```
        onHeightChange={setCompilerHeight}
        onClose={() => setCompilerOpen(false)}
      />
    )}
  </div>
);
}
```

---

ФАЙЛ 12/111: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 129 | РАЗМЕР: 5.1 K

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

/** Числовые константы TreeWalker: глобального `NodeFile
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

const escapeRe = (s: string) => s.replace(/[\.\*\+\?\^\$\{\}\(\)]/

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
```

```

    re.lastIndex = 0;
    if (!re.test(text)) continue;
    re.lastIndex = 0;

    const frag = document.createDocumentFragment();
    let last = 0;
    let m: RegExpExecArray | null;

    while ((m = re.exec(text)) !== null) {
        const surface = m[1].toLowerCase();
        const id = labelToId.get(surface);
        if (!id) continue;

        const usedTerm = perTerm.get(id) ?? 0;
        const usedSurface = perSurface.get(surface) ?? 0;
        if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
        perTerm.set(id, usedTerm + 1);
        perSurface.set(surface, usedSurface + 1);

        if (m.index > last) frag.appendChild(document.createElement("span"));
        const span = document.createElement("span");
        span.className = "term-hint";
        span.dataset.termId = id;
        span.tabIndex = 0;
        span.setAttribute("role", "button");
        span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
        span.textContent = m[1];
        frag.appendChild(span);
        last = m.index + m[1].length;
    }

    if (last === 0) continue;
    if (last < text.length) frag.appendChild(document.createElement("span"));
    textNode.parentNode?.replaceChild(frag, textNode);
}

/**

```

```
.neon-glow-blue {
  box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
}
.neon-glow-emerald {
  box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
}
.neon-glow-rose {
  box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
}
.neon-glow-yellow {
  box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
}
.no-scrollbar::-webkit-scrollbar {
  display: none;
}
.no-scrollbar {
  scrollbar-width: none;
}
}

@keyframes pulse-gold {
  0%,
  100% {
    stroke: #eab308;
    stroke-width: 5;
    filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5));
  }
  50% {
    stroke: #fef08a;
    stroke-width: 8;
    filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9));
  }
}

.animate-pulse-gold {
  animation: pulse-gold 1.5s infinite;
}
```

```
    color: #fff;
    outline: none;
}
@media (hover: none) {
    .term-hint {
        border-bottom-style: solid;
        background: rgba(99, 102, 241, 0.12);
    }
}

/* Ссылка на другой билет («...берите Форда–Беллмана (билет
.ticket-link {
    color: #a5b4fc;
    border-bottom: 1px dotted rgba(165, 180, 252, 0.7);
    text-decoration: none;
    border-radius: 3px;
    padding: 0 1px;
    cursor: pointer;
    transition:
        background-color 0.15s ease,
        color 0.15s ease;
}
.ticket-link:hover,
.ticket-link:focus-visible {
    background: rgba(99, 102, 241, 0.25);
    color: #fff;
    outline: none;
}

/* Контент главы приходит готовым HTML – чиним его под
.chapter-body {
    max-width: 100%;
    overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
    min-width: 0;
}
```



```
    line-height: 1.3 !important;
  }
  .chapter-body :where(h3) {
    font-size: 1.05rem !important;
    line-height: 1.35 !important;
  }
  .chapter-body :where(.p-6, .p-8) {
    padding: 0.9rem !important;
  }
  .chapter-body :where(.text-3xl) {
    font-size: 1.4rem !important;
  }
  .chapter-body :where(.text-2xl) {
    font-size: 1.2rem !important;
  }
  .chapter-body :where(.text-xl) {
    font-size: 1.05rem !important;
  }
}
```

```
/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}
::-webkit-scrollbar-track {
  background: #0f172a;
}
::-webkit-scrollbar-thumb {
  background: #334155;
  border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
  background: #475569;
}
```

```
/* Индикатор длительности кадра в пошаговых визуализаци  
@keyframes demoProgress {
```

```
main {
  padding: 0 !important;
  max-width: none !important;
}

/* текст темы – чёрным по белому, без теней и плашек
article.chapter-body .prose,
article.chapter-body .prose * {
  color: #111827 !important;
  background-color: transparent !important;
  background-image: none !important;
  box-shadow: none !important;
  text-shadow: none !important;
  border-color: #d1d5db !important;
}

article.chapter-body .prose pre,
article.chapter-body .prose code {
  background-color: #f3f4f6 !important;
  color: #111827 !important;
  font-size: 9pt;
}

article.chapter-body .prose table {
  border-collapse: collapse;
}

article.chapter-body .prose th,
article.chapter-body .prose td {
  border: 1px solid #9ca3af !important;
}

article.chapter-body .prose h2,
article.chapter-body .prose h3,
article.chapter-body .prose h4 {
  page-break-after: avoid;
}
```

Универсальное пособие • полный исходный код

---

ФАЙЛ 14/111: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

---

ФАЙЛ 15/111: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
  id: string;
  title: string;
  type: "html" | "markdown";
  content: string;
  description?: string;
  category?: string;
}
```

---

ФАЙЛ 16/111: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

---

```
/// <reference types="vite/client" />
```

```
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
```

```
<div class="space-y-8">
```

```
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-blue-400">
```

```
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-blue-300 italic">
      <p class="text-xl font-mono text-white">
      <p class="text-sm font-mono text-slate-300">
    ] [k] </p>
  </div>
```

```
<p class="text-slate-300 text-sm mb-4">Ячейка
лины <span class="font-mono text-emerald-300">
">i</span>. Любой отрезок накрывается <b>двумя
перация <b>идемпотентна</b>: <span class="font-mono text-emerald-300">min</span>, <span class="font-mono text-emerald-300">
, побитовые И и ИЛИ.</p>
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-indigo-300 italic">
  <p class="text-xl font-mono text-white">
  <p class="text-sm font-mono text-slate-300">
вет = op из четырёх блоков kx × ky, накрыва
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Двумя
по столбцам. Отсюда и цена: уровней становится
s="font-mono text-emerald-300">log N</span>
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
  <div class="bg-slate-900 border border-slate-700">
    <p class="text-[10px] uppercase tracking-wider">
```

```

        <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
            Аналогия 2: Торт-пирамидка
        </div>
        <p class="text-slate-300 text-sm mb-4">
            ия, зато красивый, как пирамидка: каждый из
            сех блоков  $2^{kx} \times 2^{ky}$ ), и ответ на запрос
            ой операции нет: крем обратно в торт не запы
             $n(x, x) = x$ ), и цена  $O(1)$  – сто коржей вмес
            <div class="rounded-lg bg-slate-950">
                <div><span class="text-slate-500">
                     $n \ O(1)$ </span></div>
                <div><span class="text-slate-500">
                    = 100 копий матрицы</span></div>
            </div>
            <p class="text-[12px] text-slate-400">
                шает). Менять элементы тоже нельзя. Сумму ч
            </div>
        </div>
        <p class="text-slate-400 text-sm mt-4">Поря
            ерению) → <b>2D: построение</b> (уровни <span
            ос</b> (прямоугольник раскладывается на чет
        </div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-rose-400">
    <p class="text-slate-300 text-sm mb-4">Эти
        за <span class="font-mono text-emerald-300">
        они решают разные задачи.</p>

<div class="bg-slate-800/70 p-5 rounded-lg">
    <p class="text-sm font-bold text-white">
    <p class="text-slate-300 text-sm mb-3">
        цы, у разреженной таблицы – размер, умножен
        <table class="w-full text-xs text-left">
            <thead>
                <tr class="text-slate-500 border">
                    <th class="py-1.5 pr-2 font
```

Почему префиксная тонкая

</div>

<p class="text-slate-300 text-sm mb-1" style="border: 1px solid #000; padding: 5px; display: inline-block;">P[i][j]</p>  
ругой прямоугольник выражается через четыре

</div>

<div class="bg-slate-900 p-6 rounded-lg">

<div class="absolute -top-3 left-4 border-rose-500 shadow-md">

Почему разреженная жирная

</div>

<p class="text-slate-300 text-sm mb-1" style="border: 1px solid #000; padding: 5px; display: inline-block;">ум на его части не вычислишь. Приходится хранить  
xt-emerald-300">log N × log M</span>. На ка

</div>

</div>

<p class="text-slate-300 text-sm mt-4">Вывод:  
лицу</b> – перекрывающиеся блоки посчитают  
</b> – у минимума нет обратной операции. Стр

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border">

<h3 class="text-xl font-bold text-amber-400">

<div class="bg-rose-950/30 p-5 rounded-lg border">

<ul class="list-disc pl-5 space-y-2 text">

<li><b>Сумма через разреженную таблицу пре-  
ефиксные (билет 5).</li>

<li><b>Память.</b> N = M = 1000 даём  
считайте ячейки, потом пишите код.</li>

<li><b>Один log вместо двух.</b> Вспомогательная таблица  
span class="font-mono text-emerald-300">str[i][j] = st[i][j] + st[i-1][j-1]  
</li>

<li><b>Выход за границу в запросе.</b> Вспомогательная таблица  
l</span> и <span class="font-mono text-emerald-300">str[i][j] = st[i][j] + st[i-1][j-1]

<li><b>Нужны обновления.</b> Разреженная таблица (билет 1), а не она.</li>

</ul>

```

        else
            st[r][c][kx][ky] = min(
    
```

```

// запрос минимума на прямоугольнике (R1,C1)..(R2,C2)
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 &lt;&lt; ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 &lt;&lt; kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);
    
```

Четыре запроса к четырем углам запроса и вместе с ним образуют минимальную сумму.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 = 100
- Префиксная сумма для той же матрицы

Почему это так много? Потому что для каждого запроса нужно вычислить сумму по всем уровням, а запросов очень много.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 = 100
- Префиксная сумма для той же матрицы

```
<div class="text-center my-6">
  <p class="text-slate-400 text-xs uppercase">
    <p class="my-1 font-bold leading-none">
      <span class="text-2xl text-slate-300">И</span>
    </p>
  <p class="text-slate-300 font-mono text-sm">
  </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
      Аналогия 1: Split – таможня
    </div>
    <p class="text-slate-300 text-sm">С
    х». Спуск от корня: узел, который вместе с
    скрытия</b> – досматривается только одна ветвь
  </div>
```

```
  <div class="bg-slate-900 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
      Аналогия 2: Merge – стыковка
    </div>
    <p class="text-slate-300 text-sm">М
    аг – одно сравнение: чей у у корня больше,
    сохраняет порядок х). Итого  $O(h)$  сравнений
  </div>
</div>
```

```
<div class="bg-slate-900/40 rounded-lg border">
  <p class="text-slate-300 text-sm mb-2">
  <p class="text-slate-400 text-sm mb-1">
    быстрее  $n \cdot \log n$ .</p>
  <p class="text-slate-400 text-sm mb-1">
  <p class="text-slate-400 text-sm">• «У6
```



```

<summary class="p-4 font-bold text-slate-700 border-slate-700/50">
    Код: вставка, merge, erase (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300">
    <pre class="bg-slate-950 p-4 rounded">
def insert(root, key, pri):
    if root is None:
        return Node(key, pri)
    if key <= root.key: # равные x — левый
        root.left = insert(root.left, key, pri)
    else:
        root.right = insert(root.right, key, pri)
    return root

for key, pri in pairs_sorted_by_pri_desc: # порядок э
    root = insert(root, key, pri)</pre>
    <pre class="bg-slate-950 p-4 rounded">
def merge(a, b): # все x в a < все x в b
    if not a or not b: return a or b
    if a.pri > b.pri:
        a.right = merge(a.right, b); return a
    b.left = merge(a, b.left); return b

def erase(root, key): # удалить произвольный элемент
    if key < root.key: root.left = erase(root.left, key)
    elif key > root.key: root.right = erase(root.right, key)
    else: root = merge(root.left, root.right)
    return root</pre>
    </div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "splay-tree",
        title: "4. Splay-дерево",
    },

```

по факту

: ничего не хранит, дереву разрё

```
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Стопка бумаг

```
</div>
<p class="text-slate-300 text-sm">С
тировку не ведёшь, но через неделю самое ну
ерх».</p>
```

```
</div>
<div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Аналогия 2: Тропинка через га.

```
</div>
<p class="text-slate-300 text-sm">И,
рно вдвое. Первый проход дорогой, зато опла
ы.</p>
```

```
</div>
</div>
```

```
<h4 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Повор
Среднее поддерево  $\beta$  (между  $x$  и  $p$ ) – единств
 $x$ , и под  $p$ .</p>
```

```
<pre class="bg-slate-950 p-4 rounded-lg ove
ading-relaxed">
```

```

      /  \ \
     x    y
    /  \ \
   α    β
      /  \ \
     x    y
    /  \ \
   α    β

      x выше
      -----> α
      /  \ \
     p    r
    /  \ \
   β    γ

      p выше
      <-----
      /  \ \
     p    r
    /  \ \
   β    γ
  </pre>
```

```
<p class="text-slate-300 text-sm mt-3">Обхо
омать дерево поиска поворотами невозможно в
омент «отстёжки», а врезка «Механика одного
```

```
<h4 class="text-lg font-bold text-white mt-3">
```

```
<pre class="bg-slate-950 p-4 rounded-lg">
    амбук из 8 узлов 8→7→6→5→4→3→2→1, ищем 1 (ч
наивно (x, потом x):    3 × «два зига» + Zig → высота 8
Zig-Zig (p, потом x):    3 × Zig-Zig    + Zig → высота 8
бамбук из 16 узлов:      наивно 16 → 16, Zig-Zig 16 → 10<
```

```
    <p class="text-slate-300 text-sm mt-3">
де: <code class="bg-slate-950 px-2 py-0.5 r
s="bg-slate-950 px-2 py-0.5 rounded font-mon
изации не будет.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-6">Полн
м 1. Спуск 8 узлов, затем <b>три Zig-Zig по
8 до 6 – линия перестала быть линией. Наивно
о перевёрнутой. Важно не врать себе: на кор
ван не на одном запросе, а в сумме. Следующ
по пути</b>. Отсюда амортизация: дорогой за
дом потенциала: потенциал  $\Phi$  = сумма по узла
ь его; отсюда же теорема о статической опти
ь запросов).</p>
```

```
<div class="bg-slate-900/40 rounded-lg bord
    <p class="text-slate-300 text-sm mb-2">
    <p class="text-slate-400 text-sm mb-1">
га» – просто переворот.</p>
    <p class="text-slate-400 text-sm mb-1">
иза к верху; в симуляторе оно подсвечиваетс
    <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
ое <code class="bg-slate-950 px-2 py-0.5 ro
одиночная операция – до <code class="bg-sla
>O(n)</code>; n вставок подряд – амортизиро
-300 border border-slate-800">O(n log n)</c
```

```
<p class="text-slate-300 text-sm mt-4">Все
```

```

        rotate_up(x)
    else:                                     # ZIG-ZAG: НИЖНЯЯ
        rotate_up(x)
        rotate_up(x)
    return x

def find(root, key):                         # поиск: спуск + s
    cur, last = root, None
    while cur:
        last = cur
        if key == cur.key: break
        cur = cur.left if key < cur.key else cur.right
    return splay(last) if last else None</pre>
    <p class="text-xs text-slate-400">В
        Одна строка – и вся амортизация.</p>
    </div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "prefix-sums-2d",
        title: "5. Двумерная матрица префиксных сумм",
        type: "html",
        description: "Сумма подматрицы за O(1): 1D и 2D, дв
        category: "Алгоритмы матрицы",
        content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-v
    </div>

    <div class="space-y-8">

        <div class="bg-slate-700/50 p-6 rounded-xl bord
            <h3 class="text-xl font-bold text-emerald-4

```

```

        <td class="py-2 pr-3 font-mono">
      </td>
    </tr>
  </tbody>
</table>
<p class="text-slate-400 text-xs mt-2">
  ышли за границу» – поэтому в 2D почти всегда
</div>

<div class="grid grid-cols-1 sm:grid-cols-3">
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
    <p class="text-[11px] text-slate-500
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
    <p class="text-[11px] text-slate-500
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-rose-300 t
    <p class="text-[11px] text-slate-500
  </div>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
  <h3 class="text-xl font-bold text-blue-400 m
  <div class="grid grid-cols-1 xl:grid-cols-2">
    <div class="bg-slate-800
      <div class="absolute -top-3 left-4 l
      rder border-emerald-500 shadow-md">
        Аналогия 1: Сетка из вафельных
      </div>
      <p class="text-slate-300 text-sm mb
    о накопилось от угла (0,0) до меня». Хочешь
    у и верни дважды отрезанный угол. Вычитание

```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
    Тонкая: одна матрица
  </div>
  <p class="text-slate-300 text-sm mb-2">
    одной матрицы (плюс рамка из нулей). Никаких вычитаний, только сложение. Прямоугольника от угла. Всё остальное получаем из
  </div>
  <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
    <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
      Не умеет: всё, что нельзя вычислить с помощью
    </div>
    <p class="text-slate-300 text-sm mb-2">
      максимума или gcd вычитания не существует:
      е запросы – это <a class="ticket-link" data-ticket="1" href="#">
      платит она за это памятью <span class="font-mono">
    </div>
  </div>

  <div class="bg-slate-800/70 p-5 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
    <p class="text-sm font-bold text-white">
    <table class="w-full text-xs text-left" style="width: 100%; border-collapse: collapse;">
      <thead>
        <tr class="text-slate-500 border-b border-slate-700">
          <th class="py-1.5 pr-2 font-normal">
          <th class="py-1.5 pr-2 font-normal">
          <th class="py-1.5 pr-2 font-normal">
        </tr>
      </thead>
      <tbody>
        <tr class="border-b border-slate-700">
          <td class="py-2 pr-3 font-normal">
          <td class="py-2 pr-3 font-normal">
          <td class="py-2 pr-3 font-normal">
        </tr>
        <tr class="border-b border-slate-700">

```

```

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
  -b border-slate-700/50">
    Код: 1D (тот же, что выполняет пане
  </summary>
  <div class="p-5 text-sm text-slate-300
    <pre class="bg-slate-950 p-4 rounde
      = [3, 1, 4, 1, 5, 9, 2, 6]    # исходный ма
n = len(a)                        # префиксов будет n + 1
P = [0]                          # P[0] = 0 – пустой преф
for i in range(1, n + 1):
    P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)</code></pre>
    <p class="text-slate-300 text-sm">П
  - часть демонстрации: лента подсвечивает, из
    </div>
</details>

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
  -b border-slate-700/50">
    Код: 2D – построение и запрос прямо
  </summary>
  <div class="p-5 text-sm text-slate-300
    <pre class="bg-slate-950 p-4 rounde
      = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4

S = []                            # рамка из нуле
for rr in range(n + 1):
    S.append([0] + [0] * m)
for i in range(1, n + 1):
    for j in range(1, m + 1):
        S[i][j] = A[i-1][j-1] + S[i-1][j] + S[i][j-1] -

r1, c1, r2, c2 = 1, 1, 2, 2      # прямоугольник
ans = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1]
print("итого:", ans)</code></pre>

```

```

    },
    {
      id: "graph-planar-colors",
      title: "7. Графы. Планарные. Покраска",
      type: "html",
      description: "Формула Эйлера и теорема о 4 красках.",
      category: "Графы. Теория",
      content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">Покраска
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.</p>
        <p class="text-xl font-mono text-white">
          1. Если граф планарный, то он можно вложить в плоскость так, чтобы никакие две
          его ребра не пересекались.
          2. Если граф можно вложить в плоскость, то он планарный.
          3. Если граф можно раскрасить 4 цветами, то он планарный.
          4. Если граф планарный, то он можно раскрасить 4 цветами.
        </p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Карта мира
          </div>
          <p class="text-slate-300 text-sm mb-4">
            Если карта мира раскрасить 4 цветами, то она планарная.
            Границы не пересекаются в одной точке по-прежнему не сливались цветами, Всегда можно всего 4
          </p>
        </div>
      </div>
    </div>
  </div>
</section>`,
    },
    {
      id: "top-sort",

```



```

    </div>
</section>`,
  },
  {
    id: "scc-kosaraju",
    title: "10. Графы. Компоненты сильной связности (SCC)",
    type: "html",
    description: "Разбиение орграфа на макро-вершины. Алгоритм Косарaju",
    category: "Графы. Продвинутое",
    content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">10.1. Компоненты сильной связности (SCC)
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Сильная связность графа — это свойство, при котором для любых двух вершин u и v графа существует путь из u в v и из v в u. Другими словами, граф является сильно связным, если для любых двух вершин u и v графа существует путь из u в v и из v в u.
        <p class="text-xl font-mono text-white">Алгоритм Косарaju позволяет найти все компоненты сильной связности графа, инвертированному в порядке top-sort.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Улицы с односторонним движением
          </div>
          <p class="text-slate-300 text-sm mb-4">Представьте себе карту города, где улицы имеют одностороннее движение. Если вы можете пройти по улицам в любом направлении, то город является сильно связным. В противном случае, город разделен на компоненты сильной связности.
        </div>
        <div class="bg-slate-800 p-6 rounded-lg">
          <p class="text-slate-300 text-sm mb-4">ВЫХ направлениях по односторонним улицам. Карту можно увидеть на карте.</p>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },

```

```

        description: "Вершины, удаление которых убивает свя
        category: "Графы. Продвинутые",
        content: `
<section id="graph-articulation" class="mb-12 scroll-mt
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 r
        <h2 class="text-2xl sm:text-3xl font-bold text-v
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl bord
            <h3 class="text-xl font-bold text-rose-400 m
            <div class="bg-slate-900 p-4 rounded-lg mb-4
                <p class="text-lg text-rose-300 italic m
                <p class="text-xl font-mono text-white">
тей > 1).</p>
            </div>
            <div class="grid grid-cols-1 gap-6">
                <div class="bg-slate-800 p-6 rounded-lg
                    <div class="absolute -top-3 left-4 l
                        rder border-emerald-500 shadow-md">
                            Аналогия 1: Центральный роутер
                        </div>
                        <p class="text-slate-300 text-sm mb
                            правильный роутер - и два сегмента офиса б
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>
</section>`,
    },
    {
        id: "graph-euler",
        title: "13. Графы. Эйлеров цикл",
        type: "html",
        description: "Пройти по всем ребрам ровно 1 раз.",
        category: "Графы",
        content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-blue-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-1"
      <p class="text-lg text-blue-300 italic"
      <p class="text-xl font-mono text-white"
d-длина.</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
          Аналогия: Копаем туннель под .
        </div>
        <p class="text-slate-300 text-sm mb
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площ
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "mst-kruskal",
    title: "18. Графы. Остовное дерево. Краскал",
    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-v
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-emerald-4

```

```

        которое торчит из посещенных в непосещенных
    </div>
    <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                ✦ Аналогия: Заражение вирусом
            </div>
            <p class="text-slate-300 text-sm mb-2">
                ли вирус). Мы стартуем из одного города, и
                Сеть растет только из одного связного куска
            </p>
        </div>
    </div>
</div>
</section>`,
    },
    {
        id: "mst-boruvka",
        title: "20. Графы. Остовное дерево. Борувка",
        type: "html",
        description: "",
        category: "Графы. Остовные",
        content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
                </div>
            <div class="space-y-8">
                <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
                    <h3 class="text-xl font-bold text-emerald-400">
                        <div class="bg-slate-900 p-4 rounded-lg mb-4">
                            <p class="text-lg text-emerald-300 italic">
                                <p class="text-xl font-mono text-white">
                                    ается с соседом.</p>
                                </div>
                            <div class="grid grid-cols-1 gap-6">

```

```

        rder border-emerald-500 shadow-md">
            Аналогия 1: Бумеранг (Префикс
        </div>
        <p class="text-slate-300 text-sm mb
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
меня перепроверок!</p>
    </div>
</div>
<details class="bg-slate-900/40 rounded-lg
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "string-z-func",
        title: "22. Z-функция",
        type: "html",
        description: "",
        category: "Строки",
        content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

    </div>
</section>`,
  },
  {
    id: "complexity-classes",
    title: "24. Классы сложности, сведение задач",
    type: "html",
    description: "",
    category: "Теория сложности",
    content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1 rounded-md">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
        <h3 class="text-xl font-bold text-purple-400">
          <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-purple-300 italic">
              <p class="text-base sm:text-lg font-mono">
                <b>P</b>: класс decision-задач, решаемый за полиномиальное время.
                <b>NP</b>: класс decision-задач с сертификатом.
              </p>
            </div>
            <p>
              <b>Сведение (Reduction) A ≤p B</b>: задача A сводится к задаче B.
            </p>
          </div>
          <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
            <!-- Аналогия 1 -->
            <div class="bg-slate-800 p-6 rounded-lg">
              <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                Аналогия 1: Судоку (P vs NP)
              </div>
              <p class="text-slate-300 text-sm mb-4">
                Пример сортировка чисел). Класс <b>NP</b> – класс задач, для которых существует полиномиальную сетку (ответ), он БЫСТРО (за класс P)
              </div>

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-blue-400"

  <div class="bg-slate-900 p-4 rounded-lg mb-4"
    <p class="text-lg text-blue-300 italic"
    <div class="text-left font-mono text-sm"
      <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
      <p><strong class="text-white">Эйлер
      <div class="p-3 bg-slate-950 rounded"
        <p class="text-sm text-blue-300"
        <p><strong class="text-white">П
        <p><strong class="text-white">Ц
      </div>
    </div>
  </div>
</div>

<p class="text-slate-300 text-sm">
  <strong>Билет 13 (Различие пути и цикла
  <br>• <span class="text-green-400 font-
шел-вошёл-вышел).
  <br>• <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap"
  <div class="bg-slate-800 p-6 rounded-lg bor"
    <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
      Аналогия 1: Путь – из дома в бар,
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Ты вышел из дома <strong>(нечётная)
      талый пришёл в бар <strong>(вторая нечётная
      Домой вернуться не смог, потому что
    </p>

```

```

    </div>
</section>`,
  },
  {
    id: "bridges-code",
    title: "11. Графы. Мосты",
    type: "html",
    description: "Мосты через DFS Тарьяна: tin/low, усл",
    category: "Графы. Продвинутые",
    content: `<section id="bridges-code" class="mb-20 s
<div class="flex items-center mb-6">
  <span class="bg-emerald-600 text-white px-4 py-
  <h2 class="text-3xl font-bold text-white">Мосты
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-emerald-4

    <div class="bg-slate-900 p-4 rounded-lg mb-
      <p class="text-lg text-emerald-300 ital
      <div class="text-left font-mono text-sm
        <p><strong class="text-white">tin[v
        <p><strong class="text-white">low[v
        <div class="p-3 bg-slate-950 rounde
          <span class="text-rose-400 font
          <p class="text-xl font-bold tex
          <p class="text-xs text-slate-40
        </div>
      </div>
    </div>
  </div>
  <div class="bg-slate-900 p-4 rounded-lg mb-
    <p class="text-slate-300 text-sm">
      Если из сына u и всех его потомков <str
      Единственная ниточка. Порвётся – граф р
    </p>
  </div>
</div>

```



```
-----
<pre class="text-xs font-mono text-
#include <vector>;
#include <algorithm>;

using namespace std;

int n;                // число вершин
vector<vector<int>>> adj; // список смежнос
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;           // устанавливаем
    for (int to : adj[v]) {
        if (to == p) continue;          // не идём назад
        if (visited[to]) {
            // обратное ребро: обновляем low
            low[v] = min(low[v], tin[to]);
        } else {
            // ребро дерева DFS
            dfs(to, v);                  // рекурсивный в
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v]) {
                // ЭТО МОСТ!
                // bridge_list.push_back({v, to});
            }
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
}
```

```

        <p><span class="text-emerald-400 font-l
        <p><span class="text-rose-400 font-l
    </div>
</div>

<p class="text-slate-300 text-sm">
    <strong>Следствие:</strong> Для планарн
ode>.
    Если ребер больше – граф гарантированно
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
    <div class="bg-slate-800 p-6 rounded-lg bor
        <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
            K5: Полный граф на 5 вершинах
        </div>
        <p class="text-slate-300 text-sm mb-4">
            У K5: <code class="text-white">V=5,
-white">10 &gt; 9</code> – БАХ, непланарен!
            Все 5 вершин соединены со всеми. На
            Теорема Куратовского: K5 и K3,3 – д
            Если внутри графа спрятано <b>подра
            – плоской раскладки не будет.
        </p>
    </div>

<div class="bg-slate-900 p-6 rounded-lg bor
    <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
        K3,3: 3 дома и 3 колодца
    </div>
    <p class="text-slate-300 text-sm mb-4">
        Три дома хотят соединиться с тремя
        Нарисовать без пересечений невозможно
        У него <code class="text-white">V=6
xt-white">9 ≤ 12</code> – проходит, но он в
```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Расписание экзамен

```
</div>
```

```
<p class="text-slate-300 text-sm">В
слот времени. Раскрасить граф = составить р
«берём предметы в каком-то порядке и ставим
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
border border-fuchsia-500 shadow-md">
```

Аналогия 2: Частоты вышек

```
</div>
```

```
<p class="text-slate-300 text-sm">С
цвет, интерференция = ребро. Диапазон частот
регистров в компиляторе: «живые» одновременно
</div>
```

```
</div>
```

```
<div class="overflow-x-auto my-6">
```

```
<table class="w-full text-xs text-left"
  <thead>
```

```
<tr class="text-slate-500 border-b"
  <th class="py-2 pr-3 font-b"
  <th class="py-2 pr-3 font-b"
  <th class="py-2 font-bold">I
```

```
</tr>
```

```
</thead>
```

```
<tbody class="text-slate-300">
```

```
<tr class="border-b border-slate-800"
><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta + 1$ <
а свободен</td></tr>
```

```
<tr class="border-b border-slate-800"
d><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta$ </td>
/td></tr>
```

```

        веркой цветов соседей; проверка двудольности
border border-slate-800">0(V + E)</code>; т
-300 border border-slate-800">NP-трудно</co

<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код: жадная раскраска и проверка
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounded
я раскраска: не больше  $\Delta + 1$  цвета
color = [-1] * n                # -1 = ещё не покрашен
for v in order:                  # порядок влияет на чи
    used = set()
    for to in graph[v]:
        if color[to] != -1:
            used.add(color[to]) # цвета уже покрашены
    c = 0
    while c in used:              # минимальный свободны
        c += 1
    color[v] = c
chi = max(color) + 1             #  $\chi$ , который выдал жад

# Проверка двудольности = раскраска в 2 цвета (BFS)
from collections import deque
col = [-1] * n
bipartite = True
for start in range(n):
    if col[start] != -1:
        continue
    col[start] = 0
    dq = deque([start])
    while dq:
        v = dq.popleft()
        for to in graph[v]:
            if col[to] == -1:
                col[to] = col[v] ^ 1 # второй цвет

```

```
<p class="text-slate-300 text-sm">
  <strong>Важно:</strong> Это работает то
  епятся к точкам сочленения. То есть, <strong>
  ег - это тупик со степенью 1).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-slate-900">
    <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
      Аналогия: Главный перекресток
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь город, где два района соединены
      её перекроют (удалят вершину) — районы будут
      <strong>хрупкость</strong> системы.
    </p>
  </div>

  <div class="bg-slate-900 p-6 rounded-lg border border-slate-950">
    <div class="absolute -top-3 left-4 bg-slate-900 border border-emerald-500 shadow-md">
      Телеком: Центральный свитч
    </div>
    <p class="text-slate-300 text-sm mb-4">
      У тебя несколько компьютеров в одной
      абелем (мостом). Сами свитчи — это точки со
    </p>
  </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-950">
  <summary class="p-4 font-bold text-slate-400">
    Душный мод: Код и корень DFS (Скрыто)
  </summary>
```

```

        <ul class="list-disc list-inside text-sm text-rose-400">
            <li><strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
            <li><strong class="text-emerald-400">SCC
ависимые "конгломераты".</li>
            <li><strong>Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S
        </ul>
    </div>
</div>
</section>`,
    },
];

```

ФАЙЛ 19/111: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТР0К: 145 | РАЗМ

```

import { Chapter } from "../types";
import { graphChapters } from "./graphContent";
import { additionalTickets } from "./additionalTickets";
import { tickets14to20 } from "./tickets/ticket14_20";
import { tickets21to24 } from "./tickets/ticket21_24";
import { tickets6to13 } from "./tickets/ticket6_13";
import { ticket1Segtree } from "./tickets/ticket1_segtr
import { tickets18to20Mst } from "./tickets/ticket18_20
import { chapters as newChapters } from "./content_temp

/**
 * Содержание пособия = билеты 1–24 из списка тем.
 *
 * Порядок массивов важен: при одинаковом id побеждает
 * версия главы. `newChapters` (content temp) перетирае

```

```
merged.forEach((c) => {
  if (!toRemove.includes(c.id)) {
    dedup.set(c.id, c);
  }
});

// Самые свежие версии страниц перетирают старые.
if (newChapters) {
  newChapters.forEach((c) => {
    if (!toRemove.includes(c.id)) dedup.set(c.id, c);
  });
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
  // Главы без номера билета (введение) стоят ДО билетов
  const numOf = (t: string) => {
    const m = t.match(/(\d+)/);
    return m ? parseInt(m[0], 10) : -1;
  };
  return numOf(a.title) - numOf(b.title);
});

export const chapters: Chapter[] = finalChapters;

/**
 * Билеты, закрытые каждой страницей (для шапки, экспор
 * Страница `mst`, например, закрывает сразу 18, 19 и 20
 */
export const chapterTopics: Record<string, number[]> = {
  "segment-trees": [1],
  "sparse-table": [2],
  treap: [3],
  "splay-tree": [4],
  "prefix-sums-2d": [5],
  "graph-dfs-bfs": [6],
  "planarity-euler-formula": [7],

```

```

-----
const tickets = chapterTopics[chapterId] ?? [];
// Глав без билетов (введение) относим к первому разд
if (!tickets.length) return SECTIONS[0];
const first = Math.min(...tickets);
return SECTIONS.find((sec) => first >= sec.from && fi
}

export const activeVizIds = [
  "euler-path-vs-cycle", "bridges-code", "planarity-euler
  "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se
  "dijkstra", "bellman-ford", "floyd", "aho-corasick",
  "mst", "string-kmp", "string-z-func", "segment-trees", "s
  "dynamic-programming", "johnson-algo", "graph-articulat
  "complexity-classes", "treap", "splay-tree", "prefix-sum
];

```

-----  
ФАЙЛ 20/111: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 207 | РАЗМ

```

-----
/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

```

```

export type DebugValue = null | boolean | number | string

```

```

export interface DebugStep {
  /** Номер строки в коде пользователя (1-based). */
  line: number;
  /** Имя функции-фрейма ("<module>" – верхний уровень)
  func: string;
  /** Локальные переменные: имя → герг (усечённый), для
  locals: Record<string, string>;
  /**

```



```

__steps = []
__final = {}
__final_values = {}
__CAP = ${DEBUG_STEP_CAP}
__was_truncated = False

class __TraceLimit(Exception):
    pass

def __safe(v):
    try:
        r = repr(v)
    except Exception:
        r = "<...>"
    return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
    """Небольшой JSON-снимок: UI использует его без парсера"""
    if v is None or isinstance(v, (bool, int, str)):
        return v
    if isinstance(v, float):
        return v if v == v and v not in (float("inf"), float("-inf")) else "inf/-inf"
    if depth >= 4:
        return __safe(v)
    limit = 80 if depth < 2 else 32
    if isinstance(v, (list, tuple)):
        return [__snapshot(x, depth + 1) for x in list(v)]
    if isinstance(v, (set, frozenset)):
        return [__snapshot(x, depth + 1) for x in list(v)]
    if isinstance(v, dict):
        # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
        # предел 80 незаметно обрезал уровни 4x4 и 8x8
        dict_limit = 192 if depth == 0 else limit
        return {str(k): __snapshot(x, depth + 1) for k, x in list(v.items())}
    # deque, heap-подобные и другие итерируемые учебные
    if type(v).__name__ == "deque":
        return [__snapshot(x, depth + 1) for x in list(v)]
    return __safe(v)

```

```

-----
    # PEP 709: с 3.12 компрекеншны встроены во фрейм
    # на своей строке каждую итерацию. Склеиваем по
    if __steps and __steps[-1]["line"] == frame.f_lineno:
        __steps[-1]["locals"] = loc
        __steps[-1]["values"] = values
    else:
        __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
    if len(__steps) >= __CAP:
        # Нельзя просто отключить trace: бесконечный цикл
        # тогда продолжит работать вечно. Мягко прерываем
        __was_truncated = True
        raise __TraceLimit()
    # Финальные локалы программы (после последней строки)
    if event == "return" and frame.f_code.co_filename == __FILE__:
        loc, values = __capture(frame)
        __final.update(loc)
        __final_values.update(values)
    return __tracer

__err = ""
try:
    __sys.settrace(__tracer)
    exec(compile(__src, "<user-code>", "exec"), {"__name__": "__main__"})
except __TraceLimit:
    pass
except Exception:
    import traceback as __tb
    __err = __tb.format_exc(limit=3)
finally:
    __sys.settrace(None)

__OUT = __json.dumps({
    "steps": __steps,
    "finalLocals": __final,
    "finalValues": __final_values,
    "error": __err,
    "truncated": __was_truncated,
})

```

---

ФАЙЛ 21/111: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 639 | РАЗМ:

---

```
import type { DemoKind } from "../components/hints/Mini"
```

```
/**
```

```
 * Глоссарий «википедийных» подсказок.
```

```
 *
```

```
 * Любое слово из `labels`, встреченное в тексте главы,
```

```
 * в подчёркнутый термин: при наведении (или тапе на мо
```

```
 * с объяснением на пальцах и мини-визуализацией.
```

```
 */
```

```
export interface GlossaryEntry {
```

```
  id: string;
```

```
  /** Варианты написания в тексте (регистр не важен). */
```

```
  labels: string[];
```

```
  title: string;
```

```
  /** Объяснение «на пальцах» – 1–2 фразы. */
```

```
  short: string;
```

```
  /** Строгая формулировка / что важно не забыть. */
```

```
  formal?: string;
```

```
  complexity?: string;
```

```
  /** Какая мини-анимация показывается в карточке. */
```

```
  demo?: DemoKind;
```

```
  /** id полного симулятора (см. vizRegistry) – кнопка
```

```
  simulator?: string;
```

```
}
```

```
export const GLOSSARY: GlossaryEntry[] = [
```

```
  {
```

```
    id: "bfs",
```

```
    labels: ["BFS", "обход в ширину", "поиск в ширину",
```

```
    title: "BFS – обход в ширину",
```

```
    short:
```

"Не отсортированный список, а «мешок, из которого  
formal:

"Полное бинарное дерево в массиве: дети узла  $i$  ле

complexity: "вставка/извлечение  $O(\log n)$ , «посмотре

demo: "heap",

simulator: "heap-beam-search",

},

{

id: "stack",

labels: ["стек", "стека", "стеке", "стеком", "LIFO"]

title: "Стек (LIFO)",

short: "Стопка тарелок: кладёшь сверху и берёшь свер

formal: "push / pop / top за  $O(1)$ . На стеке живёт р

demo: "stack",

simulator: "stack-dfs",

},

{

id: "queue",

labels: ["очередь", "очереди", "очередью", "FIFO"],

title: "Очередь (FIFO)",

short: "Очередь в столовой: кто первым встал, тот п

formal: "push\_back / pop\_front за  $O(1)$ . Основа BFS

demo: "queue",

simulator: "queue-bfs",

},

{

id: "dsu",

labels: ["DSU", "СНМ", "система непересекающихся мн

title: "DSU — система непересекающихся множеств",

short:

"«Кто твой староста?» Каждый элемент знает своего

formal:

"find со сжатием путей + union по рангу/размеру. I

complexity: "почти  $O(1)$  — обратная функция Аккерман

demo: "dsu",

simulator: "mst",

},

{

```

    short:
        "«Куда откатиться, если следующая буква не подошла",
        строки.",
    formal: "link(v) = go(link(parent(v)), ch). Считает",
    demo: "suffix-link",
    simulator: "aho-corasick",
},
{
    id: "toposort",
    labels: ["топологическая сортировка", "топсорт", "топ",
    title: "Топологическая сортировка",
    short:
        "Порядок «сначала штаны, потом ботинки»: каждая с",
        циклов.",
    formal: "DFS + вершины в обратном порядке выхода, л",
    complexity: "O(V + E)",
    demo: "toposort",
    simulator: "top-sort",
},
{
    id: "relaxation",
    labels: ["релаксация", "релаксацию", "релаксации",
    title: "Релаксация ребра",
    short:
        "«А через соседа не быстрее?» Если известный путь",
        ,
    formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)",
        ан и Флойд.",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "prefix-sum",
    labels: ["префиксные суммы", "префиксная сумма", "пре",
    title: "Префиксные суммы",
    short:
        "Заранее посчитанные «сколько всего набежало от н",
    formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r] - P[l-1]"

```

```

    complexity: "O(V + E) одним DFS",
    demo: "bridge",
    simulator: "bridges-code",
},
{
    id: "articulation",
    labels: ["точка сочленения", "точки сочленения", "то"],
    title: "Точка сочленения",
    short:
        "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
    formal: "Для не-корня: v — точка сочленения  Э ребро",
    demo: "articulation",
    simulator: "graph-articulation",
},
{
    id: "mst",
    labels: [
        "остовное дерево", "остовного дерева", "остовное",
        "минимальное остовное дерево", "MST",
    ],
    title: "Минимальное остовное дерево (MST)",
    short:
        "Связать все точки проводами так, чтобы суммарная",
    formal: "V-1 ребро, никаких циклов. Краскал (жадно и",
        "но).",
    demo: "mst",
    simulator: "mst",
},
{
    id: "amortized",
    labels: [
        "амортизированная", "амортизированный", "амортизир",
        "амортизированное", "амортизированного", "амортизир",
    ],
    title: "Амортизированная сложность",
    short:
        "Средняя цена операции «по абонементу»: одна опер",
    formal: "Метод потенциалов: дорогая операция оплачи
```

```
{
  id: "prefix-function",
  labels: [
    "префикс-функция", "префикс-функции", "префикс-фу",
    "префикс", "префикса", "суффиксом", "суффикс", "с",
    "подстроки", "подстроку", "подстрока",
  ],
  title: "Префикс-функция  $\pi$  (КМП)",
  short:
    "Для каждой позиции запоминаем: какой кусок начался",
    "от кусок.",
  formal:
    " $\pi[i]$  — длина наибольшего собственного префикса  $s$ ",
    "возвращая указатель по тексту.",
  complexity: "O(n + m)",
  demo: "prefix-function",
  simulator: "string-kmp",
},
{
  id: "z-function",
  labels: ["Z-функция", "Z-функции", "Z-функцию", "z-"],
  title: "Z-функция",
  short:
    "Для каждой позиции: сколько символов подряд, начи",
    "аждой букве.",
  formal:
    " $z[i]$  = длина наибольшего общего префикса  $s$  и  $s[i$ ",
  complexity: "O(n)",
  demo: "prefix-function",
  simulator: "string-z-func",
},
{
  id: "dp",
  labels: [
    "динамическое программирование", "динамики", "дин",
    "мемоизация", "мемоизацию", "подзадач", "подзадач",
  ],
  title: "Динамическое программирование",
```

```

    short:
      "На каждом шаге хватаем то, что лучше прямо сейчас"
    formal:
      "Корректность обычно доказывается «леммой о разрезе»"
    demo: "mst",
    simulator: "mst",
  },
  {
    id: "euler",
    labels: ["эйлеров цикл", "эйлеров путь", "эйлерова теорема"],
    title: "Эйлеров путь и цикл",
    short:
      "Пройти по каждому ребру ровно один раз, не отрываясь"
    formal:
      "Связный граф: эйлеров цикл  все степени чётные;"
    simulator: "euler-path-vs-cycle",
  },
  {
    id: "planarity",
    labels: ["планарный", "планарность", "планарного", "планаризация"],
    title: "Планарность",
    short:
      "Граф планарен, если его можно нарисовать на листе бумаги"
    formal:
      " $V - E + F = 2$  для связного планарного графа. Курсив — формула Эйлера"
    simulator: "planarity-euler-formula",
  },
  {
    id: "splay",
    labels: [
      "splay", "splay-дерево", "AVL", "AVL-дерево", "всестороннее"
    ],
    title: "Splay и повороты",
    short:
      "Каждый раз, когда к узлу обратились, его «вытягивают»"
    formal:

```



```

        "После k-й итерации гарантированно верны все кратчайшие пути, которые не содержат отрицательный цикл.",
        complexity: "O(V · E)",
        demo: "relax",
        simulator: "bellman-ford",
    },
    {
        id: "floyd",
        labels: [
            "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
            "промежуточная вершина", "промежуточную вершину",
        ],
        title: "Флойд-Уоршелл: разрешаем вершину k",
        short:
            "Внешний цикл — это не «шаг», а «какие пересадки сделать, чтобы пересадку в k?",
        formal:
            "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядок пересадки — k.",
        complexity: "O(V³) времени, O(V²) памяти",
        demo: "floyd",
        simulator: "floyd",
    },
    {
        id: "prim",
        labels: ["Прима", "Прим", "Prim"],
        title: "Алгоритм Прима",
        short:
            "Остов растёт как плесень из одной точки: на каждом шаге добавляем ребро, не создавая циклов.",
        formal:
            "Множество посещённых + куча рёбер на границе. Добавляем ребро с минимальным весом, не создавая циклов.",
        complexity: "O(E log V)",
        demo: "mst",
        simulator: "mst",
    },
    {
        id: "boruvka",

```

```
{
  id: "graph",
  labels: [
    "граф", "графа", "графе", "графы", "графов", "граф",
    "вершины", "вершина", "вершин", "ребра с весом",
  ],
  title: "Граф",
  short:
    "Точки (вершины) и связи между ними (рёбра). Города и дороги.",
  formal:
    " $G = (V, E)$ . Ребро бывает ориентированным (улицы с односторонним движением) и неориентированным (двусторонняя улица).",
  demo: "graph",
  simulator: "graph-dfs-bfs",
},
{
  id: "coord-compress",
  labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
  title: "Сжатие координат",
  short:
    "Координаты бывают гигантские и дробные, а нам нужно хранить их в виде целых чисел. Сжимаем координаты в один порядковый номер.",
  formal:
    "sort + unique + binary search: значение → индекс",
  complexity: " $O(n \log n)$ ",
  demo: "coord-compress",
},
{
  id: "treap",
  labels: ["Treap", "декартово дерево", "декартова д-во"],
  title: "Treap = Tree + Heap",
  short:
    "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету. Случайность и держит его сбалансированным.",
  formal:
    "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить). Ожидаемая высота  $O(\log n)$ .",
  complexity: "ожидаемо  $O(\log n)$ ",
}
```

```

    demo: "np",
  },
  {
    id: "range",
    labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
    title: "Запрос на отрезке",
    short:
      "Спрашиваем не про один элемент, а про кусок массива  

      предподсчёты.",
    formal:
      "Суммы — префиксные суммы  $O(1)$ . Минимум без изменений.",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
  },
  {
    id: "string",
    labels: ["строки", "строке", "строку", "строка", "строкам"],
    title: "Строка как массив символов",
    short:
      "Строковые алгоритмы держатся на трёх вопросах: где  

      (Z-функция) и где встречаются слова словаря (бордер).",
    formal:
      "Наивный поиск подстроки —  $O(N \cdot M)$ . Префикс-функция.",
    demo: "prefix-function",
    simulator: "string-kmp",
  },
  {
    id: "rotation",
    labels: ["поворот", "повороты", "поворота", "поворотом"],
    title: "Поворот (rotation) в дереве",
    short:
      "Локальная перевеска двух узлов: ребёнок встаёт на  

      дившемся месту. Порядок ключей при этом не ломается.",
    formal:
      "Правый поворот вокруг p:  $x = p.left; p.left = x.$   

      g и Zig-Zag.",
    complexity: " $O(1)$ ",
    demo: "zig",
  },

```

Универсальное пособие • полный исходный код

```
-----  
/** Быстрый доступ по id. */  
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {  
  
/** Все метки, отсортированные от длинных к коротким (ч  
export const GLOSSARY_LABELS: { label: string; id: string }[] =  
  e.labels.map((label) => ({ label, id: e.id }))  
}).sort((a, b) => b.label.length - a.label.length);  
  
-----
```

ФАЙЛ 22/111: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРӨК: 411 | РАЗМЕР: 1.2 КБ

```
-----  
import { Chapter } from "../types";  
  
export const graphChapters: Chapter[] = [  
  {  
    id: "intro",  
    title: "Введение. Базовая база: куча, очередь, бинарное  
    type: "html",  
    category: "Введение",  
    content: `  
      <section id="intro-content" class="mb-12">  
        <div class="flex items-center mb-6 flex-wrap gap-2">  
          <span class="bg-indigo-600 text-white px-4 py-1">Базовая база</span>  
          <h2 class="text-3xl font-bold text-white">Базовая база</h2>  
        </div>  
  
        <div class="bg-slate-900 border border-indigo-500/40 p-6">  
          <p class="text-slate-200 text-sm leading-relaxed">  
            <b class="text-indigo-300">Базовая база</b> – это структура данных  
            дерево как массив, куча, очередь и стек, бинарное дерево.  
            Кодим по минимуму: там, где за нас уже написали.  
          </p>  
        </div>  
  
        <div class="space-y-8">
```

```
h = []
heapq.heappush(h, 5); heapq.heappush(h, 1); heapq.heappop(h)
h[0] # 1 – минимум виден без всякого обхода
heapq.heappop(h) # 1, куча починилась за  $O(\log n)$ 
```

<p class="text-slate-300 text-sm">Куча – движок, который поддерживает кучу, раз, и список с <span class="font-mono text-sm"></span></p>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-t-4 border-b-4">
 <h3 class="text-xl font-bold text-sky-400 mb-4">Очереди и стеки
 <p class="text-slate-300 text-sm mb-4"><b>Очереди</b> (FIFO) и <b>стеки</b> (LIFO) отличаются одной операцией снятия элемента</span>: оба конца работают за  $O(1)$ . Обычный список сдвигает весь хвост и стоит  $O(n)$ .</p>
 <pre class="bg-slate-950 border border-slate-700 p-4">
from collections import deque
q = deque([1, 2])
q.append(3); q.popleft() # 1 – очередь (BFS)
s = [1, 2]; s.append(3); s.pop() # 3 – стек (DFS)</pre>
 <p class="text-slate-300 text-sm">Вся разница между BFS (волнами), стек – DFS (вглубь).</p>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-t-4 border-b-4">
 <h3 class="text-xl font-bold text-amber-400 mb-4">Бинарный поиск
 <p class="text-slate-300 text-sm mb-4">Если массив отсортирован, то поиск по середине – понял, в какой половине жить, – и это 20 сравнений. В Python за это отвечает модуль bisect.
 <pre class="bg-slate-950 border border-slate-700 p-4">
from bisect import bisect\_left
a = [10, 20, 30, 40, 50]
bisect\_left(a, 30) # 2 – индекс первого элемента, не меньше 30
 <p class="text-slate-300 text-sm">Без сортировки бинарный поиск не работает.</p>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-t-4 border-b-4">
 <h3 class="text-xl font-bold text-rose-400 mb-4">Сортировки
 <p class="text-slate-300 text-sm mb-4">Писать сортировки можно и вручную, но это долго и скучно. В Python есть встроенные сортировки: list.sort() и sorted(). Они используют алгоритм Timsort, который является гибридом сортировки слиянием и сортировки выбором.
 <pre class="bg-slate-950 border border-slate-700 p-4">
a = [5, 2, 6, -1, 3, 4, 0]
a.sort()
print(a) # [-1, 0, 2, 3, 4, 5, 6]
b = sorted(a)
print(b) # [-1, 0, 2, 3, 4, 5, 6]
 <p class="text-slate-300 text-sm">Сортировка в Python работает за  $O(n \log n)$ .</p>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-t-4 border-b-4">
 <h3 class="text-xl font-bold text-rose-400 mb-4">Сортировки
 <p class="text-slate-300 text-sm mb-4">Писать сортировки можно и вручную, но это долго и скучно. В Python есть встроенные сортировки: list.sort() и sorted(). Они используют алгоритм Timsort, который является гибридом сортировки слиянием и сортировки выбором.
 <pre class="bg-slate-950 border border-slate-700 p-4">
a = [5, 2, 6, -1, 3, 4, 0]
a.sort()
print(a) # [-1, 0, 2, 3, 4, 5, 6]
b = sorted(a)
print(b) # [-1, 0, 2, 3, 4, 5, 6]
 <p class="text-slate-300 text-sm">Сортировка в Python работает за  $O(n \log n)$ .</p>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-t-4 border-b-4">
 <h3 class="text-xl font-bold text-rose-400 mb-4">Сортировки
 <p class="text-slate-300 text-sm mb-4">Писать сортировки можно и вручную, но это долго и скучно. В Python есть встроенные сортировки: list.sort() и sorted(). Они используют алгоритм Timsort, который является гибридом сортировки слиянием и сортировки выбором.
 <pre class="bg-slate-950 border border-slate-700 p-4">
a = [5, 2, 6, -1, 3, 4, 0]
a.sort()
print(a) # [-1, 0, 2, 3, 4, 5, 6]
b = sorted(a)
print(b) # [-1, 0, 2, 3, 4, 5, 6]
 <p class="text-slate-300 text-sm">Сортировка в Python работает за  $O(n \log n)$ .</p>
</div>

```

left(x) = z          # z становится левым р
высоты пересчитываются только у z и x — ост
</div>
</details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
  <h3 class="text-xl font-bold text-teal-400 mb-4
  <p class="text-slate-300 text-sm mb-4">Оба обхо
    тару». Вся разница в таре: стек уходит вглуби
    ext-emerald-300">0(V + E)</span>: каждая верш
    <pre class="bg-slate-950 border border-slate-700
def go(graph, start, tank):          # tank: список-стек
  seen, tank = {start}, tank([start])
  while tank:
    v = tank.pop() if isinstance(tank, list) else t
    for to in graph[v]:
      if to not in seen:
        seen.add(to); tank.append(to)
  return seen</pre>
  <p class="text-slate-300 text-sm">Дальше эта па
    надстройки над этими восемью строками.</p>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
  <h3 class="text-xl font-bold text-slate-300 mb-
  <div class="grid grid-cols-1 xl:grid-cols-2 gap
    <div class="bg-slate-800 p-6 rounded-lg bord
      <div class="absolute -top-3 left-4 bg-s
        border-indigo-500 shadow-md">
          Аналогия 1: Схема кинозала
        </div>
        <p class="text-slate-300 text-sm">Дерев
          азателей-«usher'ов» не нужно. Место n, дети
          рху вниз и слева направо.</p>
        </div>
        <div class="bg-slate-800 p-6 rounded-lg bord
          <div class="absolute -top-3 left-4 bg-s

```

```

title: "Билет 14. Дейкстра",
type: "html",
category: "Графы",
content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 rounded
  <h2 class="text-3xl font-bold text-white">Дейкст
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border
    <h3 class="text-xl font-bold text-emerald-400">Аналогия 1: Позитивное планирование
    </h3>

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">Позитивное планирование — это когда вы знаете, что вы можете добраться до пункта назначения, но вы не знаете, как добраться до пункта назначения.
      <p class="text-xl font-mono text-white">Позитивное планирование — это когда вы знаете, что вы можете добраться до пункта назначения, но вы не знаете, как добраться до пункта назначения.
    </div>

    <div class="grid grid-cols-1 xl:grid-cols-2">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
          Аналогия 1: Позитивное планирование
        </div>
        <p class="text-slate-300 text-sm mb-4">Позитивное планирование — это когда вы знаете, что вы можете добраться до пункта назначения, но вы не знаете, как добраться до пункта назначения. (нельзя поехать и заработать). Он всегда работает.
        <div id="slot-chapter-image-dijkstra" class="h-400px">
        </div>

      <!-- Аналогия 2 -->
      <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
          Ограничения
        </div>
        <p class="text-slate-300 text-sm mb-4">Ограничения — это когда вы знаете, что вы можете добраться до пункта назначения, но вы не знаете, как добраться до пункта назначения. (нельзя поехать и заработать). Он всегда работает.
        <div id="slot-chapter-image-dijkstra" class="h-400px">
        </div>
      </div>
    </div>
  </div>
`

```

```

<div class="grid grid-cols-1 xl:grid-cols-2"
  <!-- Аналогия 1 -->
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
      ♂ Аналогия 1: Параноик
    </div>
    <p class="text-slate-300 text-sm mb-2">
      еряет ВСЕ возможные дороги V-1 раз подряд.
    <div id="slot-chapter-image-bellman"
    </div>

  <!-- Аналогия 2 -->
  <div class="bg-slate-900 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
      Отрицательный цикл
    </div>
    <p class="text-slate-300 text-sm mb-2">
      г станет ЕЩЕ короче – значит мы попали во в
    </div>
  </div>

  <div id="slot-bellman-viz" class="mt-8"></div>
</div>
</section>`
},
{
  id: "floyd",
  title: "Билет 16. Флойд-Уоршелл",
  type: "html",
  category: "Графы",
  content: `<section id="floyd-content" class="mb-12">
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
  <h2 class="text-3xl font-bold text-white">Флойд

```



```

    },
    {
      id: "aho-corasick",
      title: "Билет 23. Алгоритм Ахо-Корасик",
      type: "html",
      category: "Строки",
      content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1 round
  <h2 class="text-2xl sm:text-3xl font-bold text-white
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-l
    <h3 class="text-xl font-bold text-blue-400 mb-4">Ахо-Корасик

    <div class="bg-slate-900 p-4 rounded-lg mb-6 text
      <p class="text-sm text-blue-300 italic mb-2">Осн
      <p class="text-lg font-mono text-white">Бор (Тр
    </div>
    <p class="text-slate-300 text-sm">Позволяет найти
      -mono text-amber-300 bg-black/30 px-1 rounded">
      y.</p>
    </div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border bo
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
      Аналогия 1: Паутина (Бор)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      мы движемся по веткам. Если нужного продолж
      ("нити страховки") в самый длинный из извест
    </div>

    <div class="bg-slate-900 p-6 rounded-lg border-2

```

```

while (!q.empty()) {
    int v = q.front(); q.pop();
    for (auto const& [ch, child] : nodes[v].trie) {
        // Суффиксная ссылка ребёнка — это переход
        nodes[child].link = nodes[nodes[v].link].go

        // Если суффиксная ссылка сама является сло
        // Иначе наследуем terminal_link
        if (nodes[nodes[child].link].is_terminal) {
            nodes[child].term_link = nodes[child].l
        } else {
            nodes[child].term_link = nodes[nodes[ch
        }

        q.push(child);
    }
}
}
</div>
</div>
</details>
</div>
</section>`
},
{
    id: "alg-map",
    title: "Алгоритмы на карте",
    type: "html",
    category: "Графы",
    content: `<section id="alg-map-content" class="mb-1
<div class="flex items-center mb-6">
    <span class="bg-purple-600 text-white px-4 py-1
    <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord

```

```

    options: [
        "Да: минимум и сумма считаются одним приёмом",
        "Нет: блоки перекрываются, а вычесть перекрытие",
        "Да, если заранее хранить суммы в нулевом уровне",
    ],
    correctIndex: 1,
    explanation: "Два (в 2D – четыре) блока накрывают
        обратная операция, но применить её к перекрытию
        ммы, билет 5."
},
{
    question: "Какие операции подходят разреженной та
    options: [
        "Любые ассоциативные",
        "Идемпотентные: min, max, gcd, побитовые И и ИЛИ",
        "Только сумма и произведение"
    ],
    correctIndex: 1,
    explanation: "Нужно  $op(x, x) = x$ : тогда дважды на
        приём и уходит в префиксные суммы."
},
{
    question: "Между запросами элементы матрицы меняю
    options: [
        "Разреженную таблицу: она перестраивается за  $O(n^2)$ ",
        "Префиксную матрицу: достаточно поправить одну
        "Дерево отрезков: и таблица, и префиксы статичны",
    ],
    correctIndex: 2,
    explanation: "Обе структуры статичны: изменение э
        ксной. С обновлениями работает дерево отрезков,
    }
},
"prefix-sums-2d": [
    {
        question: "Сколько памяти занимает двумерная преф
        options: [
            "N × M – как исходная матрица, плюс рамка из ну

```

```

        ной операции нет. Для min, max и gcd берут разро
    },
    {
        question: "Изменили один элемент матрицы. Что ста
        options: [
            "Поправится одна клетка за  $O(1)$ ",
            "Нужен пересчёт  $O(N \cdot M)$ : от этой клетки зависит
            "Ничего не станет: таблица остаётся верной"
        ],
        correctIndex: 1,
        explanation: "Каждая клетка  $S[i][j]$  включает в се
            я частые, это задача дерева отрезков (билет 1).
    }
],
"euler-path-vs-cycle": [
    {
        question: "В связном графе ровно 2 вершины с нечёт
        options: [
            "Есть эйлеров цикл (вернись домой!)",
            "Есть эйлеров путь (из одной нечётной в другую)
            "Ни пути, ни цикла нет — это полный провал"
        ],
        correctIndex: 1,
        explanation: "Если нечётных вершин ровно 2 — это
            ётной, заканчиваем в другой."
    },
    {
        question: "Почему гамильтонов цикл — это NP-полная
        options: [
            "Потому что Гамильтон — болезнь, и лечить её т
            "Потому что для эйлерова цикла есть простой кри
            "Потому что Эйлер был умнее Гамильтона"
        ],
        correctIndex: 1,
        explanation: "Для эйлерова цикла работает критери
            ого критерия. Это NP-полная задача, которую не
    },
    {

```

```

        "0(2^V) – полный перебор"
    ],
    correctIndex: 1,
    explanation: "DFS обходит каждую вершину и каждое
}
],
"planarity-euler-formula": [
{
    question: "Планарный граф имеет V=6, E=12. Возможн
    options: [
        "Да, если это двудольный граф",
        "Нет, нарушается неравенство  $E \leq 3V - 6$ ",
        "Да, если нет треугольных граней"
    ],
    correctIndex: 1,
    explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$  – на грани пр
        Так что такой граф непланарен."
    },
    {
        question: "Сколько граней у планарного графа с V=
        options: [
            "3 грани",
            "5 граней",
            "10 граней"
        ],
        correctIndex: 1,
        explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$ . Не
    },
    {
        question: "Почему K5 непланарен?",
        options: [
            "Потому что  $10 > 3*5 - 6 = 9$  – нарушение",
            "Потому что Эйлер был против",
            "Потому что 5 вершин – несчастливое число"
        ],
        correctIndex: 0,
        explanation: "У K5: V=5, E=10,  $3V - 6 = 9$ .  $10 > 9$ 
    }
}

```

```

/** Транслировать реальные переменные визуализациям тек
export function emitVizState(
  chapterId: string,
  snapshot: Omit<VizStateEventDetail, "chapterId">
) {
  if (typeof window === "undefined") return;
  const detail: VizStateEventDetail = { chapterId, ...s
  latestByChapter.set(chapterId, detail);
  window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Вернуть демонстрацию в ручной режим при закрытии/ре
export function clearVizState(chapterId: string) {
  if (typeof window === "undefined") return;
  latestByChapter.delete(chapterId);
  window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Текущий снимок компилятора для прямой отрисовки мас
export function useVizRuntime(): VizStateEventDetail | n
  const chapterId = useContext(VizChapterContext);
  const [runtime, setRuntime] = useState<VizStateEventD
  chapterId ? latestByChapter.get(chapterId) ?? null
  );

  useEffect(() => {
    setRuntime(chapterId ? latestByChapter.get(chapterId
    if (!chapterId) return;
    const handler = (e: Event) => {
      const detail = (e as CustomEvent<VizStateEventDet
      if (detail?.chapterId === chapterId) {
        setRuntime(detail.variables || detail.line !==
      }
    };
    window.addEventListener(CHANNEL, handler);
    return () => window.removeEventListener(CHANNEL, ha
  }, [chapterId]);

```

```

    }, [chapterId, currentStep, maxStep, goToStep, selectStep]
  }
}

```

```

/** Утилиты без небезопасных cast-ов для визуализаторов
export const vizNumber = (value: DebugValue | undefined) => {
  typeof value === "number" && Number.isFinite(value) ? value : null;
}

```

```

export const vizString = (value: DebugValue | undefined) => {
  typeof value === "string" ? value : null;
}

```

```

export const vizArray = (value: DebugValue | undefined) => {
  Array.isArray(value) ? value : null;
}

```

```

export const vizRecord = (value: DebugValue | undefined) => {
  value !== null && typeof value === "object" && !Array.isArray(value)
    ? (value as Record<string, DebugValue>)
    : null;
}

```

```

/* — Матрицы из компилятора: СВОЯ размерность, а не дефолтная

```

```

/**

```

```

* Нормализованная матрица, пришедшая из выполненного Рун-тайма
*

```

```

* Размерность берётся из самих данных: если пользователь ввёл
* 5 вершин, визуализация рисует 5x5, а не подставляет его же
* 7x7 (раньше ячейки «не той» размерности превращались в null)
*

```

```

* Поддерживаются все учебные способы хранения:
*   list[list]      d = [[0, 4], [999, 0]]
*   list[dict]      d = [{0: 0, 1: 4}, {1: 0}]
*   dict[dict]      d = {0: {0: 0, 1: 4}}
*   dict[(i, j)]    d = {(0, 1): 4} — ключ после запятой
*/

```

```

export interface LiveMatrix {
  rows: number;
  cols: number;
  /** Значение ячейки; null — таких данных в коде пользователя нет
  at: (i: number, j: number) => number | null;
}

```

```

    if (n === null) return;
    cells.set(`${i}:${j}`, n);
    rows = Math.max(rows, i + 1);
    cols = Math.max(cols, j + 1);
};

const asArray = vizArray(value);
if (asArray) {
    asArray.slice(0, MAX_MATRIX_SIDE).forEach((row, i) => {
        const inner = vizArray(row);
        if (inner) {
            inner.slice(0, MAX_MATRIX_SIDE).forEach((cell, j) => {
                return;
            });
        }
        const record = vizRecord(row);
        if (record) {
            for (const [key, cell] of Object.entries(record)) {
                const j = Number(key);
                if (Number.isInteger(j)) put(i, j, cell);
            }
        }
    });
} else {
    const record = vizRecord(value);
    if (!record) return null;
    for (const [key, cell] of Object.entries(record)) {
        const pair = parsePairKey(key);
        if (pair) {
            put(pair[0], pair[1], cell);
            continue;
        }
        const i = Number(key);
        if (!Number.isInteger(i)) continue;
        const inner = vizArray(cell);
        if (inner) {
            inner.slice(0, MAX_MATRIX_SIDE).forEach((v, j) => {
                continue;
            });
        }
    }
}

```



-----  
/\* — Выбор демонстрации внутри страницы (вкладки 1D / 2D)

```
const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

export interface VizDemoEventDetail {
  chapterId: string;
  /** base-запись страницы – demoId пустой; подписанные */
  demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас открыта */
export function emitVizDemo(chapterId: string, demoId: string) {
  if (typeof window === "undefined") return;
  latestDemoByChapter.set(chapterId, demoId);
  window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>("viz-demo", {
    detail: { chapterId, demoId }
  }));
}

/**
 * Подписка: панель компилятора следует за активной вкладкой
 * Сразу отдаём последнее значение: компилятор часто отстает
 * как пользователь переключился с 1D на 2D.
 */
export function onVizDemo(chapterId: string, cb: (demoId: string) => void) {
  if (typeof window === "undefined") return () => {};
  const latest = latestDemoByChapter.get(chapterId);
  if (latest !== undefined) cb(latest);
  const handler = (e: Event) => {
    const detail = (e as CustomEvent<VizDemoEventDetail>).detail;
    if (detail && detail.chapterId === chapterId) cb(detail.demoId);
  };
  window.addEventListener(DEMO_CHANNEL, handler);
  return () => window.removeEventListener(DEMO_CHANNEL, handler);
}
```

## Универсальное пособие • полный исходный код

```
-----
tree = [0] * (2 * n)
i = 0                                # лист: tree[n + i]

v, l, r = 1, 0, n - 1               # вершина и её отрезок
m = (l + r) // 2                     # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
    variables: [
        { name: "n", role: "длина исходного массива; листы"},
        { name: "i", role: "индекс элемента массива; его левый и правый дети"},
        { name: "v", role: "текущая вершина дерева отрезка"},
        { name: "l, r", role: "границы отрезка, за которыми лежат левый и правый дети"},
        { name: "m", role: "середина отрезка m = (l + r) // 2"},
    ],
},
treap: {
    vizTitle: "Декартово дерево: build → split → merge",
    stepNote: "Шаг = один спуск в split/merge/erase (переход по ребру или удаление узла целиком.",
    code: `points = [[45, 8], [3, 6], [6, 5], [2, 4], [1, 3]]
n = len(points)          # сколько пар (x; y) – столько узлов
mid = 7                  # разрез для split: x <= mid y > mid
key = 9                  # какую точку удаляем в erase
step = 0                 # номер шага – листает кадры дерева
height = 0               # высота построенного дерева
root = None

# --- тело алгоритма ---
class Node:
    def __init__(s, x, y):
        s.x = x; s.y = y; s.l = None; s.r = None

def build(pts):
    """Декартово дерево из отсортированных по x точек:
    pts = sorted(pts, key=lambda p: p[0])
    stack = []
    for x, y in pts:
        cur = Node(x, y)
        last = None
```



```

while x.p:
    p = x.p; g = p.p
    step += 1
    if g is None:
        case = "zig" # родители
        rot(x)
    elif (g.l is p) == (p.l is x):
        case = "zig-zig" # x и p -
        rot(p); rot(x)
    else:
        case = "zig-zag" # змейка
        rot(x); rot(x)
    xk, pk, gk = x.k, p.k, (g.k if g else 0)
    print("шаг", step, "->", case, "| x =", xk, "p = ", pk)
    root = x
    print("    корень теперь", x.k, "| обход:", inorder(root))
    return x

```

# 1) Zig: родитель x – уже корень (дерево 40 -> 20)  
 root = N(40); root.l = N(20); root.l.p = root  
 splay(root.l)

# 2) Zig-Zig: бамбук 60 -> 40 -> 20, оба сына левые  
 root = N(60); root.l = N(40); root.l.p = root; root.l.l = N(20)  
 splay(root.l.l)

# 3) Zig-Zag: змейка 60 -> 20 -> 40 (40 – правый сын 20)  
 root = N(60); root.l = N(20); root.l.p = root; root.l.r = N(40)  
 splay(root.l.r),

```

variables: [
    { name: "case", role: "какой случай сработал: \"zig-zig\", \"zig-zag\", \"zig\""},
    { name: "step", role: "номер применения случая – 1, 2, 3"},
    { name: "xk / x", role: "ключ узла, который поднимали"},
    { name: "pk / p", role: "ключ родителя x перед поворотом"},
    { name: "gk / g", role: "ключ деда – нужен в Zig-Zig"},
    { name: "root", role: "корень дерева после расшейки"},
]

```

```

stepNote: "Шаг = заполнение очередного  $P[i] = P[i-1]$ 
code: `a = [3, 1, 4, 1, 5, 9, 2, 6]` # исходный массив
n = len(a) # длина: префиксов будет n
i = 0 # текущая граница префикса

P = [0] #  $P[0] = 0$  – пустой префикс
for i in range(1, n + 1):
    P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)`
variables: [
    { name: "a", role: "исходный массив 1D-демонстрация" },
    { name: "n", role: "длина массива a; префиксов будет n" },
    { name: "P", role: "префиксные суммы:  $P[i] = a[0] + \dots + a[i-1]$ " },
    { name: "i", role: "граница префикса: на шаге считаем до i" }
],
},
"dynamic-programming": {
    vizTitle: "Динамическое программирование (мемоизация)",
    stepNote: "Шаг = событие внутри fibМемо(k): вызов, возврат",
    code: `n = 5
memo = {}

def fib(k):
    if k in memo:
        return memo[k]
    if k <= 2:
        return 1
    print(f"считаем fib({k-1}) + fib({k-2})")
    r1 = fib(k - 1)
    r2 = fib(k - 2)
    memo[k] = r1 + r2
    return memo[k]

print("итог:", fib(n))`,
    variables: [
        { name: "n", role: "целевой номер числа Фибоначчи" },
        { name: "k", role: "аргумент текущего рекурсивного вызова" },
        { name: "memo", role: "кэш уже посчитанных значений" }
    ]
}

```

```

code: `from collections import deque

q = deque(["A"])          # очередь BFS
u = "B"                  # сосед, найденный из текущ
q.append(u)               # enqueue соседа – в конец
front = q[0]              # голова очереди: её обрабо
v = q.popleft()           # шаг: достали голову и обр
print("front =", front, "| v =", v, "| очередь:", list(
    variables: [
        { name: "front", role: "голова очереди – вершина,
        { name: "v", role: "вершина, извлечённая из очере
        { name: "u", role: "сосед вершины v, которого кла
    ],
},
"everyday-basics": {
    vizTitle: "Бытовой тренажёр: стек, очередь, куча",
    stepNote: "Шаг = одна бытовая операция: тарелка на
    code: `from collections import deque

st, q = [], deque()      # тарелки (стек) и очередь
st.append(" "); top = st.pop()
q.append(" "); q.append(" "); first = q.popleft()

i = 0                    # счётчик выполненных опера
for op in ("push", "pop", "enqueue", "dequeue"):
    i += 1
print("тарелка:", top, "| первый из очереди:", first, "
    variables: [
        { name: "top / front", role: "верх стопки (стек)
        { name: "i", role: "индекс в массиве кучи; родите
    ],
},
"graph-dfs-bfs": {
    vizTitle: "DFS и BFS на графе",
    stepNote: "Шаг = переход из текущей вершины v к неп
    code: `from collections import deque

adj = {"A": ["B", "C"], "B": ["D"], "C": [], "D": []}
```

## Универсальное пособие • полный исходный код

```
-----
v = -1                                # вершина, в которой обход начался
stack = []                            # фронт обхода (стек для DFS)
i = 0                                # номер шага — двигает по очереди

# --- тело алгоритма ---
for start in range(n):
    if comp[start] != -1:
        continue
    comps += 1                        # непокрашенная вершина => новая компонента
    c = comps
    stack = [start]
    comp[start] = c
    used[start] = True
    v = start
    while stack:
        v = stack.pop()              # для BFS: v = stack.pop(0)
        i += 1
        for to in adj[v]:
            if comp[to] == -1:
                comp[to] = c
                used[to] = True
                stack.append(to)
print("компонент:", comps)
print("разметка comp:", comp)`
variables: [
    { name: "n", role: "число вершин; в демонстрации — количество вершин графа" },
    { name: "adj", role: "списки смежности — тот же граф, что и g" },
    { name: "comp", role: "разметка: номер компоненты", range: "-1 ... comps" },
    { name: "used", role: "булев вариант разметки — для BFS" },
    { name: "comps", role: "счётчик найденных компонент" },
    { name: "c", role: "номер компоненты, которую красим" },
    { name: "v", role: "вершина, где обход находится" },
    { name: "stack", role: "фронт обхода: стек (DFS) или очередь (BFS)" },
    { name: "i / step", role: "номер шага — перематывает" },
],
},
"top-sort": {
```

```
def dfs1(v):
    used[v] = True
    for to in adj[v]:
        if not used[to]:
            dfs1(to)
    order.append(v)          # выходим из вершины — она

for v in adj:
    if not used[v]:
        dfs1(v)
    print("DFS из", v, "закончен | order:", order)

# транспонирование: каждое ребро разворачиваем
adjT = {}
for v in adj:
    adjT[v] = []
for v in adj:
    for to in adj[v]:
        adjT[to].append(v)
print("обратный граф:", adjT)

# фаза 2: идём по order С КОНЦА, DFS по adjT красит одн
comp = {}
c = 0
for i in range(len(order) - 1, -1, -1):
    v = order[i]
    if v in comp:
        continue
    stack = [v]
    comp[v] = c
    while stack:
        u = stack.pop()
        for to in adjT[u]:
            if to not in comp:
                comp[to] = c
                stack.append(to)
    print("SCC", c, "собрана из", v, ":", comp)
```



```

    },
    "bridges-code": {
        vizTitle: "Мосты: DFS + tin/low",
        stepNote: "Шаг = строка псевдокода слева; ребро (v, to) проверяется, является ли мостом. Если да, то его можно удалить, не нарушая связности графа.",
        code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1, "B": 2, "G": 3}
timer = 3
v, to = "B", "G"
is_bridge = low[to] > tin[v]
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
        variables: [
            { name: "v", role: "текущая вершина DFS", range: "A, B, G"},
            { name: "to", role: "сосед; ребро (v, to) проверяется", range: "A, B, G"},
            { name: "tin[v]", role: "время входа в вершину", range: "1, 2, 3"},
            { name: "low[v]", role: "минимум tin, куда можно вернуться", range: "1, 2, 3"},
            { name: "timer", role: "глобальный счётчик времени", range: "1, 2, 3"}
        ],
    },
    "euler-path-vs-cycle": {
        vizTitle: "Эйлеров маршрут: алгоритм Иера",
        stepNote: "Шаг = переход по неиспользованному ребру, если такой есть. Если нет, то путь закончен.",
        code: `n = 5
edges = [(0, 1), (0, 2), (1, 3), (2, 4), (1, 2), (3, 4)]
adj = {v: [] for v in range(n)}
for i, (u, v) in enumerate(edges):
    adj[u].append((v, i))
    adj[v].append((u, i))
deg = {v: len(adj[v]) for v in range(n)}
odd = [v for v in range(n) if deg[v] % 2 == 1]
start = odd[0] if odd else 0
used = [False] * len(edges)
path = [start]
last = start
stack = [start]
v = start
step = 0`
    },
}

```

```

V = n                                     # вершин для формулы
E = sum(len(a) for a in adj) // 2        # рёбер: в списке смежн
F = 2 - V + E                             # граней: из  $V - E + F = 2$ 

# --- тело алгоритма ---
print("V =", V, "| E =", E, "| F =", F, "|  $V - E + F = 2$ ")
for v in range(n):
    used = set()
    for u in adj[v]:
        if color[u] != 0:
            used.add(color[u])
    c = 1
    while c in used:                       # наименьший свободный цвет
        c += 1
    color[v] = c
    chi = max(chi, c)
    i += 1
    print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
variables: [
    { name: "n", role: "число вершин графа на демонстрации" },
    { name: "adj", role: "списки смежности — тот же граф" },
    { name: "color", role: "массив цветов вершин (0 = не покрашено)" },
    { name: "v", role: "вершина, которую красим сейчас" },
    { name: "u", role: "сосед вершины v, чей цвет уже занят" },
    { name: "delta", role: "максимальная степень  $\Delta = \max \deg v$ " },
    { name: "chi", role: "сколько цветов реально понадобится" },
    { name: "i / step", role: "номер шага — перематываем" },
    { name: "V, E, F", role: "для вкладки «Планарность»" },
],
},
dijkstra: {
    vizTitle: "Дейкстра",
    stepNote: "Шаг = старт, извлечение ближайшей вершины",
    code: `import heapq

```

## Универсальное пособие • полный исходный код

```
-----
nodes = ["S", "A", "B", "C", "D", "E", "F"]
n = len(nodes)           # 7 вершин => n - 1 = 6 работ
m = len(E)               # 9 рёбер
```

```
dist = {v: INF for v in nodes}
dist["S"] = 0             # старт – База
```

```
# --- тело алгоритма ---
```

```
last_changed = ""
for i in range(1, n):     # итерация i = 1 ... n-1
    last_changed = ""
    for u, v, w in E:     # шаг: релаксация ребра {u, v}
        if dist[u] != INF and dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            last_changed = v
    if last_changed == "":
        print(f"итерация {i}: изменений нет – сошлось для всех вершин")
        break
    print(f"итерация {i}: " + ", ".join(f"{k}={dist[k]}:" for k in nodes))
```

```
# контрольный (n-й) проход: улучшение здесь означает отрицательный цикл
cycle_edge = None
for u, v, w in E:
    if dist[u] != INF and dist[u] + w < dist[v]:
        cycle_edge = (u, v, w)
        break
```

```
if cycle_edge:
    print("отрицательный цикл: улучшается ребро", cycle_edge)
else:
```

```
    print("отрицательных циклов нет:", {k: dist[k] for k in nodes})
    variables: [
        { name: "n", role: "число вершин графа – ровно n-1"},
        { name: "m", role: "число рёбер, по которым проходят"},
        { name: "i", role: "номер итерации (после i итераций)"},
        { name: "u, v, w", role: "текущее ребро: откуда, куда"},
        { name: "dist[v]", role: "расстояние; если обновилось"},
    ],
```

```

        на-Форда" },
        { name: "u, v", role: "ребро (u, v) перевзвешивает",
        { name: "d[v]", role: "расстояние Дейкстры в новом графе"
        },
    ],
},
mst: {
    vizTitle: "Остовное дерево: Краскал и DSU",
    stepNote: "Шаг = следующее ребро из отсортированных",
    code: `n = 9 # вершин: A..I — те, что в графе
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "A", "E"),
        (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "B", "G"),
        (12, "H", "I"), (13, "F", "I")]
edges.sort() # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGH I"} # DSU: представителю
rank = {v: 0 for v in "ABCDEFGH I"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе)
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое

# --- тело алгоритма ---
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]] # сжатие путей
        x = parent[x]
    return x

for i, (w, a, b) in enumerate(edges):
    u, v = a, b
    ra, rb = find(a), find(b)
    if ra != rb: # разные компоненты
        parent[ra] = rb
        if rank[ra] == rank[rb]:
            rank[rb] += 1
        mst.append((w, a, b))
        total += w
print("вес остова:", total, "| рёбер:", len(mst))

```

```

vizTitle: "Z-функция",
stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r]",
code: `s = "abacaba"
n = len(s)
i, l, r = 1, 0, 0          # правый Z-блок [l, r]
z = [None] * n            # пустые клетки: значения по умолчанию
z[0] = 0

for i in range(1, n):      # шаг: вычисляем z[i]
    if i <= r:
        z[i] = min(r - i + 1, z[i - l]) # из блока
    else:
        z[i] = 0            # свежая клетка: с нуля
    while i + z[i] < n and s[z[i]] == s[i + z[i]]:
        z[i] += 1          # досчёт в лоб
    if i + z[i] - 1 > r:
        l, r = i, i + z[i] - 1
print(z)`
variables: [
    { name: "n", role: "длина строки s", range: "n = len(s)" },
    { name: "i", role: "позиция, для которой считаем z[i]" },
    { name: "l, r", role: "правый крайний Z-блок: отрезок [l, r]" },
    { name: "z[i]", role: "длина наибольшего общего префикса s[i:] и s[0:]" }
],
},
"aho-corasick": {
    vizTitle: "Ахо-Корасик: бор, fail-ссылки и автомат",
    stepNote: "Шаг = один символ текста: state = go(state, c).",
    code: `words = ["HERS", "HIS", "SHE", "HE"] # образцы
text = "USHERS" # текст, в котором ищем
n = len(text) # длина текста
state = 0 # текущее состояние автомата
i = 0 # индекс текущего символа
c = "" # сам символ
nodes = 1 # сколько вершин в автомате
queue = [] # очередь BFS, к которой добавляем новые вершины
fail = {} # fail[v] = вершина, с которой нужно начинать поиск

```

```

        queue.append(u)
print("вершин в боре:", nodes)
print("fail-ссылки:", fail)

def go(v, ch):
    """Переход автомата: сначала по бору, иначе спускаем
    while v != 0 and (v, ch) not in nxt:
        v = fail[v]
    return nxt.get((v, ch), 0)

for i, c in enumerate(text):
    state = go(state, c)
    v = state
    while v != 0:                                # собираем все образцы
        if v in term:
            matches.append((i, term[v]))
        v = fail[v]
    print(f"i={i} c='{c}' state={state} найдено={matches}")

print("всего вхождений:", len(matches), matches),
variables: [
    { name: "words", role: "набор образцов – по нему",
      },
    { name: "text", role: "сканируемый текст – подста",
    { name: "state", role: "текущее состояние автомата",
    { name: "i / c", role: "индекс и сам символ текста",
    { name: "nxt", role: "переходы бора: nxt[(v, ch)]",
    { name: "fail", role: "fail[v] – вершина наибольшей",
    { name: "term", role: "в каких вершинах заканчивае",
    { name: "queue", role: "очередь BFS, которой натя",
    { name: "matches", role: "найденные вхождения: (ин",
    { name: "nodes", role: "сколько вершин в боре (ко",
],
},
"complexity-classes": {
    vizTitle: "Классы сложности: рост по n и ops",
    stepNote: "Шаг = одна точка (n, ops): визуализация",
    code: `n = 1                                # размер входа

```

```

    },
    intro: {
        // мнемодкарточки без шагов алгоритма
        variables: [],
    },

    /* — Дополнительные демонстрации внутри страниц (вкл.)

    "planarity-euler-formula#greedy": {
        vizTitle: "Жадная покраска вершин ( $\chi \leq \Delta + 1$ )",
        stepNote: "Шаг = вершина v получает наименьший цвет",
        code: `n = 8                                # вершин в графе
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj)    # максимальная степень
color = [0] * n                    # 0 = ещё не покрашена
chi = 0                            # сколько цветов понадобилось
v = -1                             # вершина, которую красим сейчас
i = 0                              # номер шага

# --- тело алгоритма ---
for v in range(n):
    used = set()
    for u in adj[v]:
        if color[u] != 0:
            used.add(color[u])
    c = 1
    while c in used:                # наименьший свободный цвет
        c += 1
    color[v] = c
    chi = max(chi, c)
    i += 1
    print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
        variables: [
            { name: "color", role: "цвета вершин (0 = не покрашено)", range: "0 ... n-1" },
            { name: "v", role: "текущая вершина", range: "0 ... n-1" },
            { name: "delta / chi", role: "максимальная степень / количество цветов", range: "0 ... n" },
            { name: "i / step", role: "номер кадра", range: "0 ... n" },
        ]
    }
}
    `

```

```

    ],
},
"planarity-euler-formula#edges": {
    vizTitle: "Покраска рёбер (Визинг:  $\Delta \leq \chi' \leq \Delta + 1$ )",
    stepNote: "Шаг = ребро e получает наименьший цвет, <= \Delta",
    code: `n = 8
edges = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 4), (3, 4)]
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj)
ecolor = [0] * len(edges) # цвет каждого ребра
inc = [dict() for _ in range(n)] # inc[v][цвет] = ребро
e = -1 # номер текущего ребра
v = u = -1 # его концы
chi_e = 0

# --- тело алгоритма ---
for e, (a, b) in enumerate(edges):
    v, u = a, b
    used = set(inc[a].values()) | set(inc[b].values())
    c = 1
    while c in used:
        c += 1
    ecolor[e] = c
    inc[a][e] = c
    inc[b][e] = c
    chi_e = max(chi_e, c)
    print("ребро", (a, b), "занято:", sorted(used), "-> ", c)
print("chi' =", chi_e, "| delta =", delta, "| delta <= \Delta")
variables: [
    { name: "ecolor", role: "цвета рёбер – рёбра на графе", range: "0..delta" },
    { name: "e", role: "номер текущего ребра", range: "0..len(edges)-1" },
    { name: "v, u", role: "концы текущего ребра – подмножества V", range: "0..n-1" },
    { name: "inc", role: "для каждой вершины: какие цвета имеют инцидентные рёбра", range: "0..delta" },
    { name: "chi_e / delta", role: "χ' – сколько цветов", range: "0..delta+1" }
],
},
"graph-components#dfs": {
    vizTitle: "Компоненты связности: DFS-покраска",

```



```

    },
    "graph-components#dsu": {
        vizTitle: "Компоненты связности: DSU по рёбрам",
        stepNote: "Шаг = одно ребро (u, v): union склеивает",
        code: `n = 10                                # вершин в графе
edges = [(0, 1), (1, 2), (2, 3), (0, 3), (1, 3), (4, 5)]
parent = list(range(n))    # DSU: parent[v] = представи
rank = [0] * n             # высота дерева – по ней реш
comps = n                  # сначала каждая вершина – о
e = -1                     # номер обрабатываемого ребра
u, v = -1, -1              # концы текущего ребра
step = 0                   # номер шага – двигает по ка

# --- тело алгоритма ---
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]    # сжатие путей
        x = parent[x]
    return x

def union(a, b):
    global comps
    ra, rb = find(a), find(b)
    if ra == rb:
        return False                # ребро внутри
    if rank[ra] < rank[rb]:
        ra, rb = rb, ra
    parent[rb] = ra
    if rank[ra] == rank[rb]:
        rank[ra] += 1
    comps -= 1
    return True

for e, (a, b) in enumerate(edges):
    u, v = a, b
    step = e + 1
    union(a, b)
print("компонент:", comps)

```

```

-----
k = 3                                # размер клики, которую ищем

# --- тело алгоритма ---
# Сведение 3-SAT <=p КЛИКА: вершина = {номер дизъюнкта,
# ребро = два литерала из РАЗНЫХ дизъюнктов, не противоречивые
clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
V = [(i, l) for i, cl in enumerate(clauses) for l in cl]
E = [(a, b) for a in V for b in V if a[0] < b[0] and a[1] != b[1]]
for step in range(7):
    print("кадр", step, "| вершин:", len(V), "| рёбер:", len(E))
print("формула выполнима <=> в графе есть клика размера k")
variables: [
    { name: "step", role: "номер кадра сведения (0..6)" },
    { name: "cls", role: "класс задачи A, которую сводят к клике" },
    { name: "n", role: "размер входа x; после сведения к клике" },
    { name: "k", role: "размер клики, которую ищет рекурсия" },
],
},
"complexity-classes#verify": {
    vizTitle: "Верификатор 3-SAT: проверка сертификата",
    stepNote: "Шаг = один дизъюнкт: верификатор подставляет значения в формулу",
    code: `clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
cert = [1, 0, 1, 0]          # сертификат: x1=1, x2=0, x3=1
k = 0                        # номер проверяемого дизъюнкта
ok = True                    # вердикт верификатора

# --- тело алгоритма ---
for k, cl in enumerate(clauses):
    hit = False
    for l in cl:
        val = cert[l - 1] if l > 0 else 1 - cert[-l - 1]
        if val == 1:
            hit = True
    ok = ok and hit
    print("дизъюнкт", k + 1, cl, "выполнен" if hit else "не выполнен")
print("сертификат", "принят" if ok else "отвергнут")`,
    variables: [
        { name: "cert", role: "сертификат – назначение переменным значений 0 или 1" },
    ],
},

```

```

-----
edges.sort()                # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGHI"} # DSU: представител
rank = {v: 0 for v in "ABCDEFGHI"}   # высота дерева
mst = []                        # рёбра, взятые в остов
total = 0                      # вес остова (на этом графе 51)
i = 0                          # номер текущего ребра
u, v, w = None, None, None     # концы и вес ребра, которое рассматриваем

# --- тело алгоритма ---
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]] # сжатие путей
        x = parent[x]
    return x

for i, (w, a, b) in enumerate(edges):
    u, v = a, b
    ra, rb = find(a), find(b)
    if ra != rb:                # разные компоненты
        parent[ra] = rb
        if rank[ra] == rank[rb]:
            rank[rb] += 1
        mst.append((w, a, b))
        total += w
print("вес остова:", total, "| рёбер:", len(mst))
print(mst)`
    variables: [
        { name: "i", role: "номер текущего ребра – перебираем"},
        { name: "u, v, w", role: "концы и вес рассматриваемого ребра"},
        { name: "parent", role: "DSU-массив: цвета компонент"},
        { name: "mst", role: "рёбра, уже взятые в остов", "51"},
        { name: "total", role: "вес остова (51 на этом графе)"},
    ],
},
"mst#prim": {
    vizTitle: "Остовное дерево: Прим (рост из вершины E)",
    stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
    code: `import heapq

```

```

-----
vizTitle: "Остовное дерево: Борувка (фазы-«пятiletки»)"
stepNote: "Шаг = одна фаза: каждая компонента ОДНОВЕРХОВА"
code: `n = 9                                # вершин: A...I
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "C", "D"),
         (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "F", "G"),
         (12, "H", "I"), (13, "F", "I")]
comp = {v: v for v in "ABCDEFGH I"}          # DSU: корень компоненты
cheapest = {}                                # корень компоненты -> самое дешёвое ребро
mst = []                                     # рёбра остова
total = 0                                    # вес остова (= 51)
phase = 0                                    # номер фазы («пятiletки») — от 0 до 4
u, v, w = None, None, None                  # ребро-кандидат

# --- тело алгоритма ---
def find(x):
    while comp[x] != x:
        comp[x] = comp[comp[x]]
    x = comp[x]
    return x

while len(mst) < n - 1:
    phase += 1
    cheapest = {}
    for (w, a, b) in edges:                  # КАЖДАЯ компонента ОДНОВЕРХОВА
        ra, rb = find(a), find(b)
        if ra == rb:
            continue
        u, v = a, b
        if ra not in cheapest or w < cheapest[ra][0]:
            cheapest[ra] = (w, a, b)
        if rb not in cheapest or w < cheapest[rb][0]:
            cheapest[rb] = (w, a, b)
    if not cheapest:
        break                                # граф несвязен
    for (w, a, b) in cheapest.values():
        ra, rb = find(a), find(b)
        if ra == rb:
            continue                          # две компоненты

```

```

        { name: "k", role: "уровень: блоки  $2^k \times 2^k$  (шаг
        { name: "st2[{r,c,k,k}]", role: "минимум квадратн
    },
},
"sparse-table#2d-query": {
    vizTitle: "Разреженная таблица 2D: запрос",
    stepNote: "Демонстрация кликабельна вручную: выбери
    code: `# st2[{r, c, kx, ky}] из вкладки «построение
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
    variables: [
        { name: "r1, c1, r2, c2", role: "углы запрашиваем
        { name: "kx, ky", role: "уровень самого крупного
    ],
},
"prefix-sums-2d#2d": {
    vizTitle: "Префиксные суммы 2D (прямоугольники)",
    stepNote: "Демонстрация кликабельна вручную: наведи
    code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):
    S.append([0] + [None] * m)
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
    for j in range(1, m + 1):
        S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][j - 1]

r1, c1, r2, c2 = 1, 1, 2, 2
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`
    variables: [
        { name: "i, j", role: "текущая ячейка таблицы S (

```

```
-----
const keep: string[] = [];
for (const line of code.split("\n")) {
  const t = line.trim();
  if (t === "# --- тело алгоритма ---" || /^(for|while)/.test(t)) {
    keep.push(line);
  }
}
// убрать пустые строки с конца
while (keep.length > 0 && keep[keep.length - 1].trim() === "") {
  keep.pop();
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, chapterTitle: string, sync: Sync) {
  const sync = getPageSync(chapterId, demoId);

  const head = sync.vizTitle
    ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#`
    : `# «${chapterTitle}»\n# демо на странице нет – св

  if (!sync.code) return head;
  const init = extractInit(sync.code);
  return `${head}\n${init}\n`;
}
-----
```

## РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

=====

-----  
ФАЙЛ 26/111: src/data/tickets/ticket1\_segtree.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 217 | РАЗМЕР: 1.5 КБ

-----  
import { Chapter } from '../../../types';

```

span class="font-mono text-emerald-300">[0, 1
тям <span class="font-mono text-emerald-300
– это одиночные элементы массива. В узле л
чество единиц, И/ИЛИ – любая операция, котор
>).</p>

```

```

<p class="text-slate-300 text-sm mb-4">Отсю
а каждом уровне встречает один из трёх случ
не пересекается – возвращаем нейтральный э
склеиваем. Частичных пересечений на уровне
одного элемента – спуск к листу и пересчёт

```

```

<div class="grid grid-cols-1 sm:grid-cols-3"
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
  <div class="border border-emerald-500 shadow-md">

```

Аналогия 1: Корпоративная пре

```

  </div>
  <p class="text-slate-300 text-sm">Б
икам департаментов: «внутри всем +5 %». Нач
nt-mono text-emerald-300">lazy[v]</span>) и
свой баланс. Это и есть <b>отложенное обнов
</div>

```

```
<p class="text-slate-500 text-xs mt-3">
  кивают в строгом порядке: сначала присваива
</div>
```

```
<div class="bg-slate-900/40 rounded-lg border
  <p class="text-slate-300 text-sm mb-2">
  <p class="text-slate-400 text-sm mb-1">
за  $O(1)$ , но строятся один раз и умирают при
ми.</p>
```

```
  <p class="text-slate-400 text-sm mb-1">
идемпотентные запросы (min, gcd, И) по ста
  <p class="text-slate-400 text-sm mb-1">
n ячеек (не  $2n!$ ) – иначе на  $n = 6$  поймать п
n ...  $2n-1$ ] и  $n$  – степень двойки (или использ
  <p class="text-slate-400 text-sm">• <b>
ть лень во всё дерево – это  $O(n)$  и весь смы
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
-2 py-0.5 rounded text-xs font-mono text-em
ся один раз), запрос на отрезке <code class=
-slate-800"> $O(\log n)$ </code>, одиночное обно
-300 border border-slate-800"> $O(\log n)$ </code>
s font-mono text-emerald-300 border border-
xt-xs font-mono text-emerald-300 border bord
ounded text-xs font-mono text-emerald-300 b
```

```
<p class="text-slate-500 text-xs mt-3 leading
  в редакторах и базах, сжатие координат + «
оить», дерево отрезков по дереву (HLD), пер
</p>
```

```
<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
```

```
  Код: построение и запрос суммы (C
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```



```

        y[4 * MAXN];           // «обещание»: прибавит

void push(int v, int l, int r) {
    if (lazy[v] == 0) return;
    int m = (l + r) / 2;
    tree[2 * v]      += lazy[v] * (m - l + 1);    // п
    lazy[2 * v]      += lazy[v];
    tree[2 * v + 1] += lazy[v] * (r - m);        // и
    lazy[2 * v + 1] += lazy[v];
    lazy[v] = 0;                                // о
}

void add(int v, int l, int r, int L, int R, int x) { /
    if (r < L || R < l) return;
    if (L <= l && r <= R) {
        tree[v] += x * (r - l + 1);
        lazy[v] += x;                            // д
        return;
    }
    push(v, l, r);                                // о
    int m = (l + r) / 2;
    add(2 * v, l, m, L, R, x);
    add(2 * v + 1, m + 1, r, L, R, x);
    tree[v] = tree[2 * v] + tree[2 * v + 1];
}

```

<p class="text-slate-400 text-xs">Пр  
 г накопиться. Забыл push – получил правильн  
 </div>  
 </details>

<details class="bg-slate-900/40 rounded-lg l  
 <summary class="p-4 font-bold text-slate  
 -b border-slate-700/50">  
 Код: итеративное дерево (2n, снизу)  
 </summary>  
 <div class="p-5 text-sm text-slate-300  
 <pre class="bg-slate-950 p-4 rounde  
 # для удобств



```
ext-emerald-300">w ≥ 0</span>}, зафиксирова
ию расстояния!</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg
  <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
```

Ограничения: минус ломает жадно

```
</div>
```

```
<p class="text-slate-300 text-sm mb
lass="font-mono text-emerald-300">s→a = 2</
ono text-emerald-300">b→a = -2</span>. Верн
начала вытащит <span class="font-mono text-
o text-emerald-300">3 - 2 = 1</span> придёт
Джонсона (билет 17).</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl bord
<h3 class="text-xl font-bold text-blue-400 m
```

```
<p class="text-slate-300 text-sm mb-4">Демон
D, Дом 2 E, ТЦ F, Метро G) и 11 <b>одностор
ршины <span class="font-mono text-emerald-3
(u → v)</span>: переменные <span class="fon
>, <span class="font-mono text-emerald-300">
аут картинку напрямую.</p>
```

```
<ol class="list-decimal pl-5 space-y-2 text
  <li><b>Инициализация.</b> <span class="
text-emerald-300">∞</span>; в кучу кладём <
emerald-300">prev</span> пуст.</li>
```

```
<li><b>Извлечь минимум.</b> Достаём пар
"font-mono text-emerald-300">du</span>. Если
xt-emerald-300">done</span> – пара устарела
```

```
<li><b>Зафиксировать.</b> <span class="
меняется.</li>
```

```
</div>
```

```
<div class="bg-slate-800/70 p-5 rounded-lg"
  <p class="text-sm font-bold text-white"
  <table class="w-full text-xs text-left"
    <thead>
      <tr class="text-slate-500 border"
        <th class="py-1.5 pr-2 font"
        <th class="py-1.5 pr-2 font"
        <th class="py-1.5 font-bold"
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-slate-
y-2 pr-2">лучший известный путь  $s \rightarrow v$ </td><td class="py-2 pr-2">предок  $v$  на текущем лучшем пути</td>
      <tr class="border-b border-slate-
y-2 pr-2">предок  $v$  на текущем лучшем пути</td>
      <tr class="border-b border-slate-
lass="py-2 pr-2">вершины с окончательным  $dist[v]$ </td></tr>
      <tr class="align-top"><td class="py-2 pr-2">вершина, вершина)</td><td class="py-2">извлечение  $v$  из кучи</td></tr>
    </tbody>
  </table>
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border"
  <h3 class="text-xl font-bold text-indigo-400">Извлечение из кучи</h3>
  <p class="text-slate-300 text-sm mb-4"><b>Инициализация</b>:  $dist[s] = 0$ ,  $dist[v] = \infty$  для всех  $v \in V \setminus \{s\}$ .
  <span class="font-mono text-emerald-300"> $s \rightarrow u$ </span>. Отсюда  $dist[u] = \delta(s, u)$ .
  <span class="font-mono text-emerald-300"> $dist[v] = \delta(s, v)$ </span> для всех достижимых  $v$ .
  <span class="font-mono text-emerald-300"> $dist[v] = \infty$ </span> для недостижимых  $v$ .
```

```
<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate-300">Извлечение из кучи</summary>
  <p class="text-slate-300 text-sm mb-4"><b>Инициализация</b>:  $dist[s] = 0$ ,  $dist[v] = \infty$  для всех  $v \in V \setminus \{s\}$ .
  <span class="font-mono text-emerald-300"> $s \rightarrow u$ </span>. Отсюда  $dist[u] = \delta(s, u)$ .
  <span class="font-mono text-emerald-300"> $dist[v] = \delta(s, v)$ </span> для всех достижимых  $v$ .
  <span class="font-mono text-emerald-300"> $dist[v] = \infty$ </span> для недостижимых  $v$ .
```

```

        </div>
    </details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-slate-800">
    <h3 class="text-xl font-bold text-emerald-400">Дейкстра на графе
    <div class="overflow-x-auto my-4">
        <table class="w-full text-xs text-left">
            <thead>
                <tr class="text-slate-500 border-b">
                    <th class="py-2 pr-3 font-b">Взвешенный ребро
                    <th class="py-2 pr-3 font-b">Взвешенный узел
                    <th class="py-2 font-bold">Путь
                </tr>
            </thead>
            <tbody class="text-slate-300">
                <tr class="border-b border-slate-600">
                    <td>priority_queue (C++)</td><td class="py-2 pr-3">
                        й выбор</td></tr>
                <tr class="border-b border-slate-600">
                    <td>ease-key</td><td class="py-2 pr-3 font-mono">
                        очереди</td></tr>
                <tr class="border-b border-slate-600">
                    <td>поиск минимума</td><td class="py-2 pr-3 font-mono">
                        </tr>
                <tr class="border-b border-slate-600">
                    <td>><td class="py-2 pr-3 font-mono">0(V + E)</td></tr>
                <tr class="align-top"><td class="font-mono">0(E + V · C)</td><td class="py-2">ц
                    </tbody>
            </table>
        </div>

    <details class="bg-slate-900/40 rounded-lg border-2 border-slate-800">
        <summary class="p-4 font-bold text-slate-300 border-b border-slate-700/50">» Код: Дейкстра на
        <div class="p-5 text-sm text-slate-300"

```

```

    "E": [{"F", 1}],
    "F": [],
    "G": [{"D", 1}, {"E", 2}],
}
dist, prev = dijkstra(g, "A")
print(dist) # A 0, B 3, C 2, D 6, E 7,
print(restore(prev, "A", "F")) # ['A', 'C', 'G', 'E',
    <p class="text-slate-400 text-xs">П
        же вершины, потому что читает ваши <span c
nt-mono">done</span>.</p>
    </div>
</details>

<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">> Код: C++ (priority
    <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
    t;
#include <queue>;
using namespace std;
const long long INF = 4e18; // не INT_MAX:

int n;
vector<vector<pair<int,int>>>> g;
vector<long long> d;
vector<int> p;

void dijkstra(int s) {
    d.assign(n, INF); p.assign(n, -1); d[s] = 0;
    priority_queue<pair<long long,int>;,
        vector<pair<long long,int>>&
        greater<pair<long long,int>>
    pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du > d[u]) continue; // устарев
        for (auto [v, w] : g[u]) {

```

```
used[u] = True # фиксируем d[u]
for v in range(n): # релаксируем v
```

```
    if d[u] + w[u][v] < d[v]:
        d[v] = d[u] + w[u][v]
```

Т. Память – матрица

```
</div>
</details>
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border">
  <h3 class="text-xl font-bold text-amber-400">

```

```
<div class="bg-rose-950/30 p-5 rounded-lg border">
  <p class="text-sm font-bold text-rose-300">

```

```
<ul class="list-disc pl-5 space-y-2 text-rose-300">
  <li><b>Отрицательные веса.</b> Алгоритм релаксации вершины необратима. Нужен Форд–Беллман.
  <li><b>Отрицательные циклы.</b> При отрицательном числе. Сначала проверьте граф Фордом–Беллманом.
  <li><b>Ленивое удаление.</b> Без проверки, одна вершина обрабатывается несколько раз:
  <li><b>Переполнение INF.</b> <span class="font-mono">u</span> уходит в отрицательное число и «улетит».

```

```
<li><b>Ориентированность.</b> Демонстрация графа: каждое ребро добавляется в оба списка.
  <li><b>Дейкстра ≠ BFS.</b> BFS – чистая Дейкстра. С разными весами FIFO ломается, нужна куча.
  <li><b>«Одна пара».</b> Алгоритм с отрицательными весами и выход при извлечении цели (билет 17), но не работает.

```

```
</div>
```

```
<p class="text-slate-300 text-sm mt-4 mb-4">
  о скоростью <span class="font-mono text-emerald-300">O(V^3)</span>; б
  после перевзвешивания потенциалов). Представим
```

```

<div class="grid grid-cols-1 sm:grid-cols-3">
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
      Аналогия 1: Патруль Бэтмена на м
    </div>
    <div class="my-3 rounded-lg overflow">
      все ули
овно <b>1 выходной в году</b>, когда он отдр
-ю ночь Бэтмен делает контрольный проезд: е
о-то района снова уменьшилось – значит, Джо
</div>

```

```

  <div class="bg-slate-900 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
      Отрицательный цикл: временная п
    </div>

```



```

    </table>
    <p class="text-slate-400 text-xs mt-3">
бном» порядке: <span class="font-mono">S→A,
порядок рёбер местами – и понадобится до ше
</div>

<div class="bg-slate-800/70 p-5 rounded-lg"
    <p class="text-sm font-bold text-white"
    <p class="text-slate-300 text-sm mb-3">
тава A</span> имеет суммарный вес <span cla
веет» на 2.</p>
    <table class="w-full text-xs text-left"
        <thead>
            <tr class="text-slate-500 borde
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 font-bold
            </tr>
        </thead>
        <tbody class="text-slate-300">
            <tr class="border-b border-slate
-2 font-mono">0</td><td class="py-2 pr-2 fo
td></tr>
            <tr class="border-b border-slate
-2 font-mono">-2</td><td class="py-2 pr-2 f
/td></tr>
            <tr class="border-b border-slate
-2 font-mono">-6</td><td class="py-2 pr-2 f
</td></tr>
            <tr class="border-b border-slate
"py-2 pr-2 font-mono">-10</td><td class="py
nt-mono">-2</td></tr>
            <tr class="align-top"><td class
12</td><td class="py-2 pr-2 font-mono">-9</
ont-bold">всё ещё уменьшается → отрицательны
        </tbody>

```

```

<div class="overflow-x-auto">
  <table class="w-full text-xs text-l
    <thead>
      <tr class="text-slate-500 b
        <th class="py-1.5 pr-2
        <th class="py-1.5 pr-2
        <th class="py-1.5 pr-2
        <th class="py-1.5 font-l
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-
2">одна вершина → все</td><td class="py-2 p
      <tr class="border-b border-
font-mono">0((V+E) log V)</td><td class="p
      <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">нет</td><td c
    </tr>
      <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">молча ошибётся
emerald-300">обнаруживает (d[i][i] &lt; 0)<
      <tr class="border-b border-
ass="py-2 pr-2">список смежности</td><td cl
      <tr class="align-top"><td c
φ, веса ≥ 0</td><td class="py-2 pr-2">минус
    </tbody>
  </table>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
  <h3 class="text-xl font-bold text-emerald-4

  <div class="bg-slate-900/60 p-4 rounded-lg l
    <p class="text-xs font-bold text-amber-
    :</p>
  <pre class="bg-slate-950 p-3 rounded text-xs font-mono

```

```

if changed_vertex == -1:
    return dist, prev, None # отрицательны

# changed_vertex улучшен на n-й итерации:
# поднимаемся по prev на n шагов – гарантированно з
y = changed_vertex
for _ in range(n):
    y = prev[y]

cycle = [y]
v = prev[y]
while v != y:
    cycle.append(v)
    v = prev[v]
cycle.append(y)
cycle.reverse()
return dist, prev, cycle

# демо-граф страницы (9 рёбер, режим с отрицательным ци
nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [{"S", "A", 4}, {"S", "B", 5}, {"B", "C", -2}, {"A", "C", 1},
      {"C", "D", 3}, {"C", "E", 4}, {"D", "F", 2}, {"E", "F", 1}]
E = [(idx[u], idx[v], w) for u, v, w in E]

dist, prev, cyc = bellman_ford(len(nodes), E + [(idx["D",
print([dist[i] for i in range(len(nodes))]) # [0, 4,
print(cyc) # None –

dist2, _, cyc2 = bellman_ford(len(nodes), E + [(idx["D",
print([nodes[i] for i in cyc2]) # ['D',
print([dist2[i] for i in range(len(nodes))]) # все
    <p class="text-slate-400 text-xs">П
        имая вершина с <span class="font-mono">INF<
        ля <span class="font-mono">float('inf')</sp
    </div>
</details>

```

```

        vector<int> cycle;
        for (int v = y;; v = p[v]) {
            cycle.push_back(v);
            if (v == y && cycle.size() > 1) break;
        }
        reverse(cycle.begin(), cycle.end());
        return cycle;
    }

```

```

int main() {
    int s = 0;
    int x = bellman_ford(s);
    if (x == -1) cout <<< "отрицательных циклов не  

    else {
        cout <<< "цикл: ";
        for (int v : negative_cycle(x)) cout <<< v  

        cout <<< "\n";
    }
}
</code></pre>

```

</div>

</details>

<details class="bg-slate-900/40 rounded-lg

<summary class="p-4 font-bold text-slate

-b border-slate-700/50">» Оптимизации: ранни

<div class="p-5 text-sm text-slate-300

```

<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
    rt deque

```

```

def spfa(n, adj, s):
    """adj[u] = [(v, w), ...]. Релаксируем только «живые»
    INF = float("inf")
    dist = [INF] * n
    in_queue = [False] * n
    counter = [0] * n # сколько раз вершина
    dist[s] = 0
    q = deque([s])
    in_queue[s] = True

```

```

отрицательным циклом <span class="font-mono" style="font-family: monospace;">
    <li><b>Переполнение INF.</b> <span class="font-mono" style="font-family: monospace;">
        INT_MAX</span> превращается в отрицательное значение, что приводит к переполнению
        запасом и <span class="font-mono" style="font-family: monospace;">long long</span>
    <li><b>Неориентированный граф с минимальным циклом</b> <span class="font-mono" style="font-family: monospace;">-3</span> – это цикл длины 3, который встречается в отрицательном цикле. Минусы в неориентированном графе.
    <li><b>Цикл есть, но не от старта.</b> <span class="font-mono" style="font-family: monospace;">-3</span> </span>. Нужна проверка всего графа – добавляем ребро (1, 2) и запускаем от неё.</li>
    <li><b>«Ответ  $-\infty$ ».</b> Если отрицательный цикл, то <span class="font-mono" style="font-family: monospace;">dist[v] = -\infty</span>: печатать ответ, если отрицательный цикл, иначе -1.
    <li><b>SPFA как «быстрый Форд–Беллман».</b> Если отрицательный цикл, то <span class="font-mono" style="font-family: monospace;">V, E \le 10^5</span> в условии <span class="font-mono" style="font-family: monospace;">V, E \le 10^5</span>
    </ul>
</div>

<p class="text-slate-300 text-sm mt-4"><b>Сложности</b>
    Потенциалы <span class="font-mono text-emerald-500" style="font-family: monospace;">O(V)</span>
    Дейкстры. Флойд–Уоршелл (16) решает все задачи на кратчайшие пути.
    Порядок для DAG – билет 9.</p>
</div>
</div>
</section>`
},
{
    id: "floyd",
    title: "16. Графы. Поиск кратчайшего пути. Флойд",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">16. Графы. Поиск кратчайшего пути. Флойд</span>
        <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">16. Графы. Поиск кратчайшего пути. Флойд</h2>
    </div>

```

```

        Суть фокуса (по шагам)
    </div>
    <p class="text-slate-300 text-sm mb-4 lg:mb-6">
        Главный герой – внешний цикл
        <span class="font-mono text-emerald-300">k</span> т
        </span>, если в качестве промежуточных
        , ..., k}</span>?». Каждый шаг добавляет в ра
    </p>
    <ul class="text-sm text-slate-300 space-y-4">
        <li><b>Шаг k = -1 (старт):</b> разр
        <li><b>Шаг k = 0:</b> разрешаем пер
        <span class="font-mono text-emerald-300">i → 0 → j</span>, чем текущий <span cl
        <li><b>Шаг k = 1:</b> разрешены пер
        <span class="font-mono">l → j</span> уже может стоять
        <li><b>Шаг k = V - 1:</b> разрешены
    </ul>
</div>

<div class="grid grid-cols-1 sm:grid-cols-3 lg:grid-cols-4">
    <div class="bg-slate-900 border border-emerald-500 shadow-md">
        <p class="text-[10px] uppercase tracking-wider">Шаг 0:
        <p class="font-mono text-emerald-300">i → 0 → j
    </div>
    <div class="bg-slate-900 border border-emerald-500 shadow-md">
        <p class="text-[10px] uppercase tracking-wider">Шаг 1:
        <p class="font-mono text-emerald-300">l → j
    </div>
    <div class="bg-slate-900 border border-emerald-500 shadow-md">
        <p class="text-[10px] uppercase tracking-wider">Шаг k:
        <p class="font-mono text-emerald-300">...
    </div>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 lg:left-6 lg:right-6
        <div class="border border-emerald-500 shadow-md">
            Аналогия 1: Таблица пересадок в

```

```

                <th class="py-1.5 pr-2 font-
                <th class="py-1.5 font-bold
            </tr>
        </thead>
        <tbody class="text-slate-300">
            <tr class="border-b border-slate-
-2 font-mono">0</td><td class="py-2">в вершн
            <tr class="border-b border-slate-
-2 font-mono">4</td><td class="py-2">открыва
            <tr class="border-b border-slate-
-2 font-mono">3</td><td class="py-2">2 – вх
            <tr class="border-b border-slate-
-2 font-mono">2</td><td class="py-2">3 – вх
            <tr class="border-b border-slate-
-2 font-mono">4</td><td class="py-2">через
            <tr class="border-b border-slate-
-2 font-mono">0</td><td class="py-2">из 5 в
            <tr class="align-top"><td class="
300">10</td><td class="py-2">ребро 6 → 1 за
        </tbody>
    </table>
    <p class="text-slate-400 text-xs mt-3">
mono text-emerald-300">0 → {0, 4, 7, 2, 6,
"font-mono">0 → 3 → 4 → 6</span> (2 + 4 + 1
всего в финальной матрице 11 таких пар. Диа
</div>

<div class="bg-slate-800/70 p-5 rounded-lg
    <p class="text-sm font-bold text-white
    <table class="w-full text-xs text-left
        <thead>
            <tr class="text-slate-500 borde
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 font-bold
            </tr>
        </thead>
        <tbody class="text-slate-300">

```

```

- mono">d[k][k] < 0 и рассуждение
    </div>
</details>

```

```

<details class="bg-slate-900/40 rounded-lg
    <summary class="p-4 font-bold text-slate-
    -b border-slate-700/50">> Эксперимент: пере
    <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
M = [[0,4,INF,2,INF,INF,INF],
      [INF,0,3,INF,INF,3,INF],
      [INF,INF,0,INF,2,INF,INF],
      [INF,INF,INF,0,4,2,INF],
      [INF,INF,INF,INF,0,INF,1],
      [INF,INF,INF,INF,INF,0,5],
      [INF,1,INF,INF,INF,INF,0]]

```

```

def floyd(M, order="kij"):
    n = len(M)
    d = [row[:] for row in M]
    for a in range(n):
        for b in range(n):
            for c in range(n):
                k, i, j = {"kij": (a, b, c),
                           "ijk": (c, a, b),
                           "ikj": (c, a, b)}[order]
                if d[i][k] + d[k][j] < d[i][j]:
                    d[i][j] = d[i][k] + d[k][j]
    return d

```

```

good = floyd(M, "kij")
bad = floyd(M, "ijk")
print(good[2][1], bad[2][1])    # 4 и 999 – путь 2→4→6→1
print(sum(1 for i in range(7) for j in range(7) if good
# 4 клетки отличаются; при порядке (i,k,j) их 6
    <p class="text-slate-400 text-xs">В
угих «почти работают» (большинство клеток с
    </div>

```



```

        return d, nxt, negative

def path(nxt, i, j):
    if nxt[i][j] == -1:
        return None # j недостижим
    p = [i]
    while p[-1] != j:
        p.append(nxt[p[-1]][j])
    return p

nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [{"S", "A", 4}, {"S", "B", 5}, {"B", "C", -2}, {"A", "C", 1},
      {"C", "D", 3}, {"C", "E", 4}, {"D", "F", 2}, {"E", "F", 1}]
E = [(idx[u], idx[v], w) for u, v, w in E]

d, nxt, neg = floyd_warshall(len(nodes), E)
print([d[0][j] for j in range(len(nodes))]) # [0, 4, 5, 3, 2, 1]
print(path(nxt, idx["S"], idx["F"])) # [0, 2, 1]
print(neg) # [] — отрицательного цикла нет

```

</div>  
</details>

```

<details class="bg-slate-900/40 rounded-lg p-4">
  <summary class="p-4 font-bold text-slate-300">
    -b border-slate-700/50">» Код: C++ (матрица
    <div class="p-5 text-sm text-slate-300">
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">
    t;
#include <algorithm>
using namespace std;
const long long INF = 4e18;

int n;
vector<vector<long long>>> d(n, vector<long long>(n, INF));

void floyd() {
    for (int i = 0; i < n; ++i) d[i][i] = 0;

```

```

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
  -b border-slate-700/50">» Та же динамика дл
    <div class="p-5 text-sm text-slate-300
      <table class="w-full text-xs text-l
        <thead>
          <tr class="text-slate-500 b
            <th class="py-1.5 pr-2
            <th class="py-1.5 pr-2
            <th class="py-1.5 font-l
          </tr>
        </thead>
        <tbody class="text-slate-300">
          <tr class="border-b border-
имость)</td><td class="py-2 pr-2 font-mono">
ь ребро; r[i][i] = true</td></tr>
          <tr class="border-b border-
пути)</td><td class="py-2 pr-2 font-mono">d
= w(i,j); d[i][i] = 0</td></tr>
          <tr class="border-b border-
ой сети</td><td class="py-2 pr-2 font-mono">
и отсутствии положительных циклов</td></tr>
          <tr class="align-top"><td c
o">if d[i][k]+d[k][j] == d[i][j]: cnt[i][j]
а переполнением</td></tr>
        </tbody>
      </table>
      <p class="text-slate-400 text-xs mt
> при <span class="font-mono">r[i][k]</span>
V³)</span>.</p>
    </div>
  </details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
  <h3 class="text-xl font-bold text-amber-400

  <div class="bg-rose-950/30 p-5 rounded-lg b

```

```

title: "17. Графы. Поиск кратчайшего пути. Ускорени
type: "html",
description:
  "Джонсон (перевзвешивание потенциалов), встречный
category: "Графы. Пути",
content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-amber-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-amber-300 italic">
        <p class="text-xl font-mono text-white">
        е посещённых вершин (встречный поиск, ранни
        contraction hierarchies).</p>
      </div>

      <p class="text-slate-300 text-sm mb-4">Дейк
      в лоб. На экзамене спрашивают, <b>что сдела
      строки перевзвешивания» до промышленных у

      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-
            rder border-emerald-500 shadow-md">
              Аналогия 1: Такси с навигатором
            </div>
          <p class="text-slate-300 text-sm mb-4">
            ы одинаково. А* — тот же таксист, но с нави
            /span> прибавляется оценка остатка пути по
            т туда, где сумма меньше. Навигатор обязан
            /span>): занижать можно, завышать нельзя —
            0">h = 0</span> это снова Дейкстра.</p>
          </div>

```

```

        лько 0 и 1</td></tr>
        <tr class="border-b border-slate-500">
<td class="py-2 pr-3">куча → массив корзинок
        С небольшое</td></tr>
        <tr class="border-b border-slate-500">
<td class="py-2 pr-3">decrease-key за  $O(1)$ 
        y-2">теоретически; на практике двоичная/d-ар
        <tr class="border-b border-slate-500">
<td class="py-2 pr-3">не релаксируем все рёбра
        , на практике быстро</td><td class="py-2">о
        <tr class="align-top"><td class="py-2 pr-3">3">переносит работу в предобработку</td><td class="py-2">
        иллионы запросов (карты)</td></tr>
    </tbody>
</table>
</div>

```

```

<h3 class="text-lg font-bold text-white mt-6">
<div class="bg-slate-900 p-4 rounded-lg mb-6">
    <p class="text-xl font-mono text-white">
</div>
<div class="grid grid-cols-1 gap-6 mb-6">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-amber-500 shadow-md">
            ☞ Суть перевзвешивания
        </div>
        <p class="text-slate-300 text-sm mb-6">
            Дейкстра (билет 14) быстрая, но не оптимальный. Алгоритм Джонсона делает финт: пере-
            ь теми же.
        </p>
        <ol class="text-sm text-slate-300 space-y-2">
            <li>Добавляем <b>фиктивную вершину</b>
            <li>Запускаем от s <b>Форда-Беллмана</b>
            </ol>
            <p>важный факт:</i> Джонсон НЕ «удаляет» отрицательные
            кой и прекращает работу, так как при отрицательных весах
            <li>Перевзвешиваем: <span class="font-mono">

```

оторых расстояние только что уменьшилось

ческого прогона, но асимптотика та же,  $O(V \cdot E)$ .  
пользуют осторожно, с эвристиками SLF/LLL).  
) уменьшает высоту и на практике быстрее, к

### Навигация по графу

в **предобработку**. Сначала – **сжатие** индексами  $0 \dots N$ , и уже по ним строят граф и сетевые потоки. **Contraction hierarchies** (так и называются) и запрос идёт двунаправленным поиском вверх по дереву. В вершине заранее приписывают «метки», и запрос

тро, но не кратчайший. Переоценка = неверный ответ.

именимой к отрицательным весам и даёт все возможные кратчайшие пути.

информации. Там нужен Dial.

практике меньше бесполезных релаксаций.

**Алгоритмы поиска кратчайшего пути**

м числом раскрытий,  $0 \dots 1$  BFS  $O(V + E)$ , Dial  $O(E + V \cdot C)$ , Дейкстра с кучей  $O(E + V \log V)$ .

```

        for to, w in g[v]:
            if d[s][to] > dist + w:
                d[s][to] = dist + w
                heapq.heappush(heap, (d[s][to], to))
    for v in range(n):
        if d[s][v] < INF:
            d[s][v] = d[s][v] - h[s] + h[v]    # откорректировать
    return d

```

</div>

</details>

<details class="bg-slate-900/40 rounded-lg p-4">

<summary class="p-4 font-bold text-slate-300">

-b border-slate-700/50">

Код: встречный Дейкстра, A\*, 0-1

</summary>

<div class="p-5 text-sm text-slate-300">

<pre class="bg-slate-950 p-4 rounded">

встречный Дейкстра: остановка по min\_f + min\_g

def bidirectional(g, rg, s, t): # rg – граф с перестройкой

INF = float("inf")

df = {s: 0}; db = {t: 0}

qf = [(0, s)]; qb = [(0, t)]

mu = INF

while qf and qb:

if qf[0][0] + qb[0][0] >= mu: # вот правый

return mu

dist, v = heapq.heappop(qf)

if dist > df.get(v, INF): continue

for to, w in g[v]:

nd = dist + w

if nd < df.get(to, INF):

df[to] = nd; heapq.heappush(qf, (nd, to))

if to in db:

mu = min(mu, nd + db[to]) # встреча:

dist, v = heapq.heappop(qb)

if dist > db.get(v, INF): continue

for to, w in rg[v]:

```

        (dq.appendleft(to) if w == 0 else dq.append(to))
    return d</pre>
</div>
</details>

<details class="bg-slate-900/40 rounded-lg border-1
  <summary class="p-4 font-bold text-slate-300
  -b border-slate-700/50">
    Вопросы, которые любят задавать (и на которые
  </summary>
  <div class="p-5 text-sm text-slate-300">
    <p>• Почему после перевзвешивания весов не
    есть условие отсутствия улучшающей релаксации?
    <p>• Почему кратчайшие пути не меняются?
    иш) и одинаков для всех путей между этой парой
    <p>• Что лучше для всех пар: Флойд-Уоршелл или
    g V) выигрывает на разреженных.</p>
    <p>• Какая эвристика A* допустима на графах с
    ии скорости можно делить на максимальную скорость
    <p>• Зачем нужен ранний выход в Дейкстре?
    смысла нет.</p>
    <p>• Почему 0-1 BFS работает без кучи?
    же отсортирована.</p>
  </div>
</details>
</div>
</div>
</section>`
  },
];

```

ФАЙЛ 28/111: src/data/tickets/ticket18\_20\_mst.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 231 | РАЗМЕР: 1.2 KB

```
import { Chapter } from '../../../types';
```

MST. Алгоритмы отличаются только тем, **<b>к**

Прим – «выросшее дерево против остального графа»

```
<div class="bg-slate-800 p-5 rounded-lg border" style="border-color: #444; border-radius: 10px; padding: 10px;">
  <h4 class="text-amber-400 font-bold text-sm">Пример
  <p class="text-slate-200 text-sm font-sans">
    <p class="text-slate-300 text-sm mb-3">
      остается буква Р – остается <b>остов</b>.</p>
      <p class="text-slate-300 text-sm">Остов – это часть графа: минимум костей, чтобы всё держалось.
    </p>
  </div>
```

**<p class="text-slate-300 text-sm mb-4">Второй вариант**

никогда не входит в MST. Именно его использовали в алгоритме Крускала. В образовавшемся цикле (рёбра-то мы берём по возрастанию веса)

```
<div class="overflow-x-auto my-6">
  <table class="w-full text-xs text-left border-collapse" style="width: 100%; border-collapse: collapse;">
    <thead>
      <tr class="text-slate-500 border-bottom" style="border-bottom: 1px solid #444; text-align: left;">
        <th class="py-2 pr-3 font-bold">Веса
        <th class="py-2 pr-3 font-bold">Рёбра
        <th class="py-2 pr-3 font-bold">Минимальный остов
        <th class="py-2 pr-3 font-bold">Максимальный остов
        <th class="py-2 font-bold">Комментарии
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-slate-800" style="border-bottom: 1px solid #444; text-align: left;">
        <td class="py-2 pr-3 font-bold">Веса
        <td class="py-2 pr-3">Рёбра
        <td class="py-2 pr-3 font-medium">Минимальный остов
        <td class="py-2 pr-3 font-medium">Максимальный остов
        <td class="py-2">рёбра даны
      </tr>
      <tr class="border-b border-slate-800" style="border-bottom: 1px solid #444; text-align: left;">
        <td class="py-2 pr-3 font-bold">Веса
        <td class="py-2 pr-3">Рёбра
      </tr>
    </tbody>
  </table>
```



```
<div class="absolute -top-3 left-4 l
r border-rose-500 shadow-md">
```

Аналогия к билету 20 • Боровк

```
</div>
```

```
<p class="text-slate-300 text-sm">С
айшему чужому племени: к вечеру компонент в,
<b>log2 V</b> дней образуется единая федер
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-900/40 rounded-lg bord
```

```
<p class="text-slate-300 text-sm mb-2">
```

```
<p class="text-slate-400 text-sm mb-1">
```

состояние от одной вершины до каждой; MST ми  
куче другой: у Прима – вес ребра к дереву,   
тся.</p>

```
<p class="text-slate-400 text-sm mb-1">
```

вных весах деревьев может быть несколько, н

```
<p class="text-slate-400 text-sm mb-1">
```

дом вместо почти-константного find/union.</p>

```
<p class="text-slate-400 text-sm mb-1">
```

и Краскал построил минимальный остовный <i>

```
<p class="text-slate-400 text-sm">• <b>P
```

ин раз, DSU спасает от дубля (find(u) == fi

```
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b>с
py-0.5 rounded text-xs font-mono text-emera
```

бесплатный); Прим с двоичной кучей <code>с  
rder-slate-800">O(E log V)</code>, на плотн  
text-xs font-mono text-emerald-300 border b  
d text-xs font-mono text-emerald-300 border  
ex <code class="bg-slate-950 px-2 py-0.5 ro  
</p>

```
<details class="bg-slate-900/40 rounded-lg l
```

```
<summary class="p-4 font-bold text-slate
```

```
-b border-slate-700/50">
```

## Универсальное пособие • полный исходный код

```
-----
key = [INF] * n          # минимальный вес ребра, ведущий к v
used = [False] * n
key[0] = 0
heap = [(0, 0)]          # (вес ребра, вершина) ← вот это
total = 0
```

```
while heap:
    w, v = heapq.heappop(heap)
    if used[v]:
        continue        # устаревшая запись в куче
    used[v] = True
    total += w
    for to, weight in graph[v]:
        if not used[to] and weight < key[to]:
            key[to] = weight
            heapq.heappush(heap, (weight, to))
print(total)</pre>
```

<p class="text-slate-400 text-xs">Собираем путь от старта до каждой вершины (накапливаем путь от старта), здесь <span class="text-slate-400 text-xs">а – другая задача.</p>

</div>  
</details>

<details class="bg-slate-900/40 rounded-lg border-slate-700/50">  
 <summary class="p-4 font-bold text-slate-300">  
 -b border-slate-700/50">

Код 20: Борувка (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300">  
 <pre class="bg-slate-950 p-4 rounded">  
 = list(range(n))

```
def find(v):
    while parent[v] != v:
        parent[v] = parent[parent[v]]
        v = parent[v]
    return v
```

```
comps = n
```

```
        <p>• Что если граф несвязный? — все  
    </p>  
    <p>• Как найти второе по величине о  
ребро на образовавшемся цикле (LCA + максим  
    </div>  
  </details>  
</div>  
</div>  
</section>`,  
  },  
];
```

---

ФАЙЛ 29/111: src/data/tickets/ticket21\_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРӨК: 420 | РАЗМ

---

```
import { Chapter } from '../types';  
  
export const tickets21to24: Chapter[] = [  
  {  
    id: "string-kmp",  
    title: "21. Алгоритм Кнута-Морриса-Пратта (KMP)",  
    type: "html",  
    category: "Строки",  
    content: `  
<section id="string-kmp" class="mb-12 scroll-mt-10">  
  <div class="flex items-center mb-6 flex-wrap gap-3">  
    <span class="bg-indigo-600 text-white px-4 py-1">  
    <h2 class="text-2xl sm:text-3xl font-bold text-  
  </div>  
  <div class="space-y-8">  
    <div class="bg-slate-700/50 p-6 rounded-xl bord  
      <h3 class="text-xl font-bold text-blue-400 r  
      <div class="bg-slate-900 p-4 rounded-lg mb-  
        <p class="text-lg text-blue-300 italic r  
        <p class="text-xl font-mono text-white">
```

```

        последняя буква). Длина 3: <b>abc</b> против
        digo-300">π[6] = 0</code></li>
    </ul>
</div>
</details>
<details class="bg-slate-900/40 rounded-lg mb-4" >
    <summary class="p-4 font-bold text-slate-300" >
        -b border-slate-700/50">
            Код C++ (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300" >
        <pre class="bg-slate-950 p-4 rounded" >
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
        </div>
</details>

<h3 class="text-lg font-bold text-white mt-4">
<div class="bg-slate-900 p-4 rounded-lg mb-4" >
    <p class="text-xl font-mono text-white">
        p>
    </div>
<p class="text-slate-300 text-sm mb-4">Префикс
    для шаблона (pattern) один раз, затем идём
    пало – j += 1. Не совпало – откатываемся на
    у заново сравнивать его не нужно. Когда j д
    ждения), делаем j = π[j - 1].</p>
<p class="text-slate-300 text-sm mb-4"><b>П
    ждый откат j уменьшает. Значит суммарно отк
    классическая амортизационная оценка, её люб
    <div class="bg-slate-900/40 rounded-lg border" >
        <p class="text-slate-300 text-sm mb-2">
        <p class="text-slate-400 text-sm mb-1">

```

```

        found.append(i - m + 1)      # позиция вхождения
        j = pi[j - 1]               # ищем дальше, в
    return found

# тот же результат одной префикс-функцией:  $\pi$  от pattern
def kmp_via_concat(text, pattern):
    s = pattern + "#" + text
    pi = prefix_function(s)
    m = len(pattern)
    return [i - 2 * m for i in range(m + 1, len(s)) if pi[i] == m]

<p class="text-slate-400 text-xs">Результат поиска по границе и вхождения будут ложными.</p>
</div>
</details>
</div>
</div>
</section>`
},
{
    id: "string-z-func",
    title: "22. Z-функция — Сдвиг и наложение",
    type: "html",
    category: "Строки",
    content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">Z-функция
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Z-функция
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">Определение
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Z-функция строки  $s$  — это массив из целых чисел  $z$  той же длины, что и  $s$ , где  $z[i]$  — длина наибольшей палиндромической подстроки, начинающейся в позиции  $i$  (с индексом 0) и её суффиксом, начинающимся с  $i$ .</p>
      </div>
    </div>
  </div>
</section>`

```

```

        </summary>
        <div class="p-5 text-sm text-slate-300" style="border: 1px solid #ccc; border-radius: 10px; padding: 10px; margin-top: 10px;">
            <pre class="bg-slate-950 p-4 rounded" style="background-color: #1a2b3c; color: white; font-family: monospace; font-size: 0.9em;">
int l = 0, r = 0;
for (int i = 1; i <= n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}
            </pre>
        </div>
    </details>
</div>
</div>
</section>`
    },
    {
        id: "aho-corasick",
        title: "23. Алгоритм Ахо-Корасик",
        type: "html",
        category: "Строки",
        content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1 rounded">Ахо-Корасик
        <h2 class="text-2xl sm:text-3xl font-bold text-white">Ахо-Корасик
    </div>

    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-indigo-500">
            <h3 class="text-xl font-bold text-blue-400 mb-4">Описание
            <p class="text-sm text-slate-300">Алгоритм Ахо-Корасик (Aho-Corasick) — это алгоритм поиска всех вхождений заданного слова в текст за один проход. Он основан на автомате Ахо-Корасик, который строится на основе префиксного дерева (trie) и массива переходов по суффиксам (failure links). Алгоритм работает за время O(n * m), где n — длина текста, а m — длина слова. Он позволяет найти все вхождения слова в текст за один проход, что делает его очень эффективным для поиска по словарю.
            <p class="text-sm text-slate-300">Алгоритм Ахо-Корасик (Aho-Corasick) — это алгоритм поиска всех вхождений заданного слова в текст за один проход. Он основан на автомате Ахо-Корасик, который строится на основе префиксного дерева (trie) и массива переходов по суффиксам (failure links). Алгоритм работает за время O(n * m), где n — длина текста, а m — длина слова. Он позволяет найти все вхождения слова в текст за один проход, что делает его очень эффективным для поиска по словарю.
        </div>
        <div class="bg-slate-900 p-4 rounded-lg mb-6 text-white">
            <p class="text-sm text-blue-300 italic mb-2">Описание
            <p class="text-lg font-mono text-white">Бор (Три)
        </div>
        <p class="text-slate-300 text-sm">Позволяет найти все вхождения заданного слова в текст за один проход.</p>
    </div>
`
    }
}

```



```

        <th class="py-2 pr-3 font-b
        <th class="py-2 pr-3 font-b
        <th class="py-2 font-bold">
    </tr>
</thead>
<tbody class="text-slate-300">
    <tr class="border-b border-slate
py-2 pr-3">детерминированно за  $O(n^k)$ </td><td>
удольности, максимальный поток</td></tr>
    <tr class="border-b border-slate
py-2 pr-3">сертификат проверяется за полином
ешающая версия), коммивояжёр «есть ли тур ≤
    <tr class="border-b border-slate
s="py-2 pr-3">сертификат есть у ответа «НЕТ:
ель)</td></tr>
    <tr class="border-b border-slate
lass="py-2 pr-3">не проще любой задачи из NP
фа, TSP-оптимизация (не решающая версия)</td>
    <tr class="border-b border-slate
ass="py-2 pr-3">NP-трудные И из NP</td><td>
сумм</td></tr>
    <tr class="border-b border-slate
"py-2 pr-3">полиномиальная память, время не
й на полиномиальном поле</td></tr>
    <tr class="align-top"><td class=
за  $2^{(n^k)}$ </td><td class="py-2">обобщённые
    </tbody>
</table>
</div>

```

```

<h3 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Стандарт
ать сертификат полиномиального размера и про
    – «все вершины по разу и каждая соседняя п
ь полиномиальное сведение  $A \leq_p B$ . Тогда  $B$  –
<p class="text-slate-300 text-sm mb-3"><b>Теорема
мул SAT NP-полна. Доказывается «в лоб»: вычл
евой формулой размера  $\text{poly}(n)$ , где переменн

```



```

        ребор сертификатов <code class="bg-slate-900 p-4 border border-slate-800">0(poly(n))</code>
        ^poly(n))</code>; сведение обязано работать
        0 border border-slate-800">0(poly(n))</code>

        <details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
            <summary class="p-4 font-bold text-slate-300">
                Код: верификатор сертификата и про
            </summary>
            <div class="p-5 text-sm text-slate-300">
                <pre class="bg-slate-950 p-4 rounded">
                    ьтонов цикл NP-полон: сертификат = порядок
def verify_hamilton(n, edges, cert):
    if sorted(cert) != list(range(n)):                # каждая
        return False
    allowed = {(u, v) for u, v in edges} | {(v, u) for
    for i in range(n):
        a, b = cert[i], cert[(i + 1) % n]             # замык
        if (a, b) not in allowed:
            return False
    return True

# 3-SAT ≤p КЛИКА: формула с k дизъюнктов → граф из 3k верш
def reduce_3sat_to_clique(clauses):
    # вершина = {номер дизъюнкта, литерал}; ребро, если
    verts = [(ci, lit) for ci, cl in enumerate(clauses)
    edges = []
    for i in range(len(verts)):
        for j in range(i + 1, len(verts)):
            ci, li = verts[i]; cj, lj = verts[j]
            if ci == cj:                                # из од
                continue
            if li.lstrip("-") == lj.lstrip("-") and (li
                continue                                # x и n
            edges.append((i, j))
    return verts, edges, len(clauses)                  # клика

# Проверка задачи из P для контраста: двудольность = 2-

```

```
        облема остановки)</p>
      </div>
    </details>
  </div>
</div>
</section>`
  }
};
```

---

ФАЙЛ 30/111: src/data/tickets/ticket6\_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 576 | РАЗМ

---

```
import { Chapter } from "../../types";

export const tickets6to13: Chapter[] = [
  {
    id: "graph-dfs-bfs",
    title: "6. Графы. Представление графов, DFS, BFS",
    type: "html",
    description: "Представление графов и базовые обходы",
    category: "Графы. База",
    content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">6
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">Графы
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-slate-300">Базовые обходы
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-slate-300 italic">Глубина (DFS)
        <p class="text-xl font-mono text-white">graph.dfs(
          + E).</p>
      </div>
    </div>
  </div>
`
  }
];
```

```

        <li>Топологическая сортировка (I
        <li>Поиск мостов и точек сочлен
        <li>Компоненты сильной связности
        <li>Детектирование циклов (через
    </ul>
</div>

<!-- Аналогия BFS -->
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
    <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
        BFS (Поиск в ширину) = Волна
    </div>
    <p class="text-slate-300 text-sm mb-0">
        <b>Что делает:</b> Распространение по соседям. Работает через Очередь (FIFO). Гарантирует минимальное расстояние (в узлах) от источника.
    </p>
    <ul class="text-xs text-emerald-300">
        <li><b>Где используется:</b></li>
        <li>Кратчайший путь в Невзвешенном графе</li>
        <li>Проверка двудольности графа</li>
        <li>Алгоритм Ахо-Корасик (построение автомата)</li>
        <li>Двунаправленный BFS для поиска кратчайшего пути (в узлах) в неориентированном графе.</li>
    </ul>
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
    <summary class="p-4 font-bold text-slate-300" style="border: 1px solid #008080; border-radius: 10px 10px 0px 0px; box-shadow: 5px 5px 0px #008080;">
        <b>Код обходов C++ (Основа)</b>
    </summary>
    <div class="grid grid-cols-1 md:grid-cols-2">
        <pre class="bg-slate-950 p-3 rounded" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
// DFS (Рекурсия с возвратом)
vector<bool> vis;
vector<vector<int>>> adj;

```

```
<h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
</div>
```

```
<div class="space-y-8">
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border">
```

```
<h3 class="text-xl font-bold text-blue-400">
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
<p class="text-lg text-blue-300 italic">
```

```
<p class="text-2xl font-mono text-white">
```

```
<p class="text-xs text-slate-400 mt-1">
```

```
</div>
```

```
<div class="space-y-4 text-slate-300 text-sm">
```

```
<p><b>Главные неравенства на число рёбер</b>
```

```
<ul class="list-disc list-inside space-y-2">
```

```
<li><b>Для простого планарного графа</b>
```

```
<i>Доказательство:</i> Каждая грань
```

```
span>. Подставляя в формулу Эйлера: <span c
```

```
</li>
```

```
<li><b>Для простого двудольного план
```

```
/span>.<br>
```

```
<i>Доказательство:</i> В двудольном
```

```
class="font-mono"> $2E \geq 4F \Rightarrow F \leq E/2$ </span>
```

```
</li>
```

```
</ul>
```

```
<div class="bg-slate-950 p-4 rounded-lg">
```

```
<p class="text-amber-300 font-bold">
```

```
<p class="text-slate-300 text-xs le
```

```
1. <b> $K_5$ </b> (полный граф на 5 вершинах)  
class="font-mono"> $10 \leq 3(5) - 6 = 9$ </span>
```

```
2. <b> $K_{3,3}$ </b> (полный двудольный  $3 \times 3$  граф)  
ss="font-mono"> $9 \leq 12$ </span> – формально пр  
 $2V - 4$ </span>: <span class="font-mono"> $9 \leq$ 
```

```
</p>
```

```
</div>
```

```
<p><b>Теорема Куратовского (1930) / Вагнера</b>  
фа, являющегося <b>подразделением (subdivis
```

```

description:
  "Компоненты связности: обходами DFS/BFS за  $O(V + E)$ "
category: "Графы. База",
content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        рой вершин есть путь. Внешний цикл «для каж
        » даёт и разметку, и количество компонент.<
      </div>

      <p class="text-slate-300 text-sm mb-4">Два
      ной запускаем DFS или BFS и присваиваем все
      mp[v] = c</span>. Счётчик с увеличивается р
      Время <b> $O(V + E)$ </b>, память  $O(V)$ . <b>Чере
      каждое ребро делаем union(u, v), в конце чи
      ту</b> и после каждого добавления спрашиваю

      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
          rder border-emerald-500 shadow-md">
            Аналогия 1: Острова
          </div>
          <p class="text-slate-300 text-sm">Т
          ались серые (неисследованные) зоны? Прыгаеш
          и компонент. Краска — это comp[v], прыжок ч
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4

```

```
</section>`,
  },
  {
    id: "top-sort",
    title: "9. Графы. Топологическая сортировка",
    type: "html",
    description: "Топологическая сортировка ДАГ и детект",
    category: "Графы",
    content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
перед.</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Учеба в вузе
          </div>
          <p class="text-slate-300 text-sm mb-4">
екая сортировка – это расписание, которое
        </p>
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
            Ограничения: Циклы
          </div>
          <p class="text-slate-300 text-sm mb-4">
```

```

        has_cycle = true;
        return;
    }
}
color[v] = 2; // Вышли – вершина чёрная
ans.push_back(v); // Сохраняем в момент ВЫХОДА (pos
}

```

```

bool topological_sort(int n) {
    fill(color, color + n + 1, 0);
    has_cycle = false;
    ans.clear();
    for (int i = 1; i <= n; ++i) {
        if (color[i] == 0) dfs(i);
    }
    if (has_cycle) return false; // Граф содержит цикл!
    reverse(ans.begin(), ans.end()); // Разворачиваем р
    return true;
}

```

```

        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "scc-kosaraju",
        title: "10. Графы. Компоненты сильной связности (SCC)",
        type: "html",
        description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
        category: "Графы. Продвинутое",
        content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">10. Графы. Компоненты сильной связности (SCC)</span>
        <h2 class="text-2xl sm:text-3xl font-bold text-white">10. Графы. Компоненты сильной связности (SCC)</h2>
    </div>

```

```

    },
    {
      id: "graph-bridges",
      title: "11. Графы. Мосты",
      type: "html",
      description: "Рёбра, удаление которых увеличивает число мостов",
      category: "Графы. Продвинутое",
      content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">Мосты
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-rose-400">Определение
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">Мостом в графе называют ребро, удаление которого увеличивает количество компонент связности графа.
        <p class="text-xl font-mono text-white">Пример:
        <p class="text-xs text-slate-400 mt-1">
          <img alt="Diagram of a graph with 6 vertices and 7 edges. The graph consists of a cycle of 5 vertices with an additional vertex connected to one of the cycle vertices." data-bbox="100 100 250 250"/>
        </p>
      </div>
      <div class="bg-slate-950 p-4 rounded-lg border border-slate-800">
        <p class="text-amber-300 font-bold text-sm">Определение
        <p class="text-slate-300 text-xs leading-relaxed">
          Проверка <span class="font-mono text-white">u
          тиграфе между u и v может быть 2 параллельных ребра.
          азуют цикл длиной 2.<br>
          <b>Решение:</b> передавать в DFS не только id вершины, но и id ребра.
          rald-300">p_edge_id</span> (индекс ребра, по которому пришли)
        </p>
      </div>
      <div class="bg-slate-900/40 rounded-lg border border-slate-700/50">
        <summary class="p-4 font-bold text-slate-200">
          <b>Код поиска мостов C++ (Скрыто)
        </summary>
      </div>
    </div>
  </div>
`
    }
  ]
}
```



```
</div>
```

```
<div class="space-y-8">
```

```
  <div class="bg-slate-700/50 p-6 rounded-xl border">
```

```
    <h3 class="text-xl font-bold text-rose-400">
```

```
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
      <p class="text-lg text-rose-300 italic">
```

```
      <div class="text-left font-mono text-sm">
```

```
        <p><strong class="text-white">Точка  
рой увеличивает число компонент связности.</p>
```

```
      <div class="p-3 bg-slate-950 rounded">
```

```
        <span class="text-rose-400 font">
```

```
        <p class="text-xl font-bold tex">
```

```
        <p class="text-xs text-slate-40">
```

```
      </div>
```

```
    </div>
```

```
</div>
```

```
<p class="text-slate-300 text-sm">
```

```
  <b>Особый случай:</b> Корень дерева DFS  
мого ребёнка</b> в дереве DFS.
```

```
</p>
```

```
<details class="bg-slate-900/40 rounded-lg">
```

```
  <summary class="p-4 font-bold text-slate-300  
-b border-slate-700/50">
```

```
    Код поиска точек сочленения (Скры
```

```
  </summary>
```

```
  <div class="p-5 text-sm text-slate-300">
```

```
    <pre class="bg-slate-950 p-4 rounded">
```

```
void dfs_cutpoints(int v, int p = -1) {
```

```
  tin[v] = low[v] = ++timer;
```

```
  int children = 0;
```

```
  for (int to : adj[v]) {
```

```
    if (to == p) continue;
```

```
    if (tin[to]) {
```

```
      low[v] = min(low[v], tin[to]);
```

```
</div>
```

```
<div class="grid grid-cols-1 md:grid-cols-2"
  <!-- Неориентированные графы -->
  <div class="bg-slate-800 p-5 rounded-lg"
    <p class="text-indigo-300 font-bold"
      <ul class="list-disc list-inside sp
        <li><b>Эйлеров цикл:</b> связан
-emerald-300">deg[v] % 2 == 0</span>).</li>
        <li><b>Эйлеров путь:</b> связан
о 2 – путь начинается в одной нечётной и за
      </ul>
    </div>
```

```

  <!-- Ориентированные графы -->
  <div class="bg-slate-800 p-5 rounded-lg"
    <p class="text-indigo-300 font-bold"
      <ul class="list-disc list-inside sp
        <li><b>Эйлеров цикл:</b> слабос
ald-300">in[v] == out[v]</span>).</li>
        <li><b>Эйлеров путь:</b> слабос
s] = 1</span> (старт), у одной <span class=
an class="font-mono text-emerald-300">in[v]
      </ul>
    </div>
  </div>
```

```
<h3 class="text-lg font-bold text-white mt-1"
<p class="text-slate-300 text-sm mb-3">Исполн
  → рекурсивный переход. При возврате (на вы
ты: <span class="font-mono text-emerald-300
```

```
<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate-300"
  -b border-slate-700/50">
    Код построения Эйлерова цикла C++
  </summary>
  <div class="p-5 text-sm text-slate-300
```

```

    y: number;
    label: string;
}

interface Edge {
    u: number;
    v: number;
}

const nodes: Node[] = [
    { id: 0, x: 250, y: 50, label: "0" },
    { id: 1, x: 100, y: 150, label: "1" },
    { id: 2, x: 250, y: 250, label: "2" },
    { id: 3, x: 400, y: 150, label: "3" },
    { id: 4, x: 550, y: 150, label: "4" },
    { id: 5, x: 700, y: 50, label: "5" },
    { id: 6, x: 700, y: 250, label: "6" },
];

const edges: Edge[] = [
    { u: 0, v: 1 },
    { u: 1, v: 2 },
    { u: 2, v: 0 },
    { u: 2, v: 3 },
    { u: 3, v: 4 }, // 3 and 4 are articulation points (3
    { u: 4, v: 5 },
    { u: 5, v: 6 },
    { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
    { length: Object.keys(nodes).length },
    () => [],
);

edges.forEach((e) => {
    adj[e.u].push(e.v);
    adj[e.v].push(e.u);
});

```

```
};
```

```
recordStep("Начало DFS (Тарьян) для поиска точек со
```

```
const dfs = (v: number, p: number = -1) => {
  visited.add(v);
  currentTin[v] = currentLow[v] = timer++;
  let children = 0;
```

```
  recordStep(`Заходим в вершину ${v}. tin[${v}]=${c
```

```
  for (const to of adj[v]) {
    if (to === p) continue;
```

```
    if (visited.has(to)) {
      currentLow[v] = Math.min(currentLow[v], curren
      recordStep(
        `Обратное ребро ${v}-${to}. Обновляем low[${v}
        v,
      );
    } else {
```

```
      children++;
      dfs(to, v);
      currentLow[v] = Math.min(currentLow[v], curren
```

```
      recordStep(
        `Возврат в ${v} из ${to}. Обновляем low[${v}
        v,
      );
```

```
      if (currentLow[to] >= currentTin[v] && p !==
        ap.add(v);
        recordStep(
          `Найдена точка сочленения: вершина ${v}
          v,
        );
      }
    }
  }
```

```
}
```

```

        interval = setInterval(() => {
            setCurrentStepIndex((prev) => prev + 1);
        }, 1500);
    } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
    }
    return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
    setIsPlaying(false);
    setCurrentStepIndex(0);
};

return (
    <div className="bg-slate-900 rounded-xl border border-
        <div className="bg-slate-800 p-4 border-b border-
            <h3 className="font-bold text-white flex items-
                <Info className="w-5 h-5 text-rose-400" />
                Моделирование поиска точек сочленения
            </h3>

        <div className="flex items-center gap-2 bg-slate-
            <button
                onClick={togglePlay}
                className="flex items-center gap-2 px-3 py-
                text-white"
            >
                {isPlaying ? (
                    <Pause className="w-4 h-4 ml-1" />
                ) : (
                    <Play className="w-4 h-4 ml-1" />
                )}
                {isPlaying ? "Пауза " : "Старт "}
            </button>

            <button

```

```

{nodes.map((node) => {
  const isCurrent = currentStep.currentNode === node.id
  const isVisited = currentStep.visited.has(node.id)
  const isAp = currentStep.ap.has(node.id);

  const fill = isCurrent
    ? "#3b82f6"
    : isAp
    ? "#e11d48"
    : isVisited
    ? "#10b981"
    : "#1e293b";
  const stroke = isCurrent
    ? "#60a5fa"
    : isAp
    ? "#f43f5e"
    : isVisited
    ? "#34d399"
    : "#334155";
  const r = isAp ? 24 : 20;

  return (
    <g key={node.id}>
      <circle
        cx={node.x}
        cy={node.y}
        r={r}
        fill={fill}
        stroke={stroke}
        strokeWidth="3"
        className="transition-all duration-300"
      />
      <text
        x={node.x}
        y={node.y}
        dy=".3em"
        textAnchor="middle"
        fill="white"
      />
    </g>
  )
})}

```

```
    </svg>
  </div>

  <div className="bg-slate-950 p-6 border-t lg:bo
    <h4 className="text-sm font-bold text-slate-4
      Статус алгоритма
    </h4>

    <div className="bg-slate-900 border border-sl
      <p className="text-white font-medium min-h-
        {currentStep.desc}
      </p>
    </div>

    <div className="space-y-4 flex-1 overflow-y-a
      <div className="mb-4">
        <h5 className="text-xs text-slate-500 fon
        <div className="flex items-center gap-2 t
          <div className="w-3 h-3 rounded-full bg
            Точка сочленения
          </div>
        <div className="flex items-center gap-2 t
          <div className="w-3 h-3 rounded-full bg
            Посещена
          </div>
        <div className="flex items-center gap-2 t
          <div className="w-3 h-3 rounded-full bg
            Текущая
          </div>
        <div className="flex items-center gap-2 t
          <div className="w-8 h-1 bg-rose-500 bor
            Мост
          </div>
        </div>
      </div>

      <div>
        <h5 className="text-xs text-slate-500 fon
          ЛОГ ДЕЙСТВИЙ:
```

```

import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord, vizString } from 'viz-runtime';
import { LiveDistTable } from './LiveDistTable';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
  iteration: number;
  checkingEdge: { u: string; v: string; w: number } | null;
  distances: { [key: string]: number };
  previous: { [key: string]: string | null };
  log: string;
  hasNegativeCycle?: boolean;
  activeCodeLine: number;
}

/** Подсказка для кнопок, когда шаги ведёт компилятор.
 *  const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
 *  и листать вручную.";
 */

export default function BellmanFordViz() {
  const [hasCycle, setHasCycle] = useState(false);
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  const runtime = useVizRuntime();
  const [isPlaying, setIsPlaying] = useState(false);

  const nodes: Node[] = [
    { id: 'S', x: 60, y: 150, name: 'База (S)' },
    { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
    { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
    { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
    { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
    { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
    { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
  ];
};

```



```

    { line: 10, text: '    return dist' },
  ];

useEffect(() => {
  const simulationSteps: Step[] = [];
  let dist: { [key: string]: number } = { S: 0, A: 999 };
  let prev: { [key: string]: string | null } = { S: null };

  simulationSteps.push({
    iteration: 0, checkingEdge: null,
    distances: { ...dist }, previous: { ...prev },
    log: '    Фура заведена. Начинаем с Базы (S). Расстояние до А: 999',
    activeCodeLine: 2,
  });

  const vCount = nodes.length;
  for (let iter = 1; iter < vCount; iter++) {
    let anyUpdate = false;
    for (const edge of edges) {
      const uDist = dist[edge.u];
      const vDist = dist[edge.v];

      let updated = false;
      if (uDist !== 999 && uDist + edge.w < vDist) {
        dist = { ...dist, [edge.v]: uDist + edge.w };
        prev = { ...prev, [edge.v]: edge.u };
        updated = true;
        anyUpdate = true;
      }

      simulationSteps.push({
        iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
        distances: { ...dist }, previous: { ...prev },
        log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}. ${
          updated ? `Релаксация успешна! dist[${edge.v}] = ${dist[edge.v]} ` : ` `
        }`,
        activeCodeLine: updated ? 6 : 5,
      });
    }
  }
}

```

```

        setCurrentStepIndex(0);
        setIsPlaying(false);
    }, [hasCycle]));

useEffect(() => {
    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length - 1) {
        timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
    } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
    }
    return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Зарядка...</div>;
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveW = vizNumber(vars?.w);
const liveDist = vizRecord(vars?.dist);
const compilerLinked = liveI !== null || liveU !== null || liveV !== null || liveW !== null || liveDist !== null;
const currentStep: Step = compilerLinked
    ? {
        ...baseStep,
        iteration: liveI ?? 0,
        checkingEdge: liveU && liveV ? { u: liveU, v: liveV },
        distances: Object.fromEntries(nodes.map((node) => [node.id, liveDist[node.id]])),
        log: `Компилятор: i=${liveI ?? "-"}, ребро ${liveU} -> ${liveV}, вес ${liveW}, расстояние ${liveDist[liveU]} -> ${liveDist[liveV]}`
    }
    : baseStep;

return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
        <div className="flex flex-wrap items-center justify-center">
            <div className="flex items-center gap-3">
                <div className="p-2.5 bg-blue-600 text-white rounded-md">Шаг {currentStepIndex + 1} из {steps.length}</div>
                <div className="p-2.5 bg-blue-600 text-white rounded-md">Итерация {currentStep.iteration}</div>
            </div>
            <div className="flex items-center gap-3">
                <div className="p-2.5 bg-blue-600 text-white rounded-md">Ребро {currentStep.checkingEdge?.u} -> {currentStep.checkingEdge?.v}</div>
                <div className="p-2.5 bg-blue-600 text-white rounded-md">Вес {currentStep.checkingEdge?.weight}</div>
            </div>
            <div className="p-2.5 bg-blue-600 text-white rounded-md">Расстояние {currentStep.distances[currentStep.checkingEdge?.u]} -> {currentStep.distances[currentStep.checkingEdge?.v]}</div>
        </div>
        <div className="p-4 mt-4">
            <div>Компилятор: i={currentStep.iteration}, ребро {currentStep.checkingEdge?.u} -> {currentStep.checkingEdge?.v}, вес {currentStep.checkingEdge?.weight}, расстояние {currentStep.distances[currentStep.checkingEdge?.u]} -> {currentStep.distances[currentStep.checkingEdge?.v]}</div>
            <div>Шаг {currentStepIndex + 1} из {steps.length}</div>
        </div>
    </div>
);

```

```

y-center min-h-[400px]">
<div className="absolute top-3 left-3 bg-blue
  ld shadow-sm">
  Итерация {currentStep.iteration} (Шаг {currentStep.step})
</div>

{currentStep.hasNegativeCycle && (
  <div className="absolute top-3 right-3 bg-red
    animate-pulse shadow-lg shadow-rose-600/50">
    ⚠ Отрицательный цикл!
  </div>
)}

<svg className="w-full h-auto max-h-[500px]" >
  <defs>
    <marker id="arrow-bf-normal" markerWidth="10" markerHeight="10"
    <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
    <marker id="arrow-bf-active" markerWidth="10" markerHeight="10"
    <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
    <marker id="arrow-bf-cycle" markerWidth="10" markerHeight="10"
    <path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
  </defs>

  {edges.map((edge, idx) => {
    const uNode = nodes.find((n) => n.id === edge.u);
    const vNode = nodes.find((n) => n.id === edge.v);
    const isChecking = currentStep.checkingEdges === idx;
    const isNeg = edge.w < 0;
    const isCyclePart = hasCycle && ((edge.u === 'D' || edge.v === 'D'));

    let strokeColor = '#475569';
    let marker = 'url(#arrow-bf-normal)';
    if (isChecking) { strokeColor = '#f59e0b'; }
    else if (currentStep.hasNegativeCycle && isNeg) { strokeColor = '#e11d48'; }
    else if (currentStep.hasNegativeCycle && isCyclePart) { strokeColor = '#e11d48'; }

    return (

```

```

<div className="w-full lg:w-1/3 bg-emerald-950/
  <h4 className="text-lg font-bold text-emerald
  <div className="space-y-2 text-sm font-mono t
    {Object.keys(adjList).map(u => {
      <div key={u} className="flex flex-wrap g
    -colors">
      <span className="font-bold text-white
      {adjList[u].map((edge, idx) => {
        const isChk = currentStep.checkingE
        return (
          <span key={idx} className={`px-2
edge.w < 0 ? 'bg-rose-900/80 text-rose-300
          {edge.v} ({edge.w})
        </span>
      )
    })}
  </div>
  )})
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10
    <div>
      <h4 className="text-lg font-bold text-rose-
      <div className="grid grid-cols-4 gap-2 text
        {nodes.map(n => {
          const d = currentStep.distances[n.id];
          const isTgt = currentStep.checkingEdge
          return (
            <div key={n.id} className={`p-2 round
            : 'border-slate-700/60 bg-slate-800/40'}`}>
              <div className="text-xs text-slate
              <div className={`font-mono font-bo
v>
      </div>

```

```

    { id: 2, label: '2', x: 450, y: 150 },
    { id: 3, label: '3', x: 75, y: 250 },
    { id: 4, label: '4', x: 225, y: 250 },
    { id: 5, label: '5', x: 375, y: 250 },
    { id: 6, label: '6', x: 525, y: 250 },
  ];

const EDGES = [
  { from: 0, to: 1 }, { from: 0, to: 2 },
  { from: 1, to: 3 }, { from: 1, to: 4 },
  { from: 2, to: 5 }, { from: 2, to: 6 }
];

const ADJ: { [key: number]: number[] } = {
  0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
};

interface Step {
  activeLine: number;
  queue: number[];
  visited: number[];
  currentNode: number | null;
  logs: string;
}

const generateBfsSteps = (): Step[] => {
  const steps: Step[] = [];
  const queue: number[] = [];
  const visited = new Set<number>();

  // Init
  steps.push({
    activeLine: 1,
    queue: [],
    visited: [],
    currentNode: null,
    logs: "Инициализация: начинаем с корня (0)."
  });
};

```

```
        logs: `Рассматриваем соседа: ${n}. Он еще не посещен.`
      });

      visited.add(n);
      queue.push(n);

      steps.push({
        activeLine: 9,
        queue: [...queue],
        visited: Array.from(visited),
        currentNode: curr,
        logs: `Помечаем ${n} как посещенную и добавляем в очередь.`
      });
    }
  }
} else {
  steps.push({
    activeLine: 6,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: curr,
    logs: `Соседей нет или все уже посещены.`
  });
}
}

steps.push({
  activeLine: 12,
  queue: [],
  visited: Array.from(visited),
  currentNode: null,
  logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();
```



```

        fill={fill}
        stroke={stroke}
        strokeWidth="3"
        className="transition-colors duration-200"
      />
      <text
        x={n.x} y={n.y}
        textAnchor="middle"
        dy=".3em"
        className="fill-white font-bold text-sm"
      >
        {n.label}
      </text>
    </g>
  );
}
</svg>
</div>

```

```

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
  <div className="bg-slate-900 border border-slate-800 p-4">
    <div className="bg-slate-800 text-slate-400 p-4">
      <span>bfs(graph, start)</span>
      <span className="text-blue-400">War {steps}</span>
    </div>
    <div className="p-4">
      {[
        { line: 1, code: 'queue = Queue()' },
        { line: 2, code: 'queue.push(start); visited.add(start)' },
        { line: 3, code: '' },
        { line: 4, code: 'while not queue.isEmpty():' },
        { line: 5, code: '    node = queue.pop()' },
        { line: 6, code: '    for neighbor in graph[node]:' },
        { line: 7, code: '        if neighbor not in visited:' },
        { line: 8, code: '            queue.push(neighbor)' },
        { line: 9, code: '            visited.add(neighbor)' },
      ]}
    </div>
  </div>

```



```
    </div>
  );
}
```

---

ФАЙЛ 34/111: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-freakazoid.jpg';
import bellmanImg from '../assets/bellman-ford-batman.jpg';
import floydImg from '../assets/floyd-network.jpg';

const imageMap: Record<string, { src: string; alt: string; caption: string; borderColor: string; captionColor: string; }> = {
  dijkstra: {
    src: dijkstraImg,
    alt: 'Декстер Дуглас и Фриказоид',
    caption: 'Декстер Дуглас и Фриказоид – волна вируса',
    borderColor: 'border-blue-500/30',
    captionColor: 'text-blue-400',
  },
  'bellman-ford': {
    src: bellmanImg,
    alt: 'Бэтмен на машине Генри Форда',
    caption: 'Бэтмен на машине Форда – n-1 ночей патрулирует',
    borderColor: 'border-rose-500/30',
    captionColor: 'text-rose-400',
  },
  floyd: {
    src: floydImg,
    alt: 'Все ко всем Флойд',
    caption: 'Все связаны со всеми!',
    borderColor: 'border-emerald-500/30',
    captionColor: 'text-emerald-400',
  },
};
```

```

if (compact) {
  return (
    <div className="flex items-center gap-2 text-xs">
      <button
        disabled={!prev}
        onClick={() => prev && onGo(prev.id)}
        title={prev ? prev.title : "Это первая тема"}
        className="flex items-center gap-1 px-2.5 py-1
          :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
      >
        <ChevronLeft className="w-4 h-4" />
        <span className="hidden sm:inline">Назад</span>
      </button>
      <span className="text-slate-500 font-mono tabular-nums">
        {index + 1} / {total}
      </span>
      <button
        disabled={!next}
        onClick={() => next && onGo(next.id)}
        title={next ? next.title : "Это последняя тема"}
        className="flex items-center gap-1 px-2.5 py-1
          :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
      >
        <span className="hidden sm:inline">Вперёд</span>
        <ChevronRight className="w-4 h-4" />
      </button>
    </div>
  );
}

return (
  <nav className="grid grid-cols-1 sm:grid-cols-2 gap-2">
    {prev ? (
      <button
        onClick={() => onGo(prev.id)}
        className="group text-left p-4 rounded-xl bg-white
          transition-all"
      >

```

```
    chapterId: string;
  }

/**
 * Блиц по прочитанной теме: 3–5 коротких вопросов с не
 *
 * Вопросы живут в `src/data/quizzes.ts` (по id главы).
 * компонент-викторина существовали, но не были подклю
 * контент был, а добраться до него было нельзя. Теперь
 * демонстрации: прочитал → посмотрел, как работает → пр
 *
 * В текстовой версии (`?lite=1`) и в PDF те же вопросы
 * с ответами и разбором – там интерактив не нужен.
 */
export const ChapterQuiz: React.FC<Props> = ({ chapterId
  const list = quizzes[chapterId];
  const [idx, setIdx] = useState(0);
  const [picked, setPicked] = useState<number | null>(n
  const [score, setScore] = useState(0);
  const [finished, setFinished] = useState(false);

  // другая тема – другой блиц с нуля
  useEffect(() => {
    setIdx(0);
    setPicked(null);
    setScore(0);
    setFinished(false);
  }, [chapterId]);

  if (!list || list.length === 0) return null;

  const q = list[idx];
  const answered = picked !== null;
  const correct = answered && picked === q.correctIndex

  const pick = (i: number) => {
    if (answered) return;
    setPicked(i);
```

```

        ? "Половина и больше верно – стоит пер
        : "Стоит вернуться к тексту страницы:
    </p>
    <p className="text-2xl font-bold text-white
      {score} / {list.length}
    </p>
    <button
      type="button"
      onClick={restart}
      className="inline-flex items-center gap-1
        -bold text-indigo-200 hover:bg-indigo-500/2
    >
      <RotateCcw className="w-3.5 h-3.5" /> Про
    </button>
  </div>
) : (
  <>
    <p className="text-sm font-semibold text-wh
    <div className="space-y-2">
      {q.options.map((option, i) => {
        const isRight = i === q.correctIndex;
        const chosen = picked === i;
        const tone = !answered
          ? "border-slate-600 bg-slate-800/70 t
          : isRight
            ? "border-emerald-500 bg-emerald-50
            : chosen
              ? "border-rose-500 bg-rose-500/15
              : "border-slate-700 bg-slate-800/
        return (
          <button
            key={i}
            type="button"
            onClick={() => pick(i)}
            disabled={answered}
            className={`w-full text-left flex i
            lors disabled:cursor-default ${tone}`}
          >

```

---

ФАЙЛ 37/111: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useCallback, useEffect, useRef, useState } from "react";
import { Check, Download, FileText, Loader2, Maximize2, Minimize2 } from "lucide-react";
import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
import { TermPopover, type HintAnchor } from "../hints/TermHint";
import { getViz } from "../vizRegistry";
import { VizChapterContext } from "../data/vizStepBus";
import { VizLiveStatus } from "../VizLiveStatus";
import { ChapterNav } from "../ChapterNav";
import { ChapterQuiz } from "../ChapterQuiz";
import { downloadChapterHtml } from "../utils/exportHtml";
import { useTicketLinks } from "../utils/ticketLinks";
```

```
interface Props {
  chapter: Chapter;
  prev?: Chapter;
  next?: Chapter;
  index: number;
  total: number;
  onGo: (id: string) => void;
  onOpenSimulator: (vizId: string) => void;
  /** Перейти во вкладку компилятора – там уже подставлены значения */
  onOpenCompiler?: () => void;
}
```

```
const isTouch = () => typeof window !== "undefined" && "ontouchstart" in window;
```

```
export const ChapterView: React.FC<Props> = ({ chapter, prev, next, index, total, onGo, onOpenSimulator, onOpenCompiler }) => {
  const contentRef = useRef<HTMLDivElement>(null);
  const [anchor, setAnchor] = useState<HintAnchor | null>(null);
  const hideTimer = useRef<number | null>(null);
```

```

    };
    const restore = () => {
      each().forEach((d) => {
        if (d.dataset.wasOpen === "0") d.open = false;
        delete d.dataset.wasOpen;
      });
    };
    window.addEventListener("beforeprint", openAll);
    window.addEventListener("afterprint", restore);
    return () => {
      window.removeEventListener("beforeprint", openAll);
      window.removeEventListener("afterprint", restore);
    };
  }, [chapter.id]);

const saveHtml = useCallback(async () => {
  setSaving("work");
  try {
    await downloadChapterHtml(chapter);
    setSaving("done");
    window.setTimeout(() => setSaving("idle"), 2500);
  } catch {
    setSaving("idle");
  }
}, [chapter]);

const open = useCallback(
  (el: HTMLElement) => {
    const termId = el.dataset.termId;
    if (!termId) return;
    cancelHide();
    setAnchor({ termId, rect: el.getBoundingClientRect() },
    [cancelHide]
  );
);

const handlePointerOver = (e: React.MouseEvent) => {
  if (isTouch()) return;

```

```

        onClick={saveHtml}
        disabled={saving === "work"}
        title="Сохранить эту тему одним автономным файлом"
        className="flex items-center gap-1.5 text-x-small text-white border-indigo-500 p-3"
        style={{width: 300, hover:text-white, hover:border-emerald-500}}
      >
        {saving === "work" ? (
          <Loader2 className="w-3.5 h-3.5 animate-spin" />
        ) : saving === "done" ? (
          <Check className="w-3.5 h-3.5 text-emerald-500" />
        ) : (
          <Download className="w-3.5 h-3.5" />
        )}
        {saving === "done" ? "Файл сохранён" : "Скачать"}
      </button>
      <button
        onClick={() => window.print()}
        title="Распечатать или сохранить эту тему в PDF"
        className="flex items-center gap-1.5 text-x-small text-white border-indigo-500 p-3"
        style={{width: 300, hover:text-white, hover:border-indigo-500}}
      >
        <Printer className="w-3.5 h-3.5" /> PDF этой темы
      </button>
      <a
        href={`?lite=1&topic=${encodeURIComponent(topic)}`}
        target="_blank"
        rel="noreferrer"
        title="Текстовая версия темы без навигации, меню и заголовка"
        className="flex items-center gap-1.5 text-x-small text-white border-slate-500 p-3"
        style={{width: 300, hover:text-white, hover:border-slate-500}}
      >
        <FileText className="w-3.5 h-3.5" /> Текстовая версия
      </a>
      <span className="text-[11px] text-slate-500">
        (ссылка на тему)
      </span>
    </div>
  )}

```

```

        рации"
        className="flex items-center gap-1.5
t-emerald-400 hover:text-white hover:border
        >
        <Terminal className="w-3.5 h-3.5" />
        </button>
    })
</div>
</div>
{viz.hint && <p className="text-xs text-sla
<VizChapterContext.Provider value={chapter.
    <VizLiveStatus demoTitle={viz.title} clas
    <viz.Component />
</VizChapterContext.Provider>
</section>
)}

{/* прочитал → посмотрел, как работает → провер
<ChapterQuiz chapterId={chapter.id} />

    <ChapterNav prev={prev} next={next} index={inde
</article>

<TermPopover
    anchor={anchor}
    onClose={() => setAnchor(null)}
    onOpenSimulator={(id) => {
        setAnchor(null);
        onOpenSimulator(id);
    }}
    onCardEnter={cancelHide}
    onCardLeave={scheduleHide}
/>
</>
);
};

```



```

{ id: "EXP", label: "EXP", sub: "2^(n^k)", color: "#0000FF",
  {кванторные формулы} } ],
{ id: "PSPACE", label: "PSPACE", sub: "полиномиальная
  с полной информацией" } ],
{ id: "NP", label: "NP", sub: "сертификат проверяется
  AT", "гамильтонов цикл", "раскраска в 3 цвета", "клика" },
{ id: "P", label: "P", sub: "решается за O(n^k)", color: "#00FF00",
  {ид", "MST", "2-SAT, двудольность" } ],
];

```

```

const NP_HARD_NOTE = "NP-трудные — всё, к чему сводится
  гамильтонов цикл. Проблема остановки NP-трудна, но в NP.

```

```

/* ----- Сведение -----

```

```

const REDUCE_STEPS = [
  { title: "Дано: задача A, для которой быстрого алгоритма
    — то есть B NP-трудная.", boxes: ["A"], active: 0 },
  { title: "Берём вход x задачи A", note: "x — конкретный
    \"x\"], active: 1 },
  { title: "Строим f(x) = y за полиномиальное время", note: "y
    шинами из разных дизъюнктов, если литералы не противоречивы",
    active: 2 },
  { title: "Решаем B на входе y", note: "Гипотетический алгоритм
    — но допустим, что он есть.", boxes: ["A", "x", "f(x)", "y"],
    active: 3 },
  { title: "Переводим ответ обратно: A(x) = B(f(x))", note: "A(x)
    ⇔ формула выполнима. Полиномиальное время сохранилось",
    active: 5 },
  { title: "Вывод: B не проще A", note: "Раз A ≤p B и A NP-полна,
    — полна.", boxes: ["A", "x", "f", "y", "B", "ans"], active: 6 },
];

```

```

const BOX_LABEL: Record<string, { t: string; c: string }> = {
  A: { t: "задача A\n(NP-полная)", c: "#f43f5e" },
  x: { t: "вход x\n|x| = n", c: "#64748b" },
  f: { t: "f: сведение\nO(poly(n))", c: "#f59e0b" },
  y: { t: "вход y\n|y| = poly(n)", c: "#64748b" },
  B: { t: "решатель B\n(гипотеза: быстрый)", c: "#6366f1" },
};

```

```

const runtime = useVizRuntime();
const vars = runtime?.variables;
const [tab, setTab] = useState<Tab>("hierarchy");
const [step, setStep] = useState(0);
const [playing, setPlaying] = useState(false);
const [samples, setSamples] = useState<Sample[]>([]);

useEffect(() => {
  if (chapterId) emitVizDemo(chapterId, tab);
}, [chapterId, tab]);

const maxStep = tab === "reduce" ? REDUCE_STEPS.length : 0;
useEffect(() => {
  setStep(0);
  setPlaying(false);
}, [tab]);

useEffect(() => {
  if (!playing) return;
  const t = window.setInterval(() => {
    setStep((s) => {
      if (s >= maxStep) {
        setPlaying(false);
        return s;
      }
      return s + 1;
    });
  }, 1400);
  return () => window.clearInterval(t);
}, [playing, maxStep]);

/* живые переменные компилятора */
const liveStep = vars ? (vizNumber(vars.step) ?? vizNumber(0)) : 0;
useEffect(() => {
  if (liveStep === null || !Number.isFinite(liveStep))
    setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);

```

```

    if (q.length < 2) return slope;
    const lx = q.map((p) => Math.log(p.n));
    const ly = q.map((p) => Math.log(p.ops));
    const ax = lx.reduce((a, b) => a + b, 0) / lx.length;
    const ay = ly.reduce((a, b) => a + b, 0) / ly.length;
    let nn = 0;
    let dd = 0;
    for (let i = 0; i < lx.length; i++) {
        nn += (lx[i] - ax) * (ly[i] - ay);
        dd += (lx[i] - ax) ** 2;
    }
    return dd === 0 ? slope : nn / dd;
  })());
  const cls =
    slopeTail > slope + 0.8 || slopeTail > 4.5
      ? { name: "экспонента / суперполином", color: "#f59e0b", hint: "нечисленности" }
      : slope <= 1.25
      ? { name: "≈ линейный,  $O(n)$ ", color: "#10b981", hint: "линейный" }
      : slope <= 1.8
      ? { name: "≈  $O(n \log n)$ ", color: "#06b6d4", hint: "логарифмический" }
      : slope <= 2.4
      ? { name: "≈ квадратичный,  $O(n^2)$ ", color: "#f59e0b", hint: "квадратичный" }
      : slope <= 3.4
      ? { name: "≈ кубический,  $O(n^3)$ ", color: "#fb9238", hint: "кубический" }
      : { name: "полином высокой степени", color: "#fb9238", hint: "полином" };
  return { slope, slopeTail, ...cls };
}, [samples]);

```

```

const reduceFrame = REDUCE_STEPS[Math.min(step, REDUCE_STEPS.length)];
const verifyIdx = Math.min(step, VERIFY_STEPS.length);
const verifyDone = verifyIdx >= VERIFY_STEPS.length;
const allOk = VERIFY_STEPS.every((s) => s.ok);

```

```

return (
  <div className="bg-slate-800/90 p-5 sm:p-6 rounded-2xl" >
    <div className="flex flex-wrap items-center justify-center" >
      <div className="flex items-center gap-3" >

```

```

    </div>
  })

  { /* ----- ИЕРАРХИЯ -----
  {tab === "hierarchy" && (
    <div className="space-y-4">
      <div className="bg-slate-950 rounded-xl p-3 b
        <svg viewBox="0 0 500 380" className="w-ful
          {CLASSES.map((c) => {
            const on = liveCls ? liveCls.toUpperCas
            return (
              <g key={c.id}>
                <ellipse cx={c.x} cy={c.y} rx={c.rx
              {on ? 3 : 1.5} strokeDasharray={on ? undefi
                <text x={c.x} y={c.y - c.ry + 18} t
                <text x={c.x} y={c.y - c.ry + 32} t
              </g>
            );
          })}
        { /* примеры задач – внутри своих овалов *
        {CLASSES.map((c) =>
          c.tasks.map((t, i) => {
            <text key={t} x={c.x} y={c.y + (c.id :
            tAnchor="middle" fill="#e2e8f0" fontSize="8
              · {t}
            </text>
          })
        })
        { /* NP-полные = пересечение */}
        <text x={250} y={352} textAnchor="middle"
        <text x={250} y={368} textAnchor="middle"
        рхии времени)</text>
      </svg>
    </div>
    <p className="text-xs text-slate-400 leading-
    <p className="text-[11px] text-slate-500">
      Подсказка: выполни в компиляторе <code clas
    </p>
  }
}

```

```

{ /* ----- ВЕРИФИКАТОР -----
{tab === "verify" && (
  <div className="space-y-3">
    <div className="bg-slate-950 rounded-xl p-4 b
      <p className="text-[10px] uppercase tracking
      <div className="flex flex-wrap gap-2 mb-3">
        {CERT.map((v, i) => {
          <span key={i} className={"font-mono tex
            ld-600/50 text-emerald-300" : "bg-slate-900
              x{i + 1} = {liveCert && typeof liveCe
            </span>
          )}}
          {liveCert && <span className="font-mono t
            le-950/40">cert из кода</span>}
        </div>
        <div className="space-y-2">
          {CLAUSES.map((cl, i) => {
            const checked = liveK !== null ? i < li
            const current = liveK !== null ? i ===
            const st = VERIFY_STEPS[i];
            return (
              <div
                key={i}
                className={"flex items-center gap-3
                  amber-950/20" : checked ? {st.ok ? "border-
                  slate-800 bg-slate-900/40"}}
              >
                <span className="text-[10px] font-b
                <span className="font-mono text-xs
              </span>
              <span className="font-mono text-[10
                " )}</span>
                <span className={"text-[11px] font-l
              0" )}>
                  {checked ? {st.ok ? "✓ истинно" :
                  </span>
                </div>

```

```

{GROWTH_LINES.map((g) => {
  const pts: string[] = [];
  for (let i = 0; i <= 20; i++) {
    const n = 1 + i * 0.8;
    const v = g.fn(n);
    const x = 44 + (Math.log(n) / Math.log(2)) * 30;
    const y = 200 - (Math.log(Math.max(1, v)) / Math.log(2)) * 100;
    pts.push(`${x.toFixed(1)},${Math.max(0, y.toFixed(1))}`);
  }
  return <polyline key={g.id} points={pts} />;
})}
{samples.map((s) => {
  const x = 44 + (Math.log(Math.max(1, s.v)) / Math.log(2)) * 30;
  const y = 200 - (Math.log(Math.max(1, s.v)) / Math.log(2)) * 100;
  return <circle key={` ${s.n}` } cx={x} cy={y} r={10} />;
})}
{GROWTH_LINES.map((g, i) => (
  <g key={g.id + "l"}>
    <rect x={330 + (i % 3) * 58} y={16 + i * 16} width={58} height={16} />
    <text x={343 + (i % 3) * 58} y={24 + i * 16} text={g.name} />
  </g>
))}
</svg>
{samples.length === 0 && (
  <p className="text-xs text-slate-500 text-center">
    Пусто. Запусти в компиляторе код с переменными ops и samples
  </p>
)}
</div>
<div className={"rounded-xl border p-3.5 " + (samples.length > 0 ? "border-gray-200" : "border-gray-400")}>
  <p className="text-sm font-bold text-white">
    Вердикт: {verdict} Похоже на {verdict.name}
  </p>
  <p className="text-xs text-slate-400 leading-relaxed">
    Описание: {description}
  </p>
</div>

```

ФАЙЛ 39/111: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect, useRef } from "react";
import { useVizStepSync, vizNumber, vizString } from ".";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "react-icons/ri";
```

```
interface CodeLine {
  line: string;
  highlight: string;
}
```

```
const CODE_LINES: CodeLine[] = [
  { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
  { line: "    visited[v] = true;", highlight: "normal" },
  { line: "    tin[v] = low[v] = timer++;", highlight: "normal" },
  { line: "    for (int to : adj[v]) {", highlight: "normal" },
  { line: "        if (to == p) continue;", highlight: "normal" },
  { line: "        if (visited[to]) {", highlight: "normal" },
  { line: "            low[v] = min(low[v], tin[to]);", highlight: "normal" },
  { line: "        } else {", highlight: "normal" },
  { line: "            dfs(to, v);", highlight: "normal" },
  { line: "            low[v] = min(low[v], low[to]);", highlight: "normal" },
  { line: "        }", highlight: "normal" },
  { line: "        if (low[to] > tin[v]) {", highlight: "normal" },
  { line: "            // ЭТО МОСТ!", highlight: "normal" },
  { line: "        }", highlight: "normal" },
  { line: "    }", highlight: "normal" },
  { line: "}", highlight: "normal" },
];
```

```
interface DfsStep {
  currentNode: number | null;
  visitedIndices: number[];
}
```

```

    },
    {
        currentNode: 1, visitedIndices: [0,1],
        tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridge: null,
        activeEdge: [0,1], backEdge: null,
        log: "Идём (A,B). В впервые. tin[B]=2, low[B]=2.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 6, visitedIndices: [0,1,6],
        tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
        activeEdge: [1,6], backEdge: null,
        log: "Из B идём в G. tin[G]=3, low[G]=3.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 1, visitedIndices: [0,1,6],
        tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
        activeEdge: null, backEdge: null,
        log: "У G нет соседей (кроме B). Возврат. low[G]=3 : ",
        activeCodeLine: [13,14,15]
    },
    {
        currentNode: 2, visitedIndices: [0,1,6,2],
        tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,6,2],
        activeEdge: [1,2], backEdge: null,
        log: "Из B идём в C. tin[C]=4, low[C]=4.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 2, visitedIndices: [0,1,6,2],
        tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, stack: [0,1,6,2],
        activeEdge: null, backEdge: [2,0],
        log: "Видим соседа A (уже посещён)! Обратное ребро ",
        activeCodeLine: [7,8,9]
    },
    {
        currentNode: 3, visitedIndices: [0,1,6,2,3],

```





```

        <Undo className="h-3.5 w-3.5" />
      </button>
      <button onClick={() => setDfsAuto(!dfsAuto)}
        className={`px-2.5 py-1 rounded text-[10px]
          dfsAuto ? "bg-rose-600 text-white" : "bg-
        }`} >
        {dfsAuto ? <><Pause className="h-3 w-3" /> : <><
      </button>
      <button onClick={() => { setDfsAuto(false); s
        disabled={dfsIdx === DFS_STEPS.length-1}
        aria-label="Шаг вперёд" title="Шаг вперёд"
        className="p-1.5 hover:bg-slate-700 rounded
        <SkipForward className="h-3.5 w-3.5" />
      </button>
      <button onClick={() => { setDfsAuto(false); s
        aria-label="Сбросить DFS" title="Сбросить D
        className="p-1.5 hover:bg-slate-700 rounded
        <RefreshCw className="h-3.5 w-3.5" />
      </button>
    </div>
  </div>

  <div className="grid grid-cols-1 md:grid-cols-5 g
    { /* Graph visualization */ }
    <div className="md:col-span-3 bg-slate-950 roun
      <div className="absolute inset-0 bg-[radial-g
        >
      <svg className="w-full h-[280px] z-10 relativ
        {GRAPH_EDGES.map(e => {
          const u = GRAPH_NODES.find(n => n.id ===
          const v = GRAPH_NODES.find(n => n.id ===
          const bridge = step.bridges.includes(e.ke
          const activeTree = step.activeEdge && (
            (step.activeEdge[0]===e.u && step.activ
            (step.activeEdge[0]===e.v && step.activ
          ));
          const activeBack = step.backEdge && (
            (step.backEdge[0]===e.u && step.backEdge

```

```

    }}}
  </svg>
  <div className="flex items-center justify-between" >
    <span>Шаг {dfsIdx+1}/{DFS_STEPS.length}</span>
    <div className="flex-1 mx-2 bg-slate-950 h-10" >
      <div className="bg-indigo-600 h-full transition-all duration-300" >
        <div>
        </div>
      </div>
    </div>
  </div>

  {/* Code panel + stack */}
  <div className="md:col-span-2 space-y-3">
    {/* Code panel */}
    <div className="bg-slate-950 rounded-lg border border-slate-800" >
      <div className="bg-slate-900 px-3 py-1.5 text-slate-400" >
        <div className="p-2 overflow-x-auto" >
          <pre className="text-[10px] font-mono leading-relaxed" >
            {CODE_LINES.map(({cl, i}) => {
              const hl = step.activeCodeLine.includes(cl)
              return <div key={i}
                className={`px-1 transition-all duration-300
                  ${hl ? "bg-indigo-600/20 text-indigo-400" : "text-slate-400"}
                `}>
                <span className="text-slate-600 mr-1" >{cl.line} | ` `}
              </div>;
            })}
          </pre>
        </div>
      </div>
    </div>

    {/* Stack */}
    <div className="bg-slate-900/50 rounded-xl border border-slate-800" >
      <div className="text-[10px] font-bold text-slate-400" >
        <span>Стек DFS</span>
        <span className="text-indigo-400">{step.s

```

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber, vizRecord,
import { LiveDistTable } from './LiveDistTable';

interface Node { id: string; x: number; y: number; name
interface Edge { u: string; v: string; w: number; }
interface Step {
  currentNode: string | null;
  checkingEdge: { u: string; v: string } | null;
  distances: { [key: string]: number };
  visited: { [key: string]: boolean };
  previous: { [key: string]: string | null };
  log: string;
  activeCodeLine: number;
}

/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
    и листать вручную.";

export default function DijkstraViz() {
  const nodes: Node[] = [
    { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
    { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
    { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
    { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
    { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
    { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
    { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
  ];

  const edges: Edge[] = [
    { u: 'A', v: 'B', w: 4 },
    { u: 'A', v: 'C', w: 2 },
    { u: 'C', v: 'B', w: 1 },
    { u: 'C', v: 'G', w: 3 },
    { u: 'C', v: 'E', w: 8 },
```

```

simulationSteps.push({
  currentNode: null, checkingEdge: null,
  distances: { ...dist }, visited: { ...visited },
  log: ' Дейкстра готов к доставке. Начинаем из ПИ',
  activeCodeLine: 2,
});

for (let iter = 0; iter < nodes.length; iter++) {
  let minD = 999;
  let u = null;
  for (const n of nodes) {
    if (!visited[n.id] && dist[n.id] < minD) {
      minD = dist[n.id];
      u = n.id;
    }
  }

  if (!u) break;

  visited[u] = true;
  simulationSteps.push({
    currentNode: u, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: ` Выбираем вершину ${u} с мин. непосещённой вершиной`,
    activeCodeLine: 5,
  });

  const neighbors = edges.filter(e => e.u === u);
  for (const edge of neighbors) {
    const v = edge.v;
    if (visited[v]) continue;

    simulationSteps.push({
      currentNode: u, checkingEdge: { u, v },
      distances: { ...dist }, visited: { ...visited },
      log: ` Смотрим соседа ${v}. Текущее расстояние: ${dist[u] + edge.w}.`,
      activeCodeLine: 6,
    });
  }
}

```

```

const baseStep = steps[currentStepIndex] || null;

if (!baseStep) return <div className="text-white">3arg
const vars = runtime?.variables;
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveDist = vizRecord(vars?.dist);
const livePrev = vizRecord(vars?.prev);
const liveDone = vizArray(vars?.done) ?? vizArray(var
const compilerLinked = liveU !== null || liveV !== nu
const currentStep: Step = compilerLinked
  ? {
    ...baseStep,
    currentNode: liveU,
    checkingEdge: liveU && liveV ? { u: liveU, v: l
    distances: Object.fromEntries(nodes.map((node) =>
    previous: Object.fromEntries(nodes.map((node) =>
    visited: Object.fromEntries(nodes.map((node) =>
    log: `Компилятор: u=${liveU ?? "-"}, v=${liveV
  }
  : baseStep;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl bor
    { /* Шапка симулятора */ }
    <div className="flex flex-wrap items-center justifi
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-indigo-600 text-white
          <span className="text-2xl"> </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white
            <p className="text-sm text-indigo-300">Полн
          </div>
        </div>
      </div>

    <div className="flex flex-wrap items-center jus
      <button

```

```

<div className="flex flex-col lg:flex-row gap-6 ml-6" style={{width: '100%'}}>
  {/* Граф */}
  <div className="w-full lg:w-2/3 bg-indigo-950/20 p-6 flex flex-direction-column align-items-center min-h-[400px]">
    <div className="absolute top-3 left-3 bg-indigo-950/20 p-6 font-bold shadow-sm">
      Шаг {currentStepIndex + 1} из {steps.length}
    </div>

    <svg className="w-full h-auto max-h-[500px]" style={{width: '100%'}}>
      <defs>
        <marker id="arrow-dh-normal" markerWidth="10" markerHeight="10" fill="#475569">
          <path d="M0,0 L6,3 L0,6 z" fill="#475569"/>
        </marker>
        <marker id="arrow-dh-active" markerWidth="10" markerHeight="10" fill="#f59e00">
          <path d="M0,0 L6,3 L0,6 z" fill="#f59e00"/>
        </marker>
        <marker id="arrow-dh-opt" markerWidth="6" markerHeight="6" fill="#10b981">
          <path d="M0,0 L6,3 L0,6 z" fill="#10b981"/>
        </marker>
      </defs>
      {/* Рисуем граф */}
      {edges.map((edge, idx) => {
        const uNode = nodes.find((n) => n.id === edge.u);
        const vNode = nodes.find((n) => n.id === edge.v);
        const isChecking =
          currentStep.checkingEdge &&
          (currentStep.checkingEdge.u === edge.u && currentStep.checkingEdge.v === edge.v);

        const isOptimal = currentStep.previous[edge.u] === edge.v;

        let marker = 'url(#arrow-dh-normal)';
        if (isChecking) marker = 'url(#arrow-dh-active)';
        else if (isOptimal) marker = 'url(#arrow-dh-opt)';

        return (
          <g key={idx}>

```

```

        strokeWidth={isCurrent ? 4 : 2}
        className="transition-all duration-300"
      />
      <text x={node.x} y={node.y + 5} fill={node.id}>
        {node.id}
      </text>
      <text x={node.x} y={node.y - 28} fill="white">
        {node.name}
      </text>
      <text x={node.x} y={node.y + 35} fill="white">
        {node.name.split(' ')[0]}
      </text>
    </g>
  );
}
}
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded">
  <p className="font-semibold text-amber-400">
    Список смежности
  </p>
  <p className="min-h-[40px] flex items-center">
    {currentStep.currentNode}
  </p>
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/50">
  <h4 className="text-lg font-bold text-emerald-400">
    Список смежности
  </h4>
  <div className="space-y-2.5 flex-1 overflow-y-auto">
    {Object.keys(adjList).map((u) => {
      const isCurrentNode = currentStep.currentNode === u;
      return (
        <div key={u} className={`p-3 rounded-lg ${
          isCurrentNode ? 'bg-emerald-900/60' : 'bg-slate-300'
        }`} >
          {u}
        </div>
      );
    })}
  </div>
</div>

```



```

        </thead>
        <tbody className="text-sm">
          {nodes.map((n) => {
            const dist = currentStep.distances[n.id]
            const prev = currentStep.previous[n.id]
            const vis = currentStep.visited[n.id]
            const isCurr = currentStep.currentNode === n.id

            return (
              <tr key={n.id} className={`border-1
                isCurr ? 'bg-indigo-950/60 font-medium' : ''}>
                <td className="py-2.5 px-3 flex items-center">
                  <span className={`w-2 h-2 rounded-full ${vis ? 'bg-indigo-500' : 'bg-gray-300'} mr-2`} />
                  {n.name}
                </td>
                <td className="py-2.5 px-3 font-medium">
                  {dist === 999 ? '∞' : dist}
                </td>
                <td className="py-2.5 px-3 text-sm">
                  {prev || '-'}
                </td>
              </tr>
            );
          })}
        </tbody>
      </table>
    </div>
  </div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 p-4">
  <div>
    <h4 className="text-lg font-bold text-slate-200">Алгоритм
    <div className="bg-slate-950/80 rounded-lg p-4">
      {codeLines.map((item) => {
        const isActive = currentStep.activeCodeLine === item.id
      })}
    </div>
  </div>

```

```
-----
const DP_CODE_LINES = [
  /* 1 */ "function fibMemo(n, memo = {}) {",
  /* 2 */ "    if (n in memo) return memo[n];",
  /* 3 */ "    if (n <= 2) return 1;",
  /* 4 */ "    memo[n] = fibMemo(n - 1, memo) + fibMemo",
  /* 5 */ "    return memo[n];",
  /* 6 */ "}"
];
```

```
export function DPVisualizer() {
  const [targetN, setTargetN] = useState<number>(5);
  const [steps, setSteps] = useState<DPStep[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const runtime = useVizRuntime();
  const [isPlaying, setIsPlaying] = useState<boolean>(f
  const [speed] = useState<number>(800);
  const [activeTab, setActiveTab] = useState<'table' |
```

```
useEffect(() => {
  const generatedSteps: DPStep[] = [];
  let stepId = 0;
  const memoMap: { [key: number]: number } = {};
```

```
function simulateFib(n: number): number {
  generatedSteps.push({
    id: stepId++,
    n,
    action: 'call',
    desc: `Вызов fibMemo(${n}). Проверяем наличие в
    codeLine: 2,
    memoState: { ...memoMap }
  });
```

```
if (n in memoMap) {
  generatedSteps.push({
    id: stepId++,
    n,
    action: 'memo hit',
```

```

        memoState: { ...memoMap }
    });

    return memoMap[n];
}

const finalRes = simulateFib(targetN);

generatedSteps.push({
  id: stepId++,
  n: targetN,
  action: 'done',
  desc: `  Расчет завершен! fibMemo(${targetN}) = $
  codeLine: 6,
  memoState: { ...memoMap }
});

setSteps(generatedSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [targetN]);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length -
    timer = setTimeout(() => {
      setCurrentStepIndex(prev => prev + 1);
    }, speed);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]);

const baseStep = steps[currentStepIndex] || null;
const vars = runtime?.variables;
const liveK = vizNumber(vars?.k) ?? vizNumber(vars?.n
const liveMemo = vizRecord(vars?.memo);
```

```

        disabled={isPlaying}
        className="bg-transparent text-white font
    >
        <option value={4}>N = 4</option>
        <option value={5}>N = 5</option>
        <option value={6}>N = 6</option>
        <option value={7}>N = 7</option>
    </select>
</div>

<div className="flex items-center bg-slate-950
    <button
        onClick={() => { setIsPlaying(false); set
        className="p-2 text-slate-400 hover:text-
        title="Сбросить"
    >
        <RotateCcw className="w-4 h-4" />
    </button>
    <button
        onClick={() => { setIsPlaying(false); set
        disabled={currentStepIndex === 0}
        className="p-2 text-slate-400 hover:text-
        title="Шаг назад"
    >
        <ChevronLeft className="w-5 h-5" />
    </button>
    <button
        onClick={() => setIsPlaying(!isPlaying)}
        className={`flex items-center gap-1.5 px-
            isPlaying
                ? 'bg-amber-500 text-slate-950 hover:
                : 'bg-emerald-600 text-white hover:bg
            }`}
    >
        {isPlaying ? <Pause className="w-4 h-4" />
        {isPlaying ? 'Пауза' : 'Пуск'}}
    </button>
    <button

```

```

    }}}
  </div>

  {currentStep && (
    <div className="mt-6 pt-4 border-t border-slate-400">
      <div className="text-sm font-medium text-slate-400">
        <span className="inline-block w-2 h-2 rounded-full bg-slate-400 mr-2"></span>
        <span>{currentStep.desc}</span>
      </div>
      <div className="text-xs font-mono bg-slate-400 p-2">
        Текущий N: <strong className="text-emerald-400">{currentStep.n}</strong>
      </div>
    </div>
  )}
</div>

{/* Tabs */}
<div className="flex items-center gap-2 border-b border-slate-400">
  <button
    onClick={() => setActiveTab('table')}
    className={`flex items-center gap-2 px-6 py-3
      ${activeTab === 'table'
        ? 'border-emerald-500 text-emerald-400 bg-slate-400'
        : 'border-transparent text-slate-400 hover:text-slate-300'}
    `}
  >
    <Eye className="w-4 h-4" />
    Степ-бай-степ состояние
  </button>
  <button
    onClick={() => setActiveTab('code')}
    className={`flex items-center gap-2 px-6 py-3
      ${activeTab === 'code'
        ? 'border-emerald-500 text-emerald-400 bg-slate-400'
        : 'border-transparent text-slate-400 hover:text-slate-300'}
    `}
  >
    <Code className="w-4 h-4" />
  </button>
</div>

```

```

        ? 'bg-emerald-600/20 border-l-4
        : 'text-slate-300 hover:bg-slate-
      }`}
    >
      <span className="w-8 text-xs text-slate-300">
        <span>{line}</span>
      </div>
    );
  }}}
</pre>
<div className="p-6 bg-slate-900/50 border-l-4 border-emerald-500/20">
  <span className="p-1.5 bg-emerald-500/20 text-white">
    <span>{currentStep?.desc}</span>
  </div>
</div>
)}
</div>

{/* Progress Bar */}
<div className="mt-8 pt-6 border-t border-slate-800">
  <div className="flex items-center justify-between">
    <span>Прогресс выполнения</span>
    <span>Шаг {currentStepIndex + 1} из {steps.length}</span>
  </div>
  <div className="h-2 bg-slate-950 rounded-full overflow-hidden">
    <div
      className="h-full bg-gradient-to-r from-emerald-500 to-emerald-600"
      style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
    ></div>
  </div>
</div>
</div>
);
}

```

```

export function EulerSimulator() {
  const chapterId = useContext(VizChapterContext);
  const runtime = useVizRuntime();
  const vars = runtime?.variables;

  const [c1Path, setC1Path] = useState<number[]>([]);
  const [c1VisitedEdges, setC1VisitedEdges] = useState<
  const [c1Msg, setC1Msg] = useState("Кликни на вершину
  const [c1Done, setC1Done] = useState(false);

  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, "walk");
  }, [chapterId]);

  const resetC1 = () => {
    setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
  };

  const clickC1 = (vId: number) => {
    if (c1Done && c1Path.length > 0) return;
    if (c1Path.length === 0) {
      setC1Path([vId]);
      setC1Msg("Старт! Кликай соседние вершины, чтобы п
      return;
    }
    const last = c1Path[c1Path.length - 1];
    const edge = C1_EDGES.find(e => (e.u === last && e.v
    if (!edge) {
      setC1Msg("Нет ребра между этими вершинами! Выбери
      return;
    }
    const key = ekey(last, vId);
    if (c1VisitedEdges.includes(key)) {
      setC1Msg("Это ребро уже пройдено! Эйлер не прощае
      setC1Done(true); return;
    }
    const nextEdges = [...c1VisitedEdges, key];
    const nextPath = [...c1Path, vId];
  }
}

```

```

// рёбра маршрута: пары соседних вершин
const keys: string[] = [];
for (let i = 0; i + 1 < path.length; i++) keys.push(
  usedFlags.forEach((f, i) => {
    if (f && C1_EDGES[i]) keys.push(ekey(C1_EDGES[i]));
  })
);
return {
  path,
  keys: Array.from(new Set(keys)),
  deg,
  odd,
  last: last !== null && last >= 0 && last < C1_NODES.length,
  step,
  complete: keys.length >= C1_EDGES.length,
};
}, [vars]));

```

```

const path = live && live.path.length ? live.path : c1Path;
const visitedKeys = live ? live.keys : c1VisitedEdges;
const deg = live?.deg ?? DEG;
const odd = live?.odd ?? ODD;
const lastId = live ? live.last : path.length ? path[path.length - 1] : 0;
const done = live ? live.complete : c1Done && visitedKeys.length >= C1_EDGES.length;
const msg = live
  ? live.complete
    ? `Компилятор прошёл все ${C1_EDGES.length} рёбра
      ${odd.length} вершин → это ${odd.length === 0 ? "цикл" : "маршрут"}
      : `Маршрут из кода: ${live.path.length ? live.path.join(" ") : ""}
        ${C1_EDGES.length} рёбра`
    : c1Msg;

```

```

const verdict =
  odd.length === 0
    ? { text: "0 нечётных → ЭЙЛЕРОВ ЦИКЛ есть (старт и финиш в одной вершине)" }
    : odd.length === 2
      ? { text: `2 нечётные (${odd.map((id) => C1_NODES[id].label).join(" и ")})
        в другой`, cls: "bg-amber-950/40 border-amber-950/40" }
      : { text: "2 нечётных → ЭЙЛЕРОВ ЦИКЛ нет" };

```



```

    }}}
    {C1_NODES.map(n => {
      const isLast = lastId === n.id;
      const visited = path.includes(n.id);
      const isOdd = odd.includes(n.id);
      return <g key={n.id} className="cursor-point
        {isLast && <circle cx={n.x} cy={n.y} r={1}
        <circle cx={n.x} cy={n.y} r={11}
          fill={isLast ? "#6366f1" : visited ? "#6366f1" : "#fff"}
          stroke={isLast ? "#fff" : visited ? "#6366f1"}
          strokeWidth={2.5}
          className="transition-all duration-200"
        <text x={n.x} y={n.y + 4} textAnchor="middle">{n.label}</text>
        <text x={n.x} y={n.y - 16} textAnchor="middle">{deg[n.id]}{isOdd ? " • нечёт" : ""}</text>
      </g>;
    }}}
  </svg>
  <div className="absolute top-2 left-2 bg-slate-800 text-white p-2">
    Ребёр: {visitedKeys.length}/{C1_EDGES.length}
  </div>
</div>

<div className={`p-3 rounded-xl border text-sm ${
  done && visitedKeys.length === C1_EDGES.length
    ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
    : c1Done && !live
    ? "bg-rose-950/40 border-rose-500/30 text-rose-50"
    : "bg-slate-900/50 border-slate-700 text-slate-50"}`}>
  {msg}
</div>

{/* Критерий Эйлера */}
<div className={`p-3 rounded-xl border text-xs font-mono ${
  verdict ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50" : "bg-rose-950/40 border-rose-500/30 text-rose-50"}`}>
  Критерий Эйлера (для связного графа): {verdict}
  <span className="block font-normal opacity-80 m-top-10">

```

```
-----
import { liveMatrix, pickLiveVar, useVizRuntime, vizNum
```

```
interface Step {
  k: number; i: number; j: number;
  matrix: (number | null)[][]; updated: boolean;
  log: string; activeCodeLine: number;
}
```

```
/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
    ва листать вручную.";
```

```
export default function FloydViz() {
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const runtime = useVizRuntime();
  // k/i/j/d приходят напрямую; номер строки произвольн
  const [isPlaying, setIsPlaying] = useState(false);
```

```
const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
const nodesSvg = [
  { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
  { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
  { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
  { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
  { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
  { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
  { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];
```

```
const initialMatrix = [
  [0, 4, 999, 2, 999, 999, 999],
  [999, 0, 3, 999, 999, 3, 999],
  [999, 999, 0, 999, 2, 999, 999],
  [999, 999, 999, 0, 4, 2, 999],
  [999, 999, 999, 999, 0, 999, 1],
  [999, 999, 999, 999, 999, 0, 5],
  [999, 1, 999, 999, 999, 999, 0]
```

```

        activeCodeLine: 3,
    });

    for (let i = 0; i < N; i++) {
        for (let j = 0; j < N; j++) {
            if (i === j || i === k || j === k) continue;

            const oldVal = dist[i][j];
            const viaVal = dist[i][k] + dist[k][j];
            if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal) {
                dist[i][j] = viaVal;
                simulationSteps.push({
                    k, i, j, matrix: dist.map(r => [...r]), update: {
                        log: `    Улучшение пути [${i}]→[${j}] через [${k}].`,
                        activeCodeLine: 7,
                    },
                });
            }
        }
    }

    simulationSteps.push({
        k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]), update: {
            log: `    Все пары отработаны. Алгоритм Флойда завершён.`,
            activeCodeLine: 8,
        },
    });

    setSteps(simulationSteps);
    setCurrentStepIndex(0);
    setIsPlaying(false);
}, []));

useEffect(() => {
    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length - 1) {
        timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
    } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
    }
}, [isPlaying, currentStepIndex, steps]);

```



```

const isK = currentStep.k === node.id;
const isI = currentStep.i === node.id;
const isJ = currentStep.j === node.id;
let fill = '#1e293b'; let stroke = '#64748f';
if (isK) { fill = '#4f46e5'; stroke = '#a6c9ec'; }
else if (isI) { fill = '#b45309'; stroke = '#d9a78f'; }
else if (isJ) { fill = '#047857'; stroke = '#a6c9ec'; }
return (
  <g key={node.id} className="transition-colors">
    <circle cx={node.x} cy={node.y} r={isJ ? 4 : 2} className="transition-colors duration-200">
      <text x={node.x} y={node.y + 5} fill="white" font-size="10px">{node.k}</text>
      <text x={node.x} y={node.y + 32} fill="white" font-size="10px">{node.i}</text>
    </circle>
  </g>
);
}}
</svg>

```

```

<div className="w-full bg-slate-950/80 p-4 rounded">
  <p className="font-semibold text-amber-400">Список смежности</p>
  <p className="min-h-[40px] flex items-center"></p>
</div>
</div>

```

```

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded">
  <h4 className="text-lg font-bold text-emerald-400">Список смежности</h4>
  <div className="space-y-2 text-sm font-mono">
    {Object.keys(adjList).map((uStr) => {
      const u = parseInt(uStr);
      const isI = currentStep.i === u;
      const isK = currentStep.k === u;
      return (
        <div key={u} className={`flex flex-wrap items-center justify-center gap-2`}>
          <div className={isI ? 'bg-amber-900/40 border-amber-400' : 'bg-indigo-900/40 border-indigo-400'}>

```

```

    )))
  </tr></thead>
  <tbody className="text-xs font-mono">
    {currentStep.matrix.map((row, i) => (
      <tr key={i} className="border-b borde
        <td className={`p-2 bg-rose-950/40
tStep.k === i ? 'text-amber-400' : ''}`>
          {labelOf(i)} {i}
        </td>
        {row.map((val, j) => {
          const isUpd = currentStep.i === i
          const isVia = currentStep.k !== -
tep.j === j});
          let bg = 'bg-slate-900/40 text-sl
          if (isUpd) bg = 'bg-emerald-900/8
          else if (isVia) bg = 'bg-indigo-9
          else if (currentStep.i === -1 &&
            bg = 'bg-indigo-950/60 text-ind
          else if (i === j) bg = 'bg-slate-
          const shown = val === null ? '.'
          return (<td key={j} className={`p
        })))
      </tr>
    })))
  </tbody>
</table>
</div>
</div>

```

```

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60
  <div>
    <h4 className="text-lg font-bold text-slate
    <div className="bg-slate-950/80 rounded-lg
      {codeLines.map(item => (
        <div key={item.line} className={`py-1.5
e === item.line ? 'bg-blue-900/80 text-ambe
        <span className="text-slate-600 w-5 s

```

```
type Mode = "dfs" | "dsu";
```

```
interface Node {
  id: number;
  label: string;
  x: number;
  y: number;
}
```

/\*\* Граф из 10 вершин и трёх компонент – ровно как в те

```
const NODES: Node[] = [
  { id: 0, label: "0", x: 90, y: 80 },
  { id: 1, label: "1", x: 175, y: 45 },
  { id: 2, label: "2", x: 205, y: 140 },
  { id: 3, label: "3", x: 105, y: 175 },
  { id: 4, label: "4", x: 330, y: 70 },
  { id: 5, label: "5", x: 420, y: 45 },
  { id: 6, label: "6", x: 380, y: 145 },
  { id: 7, label: "7", x: 130, y: 290 },
  { id: 8, label: "8", x: 230, y: 320 },
  { id: 9, label: "9", x: 330, y: 275 },
];
```

```
const EDGES: [number, number][] = [
  [0, 1], [1, 2], [2, 3], [0, 3], [1, 3], // компонента
  [4, 5], [5, 6], [4, 6],                // компонента
  [7, 8], [8, 9],                        // компонента
];
```

```
const ADJ: number[][] = NODES.map(() => []);
EDGES.forEach(([u, v]) => {
  ADJ[u].push(v);
  ADJ[v].push(u);
});
ADJ.forEach((a) => a.sort((x, y) => x - y));
```

/\*\* Палитра компонент. \*/

```

    }
    comp[next] = c;
    stack.push(next);
    push(next, stack, c, `${v} → ${next}: красим в цвет ${c}`,
        `кладём ${next} в стек.`, [v, next]);
  }
  push(null, [], c, `Компонента ${c} готова`, `Стек п
  } вершин. Ищем следующую серую вершину.`);
}
push(null, [], c, "Готово: компонент = " + c, `Внешни
  вязности.`);
return frames;
}

```

```

interface DsuFrame {
  title: string;
  note: string;
  parent: number[];
  comps: number;
  edge: [number, number] | null;
  merged: boolean;
}

```

```

function buildDsuFrames(): DsuFrame[] {
  const frames: DsuFrame[] = [];
  const parent = NODES.map((n) => n.id);
  const find = (v: number): number => (parent[v] === v
  let comps = NODES.length;
  const push = (title: string, note: string, edge: [num
    frames.push({ title, note, parent: [...parent], comp
  push("Старт: каждая вершина сама по себе", `parent[v]
    .`, null, false);
  EDGES.forEach(([u, v], i) => {
    const ru = find(u);
    const rv = find(v);
    if (ru === rv) {
      push(`Ребро ${i}: (${u}, ${v}) – корни совпали`,
        `кл, union ничего не меняет, comps остаётся ${co

```



```

/* — вкладка сообщает компилятору, какой код подстав.
useEffect(() => {
  if (chapterId) emitVizDemo(chapterId, mode);
}, [chapterId, mode]);

/* — автопроигрывание — */
useEffect(() => {
  if (!playing) return;
  timer.current = window.setInterval(() => {
    setStep((s) => {
      if (s >= maxStep) {
        setPlaying(false);
        return s;
      }
      return s + 1;
    });
  }, 900);
  return () => {
    if (timer.current) window.clearInterval(timer.cur
  };
}, [playing, maxStep]);

const goTo = useCallback((n: number) => setStep(Math.

/* — переменные компилятора, которые рисуются НАПРЯМ.
const vars = runtime?.variables;
/** comp / color / used из кода пользователя → своя р.
const liveComp = useMemo(() => {
  const arr = vars ? (vizArray(vars.comp) ?? vizArray
  if (!arr) return null;
  return arr.slice(0, NODES.length).map((v) => (typeo
}, [vars]);
/** parent / p из кода → DSU-множества по корням. */
const liveParent = useMemo(() => {
  const arr = vars ? (vizArray(vars.parent) ?? vizArr
  if (!arr || arr.length < NODES.length) return null;
  return arr.slice(0, NODES.length).map((v) => vizNum
}, [vars]);

```

```

const title = runtime && vars ? `${frame.title}` : 'Код'

const switchMode = (m: Mode) => {
  setMode(m);
  setStep(0);
  setPlaying(false);
};

return (
  <div className="bg-slate-800/90 p-5 sm:p-6 rounded-3xl">
    <div className="flex flex-wrap items-center justify-center">
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-emerald-600 text-white flex items-center justify-center">
          <Network className="w-5 h-5" />
        </div>
        <div>
          <h3 className="text-lg sm:text-xl font-bold">Шаг 1: DFS
          <p className="text-xs text-emerald-300">10 минут
        </div>
      </div>
      <div className="flex items-center bg-slate-950 p-5">
        <button
          onClick={() => switchMode("dfs")}
          className={"flex items-center gap-1.5 px-3 py-2 bg-emerald-600 text-white shadow" : "text-slate-400 hover:bg-emerald-700"}
        >
          <Layers className="w-3.5 h-3.5" /> DFS
        </button>
        <button
          onClick={() => switchMode("dsu")}
          className={"flex items-center gap-1.5 px-3 py-2 bg-emerald-600 text-white shadow" : "text-slate-400 hover:bg-emerald-700"}
        >
          <Network className="w-3.5 h-3.5" /> DSU (пёрл)
        </button>
      </div>
    </div>
  </div>
)

```

```

        <line
          key={i}
          x1={a.x} y1={a.y} x2={b.x} y2={b.y}
          stroke={active ? "#fbbf24" : sameComp ?
          strokeWidth={active ? 4 : sameComp ? 2.5}
          strokeDasharray={sameComp ? undefined :
          className="transition-all duration-300"
          opacity={sameComp || active ? 1 : 0.55}
        />
      );
    }
  }
  {NODES.map((n) => {
    const c = compOf(n.id);
    const color = c > 0 ? COLORS[(c - 1) % COLORS.length] : "#ccc";
    const isCur = cur === n.id;
    const inFrontier = frontier.includes(n.id);
    return (
      <g key={n.id} onClick={() => goTo(step)} >
        {isCur && <circle cx={n.x} cy={n.y} r={13} fill={color} />
        {inFrontier && !isCur && <circle cx={n.x} cy={n.y} r={13} fill="none" stroke={color} />
        <circle cx={n.x} cy={n.y} r={13} fill={color} />
        <text x={n.x} y={n.y + 4} textAnchor="middle">{n.label}</text>
        {c > 0 && <text x={n.x} y={n.y - 19} textAnchor="middle">c{c}</text>}
      </g>
    );
  })}
  {/* стек/очередь и DSU-таблица справа */}
  <g>
    <text x={470} y={24} textAnchor="end" fill={color} font-weight="bold">
      {mode === "dfs" ? "стек DFS" : "parent[]"}
    </text>
    {mode === "dfs"
      ? frontier.slice(-8).map((v, i) => (
        <g key={i}>

```

```
    ) : (  
      <span className="text-[11px] text-slate-500">  
        открой панель Python – переменные <code cla  
        erald-400">parent</code>, <code className="  
        comps</code> покрасят этот граф сами  
      </span>  
    )}  
  </div>  
</div>  
);  
};  
  
export default GraphComponentsViz;
```

---

ФАЙЛ 45/111: src/components/GraphTraversalViz.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import { useState, useEffect, useMemo } from 'react';  
import { useVizStepSync, vizArray, vizString } from '...'  
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';  
  
type Node = { id: string; label: string; x: number; y: number };  
type Edge = { u: string; v: string };  
  
const nodes: Node[] = [  
  { id: 'A', label: 'A', x: 200, y: 40 },  
  { id: 'B', label: 'B', x: 100, y: 120 },  
  { id: 'C', label: 'C', x: 300, y: 120 },  
  { id: 'D', label: 'D', x: 50, y: 200 },  
  { id: 'E', label: 'E', x: 150, y: 200 },  
  { id: 'F', label: 'F', x: 250, y: 200 },  
  { id: 'G', label: 'G', x: 350, y: 200 },  
];  
  
const edges: Edge[] = [  
  { u: 'A', v: 'B' },  
  { u: 'A', v: 'C' },  
  { u: 'B', v: 'D' },  
  { u: 'B', v: 'E' },  
  { u: 'C', v: 'F' },  
  { u: 'C', v: 'G' },  
  { u: 'D', v: 'E' },  
  { u: 'E', v: 'F' },  
  { u: 'F', v: 'G' },  
];
```

```

        dfs(u);
    }
}

    stack.pop();
    steps.push({ type: 'backtrack', node: v, desc:
        ueOrStack: [...stack], visitedNodes: Array.fr
    }

    dfs(start);
    return steps;
}

function generateBFSSteps(start: string) {
    const steps: Action[] = [];
    const vis = new Set<string>();
    const q: string[] = [];

    q.push(start);
    vis.add(start);
    steps.push({ type: 'visit', node: start, desc: `Нача
        ray.from(vis)] }));

    while (q.length > 0) {
        const v = q[0];
        steps.push({ type: 'process', node: v, desc: `Д
            q], visitedNodes: [...Array.from(vis)] }));
        q.shift();

        for (const u of adj[v]) {
            if (!vis.has(u)) {
                vis.add(u);
                q.push(u);
                steps.push({ type: 'visit', node: u, de
                    tack: [...q], visitedNodes: [...Array.from(
                }
            }
        }
    }
}

```

```

        if (p >= steps.length - 1) {
            setAutoPlay(false);
            return p;
        }
        return p + 1;
    });
}, 1200);
return () => clearInterval(t);
}
}, [autoPlay, steps.length]));

const step = steps[stepIdx] || steps[0];

const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
const isCurrent = (nodeId: string) => step.node === nodeId;

return (
    <div className="space-y-4">
        <div className="flex bg-slate-900 border border-slate-700 p-4">
            <button onClick={() => setMode("dfs")}
                className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
                DFS (В глубину)
            </button>
            <button onClick={() => setMode("bfs")}
                className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
                BFS (В ширину / Волна)
            </button>
        </div>

        <div className="flex flex-col md:flex-row gap-4">
            {/* SVG Graph View */}
            <div className="bg-slate-900 border border-slate-700 p-4 flex flex-grow-1 min-h-[300px]">
                <svg className="w-full h-full max-w-[1000px] max-h-[500px]">
                    {/* Edges */}
                    {edges.map((e, i) => {

```

```

<text x={n.x} y={n.y}
      className={`text-
        {n.label}
      </text>

      {/* Ripple for BFS */}
      {current && mode === 'bfs' ? (
        <circle cx={n.x} cy={n.y}
          className="stroke-emerald-500"
        >
      )}
    </g>
  )
  )}
</svg>
</div>

{/* Structure View (Stack / Queue) */}
<div className="bg-slate-900 border border-emerald-500
  r">
  <div className={`absolute -top-3 left-3 right-3
    ${mode === 'dfs' ? 'border border-emerald-500' : ''}
    ld-300 border border-emerald-500`} `>
    {mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
  </div>

  <div className="flex flex-col gap-2 p-3" `>
    {step.queueOrStack && step.queueOrStack.length > 0 ? (
      <div key={idx} className={`w-full
        ${mode === 'dfs' ? 'bg-slate-800' : 'bg-slate-900'}
        text-emerald-200`} `>
        {step.queueOrStack[idx]}
      </div>
    ) : ''}
    <div key={idx} className="flex items-center justify-between" `>
      <div>
        {step.queueOrStack[idx]}
      </div>
      <div>
        {step.queueOrStack[idx].label}
      </div>
    </div>
  </div>

```

```
                {step.desc}
            </div>
        </div>
    </div>
</div>
);
}
```

---

ФАЙЛ 46/111: src/components/HeapViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
import { useVizRuntime, vizArray, vizNumber } from '../utils';
```

```
interface Patient {
  id: string;
  name: string;
  emoji: string;
  symptom: string;
  priority: number; // 1 to 99 (Severity)
  animState: 'in' | 'idle' | 'winner' | 'out';
}
```

```
// Max-heap helpers for Patients
```

```
function siftUp(arr: Patient[]): Patient[] {
  const a = arr.map(x => ({ ...x }));
  let idx = a.length - 1;
  while (idx > 0) {
    const parent = Math.floor((idx - 1) / 2);
    if (a[parent].priority < a[idx].priority) {
      [a[parent], a[idx]] = [a[idx], a[parent]];
      idx = parent;
    } else break;
  }
}
```





```

    const nextHeap = heapInsert(heap, newPatient).map(x => x)
    // Mark specifically this new patient as 'in' in the heap
    const target = nextHeap.find(x => x.priority === priority)
    const marked = nextHeap.map(x => x.id === (target?.id || 0))

    setHeap(marked);
    addLog(` Поступил пациент: ${finalName} с тяжестью ${priority}`);

    setTimeout(() => {
      setHeap(nextHeap);
      setBusy(false);
    }, 500);
  }, [busy, heap]);

const handleRandomInsert = () => {
  const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)]
  const randomShift = Math.floor(Math.random() * 10)
  const prio = Math.max(10, Math.min(99, preset.priority + randomShift))
  insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
  e.preventDefault();
  if (!customName.trim()) return;
  insertPatient(customName, customPriority);
  setCustomName('');
};

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
  if (busy || heap.length === 0) {
    setShake(true);
    addLog('⚠ Нет пациентов для экстренной госпитализации');
    setTimeout(() => setShake(false), 400);
    return;
  }
  setBusy(true);

```

```

    setLog([' База данных скорой помощи перезагружена.
    setHealing(null);
    setTimeout(() => setHeap(INITIAL_PATIENTS.map(x =>
  });

  // --- Render binary tree lines ---
  const edges = visualHeap.flatMap((_, idx) => {
    const res: React.ReactElement[] = [];
    const p = nodePos(idx);
    const l = 2 * idx + 1;
    const r = 2 * idx + 2;
    if (l < visualHeap.length) {
      const lp = nodePos(l);
      res.push(
        <line key={`el${idx}`}
          x1={` ${p.x}%` } y1={` ${p.y + 4}%` }
          x2={` ${lp.x}%` } y2={` ${lp.y - 4}%` }
          stroke="#475569" strokeWidth="2"
        />
      );
    }
    if (r < visualHeap.length) {
      const rp = nodePos(r);
      res.push(
        <line key={`er${idx}`}
          x1={` ${p.x}%` } y1={` ${p.y + 4}%` }
          x2={` ${rp.x}%` } y2={` ${rp.y - 4}%` }
          stroke="#475569" strokeWidth="2"
        />
      );
    }
    return res;
  });

  return (
    <div className="flex flex-col gap-6">
      {/* МНЕМОНИКА / ПРАВИЛО */}
      <div className="bg-emerald-950/40 border border-e

```

```

<div className="flex flex-col items-center min-h-screen" style={styles.container}>
  <span className="text-[9px] font-mono text-slate-700">
    <input
      type="range"
      min="10"
      max="99"
      value={customPriority}
      onChange={e => setCustomPriority(Number(e.target.value))}
      className="w-20 accent-emerald-500 cursor-pointer"
    />
  </div>
  <button
    type="submit"
    disabled={busy || !customName.trim() || heap.length === 0}
    className="bg-teal-600 hover:bg-teal-500 disabled:bg-slate-700 text-white px-4 rounded-xl shadow-lg active:scale-95 transition-all cursor-pointer"
  >
    Везти
  </button>
</form>

{/* Забрать самого тяжелого */}
<button
  onClick={extractMax}
  disabled={busy || heap.length === 0}
  className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 text-white px-4 disabled:cursor-not-allowed text-white font-mono transition-all cursor-pointer flex items-center justify-center"
>
  <span> В реанимацию (EXTRACT MAX)</span>
</button>

<button
  onClick={reset}
  className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white cursor-pointer border border-slate-700"
>
  Сброс

```

```

        height: isRoot ? 60 : 52,
      }}
    >
      <span className="text-2xl leading-normal">
        <span className="text-[10px] font-mono">
          {patient.priority}%
        </span>

        {/* Tooltip */}
        <div className="absolute bottom-full"
          <div className="bg-slate-950 border
0 shadow-xl">
          <div className="font-bold text-wh
          <div className="text-emerald-400"
          </div>
        </div>
      </div>
    );
  }}}
</div>

<div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 round
  <div className="absolute top-4 left-4 text-[1
    Операционная койка
  </div>

  <div className="flex-1 flex flex-col justify-
    {healingPatient ? (
      <div className="flex flex-col items-cente
        <div className="relative">
          <span className="text-6xl block anim-l
          <span className="absolute -top-4 -right
          <span className="absolute -bottom-2 -
        </div>
      )
    )
  }
</div>

```

```
    }}}  
  </div>  
</div>  
  );  
};
```

---

ФАЙЛ 47/111: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { createContext, useContext, useEffect, use  
  
/** Общие примитивы для мини-визуализаций подсказок. */  
export const W = 260;  
export const H = 130;  
  
/**  
 * Общий множитель скорости. Кадры были слишком короткими  
 * заметить, что именно переехало, и анимация читалась  
 */  
export const SPEED = 2.1;  
  
/** Длительность переезда узлов. Должна быть заметно мен  
export const MOVE_MS = 800;  
export const EASE = "cubic-bezier(.34,.01,.2,1)";  
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;  
  
/** Наведение на карточку ставит анимацию на паузу – мо  
export const DemoPauseContext = createContext(false);  
  
export function useLoop(steps: number, ms = 900) {  
  const [step, setStep] = useState(0);  
  const paused = useContext(DemoPauseContext);  
  const period = Math.round(ms * SPEED);  
  
  useEffect(() => {
```

```

export const Node: React.FC<{
  x: number;
  y: number;
  r?: number;
  fill: string;
  label?: string | number;
  stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
  <g style={{ transition: MOVE }}>
    <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke} />
    {label !== undefined && (
      <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
        {label}
      </text>
    )}
  </g>
);

export const Edge: React.FC<{
  a: [number, number];
  b: [number, number];
  color?: string;
  width?: number;
  dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
  <line
    x1={a[0]}
    y1={a[1]}
    x2={b[0]}
    y2={b[1]}
    stroke={color}
    strokeWidth={width}
    strokeDasharray={dashed ? "4 3" : undefined}
    style={{ transition: MOVE }}
  />
);

```

```

</text>

    <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C}
    <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v}
    <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r}
  </Svg>
);
};

/** Компоненты связности: несколько «островов», которые
export const ComponentsDemo: React.FC = () => {
  const comps: [string, string][][] = [
    ["A", "B"], ["B", "C"],
    ["D", "E"],
    ["F", "G"], ["G", "H"], ["F", "H"],
  ];
  const pos: Record<string, [number, number]> = {
    A: [26, 40], B: [62, 92], C: [26, 92],
    D: [120, 40], E: [120, 92],
    F: [190, 34], G: [232, 78], H: [178, 96],
  };
  const colors = [C.active, C.warn, C.done];
  const step = useLoop(comps.length + 1, 900);
  const compOf = (v: string) => comps.findIndex((e) =>

  return (
    <Svg caption={step === 0 ? "Граф распался на острова"
      {comps.flatMap((edges, ci) =>
        edges.map(([u, v], i) => (
          <Edge key={`${ci}-${i}`} a={pos[u]} b={pos[v]}
        ))
      )}
      {Object.entries(pos).map(([k, [x, y]]) => {
        const ci = compOf(k);
        return <Node key={k} x={x} y={y} r={12} label={
      }}}
    </Svg>
  );
};

```



```

        {Object.entries(pos).map(([k, [x, y]]) => (
          <Node key={k} x={x} y={y} label={k} fill={C.idle}
        ))}
      </Svg>
    );
  };
};

```

/\*\* Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
  const raw = [1000, 5, 74000, 5, 300];
  const sorted = [5, 300, 1000, 74000];
  const step = useLoop(3, 1200);
  const boxW = 46;
  const startX = (260 - raw.length * (boxW + 4)) / 2;

  return (
    <Svg
      caption={
        step === 0
          ? "Исходные координаты — огромные и с дырами"
          : step === 1
            ? "Сортируем уникальные значения"
            : "Каждое число заменяем его номером → плотные индексы"
      }
    >
      {raw.map((v, i) => (
        <g key={i}>
          <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={20} fill={C.idle} />
          <text x={startX + i * (boxW + 4) + boxW / 2} y={35} text={v} />
        </g>
      ))}

      {step >= 1 &&
        sorted.map((v, i) => (
          <g key={v}>
            <rect x={30 + i * 52} y={62} width={46} height={20} fill={C.idle} />

```

```

        x1={f.pos[a][0]}
        y1={f.pos[a][1]}
        x2={f.pos[b][0]}
        y2={f.pos[b][1]}
        stroke={C.edge}
        strokeWidth={1.8}
        style={{ transition: MOVE }}
      />
    )))
    {Object.entries(f.pos).map(([id, [x, y]]) => (
      <g key={id} style={{ transform: `translate(${x}, ${y})` }>
        <circle r={13} fill={color(id)} stroke={C.idle} />
        <text textAnchor="middle" dominantBaseline="bottom">
          {id}
        </text>
      </g>
    )))
  </Svg>
);
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
  <SplayFrames
    captions={["р – корень, х под ним", "Один поворот: р → х"]}
    frames={[
      { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": "р", "x": "х"}] },
      { pos: { x: [130, 32], p: [182, 92] }, edges: [{"x": "х", "p": "р"}] }
    ]}
  />
);

```

```

/** Zig-Zig – х и р с одной стороны, крутим сначала вершину р. */
export const ZigZigDemo: React.FC = () => (
  <SplayFrames
    captions={["Линия g → р → х («бамбук»)", "Поворот В: р → х"]}
    frames={[
      { pos: { g: [176, 24], p: [126, 70], x: [76, 112] }, edges: [{"g": "g", "p": "р", "x": "х"}] }
    ]}
  />
);

```

```

const wave = step > 0 && step <= order.length ? order
return (
  <Svg caption={`Уровень ${Math.max(0, Math.min(step,
    {GRAPH_EDGES.map(([u, v], i) => {
      <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
    }}}
    {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
      <Node key={k} x={x} y={y} label={k}
        fill={wave.includes(k) ? C.active : visited.h
        stroke={wave.includes(k) ? "#a5b4fc" : undefi
      }}}
  </Svg>
);
};

export const DfsDemo: React.FC = () => {
  const path = ["A", "B", "D", "E", "F", "C"];
  const step = useLoop(path.length + 1, 750);
  const visited = path.slice(0, step);
  const head = visited[visited.length - 1];
  return (
    <Svg caption={head ? `Идём вглубь: ${visited.join("
      {GRAPH_EDGES.map(([u, v], i) => {
        const iu = visited.indexOf(u);
        const iv = visited.indexOf(v);
        const onPath = iu >= 0 && iv >= 0 && Math.abs(i
        return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_
      }}}
      {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k} fill={k ===
      }}}
    </Svg>
  );
};

export const ToposortDemo: React.FC = () => {
  const nodes = [
    { id: "трусы", x: 52, y: 32 }, { id: "штаны", x: 14

```

```

<Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь" : ""}>
  <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "black"} />
  <text x={130} y={54} textAnchor="middle" fontSize={16}>12 > 3 + 4 = 7</text>
  <Node x={60} y={70} r={20} fill={C.active} label="A" />
  <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="B" />
  <text x={60} y={106} textAnchor="middle" fontSize={16}>A</text>
  <text x={200} y={106} textAnchor="middle" fontSize={16}>B</text>
  <text x={130} y={22} textAnchor="middle" fontSize={16}>12 > 3 + 4 = 7</text>
</Svg>
);
};

export const BridgeDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
  ];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : ""}>
      {edges.map(([u, v], i) => {
        const isBridge = u === "C" && v === "D";
        if (isBridge && cut) return null;
        return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge && cut ? "red" : "black"} />
      })}
      {Object.entries(pos).map(([k, [x, y]]) => (
        <Node key={k} x={x} y={y} label={k} fill={cut ? C.done : C.active} />
      ))}
    </Svg>
  );
};

export const ArticulationDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {

```

```

                textAnchor="middle" fontSize={9} fontWeigh
            </g>
        );
    }
    }
    {Object.entries(pos).map(([k, [x, y]]) => (<Node
</Svg>
    );
};

export const SccDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
  };
  const edges: [string, string][] = [["A", "B"], ["B",
  const step = useLoop(2, 1400);
  const scc = ["A", "B", "C"];
  const inScc = (u: string, v: string) => step === 1 &&
  return (
    <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
      {edges.map(([u, v], i) => {
        <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
      })}
      {Object.entries(pos).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k} fill={step
      })}
    </Svg>
  );
};

```

ФАЙЛ 50/111: src/components/hints/demosStruct.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

```

```

const states = [["A", "B"], ["A", "B", "C"], ["B", "C"], ["C"]];
const step = useLoop(states.length, 800);
const items = states[step];
return (
  <Svg caption="pop_front слева, push_back справа – к"
    <defs>
      <marker id="mdArrow" markerWidth="6" markerHeight="6"
        <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
      </marker>
    </defs>
    <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХОД</text>
    <text x={206} y={44} fontSize={9} fill={C.dim}>ВХОД</text>
    {items.map((v, i) => (
      <g key={v} style={{ transition: "all .3s" }}>
        <rect x={30 + i * 54} y={56} width={46} height={20} />
        <text x={53 + i * 54} y={71} textAnchor="middle">{v}</text>
      </g>
    ))}
    <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
  </Svg>
);
};

```

```

export const DsuDemo: React.FC = () => {
  const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
  const step = useLoop(parents.length, 950);
  const p = parents[step];
  const pos: [number, number][] = [[50, 95], [105, 95], [160, 95], [215, 95]];
  const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#ff7f0e"];
  const root = (i: number): number => (p[i] === i ? i : root(p[i]));
  return (
    <Svg caption="find = «кто твой староста», union = объединение"
      {p.map((par, i) => (
        par !== i ? (
          <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}
            fill="none" stroke={rootColor[root(i)]} stroke-width={2} />
        ) : null
      ))}
    </Svg>
  );
};

```

```

    }}}}
    {TRIE_NODES.map((n) => {
      <Node key={n.id} x={n.x} y={n.y} label={n.ch}
        fill={wave.includes(n.id) ? C.active : doneId}
      >>>
      <text x={6} y={12} fontSize={9} fill={C.dim}>очер
    </Svg>
  );
};

export const SuffixLinkDemo: React.FC = () => {
  const links: [number, number][] = [[1, 0], [2, 0], [3, 0]];
  const step = useLoop(links.length + 1, 900);
  return (
    <Svg caption="Суффиксная ссылка: «куда откатиться, куда пойти»"
      {TRIE_NODES.filter((n) => n.parent !== null).map((n) => {
        const p = TRIE_NODES[n.parent as number];
        return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]}
      >>>
      {links.slice(0, step).map(([from, to], i) => {
        const a = TRIE_NODES[from];
        const b = TRIE_NODES[to];
        return (
          <path key={i} d={`M ${a.x} ${a.y} Q ${a.x + a.y} ${b.x + b.y}
            fill="none" stroke={C.warn} strokeWidth={1}
          >
        );
      >>>
      {TRIE_NODES.map((n) => {
        <Node key={n.id} x={n.x} y={n.y} label={n.ch} fill={C.dim}
      >>>
    </Svg>
  );
};

```

```

export const SegmentTreeDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const nodes = [
    { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
  ];

```

```

<text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
{pref.map((v, i) => {
  <g key={i}>
    <rect x={20 + i * 33} y={78} width={29} height={10}
      fill={show && i === l ? C.bad : show && i === l ? C.dim : C.dim}
      stroke={C.idleStroke} style={{ transition: 'stroke 0.5s'}}
    <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
  </g>
)}}
</Svg>
);
};

export const SparseTableDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const len = [1, 2, 4][step];
  const count = 8 - len + 1;
  return (
    <Svg caption={`Уровень j=${step}: готовые ответы для j`}>
      {Array.from({ length: 8 }).map((_, i) => {
        <g key={i}>
          <rect x={12 + i * 30} y={18} width={26} height={10}
            fill={C.dim} stroke={C.idleStroke} style={{ transition: 'stroke 0.5s'}}
          <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
        </g>
      })}
      {Array.from({ length: count }).map((_, i) => {
        <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 10}
          fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.idleStroke} style={{ transition: 'stroke 0.5s'}}
        </rect>
      })}
    </Svg>
  );
};

export const AmortizedDemo: React.FC = () => {
  const costs = [1, 1, 1, 8, 1, 1, 1, 1];
  const step = useLoop(costs.length + 1, 550);
  const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
  return (
    <Svg caption={`Шаг ${step}: сумма затрат`}>
      {Array.from({ length: costs.length }).map((_, i) => {
        <rect key={i} x={12 + i * 30} y={18} width={26} height={10}
          fill={C.dim} stroke={C.idleStroke} style={{ transition: 'stroke 0.5s'}}
        </rect>
        <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 10}
          fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.idleStroke} style={{ transition: 'stroke 0.5s'}}
        </rect>
        <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
      })}
      {Array.from({ length: count }).map((_, i) => {
        <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 10}
          fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.idleStroke} style={{ transition: 'stroke 0.5s'}}
        </rect>
      })}
    </Svg>
  );
};

```

```

export const AmortizedDemo: React.FC = () => {
  const costs = [1, 1, 1, 8, 1, 1, 1, 1];
  const step = useLoop(costs.length + 1, 550);
  const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
  return (
    <Svg caption={`Шаг ${step}: сумма затрат`}>
      {Array.from({ length: costs.length }).map((_, i) => {
        <rect key={i} x={12 + i * 30} y={18} width={26} height={10}
          fill={C.dim} stroke={C.idleStroke} style={{ transition: 'stroke 0.5s'}}
        </rect>
        <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 10}
          fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.idleStroke} style={{ transition: 'stroke 0.5s'}}
        </rect>
        <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
      })}
      {Array.from({ length: count }).map((_, i) => {
        <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 10}
          fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.idleStroke} style={{ transition: 'stroke 0.5s'}}
        </rect>
      })}
    </Svg>
  );
};

```



```

    <g key={idx}>
      <rect x={18 + idx * 33} y={26} width={28} height={24}
        fill={step > 0 && (idx < len || (idx > i - len)} ? C.warn : C.idle
        style={{ transition: "fill .25s" }} />
      <text x={32 + idx * 33} y={40} textAnchor="middle" font-size={10} fill="black">
        ext>
      <text x={32 + idx * 33} y={70} textAnchor="middle" font-size={10} fill="black">
        fill={idx <= i && step > 0 ? C.warn : C.idle} />
    </g>
  )}}
  <text x={4} y={40} font-size={8} fill={C.dim}>s</text>
  <text x={4} y={70} font-size={8} fill={C.dim}>π</text>
  <text x={130} y={106} textAnchor="middle" font-size={10} fill="black">
    π[i] – длина самого длинного «префикс = суффикс»
  </text>
</Svg>
);
};

export const DpDemo: React.FC = () => {
  const rows = 3, cols = 5;
  const step = useLoop(rows * cols + 1, 320);
  return (
    <Svg caption={`Заполняем таблицу подзадач: готово $
      {Array.from({ length: rows }).map((_, r) =>
        Array.from({ length: cols }).map((_, c) => {
          const idx = r * cols + c;
          const filled = idx < step;
          return (
            <g key={idx}>
              <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
                fill={idx === step - 1 ? C.active : filled ? C.warn : C.idle}
                style={{ transition: "fill .25s" }} />
              {filled && (
                <text x={57 + c * 38} y={36 + r * 32} textAnchor="middle" font-size={10} font-weight="700" fill="black">
                  {C.dp[idx]}
                </text>
              )}
            </g>
          );
        })}
    </Svg>
  );
};

```

```

    stack: StackDemo,
    queue: QueueDemo,
    dsu: DsuDemo,
    trie: TrieDemo,
    "trie-bfs": TrieBfsDemo,
    "suffix-link": SuffixLinkDemo,
    toposort: ToposortDemo,
    relax: RelaxDemo,
    "prefix-sum": PrefixSumDemo,
    "sparse-table": SparseTableDemo,
    "segment-tree": SegmentTreeDemo,
    bridge: BridgeDemo,
    articulation: ArticulationDemo,
    mst: MstDemo,
    amortized: AmortizedDemo,
    np: NpDemo,
    scc: SccDemo,
    "prefix-function": PrefixFunctionDemo,
    dp: DpDemo,
    floyd: FloydDemo,
    components: ComponentsDemo,
    graph: GraphBasicsDemo,
    "coord-compress": CoordCompressDemo,
    zig: ZigDemo,
    "zig-zig": ZigZigDemo,
    "zig-zag": ZigZagDemo,
  };

```

/\*\* Список работающих мини-демо — его проверяет `npm run`

```
export const MINI_DEMO_KINDS = Object.keys(REGISTRY);
```

```
export const MiniDemo: React.FC<{ kind: DemoKind }> = ({
```

```
  const [paused, setPaused] = useState(false);
```

```
  const Cmp = REGISTRY[kind];
```

```
  if (!Cmp) return null;
```

```
  return (
```

```
    <DemoPauseContext.Provider value={paused}>
```

```
      <div
```

```

        window.removeEventListener("keydown", onKey);
        document.body.style.overflow = "";
    };
}, [vizId, onClose]));

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
    <div className="fixed inset-0 z-[110] flex items-end"
      <div className="absolute inset-0 bg-slate-950/80"
      <div className="relative w-full sm:max-w-5xl max-h-
        ded-2xl shadow-2xl flex flex-col">
          <div className="flex items-center gap-3 px-4 py-4"
            <div className="min-w-0 flex-1">
              <h3 className="font-bold text-white text-sm">
                {viz.hint && <p className="text-[11px] text-white">
                  <VizLiveStatus demoTitle={viz.title} className="text-white">
                </div>
              <button
                onClick={onClose}
                className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
                aria-label="Закрыть симулятор"
              >
                <X className="w-5 h-5" />
              </button>
            </div>
          <div className="overflow-y-auto overscroll-contain">
            /* Тот же id главы: развёрнутая демонстрация
            <VizChapterContext.Provider value={vizId}>
              <Component />
            </VizChapterContext.Provider>
          </div>
        </div>
      </div>,
    document.body
  );

```

```

        return;
    }
    const vw = window.innerWidth;
    const vh = window.innerHeight;
    const width = Math.min(CARD_W, vw - 16);
    const height = cardRef.current?.offsetHeight ?? 300;

    let left = anchor.rect.left + anchor.rect.width / 2;
    left = Math.max(8, Math.min(left, vw - width - 8));

    const below = anchor.rect.top < height + GAP + 8;
    const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - GAP;

    setPos({ top: Math.max(8, Math.min(top, vh - height)), left }, [anchor]);

    useEffect(() => {
        if (!anchor) return;
        const onKey = (e: KeyboardEvent) => {
            if (e.key === "Escape") onClose();
        };
        window.addEventListener("keydown", onKey);
        window.addEventListener("scroll", onClose, true);
        return () => {
            window.removeEventListener("keydown", onKey);
            window.removeEventListener("scroll", onClose, true);
        };
    }, [anchor, onClose]);

    if (!anchor) return null;
    const entry = GLOSSARY_BY_ID.get(anchor.termId);
    if (!entry) return null;
    const viz = getViz(entry.simulator);
    const simulatorId = entry.simulator;

    return createPortal(
        <div
            ref={cardRef}

```

```

    })

    {viz && simulatorId && (
      <button
        onClick={() => onOpenSimulator(simulatorId)}
        className="w-full flex items-center justify-center text-sm font-medium rounded-lg py-2 transition-colors"
      >
        <Play className="w-3.5 h-3.5" /> Показать
      </button>
    )}
  </div>
</div>
</div>,
document.body
);
};

```

ФАЙЛ 54/111: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync, vizString } from '../data/vizSteps';

export function JohnsonViz() {
  const [step, setStep] = useState(0);
  const MAX_STEP = 7;
  useVizStepSync(step, setStep, MAX_STEP, (vars) => {
    const edge = `${vizString(vars.u)} ?? ""}${vizString(vars.v)} ?? ""`;
    return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
  });

  const nodes = [
    { id: 'S', x: 200, y: 150, label: 'S' },

```

```

        if (step === 4) return "stroke-amber-400 stroke-emerald-500";
        if (step > 4) return "stroke-emerald-500";
    }
    if (e.id === 'BC') {
        if (step === 5) return "stroke-amber-400 stroke-emerald-500";
        if (step > 5) return "stroke-emerald-500";
    }
    if (e.id === 'CA') {
        if (step === 6) return "stroke-amber-400 stroke-emerald-500";
        if (step > 6) return "stroke-emerald-500";
    }

    if (step >= 4) return "stroke-slate-600"; // otherwise
    return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
    if (e.u === 'S') {
        if (step === 0 || step === 7) return "";
        if (step === 1) return "url(#arrow-amber)";
        if (step === 2) return "url(#arrow-pink)";
        return "url(#arrow-slate-dim)";
    }
    if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
    if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
    if (e.id === 'BC' && step === 5) return "url(#arrow-pink)";
    if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
    if (e.id === 'CA' && step === 6) return "url(#arrow-slate)";
    if (e.id === 'CA' && step > 6) return "url(#arrow-slate-dim)";

    if (step >= 4) return "url(#arrow-slate-dim)";
    return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
    if (e.u === 'S') return e.weight;

    if (step < 4) return e.weight;

```

```

        </div>
    );
    case 1: return (
        <div>
            <b>Шаг 1: Точка отсчета.</b><br/>
            Добавляем фиктивную вершину S. Соединяем ее с остальными вершинами.
            Это наша база для расчета потенциалов.
        </div>
    );
    case 2: return (
        <div>
            <b>Шаг 2: Ищем потенциалы  $h(v)$ .</b><br/>
            Запускаем медленный, но мощный алгоритм поиска кратчайших путей из вершин S.
        </div>
    );
    case 3: return (
        <div>
            <b>Шаг 3: Магическая формула.</b><br/>
            Всем ребрам графа назначаем новый вес:  $w'_{ij} = w_{ij} + h(v_i) - h(v_j)$ .
            Это повышает или понижает вес ребра.
        </div>
    );
    case 4: return (
        <div>
            <b>Шаг 3.1: Пересчет ребра A → B.</b><br/>
            Старый вес:  $w_{AB} = 2$ .
            Вычисляем:  $2 + 0 - (-2) = 4$ .
        </div>
    );
    case 5: return (
        <div>
            <b>Шаг 3.2: Пересчет ребра B → C.</b><br/>
            Старый вес: 1.
            Вычисляем:  $1 + (-2) - (-1) = 0$ .
        </div>
    );

```

```

    uto-start-reverse">
        <path d="M 0 0 L 10 5 L 10 10" style="stroke-width:2px;stroke:blue;stroke-dasharray: 5 5;" />
    </marker>
    <marker id="arrow-amber" viewBox="0 0 10 10" style="position: absolute; top: 0; left: 0; width: 100%; height: 100%; pointer-events: none;">
    start-reverse">
        <path d="M 0 0 L 10 5 L 10 10" style="stroke-width:2px;stroke:blue;stroke-dasharray: 5 5;" />
    </marker>
    <marker id="arrow-pink" viewBox="0 0 10 10" style="position: absolute; top: 0; left: 0; width: 100%; height: 100%; pointer-events: none;">
    tart-reverse">
        <path d="M 0 0 L 10 5 L 10 10" style="stroke-width:2px;stroke:blue;stroke-dasharray: 5 5;" />
    </marker>
    <marker id="arrow-emerald" viewBox="0 0 10 10" style="position: absolute; top: 0; left: 0; width: 100%; height: 100%; pointer-events: none;">
    o-start-reverse">
        <path d="M 0 0 L 10 5 L 10 10" style="stroke-width:2px;stroke:blue;stroke-dasharray: 5 5;" />
    </marker>
</defs>

{/* Edges */}
{edges.map(({e, i}) => {
    const u = nodes.find(n => n.id === e.u);
    const v = nodes.find(n => n.id === e.v);

    const isS = e.u === 'S';
    if (isS && (step === 0 || step === 1)) return null;

    const midX = (u.x + v.x) / 2;
    const midY = (u.y + v.y) / 2;

    const edgeColorClass = getEdgeColorClass(e);
    const textHigh = isEdgeTextHigh(e);
    const textEmer = isEdgeTextEmer(e);

    return (
        <g key={i}>
            <line
                x1={u.x} y1={u.y}
                x2={v.x} y2={v.y}
                className={`stroke-${edgeColorClass}`}
                markerEnd={getMarkerEnd(e)}
            />
            {isS ? (
                <text
                    x={midX} y={midY}
                    text="S"
                    style={{fontSize: 12, color: 'black'}}
                />
            ) : (
                <text
                    x={midX} y={midY}
                    text={e.l}
                    style={{fontSize: 12, color: 'black'}}
                />
            )}
            {textHigh ? (
                <text
                    x={midX} y={midY + 10}
                    text="High"
                    style={{fontSize: 10, color: 'black'}}
                />
            ) : null}
            {textEmer ? (
                <text
                    x={midX} y={midY + 10}
                    text="Emer"
                    style={{fontSize: 10, color: 'black'}}
                />
            ) : null}
        </g>
    );
})}

```



```

fade-in zoom-in duration-500">
<rect x="-1
k-500 stroke-1" />
<text x="0"
font-bold fill-pink-400">
{pot}
</text>
</g>
})
</g>
)
}}
</svg>
</div>

```

```

<div className="w-full md:w-[320px] flex
  {/* Log Box */}
  <div className="bg-[#0f172a] p-5 ro
l justify-center leading-relaxed text-sm te
  {getDesc()}
</div>

```

```

  {/* Controls */}
  <div className="flex bg-slate-900 b
    <button onClick={() => setStep(1
      className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
      <RotateCcw strokeWidth={3}
    </button>
    <button onClick={() => setStep(1
      className="p-3 bg-slate-800
-slate-400 transition-colors" title="Вар на
      <Undo strokeWidth={3} size=
    </button>
    <button onClick={() => setStep(
      className={`flex-1 font-bol
        ${step === MAX_STEP
          ? 'bg-emerald-600/50

```

```

    label: string;
  }

interface GEdge {
  u: number;
  v: number;
}

const initialNodes: GNode[] = [
  { id: 0, x: 150, y: 100, label: "0" },
  { id: 1, x: 300, y: 100, label: "1" },
  { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
  { id: 3, x: 450, y: 160, label: "3" },
  { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 }, // Connection edge
  { u: 3, v: 4 },
  { u: 4, v: 3 },
];

interface AlgoStep {
  phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
  desc: string;
  visited: Set<number>;
  currentNode: number | null;
  order: number[];
  transposed: boolean;
  sccMap: Record<number, number>; // node -> sccId
  currentSccId: number;
  activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {

```

```

        : [...initialEdges],
    });
};

recordStep(
  "init",
  "Инициализация алгоритма Косарайю. Начинаем первый шаг",
  null,
  false,
  -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
  visited.add(u);
  recordStep(
    "dfs1",
    `DFS 1: зашли в вершину ${u}, помечаем посещение`,
    u,
    false,
    -1,
  );

  for (const to of adj[u]) {
    if (!visited.has(to)) {
      dfs1(to);
    }
  }

  order.push(u);
  recordStep(
    "dfs1",
    `DFS 1: вершина ${u} полностью обработана. Записываем в порядок`,
    u,
  );
};

```

```

        currentScc,
    );

    for (const to of adjT[u]) {
        if (!visited.has(to)) {
            dfs2(to, currentScc);
        }
    }
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
    const u = order[i];
    if (!visited.has(u)) {
        sccId++;
        recordStep(
            "dfs2",
            `DFS 2: берем следующую непосещенную из стека`,
            u,
            true,
            sccId,
        );
        dfs2(u, sccId);
    } else {
        recordStep(
            "dfs2",
            `DFS 2: вершина ${u} из стека уже покрашена в ${sccId}`,
            u,
            true,
            sccId,
        );
    }
}

recordStep(
    "finished",
    `Алгоритм успешно завершен! Найдено ${sccId} компонент`,
    null,

```

```

const getSccColor = (id: number) => {
  if (id === 1) return "fill-emerald-600 stroke-emera
  if (id === 2) return "fill-indigo-600 stroke-indigo
  return "fill-slate-800 stroke-slate-500";
};

return (
  <div className="bg-slate-900 rounded-xl border bord
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Layers className="w-5 h-5 text-emerald-400"
          Визуализатор Алгоритма Косарайю (Поиск SCC)
        </h3>

      <div className="flex items-center gap-2 bg-slate
        <button
          onClick={togglePlay}
          className="flex items-center gap-2 px-3 py-
            d-500 text-white"
        >
          {isPlaying ? (
            <Pause className="w-4 h-4 ml-1" />
          ) : (
            <Play className="w-4 h-4 ml-1" />
          )}
          {isPlaying ? "Пауза " : "Старт "}
        </button>

        <button
          onClick={reset}
          aria-label="Сбросить обход"
          title="Сбросить обход"
          className="flex items-center justify-center
            ors"
        >
          <RotateCcw className="w-4 h-4" />
        </button>
      </div>

```

```

>
  <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
</marker>
</defs>

{ /* Edges */ }
{initialEdges.map((edge, idx) => {
  const u = initialNodes.find((n) => n.id ==
  const v = initialNodes.find((n) => n.id ==

  // If transposed, draw reversed coordinates
  const x1 = step.transposed ? v.x : u.x;
  const y1 = step.transposed ? v.y : u.y;
  const x2 = step.transposed ? u.x : v.x;
  const y2 = step.transposed ? u.y : v.y;

  const isSccEdge =
    step.sccMap[edge.u] &&
    step.sccMap[edge.v] &&
    step.sccMap[edge.u] === step.sccMap[edge.v];
  let stroke = "#475569";
  let markerId = "arrow";

  if (step.transposed) {
    stroke = isSccEdge ? "#818cf8" : "#4f46
    markerId = "arrow-transposed";
  } else {
    if (
      step.currentNode === edge.u ||
      step.currentNode === edge.v
    ) {
      stroke = "#10b981";
      markerId = "arrow-active";
    }
  }
}

// Adjust slightly for bidirectional link
const isBidi =

```

```

        <circle
            cx={node.x}
            cy={node.y}
            r="20"
            className={`transition-all duration
            strokeWidth="3"
        />
        <text
            x={node.x}
            y={node.y}
            dy=".3em"
            textAnchor="middle"
            fill="white"
            fontSize="13px"
            fontWeight="bold"
            className="select-none"
        >
            {node.label}
        </text>

        {sccId && (
            <text
                x={node.x}
                y={node.y - 28}
                textAnchor="middle"
                fill="#10b981"
                fontSize="10px"
                fontWeight="bold"
            >
                SCC #{sccId}
            </text>
        )}
    </g>
);
}}
</svg>

{step.transposed && (

```

```

        ))
      })
    </div>
  </div>

  { /* List of actions log */ }
  <div>
    <span className="text-[10px] text-slate-500">
      ЛОГ ОПЕРАЦИЙ:
    </span>
    <div className="space-y-1.5 max-h-[150px]">
      {steps
        .slice(0, currentStepIdx + 1)
        .reverse()
        .slice(0, 5)
        .map((s, idx) => {
          <div
            key={idx}
            className={`text-[11px] p-1.5 rounded`
          >
            {s.desc}
          </div>
        })}
      </div>
    </div>
  </div>

  <div className="mt-4 pt-3 border-t border-slate-200">
    <span>
      Шаг {currentStepIdx + 1} из {steps.length}
    </span>
    <div className="flex gap-1.5">
      <button
        disabled={currentStepIdx === 0}
        onClick={() => setCurrentStepIdx((prev) => prev - 1)}
        className="bg-slate-900 border border-slate-800 text-white px-2 py-1 rounded"
      >
        Назад
      </button>
    </div>
  </div>

```



```

    u: number;
    v: number;
    weight: number;
    status: 'pending' | 'inspecting' | 'included' | 'rejection';
    color: string;
}

```

```

interface StepLog {
    title: string;
    description: string;
    type: 'info' | 'success' | 'warning' | 'error';
}

```

// 9 Vertices layout

```

const initialVertices: Vertex[] = [
    { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
    { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
    { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
    { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
    { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
    { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
    { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
    { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
    { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

```

// 12 Edges connecting the 9 vertices

```

const initialEdges: Edge[] = [
    { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
    { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
    { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
    { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
    { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
    { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
    { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
    { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
    { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
    { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
    { id: '5-8', u: 5, v: 8, weight: 12, status: 'pending' },
    { id: '7-8', u: 7, v: 8, weight: 13, status: 'pending' },
];

```

```

        description: 'Вершины A and B окрашены в красный цвет',
        type: 'success'
    }, ...prev]);
},
// Step 3: Inspect D-E (3)
() => {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
        title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
        description: 'Оно соединяет две другие бесцветные вершины',
        type: 'info'
    }, ...prev]));
},
// Step 4: Include D-E
() => {
    setVertices(prev => prev.map(v => v.id === 3 || v.id === 4 ? {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро D-E в остова',
        description: 'Вершины D и E окрашены в синий цвет',
        type: 'success'
    }, ...prev]));
},
// Step 5: Inspect B-C (4)
() => {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Шаг 3: Берем ребро B-C (вес 4)',
        description: 'Оно соединяет красную вершину B и синюю вершину C',
        type: 'info'
    }, ...prev]));
},
// Step 6: Include B-C
() => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро B-C в остова',

```

```

        description: 'Мы выбрасываем ребро В-Е. Иначе п
        type: 'error'
    }, ...prev]);
},
// Step 11: Inspect E-F (7)
() => {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
    setLogs(prev => [{
        title: 'Шаг 6: Берем ребро E-F (вес 7)',
        description: 'Оно соединяет фиолетовую вершину
        type: 'info'
    }, ...prev]));
},
// Step 12: Include E-F
() => {
    setVertices(prev => prev.map(v => v.id === 5 ? {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро E-F в осто́ве',
        description: 'Вершина F окрашена в фиолетовый ц
        type: 'success'
    }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Шаг 7: Берем ребро D-G (вес 8)',
        description: 'Оно соединяет фиолетовую вершину
        type: 'info'
    }, ...prev]));
},
// Step 14: Include D-G
() => {
    setVertices(prev => prev.map(v => v.id === 6 ? {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро D-G в осто́ве',

```

```

        type: 'success'
      }, ...prev]));
    },
    // Step 19: Inspect E-H (11) - Cycle!
    () => {
      setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setLogs(prev => [{
          title: 'Шаг 10: Берем ребро E-H (вес 11)',
          description: 'Оба конца E и H уже фиолетовые. С',
          type: 'warning'
        }, ...prev]));
    },
    // Step 20: Reject E-H
    () => {
      setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setLogs(prev => [{
          title: 'Отказ: Цикл обнаружен!',
          description: 'Выбрасываем ребро E-H.',
          type: 'error'
        }, ...prev]));
    },
    // Step 21: Inspect H-I (12)
    () => {
      setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setLogs(prev => [{
          title: 'Шаг 11: Берем ребро H-I (вес 12)',
          description: 'Соединяет фиолетовую H и последнюю',
          type: 'info'
        }, ...prev]));
    },
    // Step 22: Include H-I
    () => {
      setVertices(prev => prev.map(v => v.id === 8 ? {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setLogs(prev => [{
          title: 'Успех: MST построено Краскалом!',
          description: 'Все 9 вершин окрашены в единый фи
            + 4 + 5 + 7 + 8 + 10 + 12 = 51.',

```

```

    }, ...prev]));
},
// Step 4: Pop A-D (5)
() => {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
        title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
        description: 'Плесень расширяется в сторону верш
        type: 'info'
    }, ...prev]));
},
// Step 5: Include A-D, add A's edges to heap
() => {
    setVertices(prev => prev.map(v => v.id === 0 ? {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
        eap' } : e));
    setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
    setLogs(prev => [{
        title: 'Успех: Плесень поглотила вершину A',
        description: 'Новое ребро A-B(2) добавлено в куч
        type: 'success'
    }, ...prev]));
},
// Step 6: Pop A-B (2)
() => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
        title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
        description: 'Плесень стремительно бежит к верши
        type: 'info'
    }, ...prev]));
},
// Step 7: Include A-B, add B's edges
() => {
    setVertices(prev => prev.map(v => v.id === 1 ? {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
        eap' } : e));
    setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (

```

```

    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'E
    setLogs(prev => [{
      title: 'Отказ: Игнорируем внутреннее ребро',
      description: 'Ребро B-E отброшено. Плесень не р
      type: 'error'
    }, ...prev]));
  },
// Step 12: Pop E-F (7)
() => {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {
  setLogs(prev => [{
    title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
    description: 'Плесень из центра Токио (E) тянет
    type: 'info'
  }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
  setVertices(prev => prev.map(v => v.id === 5 ? {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {
    eap' } : e));
  setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
  setLogs(prev => [{
    title: 'Успех: Вершина F захвачена',
    description: 'Ребро F-I(13) добавлено в кучу. P
    type: 'success'
  }, ...prev]));
},
// Step 14: Pop D-G (8)
() => {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
  setLogs(prev => [{
    title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
    description: 'Плесень расширяется вниз к G.',
    type: 'info'
  }, ...prev]));
},

```

```

    }, ...prev]));
},
// Step 19: Include G-H, add H's edges
() => {
    setVertices(prev => prev.map(v => v.id === 7 ? {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
        eap' } : e));
    setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I
    setLogs(prev => [{
        title: 'Успех: Вершина H захвачена',
        description: 'В кучу добавлено H-I(12).',
        type: 'success'
    }, ...prev]));
},
// Step 20: Pop E-H (11) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setLogs(prev => [{
        title: 'Шаг 11: Куча выдает E-H (вес 11)',
        description: 'Обе вершины E и H уже в плесени.'
        type: 'warning'
    }, ...prev]));
},
// Step 21: Reject E-H
() => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setHeapQueue(['H-I (12)', 'F-I (13)']));
    setLogs(prev => [{
        title: 'Отказ: Игнорируем E-H',
        description: 'Выбрасываем из кучи.',
        type: 'error'
    }, ...prev]));
},
// Step 22: Pop H-I (12)
() => {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
        title: 'Шаг 12: Куча выдает H-I (вес 12)',

```

```

        A выбирает A-B(2), D выбирает D-E(3), а G вы
        !',
        type: 'info'
    }, ...prev]));
},
// Step 2: Concurrently merge into Kolkhozes (Component 1) => {
    // Concurrently build chosen roads. Merge into:
    // Component 1: {A, B, C} (colored red)
    // Component 2: {D, E, F, G, H, I} (colored gold/
    setVertices(prev => prev.map(v =>
        [0, 1, 2].includes(v.id) ? { ...v, color: 'red'
    }));
    setEdges(prev => prev.map(e => {
        if (['0-1', '1-2'].includes(e.id)) {
            return { ...e, status: 'included', color: 'red'
        }
        if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
            return { ...e, status: 'included', color: 'blue'
        }
        return e;
    }));
    setLogs(prev => [{
        title: 'Слияние: Деревни объединились в колхозы
        description: 'Все выбранные дороги построены ра
        оз {D, E, F, G, H, I}.',
        type: 'success'
    }, ...prev]));
},
// Step 3: Five-Year Plan 2 (Inspecting bridge between
() => {
    // Now both kolkhozes look at all outgoing roads
    // Outgoing roads between {A, B, C} and {D, E, F,
    // A-D (5), B-E (6), C-F (9).
    // Cheapest is A-D (5).
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
        title: 'Вторая пятилетка: Колхозы выбирают мост

```



```

    }
};

const introLog = {algo: 'kruskal' | 'prim' | 'boruvka'
  if (algo === 'kruskal') {
    return [{
      title: 'Введение в раскраску (Краскал)',
      description: 'Представь, что все 9 вершин графа
      type: 'info',
    }]];
  }
  if (algo === 'prim') {
    return [{
      title: 'Введение в Плесень (Прима)',
      description: 'Логика «Плесени»: выбираем старто
      ь (Heap) и захватываем соседей!',
      type: 'info',
    }]];
  }
  return [{
    title: 'Введение в Коллективизацию (Борувка)',
    description: 'Логика «Борувки»: Каждая из 9 дерев
    ы объединиться в колхоз.',
    type: 'info',
  }]];
};

const handleReset = {algo: 'kruskal' | 'prim' | 'boru
  setActiveAlgo(algo);
  setVertices(initialVertices);
  setEdges(initialEdges);
  setStepIndex(0);
  setHeapQueue([]);
  setLogs(introLog(algo));
};

/* _____ связь с Python-компилятором _____
const chapterId = useContext(VizChapterContext);

```

```

    return c ? c.charCodeAt(0) - 65 : null;
  };

  /** Разбор одного ребра из кода: (2, 'A', 'B') / [2, 'A', 'B']
  const parseEdge = (item: unknown): { u: number; v: number; w: string } | null {
    if (item === null || item === undefined) return null;
    if (typeof item === 'string') {
      const nums = item.match(/-?\d+(?:\.\d+)?/g);
      const letters = item.match(/[A-Za-z]/g);
      let u: number | null = null;
      let v: number | null = null;
      if (letters && letters.length >= 2) { u = idOf(letters[0]); v = idOf(letters[1]); }
      else if (nums && nums.length >= 2) { u = idOf(nums[0]); v = idOf(nums[1]); }
      if (u === null || v === null) return null;
      return { u, v, w: item.length > 2 ? item : null };
    }
    if (Array.isArray(item)) {
      const letters = item.filter((x) => typeof x === 'string');
      const nums = item.filter((x) => typeof x === 'number');
      if (letters.length >= 2) {
        const u = idOf(letters[0]); const v = idOf(letters[1]);
        if (u === null || v === null) return null;
        return { u, v, w: nums.length > 0 ? nums[0] : null };
      }
      if (nums.length >= 2) {
        const u = idOf(nums[nums.length - 2]); const v = idOf(nums[nums.length - 1]);
        if (u === null || v === null) return null;
        return { u, v, w: nums.length > 2 ? nums[0] : null };
      }
    }
    return null;
  };

```

```

const PALETTE = ['red', 'blue', 'purple', 'mold'];
/** Цвет по корне компоненты: одинаковый корень → одинаковый цвет
const colorOfComp = (root: string | undefined): string {
  if (!root) return 'none';
  let h = 0;

```

```

rawComp.forEach((label, id) => {
  if (parentIdx.size === 0) {
    compOfVertex.set(id, label);
    return;
  }
  let cur = id;
  const seen = new Set<number>([id]);
  for (let hop = 0; hop < 16; hop++) {
    const nxt = parentIdx.get(cur);
    if (nxt === undefined || nxt === cur || seen.has(nxt)) {
      seen.add(nxt);
      cur = nxt;
    }
    compOfVertex.set(id, String.fromCharCode(65 + cur));
  });
const takenRaw = vizArray(vars.mst) ?? vizArray(vars.mst);
const takenEdges = (takenRaw ?? []).map(parseEdge);
const takenVertices = (vizArray(vars.taken) ?? vizArray(vars.taken))
  .map((x) => idOf(typeof x === 'string' ? x : vizNumber(x)))
  .filter((x): x is number => x !== null);
const curU = idOf(vizString(vars.u) ?? vizNumber(vars.u));
const curV = idOf(vizString(vars.v) ?? vizNumber(vars.v));
const curW = vizNumber(vars.w);
const heapRaw = vizArray(vars.heap) ?? vizArray(vars.heap);
const heapEdges = (heapRaw ?? []).map(parseEdge);
const cheapestRaw = vizRecord(vars.cheapest) ?? vizRecord(vars.cheapest);
const cheapestEdges = cheapestRaw ? Object.values(cheapestRaw)
  .filter(e => e !== null) : [];
const total = vizNumber(vars.total) ?? vizNumber(vars.total);
const any = compOfVertex.size > 0 || takenEdges.length > 0 ||
  (curU !== null && curV !== null) || heapEdges.length > 0;
return any ? { compOfVertex, takenEdges, takenVertices, total,
  // eslint-disable-next-line react-hooks/exhaustive-deps
  edges, [vars] };

```

```

/** Рёбра, которые рисуем: код пользователя важнее кода библиотеки */
const effEdges: Edge[] = useMemo(() => {
  if (!live) return edges;

```

```

    }
    return 'bg-slate-700 border-slate-500 text-slate-400';
  }
};

```

```

const getEdgeStroke = (status: string, color: string) => {
  if (status === 'inspecting') return '#f59e0b';
  if (status === 'rejected') return '#ef4444';
  if (status === 'in_heap') return '#0284c7';
  if (status === 'included') {
    if (color === 'red') return '#ef4444';
    if (color === 'blue') return '#3b82f6';
    if (color === 'purple') return '#a855f7';
    if (color === 'mold') return '#10b981';
  }
  return '#475569';
};

```

```

return (
  <div className="space-y-12">

    {/* 3-in-1 MST Simulator Box */}
    <div className="bg-slate-900/90 border border-slate-800 p-6">
      <div className="flex flex-col lg:flex-row justify-between items-center">
        <div>
          <div className="flex items-center gap-2 mb-2">
            <span className="bg-gradient-to-r from-indigo-500 to-purple-500 text-white font-weight-bold uppercase tracking-wider">
              Сунер-Интерактив 3 в 1
            </span>
            <h3 className="text-2xl font-bold text-white">
              Алгоритмы Краскала
            </h3>
          </div>
          <p className="text-slate-400 text-sm">
            Наблюдай за работой Краскала (Раскраска),
          </p>
        </div>

        {/* Algorithm Tabs */}
      </div>
    </div>
  </div>
);

```

```

        <Users className="w-3.5 h-3.5" />
        <span>Борувка (Билет 20)</span>
      </button>
    </div>

    <button
      onClick={() => handleReset(activeAlgo)}
      className="flex items-center justify-cent
-200 font-semibold text-xs rounded-xl borde
    >
      <RotateCcw className="w-3.5 h-3.5" />
      <span>Сбросить</span>
    </button>

    <button
      onClick={handleNextStep}
      disabled={stepIndex >= currentSteps.length}
      className={
        "flex items-center justify-center gap-2
nitial cursor-pointer " +
        (stepIndex >= currentSteps.length
          ? "bg-slate-800 text-slate-500 border
            : activeAlgo === 'kruskal'
          ? "bg-indigo-600 hover:bg-indigo-500
            : activeAlgo === 'prim'
          ? "bg-emerald-600 hover:bg-emerald-500
            : "bg-rose-600 hover:bg-rose-500 text
        )
      }
    >
      <SkipForward className="w-4 h-4" />
      <span>{stepIndex === 0 ? 'Начать' : stepI
    </button>
  </div>
</div>

/* Что компилятор сообщил графу прямо сейчас */
<div className="mb-6 flex flex-wrap items-cente
  <span className="flex items-center gap-1.5 te

```

```

        </span>
    })
</>
) : (
    <span className="text-[11px] text-slate-500"
      открой панель Python и запусти код вкладки
      ка'}»: переменные
      {' '}<code className="font-mono text-purple-300"
e className="font-mono text-purple-300">u</code>
mono text-purple-300">w</code> перекрасят э
    </span>
  })
</div>

<div className="grid grid-cols-1 lg:grid-cols-3"
  {/* Graph Canvas */}
  <div className="lg:col-span-2 bg-slate-950 rounded-3xl
    active min-h-[480px] overflow-hidden">

    {/* Legend */}
    <div className="absolute top-4 left-4 bg-slate-950
      -center gap-3 text-xs">
      <div className="flex items-center gap-1.5">
        <div className="w-3 h-3 rounded-full bg-slate-950"
        </div>
        {activeAlgo === 'kruskal' ? (
          <div className="flex items-center gap-1.5">
            <div className="w-3 h-3 rounded-full bg-slate-950"
            </div>
            <div className="flex items-center gap-1.5">
              <div className="w-3 h-3 rounded-full bg-slate-950"
              </div>
              <div className="flex items-center gap-1.5">
                <div className="w-3 h-3 rounded-full bg-slate-950"
                </div>
              </div>
            </div>
          </div>
        ) : activeAlgo === 'prim' ? (

```

```

        return (
          <g key={edge.id}>
            <line
              x1={u.x + "%"}
              y1={u.y + "%"}
              x2={v.x + "%"}
              y2={v.y + "%"}
              stroke={strokeColor}
              strokeWidth={isInspecting ? 4.5 : 1}
              strokeDasharray={isRejected ? '4,4' : ''}
              className={"transition-all duration-300"}
            />
            <text
              x={({u.x + v.x} / 2 + "%")}
              y={({u.y + v.y} / 2 + "%")}
              fill={isInspecting ? '#f59e0b' : '#94a3b8'}
              : '#94a3b8'}
              fontSize="4.5"
              fontFamily="monospace"
              fontWeight="bold"
              textAnchor="middle"
              dominantBaseline="middle"
              className="select-none filter drop-shadow-sm"
              dy="-1.5"
            >
              {edge.weight}
            </text>
          </g>
        );
      }
    }
  </svg>

```

```

  { /* HTML Overlay for Vertices */ }
  <div className="absolute inset-0 w-full h-full" >
    {effVertices.map(vertex => (
      <div
        key={vertex.id}

```

```

    ✂ Приоритетная очередь (Min-Heap):
  </p>
  <div className="flex flex-wrap gap-1.5">
    {heapQueue.length === 0 ? (
      <span className="text-xs text-slate-500">
        Очередь пуста
      </span>
    ) : (
      heapQueue.map((item, idx) => (
        <span key={idx} className={"px-2 py-1 shadow-md animate-pulse" : "bg-slate-800 text-white"}>
          {item}
        </span>
      ))
    )}
  </div>
</div>
)}

<div className="flex-1 p-4 overflow-y-auto">
  {logs.map((log, idx) => (
    <div
      key={idx}
      className={
        "p-4 rounded-xl border transition-all" +
        (log.type === 'success'
          ? 'bg-emerald-950/40 border-emerald-500'
          : log.type === 'warning'
            ? 'bg-amber-950/40 border-amber-500'
            : log.type === 'error'
              ? 'bg-rose-950/40 border-rose-500'
              : 'bg-slate-900/60 border-slate-700')
      }
    >
      <div className="flex items-center gap-2">
        {log.type === 'success' && <CheckCircle className="text-emerald-500"/>
        {log.type === 'warning' && <HelpCircle className="text-amber-500"/>
        {log.type === 'error' && <XCircle className="text-rose-500"/>
        {log.type === 'info' && <Play className="text-slate-500"/>
      </div>
      <h5 className="font-bold text-sm text-white">{log.message}</h5>
    >
  )}
</div>

```





```

    </div>
    <div className="bg-slate-950 p-4 rounded-1">
      <p className="text-slate-400 text-xs font-mono">
        ложении:</p>
      <p className="text-xs text-slate-300 font-mono">
        не чистые вершины через самое дешевое доступ
      </p>
    </div>
  </div>
  <div className="lg:col-span-2">
    <div className="bg-slate-950 p-4 rounded-1">
      <p className="text-slate-400 text-xs font-mono">
        <Code className="w-3.5 h-3.5 text-emerald-400">
          <pre>
80 flex-1">
{`import heapq

def prim(graph, start):
    visited = [False] * len(graph)
    heap = [(0, start, -1)] # (weight, node, parent)
    mst = []

    while heap:
        w, u, prev = heapq.heappop(heap)
        if visited[u]: continue
        visited[u] = True
        if prev != -1: mst.append((prev, u, w))

        for next_w, v in graph[u]:
            if not visited[v]:
                heapq.heappush(heap, (next_w, v, u))
    return mst`}
    </pre>
    </div>
  </div>
</div>
}}

```

```
        for u, v, w in edges:
            ru, rv = find(u), find(v)
            if ru != rv:
                if cheapest[ru] == -1 or cheapest[ru][2] < w:
                    cheapest[ru] = rv, w
                if cheapest[rv] == -1 or cheapest[rv][2] < w:
                    cheapest[rv] = ru, w
# Объединяем все колхозы по выбранным мостам
for bridge in cheapest:
    if bridge != -1 and union(bridge[0], bridge[1]):
        mst.append(bridge)`}
</pre>
</div>
</div>
    })
</div>
};
};
```

---

ФАЙЛ 57/111: src/components/LiteGuide.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useMemo } from "react";
import { chapters, chapterTopics } from "../data/content";
import { getViz } from "../vizRegistry";
import { PAGE_SYNC } from "../data/vizSync";
import { linkifyTicketsHtml } from "../utils/ticketLink";
import { quizzes } from "../data/quizzes";
```

```
/**
```

```
 * «Лёгкая» текстовая версия пособия.
```

```
 *
```

```
 * Открывается по адресу `?lite=1` (весь список тем) или
```

```
 * (одна тема). Здесь НЕТ навигации, сайдбара, интерактивных
```

```

@media print {
  .lite-wrap { max-width: none; padding: 0; }
  h2 { page-break-before: always; }
  h2:first-of-type { page-break-before: avoid; }
  pre, table, details { page-break-inside: avoid; }
}
`;

/** Раскрываем все спойлеры и вычищаем служебную разметку
function toPlainTextHtml(html: string): string {
  return html
    .replace(/<details(?:[>]*\bopen\b)/g, "<details open")
    // кнопки/иконки внутри текста не нужны в текстовой версии
    .replace(/<button[\s\S]*?<\/button>/gi, "")
    .replace(/<svg[\s\S]*?<\/svg>/gi, "")
    // интерактивные подсказки терминов оставляем как есть
    .replace(/\\sdata-term-id="[^"]*" /g, "")
    .replace(/class="term-hint"[^"]*" /g, "")
    // ссылки на другие билеты должны вести в текстовую версию
    .replace(/href="\\?topic=/g, 'href="?lite=1&topic=')
    .replace(/class="ticket-link"/g, "");
}

interface Props {
  /** id одной темы; undefined – все темы подряд. */
  topicId?: string;
}

export const LiteGuide: React.FC<Props> = ({ topicId }) => {
  const list = useMemo(() => {
    if (!topicId) return chapters;
    const one = chapters.filter((c) => c.id === topicId);
    return one.length ? one : chapters;
  }, [topicId]);

  return (
    <div className="lite-wrap">
      <style dangerouslySetInnerHTML={{ __html: LIGHT_GUIDE_HTML }} />
    </div>
  );
}

```

```

    __html: linkifyTicketsHtml(toPlainTextH
  }}
/>

{viz && (
  <div className="lite-viz">
    <h4>Визуализация: {viz.title}</h4>
    {viz.hint && <p>{viz.hint}</p>}
    {syncKeys.map((k) => {
      const sync = PAGE_SYNC[k];
      const demo = k.includes("#") ? k.split(
      return {
        <div key={k}>
          <p>
            <b>{demo ? `Вкладка «${demo}»`
            {sync.vizTitle ? ` – ${sync.viz`
            {sync.stepNote ? <> Шаг демонстр
          </p>
          {sync.variables.length > 0 && (
            <ul>
              {sync.variables.map((v) => {
                <li key={v.name}>
                  <code>{v.name}</code> – {
                  {v.range ? ` (${v.range})`
                </li>
              }}}
            </ul>
          })
        </div>
      });
    })}
    <p className="lite-meta" style={{ margin
      В интерактивной версии эта демонстрац
      переменных перерисовывают её напрямую
    </p>
  </div>
)}
```

```
interface LiveDistTableProps {
  /** dist из кода пользователя: имя вершины → расстояние */
  dist: Record<string, DebugValue> | null;
  /** prev/p – предок на кратчайшем пути (необязательно) */
  prev?: Record<string, DebugValue> | null;
  /** done/visited – зафиксированные вершины (необязательно) */
  done?: DebugValue[] | null;
  /** Идентификаторы вершин, нарисованных на схеме демо */
  demoIds: string[];
  /** Подпись таблицы (по умолчанию «Ваши расстояния»). */
  title?: string;
}

const fmt = (value: DebugValue | undefined): string => {
  if (value === undefined || value === null) return "–";
  if (typeof value === "number") {
    if (value === Number.POSITIVE_INFINITY) return "∞";
    if (value === Number.NEGATIVE_INFINITY) return "−∞";
    return String(value);
  }
  if (typeof value === "string") {
    const t = value.trim().toLowerCase();
    if (t === "inf" || t === "infinity") return "∞";
    if (t === "-inf" || t === "-infinity") return "−∞";
    return value;
  }
  return String(value);
};

/**
 * Таблица «ваших» расстояний, когда имена вершин в коде
 *
 * Схема демонстрации нарисована на фиксированном наборе
 * S...F у Форда–Беллмана). Если в коде пользователя верши
 * (0...6, v1...v9), картинка физически не может их показат
 * просто пропадали, и казалось, что визуализация живёт
 * выводятся рядом, а подсказка объясняет, как связать
```

```

        : "border-slate-700 bg-slate-900/70 to
      }` }
      title={fixed ? "вершина зафиксирована (до
    >
    <b className={fixed ? "text-emerald-300"
    <span className="text-slate-500">=</span>
    <span>{fmt(dist[key])}</span>
    {parent && parent !== "-" ? <span classNa
    {fixed ? <span className="text-emerald-40
    </span>
  });
  }}}
  {keys.length > shown.length ? {
    <span className="inline-flex items-center rou
    e-400">
      +{keys.length - shown.length} ещё
    </span>
  } : null}
</div>
</div>
);
}

```

ФАЙЛ 59/111: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import dijkstraImg from '../assets/dijkstra-freakazoid.
import bellmanImg from '../assets/bellman-ford-batman.jp
import floydImg from '../assets/floyd-network.jpg';

```

```

const cards = [
  {
    img: dijkstraImg,
    alt: 'Декстер Дуглас и Фриказоид (Дейкстра)',
    title: '⚡ Фриказоид и Декстер Дуглас',
  },

```

```
    return (
      <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
        {cards.map((c, i) => (
          <div key={i} className={`bg-slate-800 rounded-2xl p-4">
            <div className="h-48 overflow-hidden">
              <img src={c.img} alt={c.alt}
                className="w-full h-full object-cover group-hover:scale-105 transition-transform duration-300"/>
            </div>
            <div>
              <h3 className="text-lg font-bold mb-1 ${c.isBook ? 'text-white' : 'text-slate-400'}">{c.title}</h3>
              <p className="text-slate-400 text-sm">{c.desc} <span className={`font-bold ${c.isBook ? 'text-white' : 'text-slate-400'}`}>{c.status}</span>
            </p>
            <div className={`mt-3 text-xs font-mono px-2 ${c.isBook ? 'text-white' : 'text-slate-400'}`}>{c.url}</div>
          </div>
        ))}
      </div>
    );
  }
}
```

---

ФАЙЛ 60/111: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Заголовок текущей темы — показываем в кнопке, что это за тема */
  currentTitle: string;
  index: number;
  total: number;
}
```



```

        </span>
      </button>
    </div>

    {open && (
      <div className="lg:hidden fixed inset-0 z-[90]">
        <div className="absolute inset-0 bg-slate-950">
          <div className="absolute inset-y-0 left-0 w-[100px] flex flex-direction-column justify-center items-center text-white font-medium">
            <div className="flex items-center justify-between p-2">
              <span className="font-bold text-white text-sm">Закреть</span>
              <button
                onClick={() => setOpen(false)}
                className="p-2 -mr-2 text-slate-400 hover:text-white"
                aria-label="Закреть содержание"
              >
                <X className="w-5 h-5" />
              </button>
            </div>
            <div className="flex-1 min-h-0">
              <Sidebar
                selectedChapterId={selectedChapterId}
                setSelectedChapterId={setSelectedChapterId}
                onNavigate={() => setOpen(false)}
              />
            </div>
          </div>
        </div>
      </div>
    )}
  </>
);
};

```

```
<div className="min-w-0 flex-1">
  <h1 className="text-lg lg:text-xl font-extrabold">
    Универсальное пособие
  </h1>
  <p className="text-xs text-indigo-400 font-mono">
    Подготовка к экзаменам: Алгоритмы, Структуры
  </p>
</div>
</div>

<div className="flex items-center flex-wrap sm:items-start sm:
late-700">
  <span className="hidden sm:flex items-center justify-center w-16 h-16
shadow-indigo-600/30 whitespace-nowrap">
    <BookOpen className="w-4 h-4" /> Учебное пособие
  </span>

  <button
    id="tab-compiler-btn"
    onClick={onToggleCompiler}
    aria-pressed={compilerOpen}
    title="Боковая панель Python-компилятора (настройка) (настройка)"
    aria-label={compilerOpen ? "Скрыть панель Python-компилятора" : "Показать панель Python-компилятора"}
    className={`shrink-0 flex items-center justify-center w-16 h-16 rounded-md
duration-200 whitespace-nowrap ${
      compilerOpen
        ? 'bg-emerald-600 text-white shadow-lg'
        : 'text-slate-400 hover:text-white bg-slate-200'
    }`}>
    >
    <Terminal className="w-4 h-4" /> Python
  </button>

  <div id="download-menu" className="flex items-center justify-center w-16 h-16">
    <a
      id="download-guide-pdf-btn"
      href={`${import.meta.env.BASE_URL}export/${
        download="guide_lite.pdf"
      }`}>
        <DownloadIcon className="w-4 h-4" />
      </a>
    </div>
  </div>
</div>
```

```
        className="flex items-center gap-2 px-4 py-2 border border-sky-500/30 transition-all duration-200"
        title="Скачать полный TXT файл со всеми файлами"
      >
        <FileText className="w-3.5 h-3.5" />
      </a>
      <button
        id="download-html-btn"
        onClick={saveBook}
        disabled={busy}
        className="flex items-center gap-2 w-3.5 h-3.5 text-white hover:bg-amber-600/20 border border-amber-500/30 rounded"
        title="Скачать всё пособие одним файлом"
      >
        {busy ? <Loader2 className="w-3.5 h-3.5" /> : <DownloadIcon />}
      </button>
    </div>
  </details>
</div>
</div>
</div>
</div>
</header>
);
};
```

---

ФАЙЛ 62/111: src/components/PlanarColoringViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useCallback, useContext, useEffect, useLayoutEffect, useState } from 'react';
import { ChevronLeft, ChevronRight, Play, Pause, RotateCcw, RotateCw } from 'lucide-react';
import { PlanarityDemo } from './PlanarityDemo';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from './VizChapterContext';
```

```
/**
```

```

if (key === "bip") {
  const edges: [number, number][] = [[0, 3], [0, 4],
  const pos = [
    { x: 80, y: 60 }, { x: 80, y: 150 }, { x: 80, y: 240 },
    { x: 380, y: 60 }, { x: 380, y: 150 }, { x: 380, y: 240 }
  ];
  const g: Graph = { key, label: "Двудольный (6 вершин)" };
  g.adj = buildAdj(g);
  return g;
}

const edges: [number, number][] = [[0, 1], [0, 2], [1, 3], [1, 4], [2, 5], [3, 6], [4, 7], [5, 8], [6, 9], [7, 10], [8, 11], [9, 12], [10, 13], [11, 14], [12, 15], [13, 16], [14, 17], [15, 18], [16, 19], [17, 20], [18, 21], [19, 22], [20, 23], [21, 24], [22, 25], [23, 26], [24, 27], [25, 28], [26, 29], [27, 30], [28, 31], [29, 32], [30, 33], [31, 34], [32, 35], [33, 36], [34, 37], [35, 38], [36, 39], [37, 40], [38, 41], [39, 42], [40, 43], [41, 44], [42, 45], [43, 46], [44, 47], [45, 48], [46, 49], [47, 50], [48, 51], [49, 52], [50, 53], [51, 54], [52, 55], [53, 56], [54, 57], [55, 58], [56, 59], [57, 60], [58, 61], [59, 62], [60, 63], [61, 64], [62, 65], [63, 66], [64, 67], [65, 68], [66, 69], [67, 70], [68, 71], [69, 72], [70, 73], [71, 74], [72, 75], [73, 76], [74, 77], [75, 78], [76, 79], [77, 80], [78, 81], [79, 82], [80, 83], [81, 84], [82, 85], [83, 86], [84, 87], [85, 88], [86, 89], [87, 90], [88, 91], [89, 92], [90, 93], [91, 94], [92, 95], [93, 96], [94, 97], [95, 98], [96, 99], [97, 100], [98, 101], [99, 102], [100, 103], [101, 104], [102, 105], [103, 106], [104, 107], [105, 108], [106, 109], [107, 110], [108, 111], [109, 112], [110, 113], [111, 114], [112, 115], [113, 116], [114, 117], [115, 118], [116, 119], [117, 120], [118, 121], [119, 122], [120, 123], [121, 124], [122, 125], [123, 126], [124, 127], [125, 128], [126, 129], [127, 130], [128, 131], [129, 132], [130, 133], [131, 134], [132, 135], [133, 136], [134, 137], [135, 138], [136, 139], [137, 140], [138, 141], [139, 142], [140, 143], [141, 144], [142, 145], [143, 146], [144, 147], [145, 148], [146, 149], [147, 150], [148, 151], [149, 152], [150, 153], [151, 154], [152, 155], [153, 156], [154, 157], [155, 158], [156, 159], [157, 160], [158, 161], [159, 162], [160, 163], [161, 164], [162, 165], [163, 166], [164, 167], [165, 168], [166, 169], [167, 170], [168, 171], [169, 172], [170, 173], [171, 174], [172, 175], [173, 176], [174, 177], [175, 178], [176, 179], [177, 180], [178, 181], [179, 182], [180, 183], [181, 184], [182, 185], [183, 186], [184, 187], [185, 188], [186, 189], [187, 190], [188, 191], [189, 192], [190, 193], [191, 194], [192, 195], [193, 196], [194, 197], [195, 198], [196, 199], [197, 200], [198, 201], [199, 202], [200, 203], [201, 204], [202, 205], [203, 206], [204, 207], [205, 208], [206, 209], [207, 210], [208, 211], [209, 212], [210, 213], [211, 214], [212, 215], [213, 216], [214, 217], [215, 218], [216, 219], [217, 220], [218, 221], [219, 222], [220, 223], [221, 224], [222, 225], [223, 226], [224, 227], [225, 228], [226, 229], [227, 230], [228, 231], [229, 232], [230, 233], [231, 234], [232, 235], [233, 236], [234, 237], [235, 238], [236, 239], [237, 240], [238, 241], [239, 242], [240, 243], [241, 244], [242, 245], [243, 246], [244, 247], [245, 248], [246, 249], [247, 250], [248, 251], [249, 252], [250, 253], [251, 254], [252, 255], [253, 256], [254, 257], [255, 258], [256, 259], [257, 260], [258, 261], [259, 262], [260, 263], [261, 264], [262, 265], [263, 266], [264, 267], [265, 268], [266, 269], [267, 270], [268, 271], [269, 272], [270, 273], [271, 274], [272, 275], [273, 276], [274, 277], [275, 278], [276, 279], [277, 280], [278, 281], [279, 282], [280, 283], [281, 284], [282, 285], [283, 286], [284, 287], [285, 288], [286, 289], [287, 290], [288, 291], [289, 292], [290, 293], [291, 294], [292, 295], [293, 296], [294, 297], [295, 298], [296, 299], [297, 300], [298, 301], [299, 302], [300, 303], [301, 304], [302, 305], [303, 306], [304, 307], [305, 308], [306, 309], [307, 310], [308, 311], [309, 312], [310, 313], [311, 314], [312, 315], [313, 316], [314, 317], [315, 318], [316, 319], [317, 320], [318, 321], [319, 322], [320, 323], [321, 324], [322, 325], [323, 326], [324, 327], [325, 328], [326, 329], [327, 330], [328, 331], [329, 332], [330, 333], [331, 334], [332, 335], [333, 336], [334, 337], [335, 338], [336, 339], [337, 340], [338, 341], [339, 342], [340, 343], [341, 344], [342, 345], [343, 346], [344, 347], [345, 348], [346, 349], [347, 350], [348, 351], [349, 352], [350, 353], [351, 354], [352, 355], [353, 356], [354, 357], [355, 358], [356, 359], [357, 360], [358, 361], [359, 362], [360, 363], [361, 364], [362, 365], [363, 366], [364, 367], [365, 368], [366, 369], [367, 370], [368, 371], [369, 372], [370, 373], [371, 374], [372, 375], [373, 376], [374, 377], [375, 378], [376, 379], [377, 380], [378, 381], [379, 382], [380, 383], [381, 384], [382, 385], [383, 386], [384, 387], [385, 388], [386, 389], [387, 390], [388, 391], [389, 392], [390, 393], [391, 394], [392, 395], [393, 396], [394, 397], [395, 398], [396, 399], [397, 400], [398, 401], [399, 402], [400, 403], [401, 404], [402, 405], [403, 406], [404, 407], [405, 408], [406, 409], [407, 410], [408, 411], [409, 412], [410, 413], [411,
```

```
    return frames;
}
```

```
/** Проверка двудольности BFS-ом: 2 краски    нет нечётных
function bipartiteFrames(g: Graph): Frame[] {
    const frames: Frame[] = [];
    const color = new Array(g.n).fill(0);
    const queue: number[] = [];
    let conflict: [number, number] | null = null;
    frames.push({
        title: "Старт: пробуем покрасить в 2 цвета",
        note: "BFS красит вершину в цвет, противоположный р
            двудольный ( $\chi > 2$ ).",
        color: [...color], v: null, u: null, e: null,
    });
    for (let start = 0; start < g.n && !conflict; start++)
        if (color[start] !== 0) continue;
        color[start] = 1;
        queue.push(start);
        frames.push({
            title: `Запускаем BFS из ${g.names[start]}: цвет
            note: `Вершина ${g.names[start]} ещё не покрашена
                и 2.`,
            color: [...color], v: start, u: null, e: null,
        });
        while (queue.length && !conflict) {
            const v = queue.shift() as number;
            for (const u of g.adj[v]) {
                if (color[u] === 0) {
                    color[u] = 3 - color[v];
                    queue.push(u);
                    frames.push({
                        title: `${g.names[v]} → ${g.names[u]}: цвет
                        note: `Сосед не покрашен: color[${g.names[u]
                            x})).join(", ") || "пуста"}].`,
                        color: [...color], v, u, e: null,
                    });
                } else if (color[u] === color[v]) {

```

```

    const used = Array.from(new Set([...usedA, ...usedB]));
    let c = 1;
    while (used.includes(c)) c += 1;
    ecolord[i] = c;
    inc[a].set(i, c);
    inc[b].set(i, c);
    frames.push({
      title: `Рёбро (${g.names[a]}, ${g.names[b]}): цве
      note: `В вершине ${g.names[a]} заняты цвета {${us
        y) => x - y}).join(", ") || "-"}}. Наименьший св
        да.`
      color: new Array(g.n).fill(0), ecolord: [...ecolor
    });
  });
const chiE = Math.max(...ecolor);
frames.push({
  title: `Готово:  $\chi' = \chiE$  при  $\Delta = \Delta$ `,
  note:
    g.key === "bip"
      ? `Двудольный граф:  $\chi' = \Delta = \Delta$  – это теор
        сс 1 по Визингу.`
      : ` $\chi' = \chiE$ . Теорема Визинга:  $\Delta \leq \chi' \leq \Delta + 1$ 
         $\chi' = \Delta + 1$  – классом 2 (например, K3 и любой
    color: new Array(g.n).fill(0), ecolord: [...ecolor],
  });
return frames;
}

```

```

const TABS: { id: Tab; label: string; icon: React.React
  { id: "planar", label: "1 · Планарность: K5 и K3,3",
  { id: "greedy", label: "2 · Покраска вершин ( $\chi \leq \Delta + 1$ )",
  { id: "bipartite", label: "3 · Двудольность (2 краски)",
  { id: "edges", label: "4 · Покраска рёбер (Визинг, Кё
];

```

```

export const PlanarColoringViz: React.FC = () => {
  const chapterId = useContext(VizChapterContext);
  const runtime = useVizRuntime();

```

```

/* живые переменные компилятора */
const liveStep = vars ? vizNumber(vars.step) ?? vizNum
useEffect(() => {
  if (liveStep === null || !Number.isFinite(liveStep))
    setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);

const liveColor = vars ? vizArray(vars.color) ?? vizA
const liveEcolor = vars ? vizArray(vars.ecolor) ?? vi
const liveV = vars ? vizNumber(vars.v) : null;
const liveU = vars ? vizNumber(vars.u) : null;
const liveE = vars ? vizNumber(vars.e) : null;
const liveChi = vars ? vizNumber(vars.chi) ?? vizNumb
const linked = Boolean(liveColor || liveEcolor || (li

const frame = frames[Math.min(step, maxStep)] ?? ({ t
const nodeColor = (id: number): number => {
  if (liveColor && typeof liveColor[id] === "number")
    return frame.color?.[id] ?? 0;
};
const edgeColor = (i: number): number => {
  if (liveEcolor && typeof liveEcolor[i] === "number")
    return frame.ecolor?.[i] ?? 0;
};
const curV = liveV !== null && Number.isFinite(liveV)
const curU = liveU !== null && Number.isFinite(liveU)
const curE = liveE !== null && Number.isFinite(liveE)
const delta = Math.max(...graph.adj.map((a) => a.leng
const usedColors = Array.from(new Set(graph.pos.map((
const usedEcolors = Array.from(new Set(graph.edges.ma

const go = useCallback((d: number) => {
  setPlaying(false);
  setStep((s) => Math.max(0, Math.min(maxStep, s + d)
}, [maxStep]);

if (tab === "planar") {
  return (

```





```

    <div className="bg-slate-900/70 border border
      {tab === "greedy" && <>Жадная покраска: <b
      : <b className="text-slate-200"> $\chi \leq \Delta$ </b>,
      в  $\chi \leq 5$  (доказывается через  $E \leq 3V - 6$  и це
      ром).</>}
      {tab === "bipartite" && <>Двудольный граф
      /DFS за  $O(V + E)$ . Распознавание 3-раскрашив
      {tab === "edges" && <>Рёберная покраска: <b
       $\Delta$  (Кёниг). Задача « $\chi' = \Delta$ ?» NP-полна даже
    </div>
  </div>
</div>

<div className="mt-3 flex flex-wrap items-center
  <span className="text-[10px] uppercase tracking
    /> из компилятора:</span>
  {vars && linked ? (
    ["v", "u", "e", "color", "ecolor", "chi"].map
    vars[name] === undefined ? null : (
      <span key={name} className="text-[11px] f
        y-1">
        {name} = {JSON.stringify(vars[name]).sl
      </span>
    )
  )
  ) : (
    <span className="text-[11px] text-slate-500">
      открой панель Python: массив <code classNam
      igo-300">ecolor</code> для рёбер) и вершина
    </span>
  )}
</div>
</div>
);
};

const Header: React.FC = () => (
  <div className="flex flex-wrap items-center justify-b

```

```

<h4 className="text-base font-bold text-white">K5
<p className="text-xs text-slate-400">Просто смотри

<div className="grid grid-cols-1 md:grid-cols-2 gap-4" style={{
  /* K5 */
  <div className="bg-slate-950 rounded-xl p-4 border border-slate-800" style={{
    <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 — НЕПЛАНАРЕН</div>
    <svg className="w-full h-full absolute inset-0" style={{
      {(() => {
        const pts = [
          {id:0,x:160,y:40},
          {id:1,x:270,y:100},
          {id:2,x:230,y:230},
          {id:3,x:90,y:230},
          {id:4,x:50,y:100}
        ];
        const edges = [
          [0,1],[0,2],[0,3],[0,4],
          [1,2],[1,3],[1,4],
          [2,3],[2,4],
          [3,4]
        ];
        function findPt(id:number) { return pts.find(p => p.id === id); }
        function intersect(a:number,b:number,c:number,d:number) {
          if (a===c||a===d||b===c||b===d) return true;
          const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
          function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
            return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
          }
          return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
            && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
        }
        const crosses: number[] = [];
        for(let i=0;i<edges.length;i++) {
          for(let j=i+1;j<edges.length;j++) {
            if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
              if (!crosses.includes(i)) crosses.push(i);
          }
        }
      })}
    </svg>
  </div>
  </div>

```

```

        [0,3],[0,4],[0,5],
        [1,3],[1,4],[1,5],
        [2,3],[2,4],[2,5]
    ];
    function findPt(id:number) { return pts.f
    function intersect(a:number,b:number,c:nu
        if (a===c||a===d||b===c||b===d) return
        const p1=findPt(a),p2=findPt(b),p3=find
        if (!p1||!p2||!p3||!p4) return false;
        function ccw(ax:number,ay:number,bx:num
            return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
        }
        return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
            && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
    }
    const crosses:number[]=[];
    for(let i=0;i<edges.length;i++) for(let j
        if (intersect(edges[i][0],edges[i][1],e
            if (!crosses.includes(i)) crosses.push
            if (!crosses.includes(j)) crosses.push
        }
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
        const p1=findPt(edges[i][0]),p2=findPt(e
        const xing=crosses.includes(i);
        lines.push(<line key={`l${i}`} x1={p1.x
            stroke={xing?"#f43f5e":"#4f46e5"} str
        }
    const labels = ["\u04141","\u04142","\u04
    for(const p of pts) {
        circles.push(<g key={`c${p.id}`}>
            <circle cx={p.x} cy={p.y} r={8} fill=
            <text x={p.x} y={p.y+3} textAnchor="m
            </g>);
    }
    return <>{lines}{circles}</>;
  })()

```

```

    Pause,
    Pencil,
    Play,
    Plus,
    RotateCcw,
    SkipBack,
    SkipForward,
    Terminal,
    Unlink,
    Variable,
    X,
} from "lucide-react";
import { buildInitTemplate, buildSyncTemplate, getPageS
import { clearVizState, emitVizState, onVizDemo } from
import {
    baseVarName,
    buildDebugRunner,
    changedVars,
    DEBUG_STEP_CAP,
    orderVars,
    type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "./Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
const PLAYBACK_SPEED_KEY = "pythonCompilerPlaybackSpeed
const PLAYBACK_SPEEDS = [0.25, 0.5, 1, 1.5, 2, 4] as co

interface PythonCompilerProps {
    /** Страница, с которой синхронизируется редактор. */
    chapterId: string;
    chapterTitle: string;
    /** Размер панели (px) управляется родителем и сохран
    width?: number;
    height?: number | null;
    onWidthChange?: (w: number) => void;

```

```

        runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_NAME)
        (module as PyodideModule).loadPyodide({ indexURL: indexURL });
    }
    return runtimePromise;
}

```

```

function errorText(error: unknown) {
    if (error instanceof Error) return error.message;
    return String(error);
}

```

```

/**
 * Предел вывода в панель «Результат».
 *
 * Трасса ограничена по ЧИСЛУ шагов (DEBUG_STEP_CAP), но
 * напечатанной строки: print("x" * 10_000_000) или print("x" * 10_000_000)
 * — это один шаг, после которого весь текст уезжал в <div>.
 * Обрезаем по границе строки и честно пишем, сколько э
 */

```

```

const OUTPUT_CHAR_CAP = 20_000;

```

```

function clipOutput(text: string): string {
    if (text.length <= OUTPUT_CHAR_CAP) return text;
    const cut = text.slice(0, OUTPUT_CHAR_CAP);
    const lastBreak = cut.lastIndexOf("\n");
    const head = lastBreak > OUTPUT_CHAR_CAP / 2 ? cut.slice(0, lastBreak) : cut;
    return `${head}\n... вывод обрезан: показаны первые ${head.length} символов`;
}

```

```

/* -----
/*  Шаблон синхронизации: короткая шапка + минимальный
/*  скелет, шаг-в-шаг с подсветкой визуализации (см. data-visualizer.ts)
/*  -----

```

```

/* -----
/*  Шпаргалка Python: команды сортировки, циклы, структуры данных
/*  «Вставить» кладёт сниппет в редактор под курсор.

```

```

    },
    {
        label: "for i, x in enumerate(...)",
        tip: "Индекс и элемент одновременно: i — позиция, x — элемент",
        code: "for i, x in enumerate(['a', 'b', 'c']):\n    print(i, x)",
    },
],
},
{
    group: "Структуры",
    items: [
        {
            label: "Стек (список)",
            tip: "LIFO, как рекурсия DFS: append = push, pop = pull",
            code: 'st = []\nst.append("A")    # push\nst.pop()    # pull',
        },
        {
            label: "Очередь (deque)",
            tip: "FIFO для BFS: popleft() за O(1). list.pop(0) за O(n)",
            code: 'from collections import deque\nq = deque()\nq.append(1)\nq.popleft()',
        },
        {
            label: "Куча (heapq)",
            tip: "Мин-куча: (вес, вершина) — как очередь приоритетов",
            code: 'import heapq\nh = []\nheapq.heappush(h, (1, 'A'))\nheapq.heappop(h)',
        },
        {
            label: "Матрица n × m",
            tip: "Таблица расстояний: dist[i][j], бесконечность — float('inf')",
            code: 'n, m = 3, 4\ndist = [[float("inf")] * m for _ in range(n)]',
        },
        {
            label: "Множество visited",
            tip: "Посещённые вершины: in/add за O(1).",
            code: 'visited = set()\nvisited.add("A")\nprint("A" in visited)',
        },
    ],
},

```

```

    [sync]
  );
  const syncVarSet = useMemo(() => new Set(syncVarBases)
  /** Полный эталон: глаз сразу вставляет его в обычный
  const template = useMemo(() => buildSyncTemplate(chap
  /** Дефолт редактора: только инициализация демо; она
  const initTemplate = useMemo(() => buildInitTemplate(

  const [code, setCode] = useState(() => {
    const saved = savedEditors.get(chapterId);
    if (!saved) return initTemplate;
    if (saved.code === saved.base || saved.code.trim()
    return saved.code;
  });
  /** «Глаз» – переключатель: эталонный код демо ↔ иници
  const [refOn, setRefOn] = useState(() => {
    const saved = savedEditors.get(chapterId);
    return !!saved && saved.code.trim() !== "" && saved
  });
  const [stdin, setStdin] = useState("");
  const [output, setOutput] = useState("Код выполнится .
  const [running, setRunning] = useState(false);
  const [runtimeReady, setRuntimeReady] = useState(false);
  const [runtimeMessage, setRuntimeMessage] = useState(
  /** Принудительный повтор (глаз / Ctrl+Enter), даже е
  const [traceRequest, setTraceRequest] = useState(0);
  const [copied, setCopied] = useState(false);
  const [stripTab, setStripTab] = useState<"vars" | "hi
  const [outputOpen, setOutputOpen] = useState(true);
  const editorRef = useRef<HTMLTextAreaElement>(null);
  /**
   * Панель жива? Заккрытие (крестик, «глаз» в шапке, см
   * компонент, но `runPythonAsync` к тому моменту може
   * этого флага доехавший до конца прогон опубликовал с
   * после закрытия: панель исчезла, а визуализация про
   * кода, которого на экране больше нет.
   */
  const aliveRef = useRef(true);

```

```

// Смена страницы (или вкладки демо): нетронутый редактор
// инициализацию; свой код не затираем – предлагаем сменить
useEffect(() => {
  const prev = prevTplRef.current;
  if (initTemplate === prev.base && template === prev.full) {
    prevTplRef.current = { base: initTemplate, full: template };
    setPlaying(false);
    setEditHold(null);
    setDebug(null); // трасса ссылается на строки старого кода
    setRefOn(false);
    if (code === prev.base || code === prev.full || code === prev.full) {
      setCode(initTemplate);
      setDesyncedFrom(null);
    } else {
      setDesyncedFrom(chapterId);
    }
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [initTemplate, template]);

const resync = useCallback(() => {
  prevTplRef.current = { base: initTemplate, full: template };
  setCode(initTemplate);
  setRefOn(false);
  setDesyncedFrom(null);
  setStripTab("vars");
  setPlaying(false);
  setEditHold(null);
  setDebug(null);
  setTraceRequest((n) => n + 1);
}, [initTemplate, template]);

const copyCode = useCallback(async () => {
  try {
    await navigator.clipboard.writeText(code);
    setCopied(true);
    setTimeout(() => setCopied(false), 1600);
  } catch {
    setRuntimeMessage("Не получилось скопировать – буфер обмена не поддерживается");
  }
});

```



```

    return {
      line: step?.line,
      func: step?.func,
      locals,
      variables,
      changed: Object.keys(changed).filter((name) => ch
    };
  }, []));

const publishDebug = useCallback(
  (result: DebugResult, idx: number, source: string) => {
    if (!aliveRef.current) return; // панель закрыта
    if (!hasViz || result.steps.length === 0) return;
    const snapshot = snapshotAt(result, idx, source);
    emitVizState(chapterId, {
      line: snapshot.line,
      func: snapshot.func,
      variables: snapshot.variables,
      changed: snapshot.changed,
    });
  },
  [chapterId, hasViz, snapshotAt]
);

// Единственная точка публикации: любое перемещение т
// визуализации согласованный снимок, без side-effect
useEffect(() => {
  if (debug) publishDebug(debug.result, debug.idx, de
}, [debug, publishDebug]);

const goDebug = useCallback(
  (nextIdx: number) => {
    setDebug((d) => {
      if (!d || d.result.steps.length === 0) return d
      const idx = Math.max(0, Math.min(d.result.steps
      return { ...d, idx };
    });
  },
);

```

```

const input = stdin;
const request = traceRequest;
setRunning(true);
setPlaying(false);
setEditHold(null);
setDebug(null);
clearVizState(chapterId);
setOutputOpen(false);

// Пустой редактор: Pyodide не гоняем, панель в реж
// связь с визуализацией снимаем — демо показывает
if (source.trim() === "") {
  setDebug({ result: { steps: [], error: "", truncat
  setOutput("В редакторе пусто — выполнять нечего.\n
    «глаз».");
  setRuntimeMessage("Пустой код: связь с визуализаци
  setOutputOpen(true);
  setRunning(false);
  return;
}

setRuntimeMessage(runtimeReady ? "Обновляю трассу..."

const stdout: string[] = [];
const stderr: string[] = [];
const inputLines = input.split(/\r?\n/);

try {
  const runtime = await getPythonRuntime();
  setRuntimeReady(true);
  runtime.setStdout({ batched: (message) => stdout.p
  runtime.setStderr({ batched: (message) => stderr.p
  runtime.setStdin({ stdin: () => (inputLines.length

  const raw = await runtime.runPythonAsync(buildDeb
  // Панель могли закрыть, пока Python считал: выхо
  if (!aliveRef.current) return;
  const result = JSON.parse(String(raw)) as DebugRe

```

```

        const message = errorText(error);
        setDebug({ result: { steps: [], error: message, t
        clearVizState(chapterId);
        setOutputOpen(true);
        setOutput(`Ошибка:\n${message}`);
        setRuntimeMessage("Не удалось выполнить код – про
    } finally {
        setRunning(false);
    }
}, [chapterId, code, stdin, traceRequest, running, ru

// Код выполняется сам: debounce даёт спокойно допечата
// изменился во время загрузки Pyodide, после завершения
useEffect(() => {
    if (running || editHold === code) return;
    if (debug?.src === code && debug.input === stdin &&
    const timer = window.setTimeout(() => void debugRun
    return () => window.clearTimeout(timer);
}, [code, stdin, traceRequest, running, runtimeReady,

// Стрелки ← → в режиме отладки листают ШАГИ (а не ст
useEffect(() => {
    if (!debug || debug.result.steps.length === 0) retu
    const onKey = (e: KeyboardEvent) => {
        const target = e.target as HTMLElement | null;
        if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
        if (e.metaKey || e.ctrlKey || e.altKey) return;
        if (e.key === "ArrowRight") {
            e.stopPropagation();
            e.preventDefault();
            setPlaying(false);
            stepBy(1);
        }
        if (e.key === "ArrowLeft") {
            e.stopPropagation();
            e.preventDefault();
            setPlaying(false);
            stepBy(-1);
        }
    }
});

```

```

const shownLocals = useMemo(() => {
  if (!debug || !curDebug?.step) return {};
  return snapshotAt(debug.result, debug.idx, debug.src);
}, [debug, curDebug, snapshotAt]);
const debugChanged = curDebug?.step ? changedVars(curDebug) : null;
const debugOrdered = curDebug?.step ? orderVars(shownLocals) : null;

/** Одна и та же кнопка запуска всегда видна: из редактора
 * проигрывает её, в отладчике переключает play/pause */
const runOrTogglePlayback = useCallback(() => {
  if (running) return;
  if (!debug || debug.result.steps.length === 0) {
    void debugRun(true);
    return;
  }
  if (playing) {
    setPlaying(false);
    return;
  }
  if (debug.idx >= debug.result.steps.length - 1) stepToNext();
  setPlaying(true);
}, [debug, debugRun, playing, running, stepTo]);

/** Вставка сниппета из шпаргалки в позицию курсора.
const insertSnippet = useCallback(({snippet: string}) => {
  const ta = editorRef.current;
  setCode((current) => {
    if (!ta) return current + (current.endsWith("\n") ? "" : "\n");
    const start = ta.selectionStart ?? current.length;
    const end = ta.selectionEnd ?? start;
    const padded = (start > 0 && !current.slice(0, start).endsWith("\n")) ? "\n" : "";
    const next = current.slice(0, start) + padded + snippet + current.slice(end, current.length);
    requestAnimationFrame(() => {
      ta.focus();
      ta.selectionStart = ta.selectionEnd = start + padded.length;
    });
    return next;
  });
});

```

```

        el.addEventListener("pointercancel", onUp as Ev)
      })
    >
    <div className="absolute left-1/2 top-1/2 h-10 w-10
      digo-400 transition-colors" />
  </div>
})

{/* Левый край меняет ширину на десктопе. */}
{onWidthChange && (
  <div
    role="separator"
    aria-orientation="vertical"
    aria-label="Потяните, чтобы изменить ширину панели"
    title="Потяните, чтобы изменить ширину панели"
    className="hidden lg:block absolute top-0 bottom-0
    onPointerDown={(e) => {
      const el = e.currentTarget;
      el.setPointerCapture(e.pointerId);
      const startX = e.clientX;
      const startW = width ?? 560;
      const onMove = (ev: PointerEvent) => {
        const next = Math.round(startW + (startX -
          onWidthChange(Math.max(380, Math.min(next
        ));
      const onUp = () => {
        el.removeEventListener("pointermove", onMove as Ev);
        el.removeEventListener("pointerup", onUp as Ev);
      };
      el.addEventListener("pointermove", onMove as Ev);
      el.addEventListener("pointerup", onUp as Ev);
    })
  >
    <div className="absolute left-1/2 top-1/2 -tr
      digo-500 transition-colors" />
  </div>
)}
}

{/* — Заголовок панели —————

```

```

</Tooltip>
<Tooltip side="bottom" content="Вернуть коммен
  <button
    type="button"
    onClick={resync}
    className="p-1.5 rounded-lg text-slate-400"
    aria-label="Сбросить к шаблону страницы"
  >
    <RotateCcw className="w-4 h-4" />
  </button>
</Tooltip>
<Tooltip side="bottom" content="Скрыть панель
  <button
    type="button"
    onClick={onClose}
    className="p-1.5 rounded-lg text-slate-400"
    aria-label="Закрыть компилятор"
  >
    <X className="w-4 h-4" />
  </button>
</Tooltip>
<Tooltip
  side="bottom"
  content={
    refOn
      ? "Убрать эталонный код: редактор стане
обственному встроенному демо. Кнопка ↺ верн
      : "Вставить полный эталонный код этого
удет заменён."
    }
  >
  <button
    type="button"
    onClick={() => {
      const next = !refOn;
      setRefOn(next);
      // «Убрать эталон» – значит убрать код
      // выполняется, поэтому демонстрация бо

```

```

        )))
      </select>
    </label>
  </div>
</div>

/* — Мини-вкладки: переменные страницы / шпаргалка
<div className="shrink-0 border-b border-slate-800" >
  <div className="flex items-center gap-1 px-2 pt-2" >
    <button
      type="button"
      onClick={() => setStripTab("vars")}
      className={`inline-flex items-center gap-1.5
        stripTab === "vars" ? "bg-slate-800 text-white" : ""
      }` >
      >
      <Variable className="w-3.5 h-3.5" /> Переменные
    </button>
    <button
      type="button"
      onClick={() => setStripTab("hints")}
      className={`inline-flex items-center gap-1.5
        stripTab === "hints" ? "bg-slate-800 text-white" : ""
      }` >
      >
      <HelpCircle className="w-3.5 h-3.5" /> Подсказки
    </button>
    <div className="ml-auto pr-1" >
      <Tooltip
        side="bottom"
        content={
          <div>
            hasViz
              ? "Связь идёт по именам ниже: реальный пример"
              : "На выбранной странице нет интерактивных элементов"
            </div>
        }
      >
      <span>

```

```

    >
    <button
      type="button"
      onClick={resync}
      className="flex items-center gap-1.5
0/40 text-amber-400 hover:bg-amber-500/20 t
    >
      <Unlink className="w-3.5 h-3.5 shrink
        Страница сменилась – подставить её пер
      </button>
    </Tooltip>
  })
  {hasViz && (
    <div className="flex flex-wrap items-centr
      {sync.variables.map((v) => (
        <Tooltip
          key={v.name}
          side="bottom"
          content={
            <span>
              <span className="block font-bol
                <code className="font-mono te
              </span>
              <span className="block">{v.role
                {v.range && <span className="bl
            </span>
          }
        >
        <code
          tabIndex={0}
          className="cursor-help rounded-md
font-bold text-emerald-300 hover:bg-emeral
        >
          {v.name}
        </code>
      </Tooltip>
    )})
    {sync.stepNote && (

```



```

        </div>
    })
</div>

{/* — Редактор (всегда на месте) + трасса + ввод
<div className="flex-1 min-h-0 flex flex-col">
  {/* Заголовок редактора: сразу видно, что выполн
  <div className="shrink-0 flex items-center gap-1">
    <Pencil className="w-3.5 h-3.5 text-emerald-400">
    <span className="text-[10px] font-bold uppercase">
      Редактор • выполняется этот текст
    </span>
    {state && (
      <span
        className="inline-flex items-center gap-1"
        ont-bold text-amber-300 shrink-0"
        title="Трасса ниже построена по предыдущей"
      >
        <Loader2 className="w-2.5 h-2.5 animate-spin">
      </span>
    )}
    <span className="ml-auto text-[10px] text-slate-500">
      {code.trim() === "" ? "пусто — ничего не выведено" : ""}
    </span>
  </div>

  <textarea
    ref={editorRef}
    value={code}
    onChange={(event) => {
      setEditHold(null);
      setRefOn(false);
      setCode(event.target.value);
    }}
    onKeyDown={(event) => {
      if ((event.ctrlKey || event.metaKey) && event.key === 'Enter') {
        event.preventDefault();
        void debugRun();
      }
    }}
  />
</div>

```

```

tion-colors">
    <ChevronLeft className="w-4 h-4" />
  </button>
</Tooltip>
<Tooltip content="Шаг вперёд (→) – автопродолжить">
  <button type="button" aria-label="Шаг вперёд" disabled={!curDebug?.step || debug.idx >= debug.steps.length}
    className="w-4 h-4 text-slate-400 disabled:opacity-40 transition-colors"
    >
    <ChevronRight className="w-4 h-4" />
  </button>
</Tooltip>
<Tooltip content="В конец трассы">
  <button type="button" aria-label="В конец трассы" disabled={!curDebug?.step ||
    debug.idx >= debug.steps.length - 1}; } disabled={!curDebug?.step ||
    debug.idx >= debug.steps.length - 1} className="w-4 h-4 text-slate-400
    disabled:opacity-40 transition-colors" >
    <SkipForward className="w-4 h-4" />
  </button>
</Tooltip>
<span className="ml-1.5 text-[10px] text-slate-400" >
  {curDebug?.step ? (
    <div>
      Шаг {debug.idx + 1}/{debug.steps.length}
      {curDebug.step.func !== "<module>" ? (
        </div>
      ) : (
        "шагов нет – исправьте ошибку и начните заново"
      )}
    </span>
  ) : (
    <div>
      {debug.result.truncated && (
        <Tooltip content={`Выполнение остановлено`}>
          <span tabIndex={0} className="cursor-default rounded-full px-1.5 py-0.5">
            Остановлено
          </span>
        </Tooltip>
      )}
    </div>
  )}
  <Tooltip content={hasViz ? "Визуализация" : "Стандартный текст"}>
    {hasViz ? "Визуализация" : "Стандартный текст"}
  </Tooltip>
</div>

```

```
</div>
```

```
{traceOpen && {
```

```
<>
```

```
{/* листинг с подсветкой текущей строки */}
```

```
<div ref={listRef} className="flex-1 min-h-
```

```
  {debug.src.split("\n").map(({line, i}) => {
```

```
    const ln = i + 1;
```

```
    const active = ln === curDebug?.step?.l
```

```
    return {
```

```
      <div
```

```
        key={i}
```

```
        data-line={ln}
```

```
        className={`flex whitespace-pre px-
```

```
          active ? "bg-emerald-500/15 text-
```

```
        }`}
```

```
      >
```

```
        <span
```

```
          className={`w-7 shrink-0 select-n
```

```
          active ? "text-emerald-300" : "
```

```
        }`}
```

```
      >
```

```
        {active ? "►" : ln}
```

```
      </span>
```

```
      <span>{line || " "}</span>
```

```
    </div>
```

```
  });
```

```
  }}}
```

```
</div>
```

```
{/* панель переменных – «Variables», как в
```

```
<div className="shrink-0 max-h-[34%] overfl
```

```
  <div className="flex items-center gap-1.5
```

```
    <Variable className="w-3 h-3 text-indig
```

```
    Переменные на этом шаге
```

```
    <span className="normal-case tracking-n
```

```
  </div>
```

```
  {debug.result.error && {
```

```

        {debug.result.error}
      </pre>
    ) : null}
  <div className="shrink-0 border-t border-slate-800" >
    <label htmlFor="python-stdin" className="text-slate-400">
      stdin для input()
    </label>
    <input
      id="python-stdin"
      value={stdin}
      onChange={(event) => {
        setEditHold(null);
        setStdin(event.target.value);
      }}
      spellCheck={false}
      placeholder="строки ввода через ↵"
      className="w-full bg-transparent font-mono"
    />
  </div>
</div>

{/* — Вывод ————— */}
<div className="shrink-0 border-t border-slate-800" >
  <button
    type="button"
    onClick={() => setOutputOpen((o) => !o)}
    className="flex items-center justify-between gap-2 text-slate-400"
  >
    <span className="inline-flex items-center gap-2">
      <Terminal className="w-3.5 h-3.5 text-emerald-500" />
    </span>
    <span className="inline-flex items-center gap-2">
      {runtimeReady && <CheckCircle2 className="w-3.5 h-3.5 text-emerald-500" />}
      <span className="text-slate-600">{outputOpen ? "Закрыть" : "Открыть"}</span>
    </span>
  </button>
  {outputOpen && (

```

```

    { name: 'Кот Борис',      emoji: ' ', color: 'from-amber-500/20 to-amber-600/20', bubbleText: 'Помоги коту Борису, пожалуйста.' },
    { name: 'Бабушка Люба',   emoji: ' ', color: 'from-rose-500/20 to-rose-600/20', bubbleText: 'Помоги бабушке Любе, пожалуйста.' },
    { name: 'Инопланетянин',   emoji: ' ', color: 'from-emerald-500/20 to-emerald-600/20', bubbleText: 'Помоги инопланетянину, пожалуйста.' },
    { name: 'Бизнесмен',       emoji: ' ', color: 'from-slate-500/20 to-slate-600/20', bubbleText: 'Помоги бизнесмену, пожалуйста.' },
  ];

```

```
let counter = 3;
```

```

export const QueueViz: React.FC = () => {
  const [queue, setQueue] = useState<Customer[]>([
    { id: 'q1', name: 'Студент Вася', emoji: ' ', color: 'from-amber-500/20 to-amber-600/20', bubbleText: 'Помоги студенту Васе, пожалуйста.', animState: 'idle' },
    { id: 'q2', name: 'Кодер Семён', emoji: ' ', color: 'from-emerald-500/20 to-emerald-600/20', bubbleText: 'Помоги кодеру Семёну, пожалуйста.', animState: 'idle' },
    { id: 'q3', name: 'Кот Борис', emoji: ' ', color: 'from-amber-500/20 to-amber-600/20', bubbleText: 'Помоги коту Борису, пожалуйста.', animState: 'idle' },
  ]);

```

```

  const runtime = useVizRuntime();
  const liveQ = vizArray(runtime?.variables?.q);
  const liveV = vizString(runtime?.variables?.v);
  const displayQueue: Customer[] = liveQ
    ? liveQ.map((value, index) => ({
        id: `python-${index}`,
        name: String(value),
        emoji: " ",
        color: "from-emerald-500/20 to-emerald-600/20",
        bubbleText: `q[${index}] из Python`,
        animState: "idle",
      }))
    : queue;

```

```

  const [servedHistory, setServedHistory] = useState<string[]>([]);
  const [isServing, setIsServing] = useState(false);

```

```

        setTimeout(() => setShake(false), 400);
        return;
    }

    setIsServing(true);
    const headGuest = queue[0];

    addLog(`  DEQUEUE: Повар наливает суп первому – ${headGuest.name}`);

    // Step 1: Head guest gets eating animation
    setQueue(prev => prev.map((c, idx) => idx === 0 ? {
        ...c,
        animState: 'eating'
    } : c));

    setTimeout(() => {
        // Step 2: Guest leaves with happy face, queue sh
        setQueue(prev => prev.map((c, idx) => idx === 0 ? {
            ...c,
            emoji: '😄'
        } : c));

        setServedHistory(prev => [`${headGuest.emoji} ${headGuest.name} поел`]);

        setTimeout(() => {
            setQueue(prev => prev.slice(1));
            setIsServing(false);
            addLog(`  ${headGuest.name} поел и ушел сытым.`, 400);
        }, 900);
    }, [queue, isServing]);

    const reset = () => {
        counter = 3;
        setQueue([
            { id: 'r1', name: 'Студент Вася', emoji: '😞', color: 'red', col: 1, animState: 'idle', text: 'Я голода после матана!', animState: 'in' },
            { id: 'r2', name: 'Кодер Семён', emoji: '😞', color: 'blue', col: 2, animState: 'idle', text: 'Напишу ИИ за порцию борща!', animState: 'in' },
            { id: 'r3', name: 'Кот Борис', emoji: '😞', color: 'green', col: 3, animState: 'idle', text: 'Дайте котика без сметаны, плиз.', animState: 'in' },
        ]);
        setServedHistory([]);
        setIsServing(false);
    };

```

```

        :scale-95 transition-all cursor-pointer flex-shrink-0
      >
      <span> Налить суп первому (DEQUEUE)</span>
    </button>

    <button
      onClick={reset}
      className="px-5 py-3.5 rounded-2xl bg-slate-800
        cursor-pointer border border-slate-700"
    >
      Сброс
    </button>
  </div>

```

```

  { /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
  <div className="grid grid-cols-1 md:grid-cols-12 gap-10">

    { /* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */ }
    <div className={`md:col-span-9 bg-slate-900 rounded-3xl
      -[300px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`>
      <div className="absolute top-4 left-4 text-[1.2em] font-medium">
        Очередь голодных гостей (Буфер FIFO / BFS)
        {liveQ && <span className="ml-2 text-emerald-400">{liveQ.length}</span>}
      </div>

      <div className="flex-1 flex flex-col justify-between p-4">
        <div className="flex items-end justify-between">
          { /* ПОВАРИХА И КОТЕЛ С СУПОМ */ }
          <div className="flex flex-col items-center justify-between flex-grow-1">
            <div className="relative h-100">
              <div className="absolute -top-10 left-10 right-10 bottom-10">
                <div className="w-1.5 h-6 bg-slate-400 rounded-3xl"></div>
                <div className="w-2 h-4 bg-slate-400 rounded-3xl"></div>
              </div>
              <span className="text-5xl block animate-pulse">Повар</span>
            </div>
            <div className="text-center">
              <div className="text-2xl font-medium">Суп</div>
              <div className="text-3xl font-medium">Варится</div>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>

```





```
        {/* Пол */}
        <div className="w-full h-3 bg-gradient-to-r f
    </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
    <div className="text-[10px] font-mono text-slate
        {log.map((entry, idx) => (
            <div
                key={idx}
                className={`text-xs font-mono flex items-ce
            >
                {idx === 0 ? <span className="text-indigo-5
                <span>{entry}</span>
            </div>
        )))}
    </div>
</div>
);
};
```

---

ФАЙЛ 66/111: src/components/SalmonAutomatonWidget.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```
import React, { useState, useEffect, useContext, useMem
import { Terminal } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime,
import { Play, RotateCcw, Plus, Volume2, Info } from 'l
```

```
interface TrieNode {
  id: number;
  char: string;
  parent: number;
  children: { [key: string]: number };
```

```
const liveQueue = vars ? vizArray(vars.queue) : null;
const liveNodes = vars ? vizNumber(vars.nodes) : null;
const liveFail = vars ? vizRecord(vars.fail) : null;
```

```
// набор образцов из кода перестраивает бор целиком
useEffect(() => {
  if (!liveWordsSig) return;
  const parsed = (JSON.parse(liveWordsSig) as unknown[]).
    .map((w) => (typeof w === "string" ? w : typeof w !== "string" ? "" : w))
    .filter((w) => Boolean(w))
    .map((w) => w.toUpperCase().replace(/[^A-ZА-Я]/g, ""))
    .filter((w) => w.length > 0)
    .slice(0, 8);
  if (!parsed.length) return;
  if (parsed.join("|") === words.join("|")) return;
  setWords(parsed);
  buildInitialTrie(parsed);
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [liveWordsSig]);
```

```
// состояние автомата и шаг BFS – подсвечиваем вершины
useEffect(() => {
  if (liveState !== null && liveState >= 0 && nodes[liveState] !== null) {
    if (liveStep !== null && liveStep >= 0) {
      // simulationStep – грубый статус: 0 = бор готов,
      const total = Object.keys(nodes).length;
      const failReady = liveFail ? Object.keys(liveFail).length : 0;
      setSimulationStep(liveStep <= 0 ? 0 : failReady ? failReady : total);
    }
    if (liveQueue) {
      const q = liveQueue.map((x) => (typeof x === "number" ? x : undefined));
      if (q.length) setBfsQueue(q);
    }
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [vars]);
```

```
const compilerLinked = Boolean(liveWordsSig || liveState);
```

```

        is_terminal: false,
        words: [],
        depth: newNodes[curr].depth + 1
    };
    }
    curr = newNodes[curr].children[c];
}
if (!newNodes[curr].words.includes(word)) {
    newNodes[curr].words.push(word);
    newNodes[curr].is_terminal = true;
}
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
    const childKeys = Object.values(newNodes[u].children);
    newNodes[u].y = 40 + depth * paddingY;

    if (childKeys.length === 0) {
        newNodes[u].x = 40 + currentX * paddingX;
        currentX++;
    } else {
        let leftX = -1;
        let rightX = -1;
        childKeys.forEach(v => {
            layoutDFS(v, depth + 1);
            if (leftX === -1) leftX = newNodes[v].x!;
            rightX = newNodes[v].x!;
        });
        newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : leftX + paddingX;
        if (leftX === -1) currentX++;
    }
};

layoutDFS(0, 0);

```

```
// Запуск BFS
const startBFS = () => {
  const rootChildren = Object.values(nodes[0].children);
  const queue = [...rootChildren];
  const updatedNodes = { ...nodes };
  rootChildren.forEach((childId) => {
    updatedNodes[childId].fail = 0;
  });

  setNodes(updatedNodes);
  setBfsQueue(queue);
  setSimulationStep(1);
  setCurrentNode(null);
  setSpeechText(
    'Начинаем обход в ширину (BFS)! Все вершины на графике  

    уже суффикса просто нет. Жми «Следующий шаг»!'
  );
};

// Один шаг BFS
const stepBFS = () => {
  if (bfsQueue.length === 0) {
    setSimulationStep(2);
    setCurrentNode(null);
    setSpeechText(
      'Ура! Очередь пуста! Мы успешно построили полный  

      граф. Теперь можно полюбоваться таблицей переходов!'
    );
    return;
  }

  const r = bfsQueue[0];
  const newQueue = bfsQueue.slice(1);
  const updatedNodes = { ...nodes };
  const rNode = updatedNodes[r];
  const childrenIds = Object.values(rNode.children);
```

```

        <h4 className="text-2xl font-bold text-white">
            Интерактивный конструктор: Говорящий Эхо-
        </h4>
        <p className="text-sm text-slate-400">
            Построение дерева и расчет суффиксных ссы
        </p>
    </div>
</div>
<div className="flex items-center gap-2">
    <button
        onClick={() => buildInitialTrie(words)}
        className="flex items-center gap-1 bg-slate
            transition-colors"
        title="Сбросить автомат"
    >
        <RotateCcw className="w-4 h-4" /> Сбросить
    </button>
</div>
</div>

{/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 g
    {/* Аватар Лосося (Сеня) с анимацией моргания и
    <div className="flex flex-col items-center just
        <div className="w-40 h-40 relative flex items
            full border border-blue-500/30 shadow-inner
            {/* SVG Лосося */}
        <svg
            viewBox="0 0 200 200"
            className={`w-36 h-36 transition-transform
                isTalking ? 'scale-105 drop-shadow-[0_0_
            ]`}
        >
            {/* Тело лосося */}
            <ellipse cx="100" cy="100" rx="70" ry="45
            <path d="M 40 85 Q 100 55 160 85 Q 100 115
            </path>

            {/* Хвост */}

```

```

        { /* Декоративные пузыри */ }
        <div className="absolute -top-1 right-4 text-slate-400">
        <div className="absolute top-8 right-1 text-slate-400">
        </div>
        <span className="mt-3 bg-rose-600/30 border border-slate-400 px-2 py-1">
        Эхо-Лосось Сеня
        </span>
    </div>

    { /* Пузырь с текстом (Баббл) */ }
    <div className="md:col-span-2 bg-slate-800 border border-slate-400 p-4
    tween min-h-[140px]">
        { /* Треугольник баббла */ }
        <div className="absolute -left-3 top-1/2 -translate-x-3
        slate-800 border-b-[12px] border-b-transparent">

        <div className="flex items-center space-x-2 text-slate-400">
            <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}` />
            <span>Прямое вещание из аквариума:</span>
        </div>

        <p className="text-slate-200 text-base leading-relaxed">
            {speechText}
        </p>

        <div className="mt-4 pt-3 border-t border-slate-400">
            <div className="text-xs text-slate-400 flex items-center">
                <Info className="w-3.5 h-3.5 text-blue-400" />
                Статус: {simulationStep === 0 ? 'Бор готов к
                оен!'}
            </div>

            <div className="flex gap-2">
                {simulationStep === 0 && (
                    <button
                        onClick={startBFS}
                        className="bg-emerald-600 hover:bg-emerald-500
                        -1 shadow-lg shadow-emerald-900/40 transition-colors">

```

```

        if (n.x && n.x > maxX) maxX = n.x;
        if (n.y && n.y > maxY) maxY = n.y;
    });
    const svgWidth = Math.max(maxX + 80, 400);
    const svgHeight = Math.max(maxY + 60, 200);

    return (
      <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
        <defs>
          <marker id="arrowhead" markerWidth="6"
            <polygon points="0 0, 6 3, 0 6" fill=
          </marker>
          <marker id="fail-arrow" markerWidth="6"
            <polygon points="0 0, 6 3, 0 6" fill=
          </marker>
        </defs>

        {/* Draw Edges */}
        {Object.values(nodes).map(node => {
          if (node.id === 0) return null;
          const parent = nodes[node.parent];
          if (!parent.x || !parent.y || !node.x ||

          const isCurrent = currentNode === node.

          return (
            <g key={`edge-${node.id}`}>
              <line
                x1={parent.x}
                y1={parent.y + 16}
                x2={node.x}
                y2={node.y - 16}
                stroke={isCurrent ? '#60a5fa' : ''}
                strokeWidth="2"
                markerEnd="url(#arrowhead)"
              />
              <text
                x={({parent.x + node.x} / 2 - 10}

```

```

    { /* Draw Nodes */
    Object.values(nodes).map(node => {
      if (!node.x || !node.y) return null;
      const isRoot = node.id === 0;
      const isCurrent = currentNode === node;
      const isInQueue = bfsQueue.includes(node);

      let fill = '#1e293b';
      let stroke = '#475569';

      if (isCurrent) {
        fill = '#2563eb';
        stroke = '#60a5fa';
      } else if (isInQueue) {
        fill = '#b45309';
        stroke = '#fbbf24';
      } else if (node.is_terminal) {
        fill = '#059669';
        stroke = '#34d399';
      }

      return (
        <g key={`node-${node.id}`}>
          <circle
            cx={node.x}
            cy={node.y}
            r="16"
            fill={fill}
            stroke={stroke}
            strokeWidth="2"
          />
          <text
            x={node.x}
            y={node.y + 4}
            fill="white"
            fontSize={isRoot ? "10" : "12"}
            fontWeight="bold"
            textAnchor="middle"
          />
        </g>
      );
    });
  }
);

```



```

        className="text-slate-500 hover:text-slate-400"
        title="Удалить"
      >
        ✕
      </button>
    </span>
  )))
</div>
</div>

<form onSubmit={handleAddWord} className="flex flex-col gap-2" >
  <input
    type="text"
    value={newWord}
    onChange={(e) => setNewWord(e.target.value)}
    placeholder="НОВОЕ СЛОВО..."
    maxLength={8}
    className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
  />
  <button
    type="submit"
    className="bg-slate-700 hover:bg-slate-600 text-white transition-colors"
  >
    <Plus className="w-3.5 h-3.5" /> Добавить
  </button>
</form>
</div>

{/* Таблица Вершин и суффиксных ссылок */}
<div className="lg:col-span-2 bg-slate-900/50 p-4" >
  <h5 className="text-white font-bold mb-3 flex items-center" >
    <span className="flex items-center gap-1.5" >
      <span> </span> Таблица переходов и fail-
    </span>
    <span className="text-xs text-slate-400 font-mono" >
      Всего вершин: {Object.keys(nodes).length}
    </span>
  </h5>
  <table>
    <thead>
      <tr>
        <th>Вершина</th>
        <th>Ссылки</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>{Object.keys(nodes).length}</td>
        <td>{Object.values(nodes).length}</td>
      </tr>
    </tbody>
  </table>
</div>

```



```

        {liveQueue && <span className="text-[11px] text-slate-500">
        -1">queue = {JSON.stringify(liveQueue).slice(0, 100)}
        </>
      ) : (
        <span className="text-[11px] text-slate-500">
        открой панель Python: массив <code className="font-
        nt-mono text-cyan-300">state</code>/<code className="font-
        </span>
      )}
    </div>
  </div>
);
};

```

ФАЙЛ 67/111: src/components/SegmentTreeVisualizer.tsx  
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
import { useVizRuntime, vizNumber } from '../data/vizState';

// ===== TYPES =====
interface Step {
  id: number;
  desc: string;
  codeLine: number;
  treeState: Record<number, number>; // node index to value
  highlightNodes: number[]; // current highlighted nodes
  mergeChildren?: [number, number]; // pair being merged
  arrayHighlight?: [number, number]; // [L, R] range in array
  result?: number; // partial result
}

// ===== PURE LOGIC: build tree =====
function buildTreeFull(arr: number[]): Record<number, number> {
  const tree: Record<number, number> = {};
  const n = arr.length;
  // leaf nodes
  for (let i = 0; i < n; i++) {
    tree[i] = arr[i];
  }
  // internal nodes
  for (let i = n; i < 2 * n - 1; i++) {
    tree[i] = 0;
  }
  // build the tree
  for (let i = n - 1; i > 0; i--) {
    const left = 2 * i;
    const right = 2 * i + 1;
    tree[i] = tree[left] + tree[right];
  }
  return tree;
}

// ===== BUILD STEP =====
function buildStep(arr: number[], i: number, desc: string, codeLine: number): Step {
  const step: Step = {
    id: i,
    desc,
    codeLine,
    treeState: buildTreeFull(arr),
    highlightNodes: [],
    mergeChildren: undefined,
    arrayHighlight: undefined,
    result: undefined,
  };
  return step;
}

// ===== VISUALIZATION =====
function VizStep({ step, index, totalSteps }: { step: Step; index: number; totalSteps: number }) {
  const { id, desc, codeLine, treeState, highlightNodes, mergeChildren, arrayHighlight, result } = step;
  const [array, setArray] = useState<number[]>(arr);
  const [tree, setTree] = useState<Record<number, number>>(treeState);
  const [highlighted, setHighlighted] = useState<number[]>(highlightNodes);
  const [merge, setMerge] = useState<[number, number]|undefined>(mergeChildren);
  const [arrayHighlight, setArrayHighlight] = useState<[number, number]|undefined>(arrayHighlight);
  const [result, setResult] = useState<number|undefined>(result);

  // update array
  const updateArray = (start: number, end: number, value: number) => {
    setArray(arr.map((v, i) => i > end ? v : i < start ? v : value));
  };

  // update tree
  const updateTree = (node: number, value: number) => {
    setTree({ ...tree, [node]: value });
  };

  // update highlighted nodes
  const updateHighlighted = (nodes: number[]) => {
    setHighlighted(nodes);
  };

  // update merge
  const updateMerge = (left: number, right: number) => {
    setMerge([left, right]);
  };

  // update array highlight
  const updateArrayHighlight = (left: number, right: number) => {
    setArrayHighlight([left, right]);
  };

  // update result
  const setResultValue = (value: number) => {
    setResult(value);
  };

  // actions
  const play = () => {
    // ...
  };
  const pause = () => {
    // ...
  };
  const reset = () => {
    // ...
  };
  const left = () => {
    // ...
  };
  const right = () => {
    // ...
  };

```

```

        , highlightNodes: [v], arrayHighlight: [l, r] }
    tree[v] = arr[l];
    steps.push({ id: id++, desc: `tree[${v}] = ${arr[
        l, r] }`});
    return;
}
const m = (l + r) >> 1;
steps.push({ id: id++, desc: `build[${v}], ${l}, ${r}
    { ...tree }, highlightNodes: [v], arrayHighlight:
go(2 * v, l, m);
steps.push({ id: id++, desc: `Вернулись в build[${v}
    lightNodes: [v], arrayHighlight: [l, r] }`});
go(2 * v + 1, m + 1, r);
tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
steps.push({ id: id++, desc: `Объединяем: tree[${v}]
    tate: { ...tree }, highlightNodes: [v], mergeChild
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: `    Дерево построено! Корень
    Highlight: [0, n - 1] }`});
return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
    const n = arr.length;
    const tree = buildTreeFull(arr);
    const steps: Step[] = [];
    let id = 0;

    function go(v: number, l: number, r: number, ql: number, qr: number) {
        if (ql > r || qr < l) {
            steps.push({ id: id++, desc: `query[${v}], [${l}...${r}]
                odeLine: 2, treeState: tree, highlightNodes: [v]
            return 0;
        }
        if (ql <= l && r <= qr) {
            steps.push({ id: id++, desc: `query[${v}], [${l}...${r}]

```

```

    if (l & 1) {
      res += tree[l] ?? 0;
      steps.push({ id: id++, desc: `l=${l} нечётный → l
        0}. Сумма = ${res}. l++.` , codeLine: 5, treeSta
      highlights.push(l);
      l++;
    }
    if (r & 1) {
      r--;
      res += tree[r] ?? 0;
      steps.push({ id: id++, desc: `r=${r + 1} нечётный
        , treeState: tree, highlightNodes: [r], arrayHi
      highlights.push(r);
    }
    l >>= 1;
    r >>= 1;
    if (l < r) {
      steps.push({ id: id++, desc: `Поднимаемся: l = ${
        , arrayHighlight: [qL, qR], result: res }));
    }
  }
  steps.push({ id: id++, desc: ` Запрос sum([${qL}..${
    ighlightNodes: [1], arrayHighlight: [qL, qR], result
  return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val:
```

```

function buildVisualTree(treeState: Record<number, number>) {
  if (l === r) return { v, l, r, val: treeState[v] ?? null };
  const m = (l + r) >> 1;
  return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(
    v + 1, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```
const BUILD_CODE = [
```

```

-----
const [speed, setSpeed] = useState(700);

useEffect(() => { setIdx(0); setPlaying(false); }, [s

useEffect(() => {
  if (!playing || idx >= steps.length - 1) { if (idx :
  const t = setTimeout(() => setIdx(i => i + 1), speed
  return () => clearTimeout(t);
}, [playing, idx, steps, speed]);

const step = steps[idx] || steps[0];
if (!step) return null;

const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-
border-indigo-500 bg-indigo-600', childRing: 'ring-
-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-e
30 border-emerald-500 bg-emerald-600', childRing: '
-slate-900' }, amber: { btn: 'bg-amber-600 hover:bg
der-amber-500 bg-amber-600', childRing: 'ring-rose-!
[color];

return { step, idx, setIdx, playing, setPlaying, speed
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr
  | 'emerald' | 'amber'; arr: number[] }) {
  const p = Player({ steps, color });
  if (!p) return null;
  const { step, idx, setIdx, playing, setPlaying, speed
  const n = arr.length;

  const vTree = useMemo(() => buildVisualTree(step.tree

  const renderNode = useCallback((nd: VNode): React.Rea
    const isHigh = step.highlightNodes.includes(nd.v);
    const isChild = step.mergeChildren?.includes(nd.v);
    const has = nd.val !== null;

```

```

<div className="px-3 py-2 bg-slate-950 border-b b
  {arr.map((v, i) => {
    const inRange = step.arrayHighlight && i >= s
    return (
      <div key={i} className={`flex flex-col item
        go' ? 'bg-indigo-950 border-indigo-500' : c
        er-500'} -translate-y-0.5` : 'bg-slate-900 l
        <span className="text-[7px] text-slate-500
        <span className="text-[11px] font-bold fo
      </div>
    );
  })}
</div>

```

```

{/* Tree */}
<div className="flex-1 p-2 overflow-x-auto bg-sla
  <div className="scale-[0.85] origin-top min-w-m
</div>

```

```

{/* Code */}
<pre className="px-3 py-2 text-[9px] md:text-[10p
  x">
  {code.map((line, i) => {
    const ln = i + 1;
    const active = step.codeLine === ln;
    return (
      <div key={ln} className={`px-2 py-0.5 round
        2 border-indigo-500 text-indigo-200' : colo
        ' : 'bg-amber-600/20 border-l-2 border-ambe
        <span className="inline-block w-4 text-sl
      </div>
    );
  })}
</pre>

```

```

{/* Description + result */}
<div className="px-3 py-2.5 bg-slate-900/60 borde
  px]">

```

```

    </div>
  );
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
  const runtime = useVizRuntime();
  const liveI = vizNumber(runtime?.variables?.i);
  const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);
  const [customInput, setCustomInput] = useState('5, 8');
  const [qL, setQL] = useState(1);
  const [qR, setQR] = useState(4);
  const [view, setView] = useState<'build' | 'queries'>

  // Clamp query range when array changes
  useEffect(() => {
    setQL(prev => Math.min(prev, arr.length - 1));
    setQR(prev => Math.min(prev, arr.length - 1));
  }, [arr]);

  const handleApply = (e: React.FormEvent) => {
    e.preventDefault();
    const parsed = customInput.split(',').map(s => parseInt(s));
    if (parsed.length >= 2 && parsed.length <= 16) {
      setArr(parsed);
    }
  };

  const randomize = () => {
    const size = arr.length;
    const a = Array.from({ length: size }, () => Math.floor(Math.random() * 100));
    setArr(a);
    setCustomInput(a.join(', '));
  };

  const buildSteps = useMemo(() => genBuildSteps(arr),

```



```

    </form>

    {/* Query range */}
    <div className="bg-slate-950 p-4 rounded-xl b
      <label className="text-[10px] font-bold upp
      <div className="flex items-center gap-2">
        <div className="flex items-center gap-1">
          <span className="text-xs text-slate-400
          <input
            type="number"
            min={0}
            max={arr.length - 1}
            value={qL}
            onChange={e => setQL(Math.min(Number(
            className="w-14 bg-slate-900 border b
        focus:border-indigo-500"
      />
    </div>
    <div className="flex items-center gap-1">
      <span className="text-xs text-slate-400
      <input
        type="number"
        min={0}
        max={arr.length - 1}
        value={qR}
        onChange={e => setQR(Math.min(Number(
        className="w-14 bg-slate-900 border b
      focus:border-indigo-500"
    />
  </div>
</div>
<div className="text-[10px] text-slate-500 p
  Запрос: sum(A[{qL}..{qR}]) = {arr.slice(q
</div>
</div>
</div>
</div>

```

```

        steps={queryTDSteps}
        code={QUERY_TD_CODE}
        color="emerald"
        arr={arr}
    />
    <SimPanel
        key={`qbu-${arr.join(' - ')}-${qL}-${qR}`}
        title={`Запрос sum([${qL}..${qR}]) — Снизу-вверх`}
        icon="↑"
        steps={queryBUSteps}
        code={QUERY_BU_CODE}
        color="amber"
        arr={arr}
    />
</div>
))

{view === 'theory' && (
    <div className="space-y-5">
        {/* Why it splits this way */}
        <div className="bg-slate-950 p-5 rounded-xl border border-slate-800">
            <h4 className="text-base font-bold text-white">Почему разбиение?
            </h4>
            <p>Разбиение происходит именно так для N={arr.length}?.
            </p>
            <p>Каждый узел берёт <code>className="bg-slate-900 px-1 rounded font-mono text-emerald-400"</code>
            </p>
            <p>Каждый узел получает <code>className="bg-slate-900 px-1 rounded font-mono text-emerald-400"</code>
            </p>
            <p>Каждая часть забирает на 1 больше (округление вправо).</p>
            <div className="bg-slate-900 p-4 rounded-lg">
                <div className="text-slate-400">Корень: L</div>
                <div className="text-slate-300">mid = ⌊(0 + L)</div>
                <div className="text-indigo-300">→ Левый: L</div>
                <div className="text-emerald-300">→ Правый: L</div>
            </div>
            <div className="text-slate-500 mt-2">Всего узлов: N</div>
        </div>
    </div>
)
```

```

        </div>
      </div>
    </div>

    { /* Verdict */
    <div className="bg-slate-950 p-5 rounded-xl b
      <h4 className="font-bold text-blue-400 mb-2
      <p className="text-sm text-slate-300 leading
        <strong className="text-white">Итеративны
        кальности. Но если нужны <strong className=
        ame="text-white">рекурсивный (сверху-вниз)<
      </p>
    </div>
  </div>
    )}
  </div>
);
}

```

ФАЙЛ 68/111: src/components/Sidebar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useMemo, useState } from "react";
import { ChevronRight, Compass, Search, Sparkles, X } from "lucide-react";
import { chapters, SECTIONS, sectionOf } from "../data/chapters";
import { VIZ_REGISTRY } from "../vizRegistry";

interface SidebarProps {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Мобильный режим: список живёт в выдвижной панели */
  onNavigate?: () => void;
  /** Узкий режим: номер билета и название в одну строку */
  compact?: boolean;
}

```

```

e-500 focus:outline-none focus:border-indigo
/>
{query && {
  <button
    onClick={() => setQuery("")}
    className="absolute right-2 top-1/2 -tran
    aria-label="Очистить поиск"
  >
    <X className="w-3.5 h-3.5" />
  </button>
}}
</div>
})
</div>

<div className="flex-1 overflow-y-auto px-3 pb-4
  {groups.length === 0 && <p className="text-sm t

{groups.map(({ section, items }) => {
  <div key={section.id}>
    <div className="flex items-baseline gap-1.5
      <span className="text-[10px] font-bold up
        {section.from > 0 && {
          <span className="text-[10px] text-slate
        }}
      </div>
    <div className="space-y-1">
      {items.map((chapter) => {
        const isSelected = chapter.id === selec
        const hasViz = Boolean(VIZ_REGISTRY[cha
        // «18–20. Графы. Основное дерево...» → з
        const num = chapter.title.match(/^(\d+{
        const label = num ? chapter.title.slice
        return {
          <button
            key={chapter.id}
            onClick={() => {
              setSelectedChapterId(chapter.id);

```

```

<div className={`shrink-0 m-3 mt-0 bg-gradient-to
  x] text-slate-300 space-y-1.5 hidden lg:block $
  <div className="font-bold text-indigo-400 flex
    <Sparkles className="w-3.5 h-3.5" /> Как поль
  </div>
  <p className="leading-relaxed">
    <b className="text-slate-200">Порядок чтения
    номерам билетов 1 → 24, каждая опирается на п
    Следующая тема» продолжают этот же порядок.
  </p>
  <p className="leading-relaxed">
    Подчёркнутые термины в тексте раскрываются по
    симулятора.
  </p>
  <p className="text-slate-400 italic">Стрелки ←
  </div>
</aside>
  );
};

```

ФАЙЛ 69/111: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar

```

```

interface Plate {
  id: string;
  name: string;
  icon: string;
}

```

```

interface Person {
  id: string;
  name: string;
}

```

```
// Stack handlers
const addPlate = () => {
  const presets = [
    { name: 'Сковорода от омлета', icon: ' ' },
    { name: 'Кастрюля от супа', icon: ' ' },
    { name: 'Чашка от кофе', icon: '=' },
    { name: 'Миска с остатками салата', icon: ' ' },
    { name: 'Тарелка от тако', icon: ' ' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPlate = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon
  };
  setStack(prev => [...prev, newPlate]);
  setPopMessage(` Положили поверх стопки: ${newPlate.name} `);
};

const popPlate = () => {
  if (stack.length === 0) {
    setPopMessage("⚠ Стопка пуста! Все тарелки вымыты.");
    return;
  }
  const topPlate = stack[stack.length - 1];
  setStack(prev => prev.slice(0, prev.length - 1));
  setPopMessage(` Вымыта верхняя тарелка (LIFO): ${topPlate.name} `);
};

const resetStack = () => {
  setStack([
    { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
    { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
    { id: '3', name: 'Блюдец от торта', icon: ' ' },
  ]);
  setPopMessage(" Стопка тарелок восстановлена.");
};
```

```

const addHypothesis = (e: React.FormEvent) => {
  e.preventDefault();
  if (!newCustomText.trim()) return;
  const item: Hypothesis = {
    id: Date.now().toString(),
    text: `Кандидат: "${newCustomText}"`,
    probability: Number(newCustomProb)
  };
  setHeap(prev => [...prev, item]);
  setNewCustomText('');
  setHeapMessage(`Добавлена гипотеза с вероятностью`);
};

const popHeap = () => {
  if (heap.length === 0) {
    setHeapMessage("⚠ Куча пуста! Нет кандидатов для");
    return;
  }
  // Find highest probability
  let maxIdx = 0;
  for (let i = 1; i < heap.length; i++) {
    if (heap[i].probability > heap[maxIdx].probability) {
      maxIdx = i;
    }
  }
  const best = heap[maxIdx];
  setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
  setHeapMessage(`Выбран самый "тяжелый" кандидат: ${best.text}`);
};

const resetHeap = () => {
  setHeap([
    { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
    { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
    { id: '3', text: 'Кандидат: "Привет, я испанский язык"' }
  ]);
  setHeapMessage("Гипотезы восстановлены.");
};

```

```

<button
  onClick={() => setActiveTab('queue')}
  className={`flex items-center justify-between
    activeTab === 'queue'
      ? 'bg-indigo-600/20 border-indigo-500 text-white'
      : 'bg-slate-800/60 border-slate-700/60 text-slate-300'}
>
  <div className="flex items-center gap-3">
    <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'text-white' : 'text-slate-300'} opacity-80`} />
    <div className="text-left">
      <div className="font-bold text-sm text-white">Queue</div>
      <div className="text-xs opacity-80">FIFO</div>
    </div>
  </div>
  <span className="text-xs bg-indigo-500/20 text-white">BFS</span>
</button>

<button
  onClick={() => setActiveTab('heap')}
  className={`flex items-center justify-between
    activeTab === 'heap'
      ? 'bg-emerald-600/20 border-emerald-500 text-white'
      : 'bg-slate-800/60 border-slate-700/60 text-slate-300'}
>
  <div className="flex items-center gap-3">
    <Award className={`w-5 h-5 ${activeTab === 'heap' ? 'text-white' : 'text-slate-300'} opacity-80`} />
    <div className="text-left">
      <div className="font-bold text-sm text-white">Heap</div>
      <div className="text-xs opacity-80">Приоритет</div>
    </div>
  </div>
  <span className="text-xs bg-emerald-500/20 text-white">Beam Search</span>
</button>

```



```

        <RotateCcw className="w-5 h-5" />
      </button>
    </div>
  </div>

  {popMessage && (
    <div className="bg-blue-950/60 border border-
imate-fade-in">
      <span className="inline-block w-2 h-2 round
        {popMessage}
      </div>
    )}

  {/* Visualization */}
  <div className="bg-slate-950/60 p-6 rounded-2
    >
    {stack.length === 0 ? (
      <div className="text-center py-12 text-sl
        <div className="text-5xl mb-3">    </div>
        <p className="text-base font-bold">Стор
        <p className="text-xs mt-1">Добавьте гр
      </div>
    ) : (
      <div className="w-full max-w-sm flex flex
        <div className="absolute top-0 text-xs
full flex items-center gap-1">
        <span>ВЕРШИНА СТЕКА (TOP)</span>
        <ArrowDown className="w-3.5 h-3.5 anim
      </div>

      {/* Reversed map so top of stack is vis
      {stack.slice().reverse().map((plate, ind
        const isTop = index === 0;
        return (
          <div
            key={plate.id}
            className={`w-full flex items-cen
              isTop

```

```

    </h3>
    <p className="text-slate-400 text-xs mt-1">
      Кто первым пришел, тот первым получил суп
    </p>
  </div>
  <div className="flex flex-wrap items-center">
    <button
      onClick={addPerson}
      className="flex-1 md:flex-none flex item
-2.5 rounded-xl text-sm font-bold shadow-lg
    >
      <Plus className="w-4 h-4" /> Встать в очередь
    </button>
    <button
      onClick={servePerson}
      className="flex-1 md:flex-none flex item
er border-indigo-500/40 px-4 py-2.5 rounded
    >
      Налить суп первому
    </button>
    <button
      onClick={resetQueue}
      title="Сбросить"
      className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
    >
      <RotateCcw className="w-5 h-5" />
    </button>
  </div>
</div>

{dequeueMessage && (
  <div className="bg-indigo-950/60 border border-
p-3 animate-fade-in">
    <span className="inline-block w-2 h-2 round
      {dequeueMessage}
    </div>
  )}

```

```

        </span>
      })
      <span className="text-4xl mb-2">{
        <span className={`font-bold text-
          {person.name}
        </span>
        <span className={`text-[10px] font
          isFirst ? 'bg-indigo-500/30 tex
        }`}>
          {isFirst ? 'Получает сын' : `В
        </span>
      </div>
    });
  }}}

  <div className="flex flex-col items-cen
-600 min-w-[120px] h-[120px]">
    <Plus className="w-6 h-6 mb-1" />
    <span className="text-xs font-mono up
  </div>
</div>
})
<div className="mt-6 text-xs text-slate-500
  ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
</div>
</div>
</div>
}}
```

```

{/* HEAP TAB */}
{activeTab === 'heap' && {
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x
      tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-whi
          <span> Приоритетная очередь / Куча</sp
          <span className="text-xs font-mono bg-e
```

```

    </div>
    <div className="md:col-span-3">
      <label className="block text-xs font-bold">
        Вероятность: <span className="text-emerald-500">
      </label>
      <input
        type="range"
        min="0.01"
        max="0.99"
        step="0.01"
        value={newCustomProb}
        onChange={e => setNewCustomProb(Number(e.target.value))}
        className="w-full accent-emerald-500 cursor-pointer"
      />
    </div>
    <div className="md:col-span-3">
      <button
        type="submit"
        disabled={!newCustomText.trim()}
        className="w-full flex items-center justify-center gap-2 text-sm font-medium text-white bg-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed rounded-md p-2"
      >
        <Plus className="w-4 h-4" /> Добавить в список
      </button>
    </div>
  </form>

  {heapMessage && (
    <div className="bg-emerald-950/60 border border-emerald-500/40 p-3 animate-fade-in">
      <span className="inline-block w-2 h-2 rounded-full bg-emerald-500/40" />
      {heapMessage}
    </div>
  )}

  {/* Visualization */}
  <div className="bg-slate-950/60 p-6 rounded-2xl">

```

```

        <span className="text-[10px
-0.5 rounded inline-block mt-1">
            Вершина Кучи (Root) • Буд
        </span>
    }}
</div>
</div>

<div className="flex items-center
    {/* Custom progress bar */}
    <div className="hidden sm:block
        <div
            className={`h-full rounded-
            style={{ width: `${percent}%
        ></div>
    </div>
    <span className={`font-mono font
        isTop ? 'text-emerald-400 tex
    }`}>
        {percent}%
    </span>
    </div>
</div>
    );
    }}}}
</div>
)}
<div className="mt-6 text-xs text-slate-500
    НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
</div>
</div>
</div>
    )}
</div>
);
};
```

```

        <h3 className="text-2xl font-bold text-white">
        <p className="text-sm text-indigo-300">Один
      </div>
    </div>
  </div>

  <div className="flex flex-wrap gap-2 mb-5">
    {[
      { id: "cases" as Tab, label: "1 · Механика: Z",
      { id: "big" as Tab, label: "2 · Большое дерево",
      { id: "play" as Tab, label: "3 · Собери сам
    ]).map((t) => {
      <button
        key={t.id}
        onClick={() => setTab(t.id)}
        className={`px-4 py-2 rounded-lg text-xs font-medium
          tab === t.id ? "bg-indigo-600 text-white" : "bg-white text-indigo-600"
        }`>
      >
        {t.label}
      </button>
    })}
  </div>

  {tab === "cases" && <SplayRotationsViz />}
  {tab === "big" && <SplayWalkViz />}
  {tab === "play" && <SplayRotationSandbox />}

  {tab === "play" && {
    <p className="text-xs text-slate-500 mt-3">
      Порядок поворотов в песочнице сверяется с правым
    </p>
  }}
</div>
);
};

export default SplayPlayground;

```

```

    const n2: TNode = { key: 2, left: n1, right: leaf(7) };
    const n4: TNode = { key: 4, left: n2, right: leaf(7) };
    return { key: 6, left: n4, right: leaf(7) };
  },
},
{
  name: "Змейка",
  hintCase: "Zig-Zag",
  target: 3,
  build: () => {
    const n3 = leaf(3);
    const n2: TNode = { key: 2, left: leaf(1), right: leaf(7) };
    const n4: TNode = { key: 4, left: n2, right: leaf(7) };
    return { key: 6, left: n4, right: leaf(7) };
  },
},
{
  name: "Один уровень",
  hintCase: "Zig",
  target: 2,
  build: () => ({ key: 4, left: { key: 2, left: leaf(1), right: leaf(7) }, right: leaf(7) }),
},
{
  name: "Смешанный",
  hintCase: "Zig-Zag, затем Zig-Zig",
  target: 5,
  build: () => {
    const n5 = leaf(5);
    const n6: TNode = { key: 6, left: n5, right: leaf(7) };
    const n4: TNode = { key: 4, left: leaf(3), right: leaf(7) };
    return { key: 8, left: n4, right: leaf(9) };
  },
},
];

```

/\* ----- операции над деревом -----

export const findPath = (root: TNode | null, key: number): TNode | null => {

```

-----
const p = path[path.length - 2];
const g = path.length >= 3 ? path[path.length - 3] : null;

if (!g) return { node: x.key, kind: "Zig" };

const xLeft = p.left === x;
const pLeft = g.left === p;
if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
return { node: x.key, kind: "Zig-Zag (сначала нижняя ветвь)" };
}

/* ----- раскладка ----- */

interface Placed {
  key: number;
  x: number;
  y: number;
  depth: number;
}

function layout(root: TNode | null): { nodes: Placed[]; edges: [number, number][] } {
  const nodes: Placed[] = [];
  const edges: [number, number][] = [];
  let order = 0;

  const walk = (n: TNode | null, depth: number) => {
    if (!n) return;
    walk(n.left, depth + 1);
    nodes.push({ key: n.key, x: order++, y: depth, depth: depth });
    if (n.left) edges.push([n.key, n.left.key]);
    if (n.right) edges.push([n.key, n.right.key]);
    walk(n.right, depth + 1);
  };
  walk(root, 0);

  const cols = Math.max(1, order);
  const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth + 1)));
  const colW = 46;

```





```

        style={{ transition: "all 700ms cubic-bezier(0.4, 0, 0.2, 1)"
      }}
    );
  }
}

{nodes.map((n) => {
  const isTarget = n.key === target;
  const clickable = !solved && n.depth > 0;
  return (
    <g
      key={n.key}
      onClick={() => clickable && clickNode(n)}
      style={{
        transform: `translate(${n.x}px, ${n.y}px)`,
        transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
        cursor: clickable ? "pointer" : "default",
      }}
    >
      {isTarget && (
        <circle r={22} fill="none" stroke="#6366f1" strokeWidth={2} />
      )}
      <circle
        r={16}
        fill={isTarget ? "#6366f1" : path.has(n.key) ? "#a5b4fc" : "#475568"}
        stroke={isTarget ? "#a5b4fc" : "#475568"}
        strokeWidth={2}
      />
      <text textAnchor="middle" dominantBaseline="middle">
        {n.key}
      </text>
    </g>
  );
})}
</svg>
</div>

<div className="flex flex-wrap items-center gap-2">
  <button

```

```

    {solved && (
      <div
        className={`mt-3 rounded-lg border px-3 py-2`}
        wrong === 0
          ? "bg-emerald-950/40 border-emerald-700/60"
          : "bg-rose-950/30 border-rose-800/60 text-rose-50"
        `)}
      >
        <div className="flex items-center gap-2 font-mono">
          {wrong === 0 ? <CheckCircle2 className="w-4 h-4 text-emerald-500"/> : <X className="w-4 h-4 text-rose-500"/>}
          {wrong === 0
            ? `Узел ${target} в корне за ${moves.length} ходов`
            : `Узел ${target} в корне за ${moves.length} ходов`
          }
        </div>
        {wrong > 0 && (
          <ul className="list-disc pl-5 space-y-0.5">
            {moves.map(
              (m, i) =>
                !m.correct && (
                  <li key={i}>
                    ход {i + 1}: крутили <span className="font-mono">{m.expected}</span>
                  </li>
                )
            )}
          </ul>
        )}
      </div>
    )}
    {wrong === 0 && moves.length > minimal && (
      <div className="opacity-80">Порядок верный,
    </div>
    )}
  </div>
);
};

```

---

```

interface Case {
  key: "zig" | "zig-zig" | "zig-zag";
  label: string;
  when: string;
  rotations: string;
  /** Номера узлов: роль → число. */
  keys: Partial<Record<NodeId, number>>;
  /** Обход слева направо – одинаковый на всех кадрах. */
  inorder: string[];
  /** Что означают поддеревья A..D. */
  ranges: string;
  frames: Frame[];
}

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
  pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
  edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {
  key: "zig",
  label: "Zig",
  when: "родитель x – уже корень, деда нет",
  rotations: "1 поворот",
  keys: { x: 20, p: 40 },
  inorder: ["A", "20", "B", "40", "C"],
  ranges: "A < 20 · B между 20 и 40 · C > 40",
  frames: [
    {
      ...ZIG_BEFORE,
      kind: "start",

```

```

label: "Zig-Zig",
when: "х и р — дети с ОДНОЙ стороны (оба слева или оба справа)",
rotations: "2 поворота, первым — верхний",
keys: { х: 20, р: 40, г: 60 },
inorder: ["A", "20", "B", "40", "C", "60", "D"],
ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
frames: [
    {
        ...ZZ_S0,
        kind: "start",
        title: "Было: прямая линия 60 → 40 → 20",
        text: "Три узла выстроились в «бамбук»: каждый следующий узел  
зглаживает.",
        ms: RESULT_MS,
    },
    {
        ...ZZ_S0,
        kind: "aim",
        title: "Прицел 1: ВЕРХНЕЕ ребро 60 — 40",
        text: "Главная хитрость этого случая: первым крутит 60, а не 40.",
        focus: ["g", "p"],
        ms: AIM_MS,
    },
    {
        ...ZZ_S1,
        kind: "result",
        title: "Поворот 1: 40 поднялось над 60",
        text: "40 встало в корень, 60 опустилось к нему по  
убине 1, а не 2.",
        moved: ["p", "g", "C"],
        ms: RESULT_MS,
    },
    {
        ...ZZ_S1,
        kind: "aim",
        title: "Прицел 2: ребро 40 — 20",
        text: "А теперь обычный одиночный поворот — тот же самый, что и раньше.",
        focus: ["p", "x"],
    },
]

```

```

        kind: "start",
        title: "Было: 60 → влево на 20 → вправо на 40",
        text: "Узлы идут «змейкой»: сначала шаг влево, по
            ",
        ms: RESULT_MS,
    },
    {
        ...ZG_S0,
        kind: "aim",
        title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
        text: "Дед (60) стоит на месте и в этом повороте
        focus: ["p", "x"],
        ms: AIM_MS,
    },
    {
        ...ZG_S1,
        kind: "result",
        title: "Поворот 1: змейка выпрямилась",
        text: "40 заняло место 20 и забрало его к себе ле
        moved: ["x", "p", "C"],
        ms: RESULT_MS,
    },
    {
        ...ZG_S1,
        kind: "aim",
        title: "Прицел 2: ребро 60 – 40",
        text: "Остался одиночный поворот вокруг деда – и
        focus: ["g", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [180, 50], p: [90, 130], g: [275, 130],
        edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
        kind: "result",
        title: "Стало: 20 и 60 – два сына сорока",
        text: "Дерево стало идеально симметричным: все че
            очку». ",
        moved: ["x", "p", "g", "C"],
    }

```

```

    const to = pos[id]!;
    const r = isSubtree(id) ? 15 : 20;
    const dx = to[0] - from[0];
    const dy = to[1] - from[1];
    const len = Math.hypot(dx, dy) || 1;
    const pad = r + 8;
    return (
      <g key={`ghost-${id}`} opacity={0.55}>
        <circle cx={from[0]} cy={from[1]} r={r} fill="white">
          {len > pad * 2 + 6 && (
            <line
              x1={from[0] + (dx / len) * pad}
              y1={from[1] + (dy / len) * pad}
              x2={to[0] - (dx / len) * pad}
              y2={to[1] - (dy / len) * pad}
              stroke="#a5b4fc"
              strokeWidth={1.5}
              strokeDasharray="4 3"
              markerEnd="url(#splay-arrow)"
            />
          )}
        </g>
      );
    )}
  )}
}

```

```

{edges.map(([a, b], i) => {
  const pa = pos[a];
  const pb = pos[b];
  if (!pa || !pb) return null;
  const hot = Boolean(focus && focus.includes(a));
  return (
    <line
      key={`-${a}-${b}-${i}`}
      x1={pa[0]}
      y1={pa[1]}
      x2={pb[0]}
      y2={pb[1]}
      stroke={hot ? "#fbbf24" : "#475569"}
    />
  );
})}

```

```

        }}
      </g>
    );
  }}}
</svg>
);
};

```

```

/** Имя случая из кода пользователя → вкладка демонстрация
const CASE_BY_NAME: Record<string, number> = {
  zig: 0, "zig-zig": 1, zigzig: 1, "zig zig": 1, "zig-z
};

```

```

export const SplayRotationsViz: React.FC = () => {
  const [caseIdx, setCaseIdx] = useState(0);
  const [frame, setFrame] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [slow, setSlow] = useState(false);

```

```

/* — связь с Python-компилятором: case переключает в
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveCase = vars ? vizString(vars.case) ?? vizSt
const liveStep = vars ? vizNumber(vars.step) ?? vizNum
const liveX = vars ? vizNumber(vars.xk) ?? vizNumber(
const liveP = vars ? vizNumber(vars.pk) ?? vizNumber(
const liveG = vars ? vizNumber(vars.gk) ?? vizNumber(
const lastSig = useRef<string | null>(null);

```

```

useEffect(() => {
  if (!liveCase && liveStep === null) return;
  const norm = (liveCase ?? "").trim().toLowerCase();
  const idx = CASE_BY_NAME[norm] ?? caseIdx;
  const sig = `${idx}:${liveStep ?? ""}`;
  if (sig === lastSig.current) return;
  const first = lastSig.current === null;
  lastSig.current = sig;
  setPlaying(false);

```



```

        >
          {c.label}
        </button>
      )))
    </div>

    <div className="mb-3 text-xs sm:text-[13px] text-
      <span className="font-bold text-indigo-300">Кор,
      <span className="text-slate-500">({active.rotat
    </div>

    {/* шаг + пояснение стоят НАД картинкой: сначала
    <div className="mb-3 rounded-xl border border-sla
      <div className="flex items-center gap-2 mb-1 fl
        <span
          className={`text-[10px] font-bold uppercase
            current.kind === "aim"
              ? "bg-amber-500/20 text-amber-300"
              : current.kind === "start"
                ? "bg-slate-700 text-slate-300"
                : "bg-indigo-500/20 text-indigo-300"
            }`}
        >
          {current.kind === "aim" ? "прицел" : current
        </span>
        <span className="text-sm font-bold text-white
          Шаг {idx + 1} из {total}. {current.title}
        </span>
      </div>
      <p className="text-[13px] text-slate-400 leading
    </div>

    <div className="bg-slate-900 rounded-xl border bo
      <Tree frame={current} prev={prevPos} keys={acti

    {/* обход слева направо – одинаковый на всех ка
    <div className="mt-2 pt-2 border-t border-slate
      <div className="flex items-center gap-1.5 fle

```

```

        d: hover: text-slate-300 text-xs font-bold transition-colors
      >
        ← Назад
    </button>
    <button
      onClick={() => {
        setPlaying(false);
        setFrame((f) => (f + 1) % total);
      }}
      className="px-4 py-2 rounded-lg bg-indigo-600
        w-indigo-900/40"
    >
      {last ? "⏮ Сначала" : "Дальше →"}
    </button>
    <button
      onClick={() => setPlaying((p) => !p)}
      className="flex items-center gap-1.5 px-3 py-2
        text-xs font-bold transition-colors"
    >
      {playing ? <Pause className="w-3.5 h-3.5" />
        : {playing ? "Стоп" : "Авто"}}
    </button>
    <button
      onClick={() => setSlow((s) => !s)}
      title="Автопрокрутка ещё медленнее"
      className={`flex items-center gap-1.5 px-3 py-2
        slow
        ? "bg-emerald-600/20 border-emerald-500 transition-colors"
        : "bg-slate-800 border-slate-700 text-slate-300"}
    >
      <Gauge className="w-3.5 h-3.5" /> {slow ? "0.5x" : "1x"}
    </button>
    <button
      onClick={() => {
        setFrame(0);
        setPlaying(false);
      }}

```

```

        ) : (
            <span className="text-[11px] text-slate-500">
                запусти код из панели Python: переменная <с
                случай,
                а <code className="font-mono text-indigo-300">
            </span>
        )}
    </div>

    <p className="mt-2 text-[11px] text-slate-500">
        Листай кнопкой «Дальше» в своём темпе – кадры н
    </p>
</div>
);
};

```

ФАЙЛ 73/111: src/components/SplayWalkViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React, { useEffect, useMemo, useRef, useState }

/**
 * Splay на БОЛЬШОМ дереве – по урокам treap-прохода.
 *
 * Главный принцип: НИКАКИХ «хоп». Каждый одиночный пов
 *   aim – прицел на тройку g–p–x, назван случай;
 *   cut – отстёгиваем среднее поддереву β (пунктир, по
 *   rot – поворот: зелёным подсвечены рёбра, которых н
 * Рядом с логом всегда висит врезка «Механика одного п
 * На каждом кадре – живая автопроверка in-order (сорти
 *
 * Чистые функции (splaySteps, rotateUp, bstInsert, col
 */

export interface SN {

```

```

        cur = cur.right;
    }
}

```

/\*\* Поднять child над его родителем (один атомарный пов

```

export function rotateUp(child: SN): void {
    const p = child.parent;
    if (!p) return;
    const g = p.parent;
    if (p.left === child) {
        p.left = child.right;
        if (child.right) child.right.parent = p;
        child.right = p;
        p.parent = child;
    } else {
        p.right = child.left;
        if (child.left) child.left.parent = p;
        child.left = p;
        p.parent = child;
    }
    child.parent = g;
    if (!g) return;
    if (g.left === p) g.left = child;
    else g.right = child;
}

```

/\*\* Рёбра в виде "p→c" — для diff между кадрами. \*/

```

export function collectEdges(t: SN | null): string[] {
    const es: string[] = [];
    const walk = (n: SN | null) => {
        if (!n) return;
        if (n.left) es.push(`${n.key}→${n.left.key}`);
        if (n.right) es.push(`${n.key}→${n.right.key}`);
        walk(n.left);
        walk(n.right);
    };
    walk(t);
}

```

```

const steps: SplayStep[] = [];
if (!root)
    return { steps: [{ kind: "done", root: null, note:

// --- спуск ---
let cur: SN | null = root;
const pathKeys: number[] = [];
let found = false;
let last: SN = root;
while (cur) {
    last = cur;
    pathKeys.push(cur.key);
    if (key === cur.key) {
        found = true;
        break;
    }
    cur = key < cur.key ? cur.left : cur.right;
}
const x = found ? cur : last;
steps.push({
    kind: "search",
    root: cloneWithParents(root)!,
    pathKeys: [...pathKeys],
    note: isInsert
        ? `Вставили ${key} обычным BST-спуском. Теперь по
        : found
        ? `Спуск по ключу ${key}: путь ${pathKeys.join(
        : `Ключа ${key} нет. Спуск упёрся в ${last.key}
    rotCount: 0,
});

// --- подъём ---
let rotCount = 0;
let target: SN = x as SN;
const fixRoot = () => {
    while (root && root.parent) root = root.parent;
};
/** Один поворот = три кадра: aim → cut → rot. */

```

```

while (target.parent) {
  const p = target.parent;
  const g = p.parent;
  if (!g) {
    emitRotation(
      target,
      "Zig",
      [p.key, target.key],
      `Zig: у ${target.key} нет деда — хватит одного
    );
    continue;
  }
  const pIsLeft = g.left === p;
  const xIsLeft = p.left === target;
  if (pIsLeft === xIsLeft) {
    const side = pIsLeft ? "оба слева" : "оба справа"
    emitRotation(
      p,
      "Zig-Zig",
      [g.key, p.key],
      `Zig-Zig (${side}: линия ${g.key}—${p.key}—${ta
        ${g.key}, а ${target.key} пока не трогаем.`
    );
    steps.push({
      kind: "aim",
      root: cloneWithParents(root)!,
      aimEdge: [target.parent.key, target.key],
      caseLabel: "Zig-Zig",
      note: `Теперь обычный зиг: ребро ${target.parent
      rotCount,
    });
    emitRotation(
      target,
      "Zig-Zig",
      [target.parent.key, target.key],
      `Доводим: ${target.key} над ${target.parent.key
    );
  } else {

```

```

while (cur) {
  if (key === cur.key) return d;
  cur = key < cur.key ? cur.left : cur.right;
  d += 1;
}
return -1;
};

const heightOf = (t: SN | null): number => (t ? 1 + Math.max(
const inorderKeys = (t: SN | null): number[] => (t ? [...
const INITIAL_KEYS = [50, 40, 35, 32, 30, 28, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1];

const buildInitial = (): SN => {
  let t: SN | null = null;
  for (const k of INITIAL_KEYS) t = bstInsert(t, k);
  return t!;
};

const CODE_LINES = [
  { line: 1, text: "while x has parent:" },
  { line: 2, text: "    p = parent(x); g = parent(p)" },
  { line: 3, text: "    if g is None: # Z" },
  { line: 4, text: "        rotate(x over p)" },
  { line: 5, text: "    elif p,g,x на одной линии: # " },
  { line: 6, text: "        rotate(p over g) # B" },
  { line: 7, text: "        rotate(x over p)" },
  { line: 8, text: "    else: # Z" },
  { line: 9, text: "        rotate(x over p) # H" },
  { line: 10, text: "        rotate(x over g)" },
];

/* ----- врезка «механика одного по

const MechanismInset: React.FC<{ step: SplayStep | null
const isCut = step?.kind === "cut";
const isRot = step?.kind === "rot";
return (
  <div className="bg-slate-950/80 rounded-lg border b

```

```

<g transform="translate(186,6)">
  <text y="10" fill="#94a3b8" fontSize="11" font-weight="bold">
  <line x1="70" y1="46" x2="45" y2="86" stroke="black" stroke-width="2" />
  <line x1="70" y1="46" x2="95" y2="86" stroke="black" stroke-width="2" />
  <line x1="45" y1="86" x2="27" y2="126" stroke="black" stroke-width="2" />
  <line
    x1="45" y1="86" x2="63" y2="126"
    stroke={isRot ? "#10b981" : "#64748b"}
    strokeWidth={isRot ? 3 : 2}
  />
  <circle cx="70" cy="46" r="15" fill="#059669" />
  <text x="70" y="50" textAnchor="middle" fill="black" />
  <circle cx="45" cy="86" r="15" fill="#1e293b" />
  <text x="45" y="90" textAnchor="middle" fill="black" />
  <circle cx="27" cy="126" r="13" fill="#0f172a" />
  <text x="27" y="130" textAnchor="middle" fill="black" />
  <circle
    cx="63" cy="126" r="13"
    fill={isRot ? "#78350f" : "#0f172a"}
    stroke={isRot ? "#10b981" : "#64748b"}
    strokeWidth={isRot ? 2.5 : 1.5}
  />
  <text x="63" y="130" textAnchor="middle" fill="black" />
  <circle cx="95" cy="126" r="13" fill="#0f172a" />
  <text x="95" y="130" textAnchor="middle" fill="black" />
  <text x="112" y="92" fill={isRot ? "#10b981" : "black"}
    {isRot ? "↓ β теперь при p" : "β при p"}
  </text>
  <text x="30" y="160" fill="#64748b" font-size="12" />
</g>
</svg>
<p className="text-[10px] text-slate-500 leading-5">
  Обход слева-направо α x β p γ не меняется никог
  числа поворотов. Единственный, кто меняет хозяин
</p>
</div>
);
};

```



```

useEffect(() => {
  if (!playing || !steps) return;
  if (idx >= steps.length - 1) {
    setPlaying(false);
    baseRef.current = steps[steps.length - 1].root ??
    setTreeVers((v) => v + 1);
    return;
  }
  timer.current = setTimeout(() => setIdx((v) => v +
  return () => {
    if (timer.current) clearTimeout(timer.current);
  };
}, [playing, idx, steps]);

const step = steps ? steps[Math.min(idx, steps.length
const viewRoot = step ? step.root : baseRef.current;
const pos = useMemo(() => layout(viewRoot), [viewRoot
const nodes = useMemo(() => [...pos.entries()].map(([
const edges = useMemo(() => {
  const es: { a: number; b: number }[] = [];
  const walk = (n: SN | null) => {
    if (!n) return;
    if (n.left) es.push({ a: n.key, b: n.left.key });
    if (n.right) es.push({ a: n.key, b: n.right.key });
    walk(n.left);
    walk(n.right);
  };
  walk(viewRoot);
  return es;
}, [viewRoot, treeVers, idx]);

const autoW = Math.max(VIEW_W, ...nodes.map((n) => n.p
const autoH = Math.max(300, ...nodes.map((n) => n.py

const dBefore = useMemo(() => (steps ? depthOf(steps[
const dNow = steps ? depthOf(viewRoot, steps[steps.le
const hBefore = useMemo(() => (steps ? heightOf(steps
const hNow = heightOf(viewRoot);

```

```

    if (step.kind === "search" && step.pathKeys?.includes(k)) return "#f59e0b";
    if ((step.kind === "aim" || step.kind === "cut") &&
        const e = step.aimEdge ?? step.cutEdge!;
        if (e.includes(k)) return "#f59e0b";
    }
    if (step.pathKeys?.includes(k)) return "#38bdf8";
    return "#64748b";
};

```

```

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border" >
    {/* Шанка */}
    <div className="flex flex-wrap items-center justify-center" >
      <div className="flex items-center gap-3" >
        <div className="p-2.5 bg-indigo-600 text-white" >
          <span className="text-2xl" > </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white" >
            <p className="text-sm text-indigo-300" >Клики
          </p>
        </div>
      </div>
      <div className="flex flex-wrap items-center gap-3" >
        <input
          value={input}
          onChange={(e) => setInput(e.target.value)}
          onKeyDown={(e) => {
            if (e.key === "Enter") {
              const v = parseInt(input, 10);
              if (!Number.isNaN(v)) {
                const has = inorderKeys(baseRef.current);
                if (has) runKey(v);
                else doInsert();
                setInput("");
              }
            }
          }}
          placeholder="ключ + Enter"
        </input>
      </div>
    </div>
  </div>
)

```

```

        В начало
      </button>
      <button onClick={() => { setPlaying(false); set
        -lg text-xs font-bold bg-slate-700 text-white
        ← Шаг
      </button>
      <button onClick={() => setPlaying(!playing)} cl
        er-600" : "bg-emerald-600"}`}>
        {playing ? "  Пауза" : "▶ Пуск"}
      </button>
      <button onClick={() => { setPlaying(false); set
        className="px-3 py-1.5 rounded-lg text-xs font
        Шаг →
      </button>
      <span className="text-xs text-slate-400">
        Шаг {idx + 1} из {steps.length} • поворотов
      </span>
      {step?.caseLabel && (
        <span className="px-2.5 py-1 rounded-full text
        00/60">
          {step.caseLabel}
        </span>
      )}
    </div>
  )}

  {/* Дерево */}
  <div className="bg-indigo-950/20 rounded-xl p-4 b
    {!steps && (
      <div className="absolute top-3 left-3 bg-indi
        ont-bold">
        Кликни по любому узлу – splay покадрово
      </div>
    )}
  <svg viewBox={`0 0 ${autoW} ${autoH}`} classNam
    {edges.map((e) => {
      const a = pos.get(e.a);
      const b = pos.get(e.b);

```

```

        <circle
          r={isAimNode ? 24 : 20}
          fill={stepColor(n.key)}
          stroke={strokeColor(n.key)}
          strokeWidth={isAimNode ? 4 : 2}
          className="transition-all duration-300"
        />
        <text y={4} fill="rgba(0,0,0,0.1)" fontSize="13"
      />
    )
    {n.key}
  </text>
</g>
);
}}}
</svg>

```

```

{/* Лог + механика */}
<div className="w-full flex flex-col lg:flex-row" >
  <div className="flex-1 bg-slate-950/80 p-4 rounded" >
    <p className="font-semibold text-amber-400" >
      {step?.kind === "aim"
        ? " Прицел:"
        : step?.kind === "cut"
        ? " ∞ Отстёгиваем среднее поддерево:"
        : step?.kind === "rot"
        ? " Поворот:"
        : step?.kind === "search"
        ? " Спуск:"
        : "Статус:"}
    </p>
    <p className="min-h-[40px] flex items-center" >
      {step ? step.note : "Клики по узлу или в
      ке справа – механика одного поворота."}
    </p>
    {steps && step && step.kind !== "search" &&
      <p className={`text-xs mt-2 ${checkOk ? "text-amber-400" : "text-slate-400"}`} >
        {checkOk ? " " : " "} проверка кадра: 0
        {inorderKeys(viewRoot).length}
      </p>
    }
  </div>
  <div className="flex-1" >

```

```

<h4 className="text-sm font-bold text-slate-200">
  <div className="bg-slate-950/80 rounded-lg p-4">
    {CODE_LINES.map((l) => {
      const zig = step?.caseLabel === "Zig" &&
      const zz1 = step?.caseLabel === "Zig-Zig" &&
      const zz2 = step?.caseLabel === "Zig-Zig" &&
      const za1 = step?.caseLabel === "Zig-Zag" &&
      const za2 = step?.caseLabel === "Zig-Zag" &&
      const active = zig || zz1 || zz2 || za1 || za2
      return (
        <div key={l.line} className={`py-1 px-2 text-slate-400`} >
          <span className="text-slate-600 select-none">
            {l.text}
          </span>
        </div>
      )
    })}
  </div>
</div>
</div>
</div>
);
};

export default SplayWalkViz;

```

ФАЙЛ 74/111: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray } from '../data/vizSteps';

interface Plate {
  id: string;

```

```

    )))
    : dirtyStack;

const [cleanRack, setCleanRack] = useState<Plate[]>([]);
const [isWashing, setIsWashing] = useState(false);
const [showSoap, setShowSoap] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>([]);

const addLog = (msg: string) => setLog(prev => [...prev, msg]);

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length >= 7) {
    addLog('⚠ Стопка слишком высокая, тарелки могут упасть');
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }

  const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
  counter++;

  const newPlate: Plate = {
    id: Date.now().toString(),
    ...preset,
    isDirty: true,
    animState: 'in',
  };

  setDirtyStack(prev => [...prev, newPlate]);
  addLog(`  PUSH: Положили ${newPlate.emoji} сверху с ${newPlate.color} мылом`);

  setTimeout(() => {
    setDirtyStack(prev => prev.map(p => p.id !== newPlate.id ? p : { ...p, animState: 'out' }));
  }, 500);
}, [dirtyStack, isWashing]);

```

```

    setDirtyStack([
      { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', State: 'in' },
      { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', imState: 'in' },
      { id: 'r3', name: 'Блюдец от торта', emoji: '🍰', mState: 'in' },
    ]);
    setCleanRack([]);
    setIsWashing(false);
    setShowSoap(false);
    setLog([' Стол сброшен. Посуда снова грязная!']);
  };

```

```

const topIndex = displayStack.length - 1;

```

```

return (
  <div className="flex flex-col gap-6">
    {/* МНЕМОНИКА / ПРАВИЛО */}
    <div className="bg-blue-950/40 border border-blue-950/40 p-6">
      <div className="text-3xl"> </div>
      <div>
        <h3 className="font-black text-blue-400 text-2xl">ПРАВИЛО</h3>
        <p className="text-xs text-slate-300 mt-1 leading-tight">
          Ты складываешь грязные тарелки друг на друга.
          А когда приходит время мыть, ты берёшь именно последнюю.
          Попробуй вытащить нижнюю – всё рухнет!
        </p>
      </div>
    </div>

    {/* УПРАВЛЕНИЕ */}
    <div className="flex items-center gap-3 flex-wrap">
      <button
        onClick={pushPlate}
        disabled={isWashing}
        className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
          disabled:cursor-not-allowed text-white"
      >
        Добавить тарелку
      </button>
    </div>
  </div>
)

```

```

        <span>РАКОВИНА    </span>
    </div>

    {/* Мыльные пузыри при мытье */}
    {showSoap && {
        <div className="absolute inset-0 pointer-events-none" >
            <div className="absolute w-6 h-6 rounded-full bg-white border-1 border-gray-200" >
            </div>
            <div className="absolute w-4 h-4 rounded-full bg-white border-1 border-gray-200" >
            </div>
            <div className="absolute w-5 h-5 rounded-full bg-white border-1 border-gray-200" >
            </div>
            <div className="absolute text-4xl anim-crash" >
            </div>
        </div>
    }}

    {displayStack.length === 0 ? {
        <div className="flex-1 flex flex-col items-center justify-center" >
            <span className="text-6xl mb-3 animate-pulse" >
            <p className="text-white font-black text-2xl" >
            <p className="text-slate-500 text-xs mt-1" >
            </div>
        } : {
            <div className={`flex-1 flex flex-col justify-center` >

                {/* Указатель TOP */}
                <div
                    className="absolute right-8 flex items-center justify-center"
                    style={{ bottom: `${24 + topIndex * 44}px` }} >
                    <span className="text-xs font-mono text-white" >
                    er gap-1">
                        <ArrowUp className="w-3.5 h-3.5" />
                        <span>ВЕРШИНА (TOP)</span>
                    </span>
                    <span className="text-blue-400 animate-pulse" >
                    </div>

```



```

<div className="absolute top-4 right-4 flex items-center justify-center"
  order-emerald-800/80 text-[9px] font-mono">
  <Sparkles className="w-3.5 h-3.5 animate-pulse" />
  <span>ЧИСТО </span>
</div>

<div className="flex-1 flex flex-col justify-between"
  {cleanRack.length === 0 ? {
    <div className="flex-1 flex flex-col item-center justify-center"
      <span className="text-5xl mb-2"> </span>
      <p className="text-xs font-bold font-mono">Пусто!
      <p className="text-[10px] mt-1">Здесь будет сушилка
    </div>
  ) : {
    <div className="flex flex-col gap-2 w-full"
      {cleanRack.map(plate => {
        <div
          key={plate.id}
          className="w-full h-8 rounded-full border border-slate-200"
          items-center justify-center gap-2 opacity-0.8"
        >
          <span className="text-sm"> {plate.name}
          <span className="text-[10px] text-emerald-500"> {plate.status}
        </div>
      })}
    </div>
  }}
</div>

{/* Столешница сушки */}
<div className="w-full h-4 bg-gradient-to-r from-emerald-500 to-teal-500" />
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4"
  <div className="text-[10px] font-mono text-slate-400" />

```

```

        while (j > 0 && s[i] !== s[j]) {
            steps.push({ i, j, pi: [...pi], highlight:
                'Сходство уменьшается. Откат назад: откат j = pi[j - 1] = ' + pi[j - 1] + '
                j = pi[j - 1];
        }

        if (s[i] === s[j]) {
            steps.push({ i, j, pi: [...pi], highlight:
                'Сходство увеличивается.' });
            j++;
        } else {
            steps.push({ i, j, pi: [...pi], highlight:
                'Схождений нет.' });
        }

        pi[i] = j;
        steps.push({ i, j, pi: [...pi], desc: `Записываем` });

        if (j === pattern.length) {
            steps.push({ i, j, pi: [...pi], found: true });
        }
    }
    return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string) {
    if (!pattern) pattern = " ";
    const s = pattern + "#" + text;
    const n = s.length;
    const z = new Array(n).fill(0);
    const steps: any[] = [];

    let l = 0, r = 0;
    steps.push({ i: 0, l, r, z: [...z], desc: `Начало.` });

    for (let i = 1; i < n; i++) {
        steps.push({ i, l, r, z: [...z], desc: `Шаг i=${i}` });
    }
}

```

```

-----
export function StringAlgorithmsViz({ defaultMode = "kmp"
  const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
  const [pattern, setPattern] = useState("aba");
  const [text, setText] = useState("abacaba");

  // Ensure inputs are valid dynamically
  const safePattern = pattern || " ";
  const safeText = text || " ";

  const { s: demoString, steps, patternLength: demoPatternLength } =
    if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
    else return generateZSteps(safePattern, safeText);
  }, [mode, safePattern, safeText]);
  const runtime = useVizRuntime();
  const runtimeString = vizString(runtime?.variables?.runtimeString);
  const s = runtimeString ?? demoString;
  const patternLength = runtimeString ? runtimeString.length : demoPatternLength;

  const [autoPlay, setAutoPlay] = useState(false);
  const [stepIdx, setStepIdx] = useState(0);
  const timer = useRef<NodeJS.Timeout | null>(null);

  // Reset when inputs or mode change
  useEffect(() => {
    setStepIdx(0);
    setAutoPlay(false);
  }, [s, mode]);

  // Timer loop for autoPlay
  useEffect(() => {
    if (autoPlay) {
      timer.current = setInterval(() => {
        setStepIdx(prev => {
          if (prev >= steps.length - 1) { setAutoPlay(false); }
          return prev + 1;
        });
      }, 1200);
    }
  }, [steps]);
}

```

```

        <button onClick={() => setMode("z")}
          className={`px-3 py-1.5 text-xs
            " : "text-slate-400 hover:text-slate-300"}`
            Z-ФУНКЦИЯ
        </button>
      </div>

      <div className="flex items-center gap-2">
        <div className="bg-slate-800 p-1 flex items-center gap-2">
          <span className="text-[10px] text-white font-bold font-mono px-2 py-1">
            <input value={pattern} onChange={pattern} />
          </span>
        </div>
        <div className="bg-slate-800 p-1 flex items-center gap-2">
          <span className="text-[10px] text-white font-bold font-mono px-2 py-1">
            <input value={text} onChange={text} />
          </span>
        </div>
      </div>
    </div>

    <div className="bg-slate-950 rounded-xl border-[1px] flex items-center justify-start">
      <div className="flex gap-1.5 px-4 h-full">
        {s.split("").map((char, index) => {
          const isPattern = index < pattern.length;
          const isSeparator = index === pattern.length;

          let borderColor = "border-slate-900";
          let bgColor = "bg-slate-900";
          let textColor = isSeparator ? "text-slate-400" : "text-white";

          // KMP Logic
          if (mode === "kmp") {
            if (step.highlight?.include(char)) {
              borderColor = "border-amber-400";
              bgColor = step.match === char ? "bg-amber-400" : "bg-slate-900/40";
            }
          }
        })}
      </div>
    </div>
  </div>
)

```

```

                                {(mode == "kmp" ?
</div>

                                {/* KMP fingers */}
                                {mode == "kmp" && step
                                  <div className="abs
z-20">
                                  <span className
                                  </div>
                                }}
                                {mode == "kmp" && step
                                  <div className="abs
                                  <span className
                                  </div>
                                }}
                                {mode == "kmp" && step
                                  <div className="abs
20">
                                  <span className
                                  </div>
                                }}

                                {/* Z fingers */}
                                {mode == "z" && step.i
                                  <div className="abs
e z-20">
                                  <span className
                                  </div>
                                }}
                                {mode == "z" && step.z
                                  <div className="abs
                                  <span className
                                  </div>
                                }}
                                {mode == "z" && step.z
                                  <div className="abs
                                  <span className
                                  </div>
                                }}

```



```

*/
export const Tooltip: React.FC<TooltipProps> = ({ content, side }) => {
  const [open, setOpen] = useState(false);

  const hiddenShift = side === "top" ? "translate-y-1" : "translate-y-0";

  return (
    <span
      className={`relative inline-flex ${className ?? ""}
        onMouseEnter={() => setOpen(true)}
        onMouseLeave={() => setOpen(false)}
        onFocus={() => setOpen(true)}
        onBlur={() => setOpen(false)}
      >
      {children}
      <span
        role="tooltip"
        className={`pointer-events-none absolute left-1/2 top-1/2
          bg-slate-900 px-3 py-2 text-left text-[11px]
          transition duration-150 ${
            side === "top" ? "bottom-full mb-2" : "top-full mt-2"
          } ${open ? "visible translate-y-0 opacity-100" : "invisible"}
        >
        {content}
        <span
          className={`absolute left-1/2 top-1/2 -translate-x-50% -translate-y-50%
            ${side === "top" ? "-bottom-[5px] border-b border-slate-900" : ""}
          >
        </span>
      </span>
    </span>
  );
};

```

```
];
```

```
interface TStep {
  desc: string;
  visited: Set<number>;
  fullyProcessed: Set<number>;
  currentNode: number | null;
  topoList: number[];
  dfsStack: number[];
}
```

```
export const TopologicalSortViz: React.FC = () => {
  const [steps, setSteps] = useState<TStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
  const v = vizNumber(vars.v);
  const orderLength = vizArray(vars.order)?.length;
  if (v === null && orderLength === undefined) return null;
  const candidates = steps
    .map((step, index) => ({ step, index }))
    .filter(({ step }) => (v === null || step.currentNode !== v));
  return candidates.at(-1)?.index ?? null;
});
```

```
const [isPlaying, setIsPlaying] = useState(false);
```

```
useEffect(() => {
  const listSteps: TStep[] = [];
  const visited = new Set<number>();
  const fullyProcessed = new Set<number>();
  const topoList: number[] = [];
  const dfsStack: number[] = [];
```

```
  const recordStep = (desc: string, currentNode: number) => {
    listSteps.push({
      desc,
      visited: new Set(visited),
      fullyProcessed: new Set(fullyProcessed),
```



```

        recordStep(
            `DFS: закончили обработку '${dagNodes[u].name}'
            ,
            u,
        );
    };
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
    if (!visited.has(i)) {
        recordStep(
            `Запускаем новый обход DFS от нетронутой верши
            null,
        );
        dfs(i);
    }
}

recordStep(
    "Топологическая сортировка завершена! Полученный
    null,
);
setSteps(listSteps);
setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
    desc: "Запуск...",
    visited: new Set(),
    fullyProcessed: new Set(),
    currentNode: null,
    topoList: [],
    dfsStack: [],
};

useEffect(() => {
    let timer: NodeJS.Timeout;
    if (isPlaying && currentStepIdx < steps.length - 1)

```

```

        ) : {
            <Play className="w-4 h-4 ml-1" />
        }}
        {isPlaying ? "Пауза " : "Старт "}
    </button>

    <button
        onClick={reset}
        aria-label="Сбросить сортировку"
        title="Сбросить сортировку"
        className="flex items-center justify-center
        ors"
    >
        <RotateCcw className="w-4 h-4" />
    </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12 "
    {/* Playfield SVG */}
    <div className="lg:col-span-8 bg-[#0B1120] rela
        <svg className="w-full h-full max-w-[650px]"
            <defs>
                <marker
                    id="dag-arrow"
                    viewBox="0 0 10 10"
                    refX="24"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"
                >
                    <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
                </marker>
                <marker
                    id="dag-arrow-active"
                    viewBox="0 0 10 10"
                    refX="24"

```

```
let fill = "#1e293b";
let stroke = "#334155";
let textColor = "text-slate-400";
void textColor;

if (isCurrent) {
  fill = "#312e81";
  stroke = "#818cf8";
  textColor = "text-white font-bold";
} else if (isProcessed) {
  fill = "#022c22";
  stroke = "#059669";
  textColor = "text-emerald-300";
} else if (isVisited) {
  fill = "#3730a3";
  stroke = "#6366f1";
  textColor = "text-indigo-200";
}

return (
  <g key={node.id}>
    <rect
      x={node.x - 55}
      y={node.y - 22}
      width="110"
      height="44"
      rx="8"
      fill={fill}
      stroke={stroke}
      strokeWidth="2"
      className="transition-all duration-500"
    />
    <text
      x={node.x}
      y={node.y - 4}
      textAnchor="middle"
      fill="white"
    />
  </g>
)
```

```

        <span className="text-[10px] text-slate-500">
            РЕЗУЛЬТАТ (МАТРУТ ОБУЧЕНИЯ):
        </span>
        <div className="bg-slate-900/60 p-2.5 rounded-3xl
pr-1 text-slate-300">
            {step.topoList.length === 0 ? (
                <span className="text-slate-600 text-[10px]">
                    ожидаем обработку узлов...
                </span>
            ) : (
                getTopoListString()
            )}
        </div>
    </div>

    {/* Color key */}
    <div className="space-y-1">
        <span className="text-[10px] text-slate-500">
            ВЕРШИНЫ В DFS:
        </span>
        <div className="flex items-center gap-2">
            <span className="w-3.5 h-3.5 rounded bg-slate-400">
                <span className="text-slate-400 text-[10px]">
                    Серые (зашли)
                </span>
            </div>
            <div className="flex items-center gap-2">
                <span className="w-3.5 h-3.5 rounded bg-slate-400">
                    <span className="text-slate-400 text-[10px]">
                        Черные (готово)
                    </span>
                </div>
            </div>
        </div>
    </div>

    <div className="mt-4 pt-3 border-t border-slate-200">
        <span>
            Шаг {currentStepIdx + 1} из {steps.length}
        </span>
    </div>

```

## Универсальное пособие • полный исходный код

```
-----
*   build   – сборка: ВИДИМАЯ сортировка точек (и чем с
*             координатная плоскость НАВЕРХУ (ассоциаци
*             чек-лист инвариантов (автопроверка, как у
*   split   – разрез по x с двумя корзинами (грамотная
*   merge   – склейка двух деревьев по y.
*   erase   – удаление ПРОИЗВОЛЬНОЙ точки: найти по x,
*   layers  – миф «дети в массиве по  $2k/2k+1$ »: работает
*   search  – «отсортирую бинарным поиском»: поиск не с
*             сортировка платит  $\geq n \cdot \log n$  сравнений, со
*
* Канонический набор: все x и все y различны  $\Rightarrow$  дерево
* Узел идентифицируется парой  $id = "x:y"$ .
*/
```

```
export interface Pair {
  key: number;
  grade: number;
}
type NodeId = string;
const pairId = (key: number, grade: number): NodeId =>

interface TNode {
  id: NodeId;
  key: number;
  grade: number;
  left: TNode | null;
  right: TNode | null;
}

export const POINTS: Pair[] = [
  { key: 45, grade: 8 },
  { key: 3, grade: 6 },
  { key: 6, grade: 5 },
  { key: 2, grade: 4 },
  { key: 9, grade: 3 },
  { key: 7, grade: 2 },
  { key: 12, grade: 0 },
  { key: 1, grade: -4 },
```

```

}
export function splitTree(root: TNode | null, x: number) {
  const trace: SplitResult["trace"] = [];
  const go = (t: TNode | null): { l: TNode | null; r: TNode | null } {
    if (!t) return { l: null, r: null };
    if (t.key <= x) {
      const sub = go(t.right);
      trace.push({ atId: t.id, goLeft: true, note: `(${t.key} <= ${x})
        о правое поддерево.` });
      t.right = sub.l;
      return { l: t, r: sub.r };
    }
    const sub = go(t.left);
    trace.push({ atId: t.id, goLeft: false, note: `(${t.key} > ${x})
      о левое поддерево.` });
    t.left = sub.r;
    return { l: sub.l, r: t };
  };
  const res = go(root);
  return { l: res.l, r: res.r, trace };
}

export interface MergeResult {
  root: TNode | null;
  trace: { aId: NodeId | null; bId: NodeId | null; chosenId: NodeId | null };
}
export function mergeTree(a: TNode | null, b: TNode | null) {
  const trace: MergeResult["trace"] = [];
  const go = (a: TNode | null, b: TNode | null): TNode | null {
    if (!a || !b) {
      trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, chosenId: null,
        note: `о одно из деревьев целиком.` });
      return a ?? b;
    }
    if (a.grade > b.grade) {
      trace.push({ aId: a.id, bId: b.id, chosenId: a.id,
        note: `выбрано левое поддерево с (${b.key}, ${b.grade})` });
      a.right = go(a.right, b);
    } else {
      trace.push({ aId: a.id, bId: b.id, chosenId: b.id,
        note: `выбрано правое поддерево с (${a.key}, ${a.grade})` });
      b.left = go(a, b.left);
    }
    return { l: go(a.left, b.left), r: go(a.right, b.right) };
  };
  const res = go(a, b);
  return { root: res, trace };
}

```

```

-----
export function checkTreap(root: TNode | null): { bst: boolean; heap: boolean } {
  let bst = true;
  let heap = true;
  let prev: number | null = null;
  const walk = (n: TNode | null, min: number, max: number, parentGrade: number) => {
    if (!n) return;
    if (n.key <= min || n.key > max) bst = false;
    if (parentGrade !== null && n.grade > parentGrade) heap = false;
    walk(n.left, min, n.key, n.grade);
    if (prev !== null && n.key < prev) bst = false;
    prev = n.key;
    walk(n.right, n.key, max, n.grade);
  };
  walk(root, -Infinity, Infinity, null);
  return { bst, heap };
}

```

/\* ----- раскладка сцены дерева ----- \*/

```

const layoutTree = (root: TNode | null) => {
  const pos = new Map<NodeId, { px: number; py: number }>();
  let slot = 0;
  const walk = (n: TNode | null, depth: number) => {
    if (!n) return;
    walk(n.left, depth + 1);
    pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 20 });
    slot += 1;
    walk(n.right, depth + 1);
  };
  walk(root, 0);
  return pos;
};

```

```

const treeEdges = (root: TNode | null): { from: NodeId; to: NodeId }[] => {
  const es: { from: NodeId; to: NodeId }[] = [];
  const walk = (n: TNode | null) => {
    if (!n) return;
    if (n.left) { es.push({ from: n.id, to: n.left.id }); }
    if (n.right) { es.push({ from: n.id, to: n.right.id }); }
    walk(n.left);
    walk(n.right);
  };
  walk(root);
  return es;
};

```

```

        stroke={edgeColor ? edgeColor(e.from, e.to)}
        strokeWidth={2.5}
        strokeDasharray={edgeDash && edgeDash(e.from, e.to)}
        className="transition-all duration-500"
      />
    );
  }
}
}
nodes.map((n) => {
  const p = pairParts(n.id);
  const pulse = pulseId === n.id;
  return (
    <g key={`tn-${n.id}`} transform={`translate(${p[0]}, ${p[1]})`} >
      <circle r={pulse ? 24 : 20} fill={nodeColor(n)}
        > strokeWidth={pulse ? 4 : 2} className="transition-all duration-500" />
      <text y={-2} fill="#fff" fontSize="13" font-weight="bold">{n.id}</text>
      <text y={13} fill="#cbd5e1" fontSize="10">{p[0]} - {p[1]}</text>
    </g>
  );
}
}
}
</svg>
{subtitle && <p className="text-xs text-slate-400">{subtitle}</p>}
</div>
);
};

const pairParts = (id: NodeId): [number, number] => {
  const [a, b] = id.split(":").map(Number);
  return [a, b];
};

/* ----- плоскость (клетчатая бумага) ----- */

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 6) / 8);
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H - 24) / 8);

interface PlaneProps {

```



```

{drawnEdges.map((e, i) => {
  const a = byPair(e.a);
  const b = byPair(e.b);
  const isBad = badEdge && ((badEdge.a.key === e.a.key &&
    badEdge.b.key === e.b.key) || (badEdge.a.key === e.b.key &&
    badEdge.b.key === e.a.key));
  return <line key={`pe-${i}`} x1={a.x} y1={a.y} x2={b.x} y2={b.y}
    strokeDasharray={isBad ? "4,3" : "none"} />
})}

{/* временная линия игры */}
{interactive && selected && cursor && (
  <line
    x1={PX(selected.key)} y1={PY(selected.grade)}
    stroke="#818cf8" strokeWidth={2.5} strokeDasharray={[]} />
)}

{points.map((p) => {
  const done = drawnEdges.some((e) => (e.a.key === p.key && e.b.key === p.key))
  const isSel = selected && selected.key === p.key
  return (
    <g
      key={`pp-${p.key}:${p.grade}`}
      className="transition-all duration-500"
      onPointerDown={interactive && onPointDown ? () => {}}
      onPointerUp={interactive && onPointUp ? () => {}}
      style={interactive ? { cursor: "pointer" } : {}}
    >
      <circle cx={PX(p.key)} cy={PY(p.grade)} r={10}
        stroke={isSel ? "#a5b4fc" : done ? "#38bdf8" : "#ccc"}
        strokeWidth={2} />
      <text x={PX(p.key)} y={PY(p.grade) + 3.5} font-size={12}
        fill="black">{p.key}</text>
    </g>
  );
})}

{pending && (

```

```

    if (seen.has(gpid(cur))) return false;
    seen.add(gpid(cur));
    cur = parentOf(edges, gpid(cur)).a;
  }
  return false;
};

/** Проверка BST леса: у узла ≤1 ребёнка слева и ≤1 справа
export function forestIsBst(edges: GEdge[], all: Pair[]) {
  const kids = new Map<string, Pair[]>();
  for (const e of edges) kids.set(gpid(e.a), [...(kids.get(gpid(e.b)) || []), e]);
  const hasParent = new Set(edges.map((e) => gpid(e.b)));
  const roots = all.filter((p) => !hasParent.has(gpid(p.a)));
  const ok = (n: Pair | undefined, lo: number, hi: number) => {
    if (!n) return true;
    if (n.key <= lo || n.key > hi) return false;
    const ch = (kids.get(gpid(n.a)) ?? []).slice().sort((a, b) => a.key - b.key);
    const left = ch.filter((c) => c.key < n.key);
    const right = ch.filter((c) => c.key > n.key);
    if (left.length > 1 || right.length > 1) return false;
    return ok(left[0], lo, n.key) && ok(right[0], n.key, hi);
  };
  return roots.every((r) => ok(r, -Infinity, Infinity));
}

/** Полный вердикт по текущим линиям. */
export function gameVerdict(edges: GEdge[], all: Pair[]) {
  const heap = edges.every((e) => e.a.grade > e.b.grade);
  const oneParent = new Set(edges.map((e) => gpid(e.b)));
  const acyclic = !edges.some((e) => reaches(edges, e.a, e.b));
  const bst = forestIsBst(edges, all);
  const roots = all.filter((p) => !edges.some((e) => gpid(e.a) === gpid(p.a)));
  const complete = edges.length === all.length - 1 && roots.length === 1;
  return { heap, oneParent, acyclic, bst, complete, win: heap && oneParent && acyclic && bst && complete };
}

/** Попытка хода: почему нельзя – или ОК. */
export function tryConnect(edges: GEdge[], a: Pair, b: Pair) {

```

```

        seen.add(id);
        pairs.push({ key: nums[0], grade: nums[1] });
    }
    const xs = pairs.map((p) => p.key);
    const ys = pairs.map((p) => p.grade);
    return { pairs, errors, equalX: new Set(xs).size !== 1 };
};

const DEMO_RAW = "45 8, 3 6, 6 5, 2 4, 9 3, 7 2, 12 0,

/* ----- игра «соедини мышкой»: чистая логика

/** Ребро игры: a = родитель (выше по y), b = ребёнок.
export interface GameEdge {
    a: Pair;
    b: Pair;
}
const pid = (p: Pair) => `${p.key}:${p.grade}`;

export function canConnect(
    edges: GameEdge[],
    x: Pair,
    y: Pair,
): { ok: boolean; edge?: GameEdge; reason?: string } {
    if (x.key === y.key && x.grade === y.grade) return { ok: true };
    for (const e of edges) {
        if ((pid(e.a) === pid(x) && pid(e.b) === pid(y)) ||
            pid(e.a) === pid(y) && pid(e.b) === pid(x))
            return { ok: false, reason: "такая линия уже проведена" };
    }
    if (x.grade === y.grade) return { ok: false, reason: "не может быть" };
    const [parent, child] = x.grade > y.grade ? [x, y] : [y, x];
    if (edges.some((e) => pid(e.b) === pid(child)))
        return { ok: false, reason: `у точки (${child.key});` };
    // цикл: родитель уже сидит в компоненте ребёнка (ребёнок не может быть своим родителем)
    const childComp = new Set<string>([pid(child)]);
    let grew = true;
    while (grew) {

```

```

    const rb = find(pid(e.b));
    if (ra !== rb) comp.set(ra, rb);
}
const comps = new Set(points.map((p) => find(pid(p))))
const connected = comps.size === 1;
const countOk = edges.length === n - 1;
// BST: дети по x, inorder не убывает; корень один; э
const children = new Map<string, Pair[]>();
for (const e of edges) children.set(pid(e.a), [...(ch
const roots = points.filter((p) => !parentCount.has(p
let bstOk = roots.length === 1;
if (bstOk) {
    const seen = new Set<string>();
    const inorder: number[] = [];
    const walk = (p: Pair): void => {
        if (seen.has(pid(p))) {
            bstOk = false; // цикл
            return;
        }
        seen.add(pid(p));
        const kids = children.get(pid(p)) ?? [];
        if (kids.length > 2) {
            bstOk = false;
            return;
        }
        const sorted = [...kids].sort((u, v) => u.key - v
        let left: Pair | null = null;
        let right: Pair | null = null;
        if (sorted.length === 2) {
            if (sorted[0].key === sorted[1].key || sorted[0
                bstOk = false; // двое детей с равным x или с
                return;
            }
            left = sorted[0];
            right = sorted[1];
        } else if (sorted.length === 1) {
            // одиночный ребёнок: сторона важна – определяет
            if (sorted[0].key <= p.key) left = sorted[0];

```

```

    if (left.length > 1 || right.length > 1) return node;
    if (left[0]) node.left = build(left[0]);
    if (right[0]) node.right = build(right[0]);
    return node;
  };
  return build(rootPair);
}

```

```

const BuildMode: React.FC = () => {
  const [raw, setRaw] = useState(DEMO_RAW);
  const live = useTreapLive();

  // набор точек из Python-кода пересобирает плоскость
  useEffect(() => {
    if (!live.pairsText || live.pairsText === raw) return;
    setRaw(live.pairsText);
    setPhase("connect");
    setOrder("y-desc");
    setSortIdx(3);
    setConnIdx(10000); // все рёбра проведены – ср
    setPlaying(false);
    // eslint-disable-next-line react-hooks/exhaustive-
  }, [live.pairsText]);
  const [phase, setPhase] = useState<BuildPhase>("sort");
  const [algo, setAlgo] = useState<SortAlgo>("none");
  const [order, setOrder] = useState<ConnectOrder>("y-d
  const [sortIdx, setSortIdx] = useState(0);
  const [connIdx, setConnIdx] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [sortCost, setSortCost] = useState<string | nul

  const parsed = useMemo(() => parsePairs(raw), [raw]);
  const userPairs = parsed.errors.length ? [] : parsed.p

  const n = userPairs.length;

  const userTree = useMemo(() => (userPairs.length >= 1
  const canonEdges = useMemo(() => {

```

```

const connEdges = connEdgesAll.slice(0, connIdx);
const connDone = phase === "connect" && n >= 1 && conn

// --- игра «соедини сам» ---
const [gameEdges, setGameEdges] = useState<GameEdge[]>([]);
const [sel, setSel] = useState<Pair | null>(null);
const [cursor, setCursor] = useState<{ x: number; y: number }>({ x: 0, y: 0 });
const [gameMsg, setGameMsg] = useState<string | null>(null);
const [badEdge, setBadEdge] = useState<GameEdge | null>(null);
const [hintUsed, setHintUsed] = useState(false);

const resetGame = () => {
  setGameEdges([]);
  setSel(null);
  setCursor(null);
  setGameMsg(null);
  setBadEdge(null);
  setHintUsed(false);
};

const startGame = () => {
  resetGame();
  setPhase("game");
  setPlaying(false);
};

const verdict = useMemo(() => checkGameState(userPair), [userPair]);
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree(gameEdges) : null), [gameEdges, gameWon]);

const attemptConnect = (x: Pair, y: Pair) => {
  const r = canConnect(gameEdges, x, y);
  if (r.ok && r.edge) {
    setGameEdges((prev) => [...prev, r.edge!]);
    setGameMsg(r.reason ?? null);
    setBadEdge(null);
  } else {
    setGameMsg(r.reason ?? "так нельзя");
    setBadEdge({ a: x, b: y });
  }
};

```

```

return (
  <div className="space-y-5">
    {/* Произвольный список */}
    <div className="bg-slate-900/60 rounded-xl p-4 border border-slate-800">
      <div className="flex flex-wrap items-center justify-between">
        <h4 className="text-sm font-bold text-slate-300">Демо-набор
        <button onClick={() => { setRaw(DEMO_RAW); setRaw(DEMO_RAW); }}
          className="py-1 rounded-lg text-xs font-bold bg-slate-800 text-slate-200">
          демо-набор
        </button>
      </div>
      <textarea
        value={raw}
        onChange={(e) => { setRaw(e.target.value); setRaw(e.target.value); }}
        rows={2}
        className="w-full bg-slate-950 border border-slate-800 border-emerald-500"
        placeholder="3 5, 45 8, 1 -4 ..."
      />
      {parsed.errors.length > 0 && (
        <div className="mt-2 text-xs text-rose-300 bg-slate-900/60 rounded-xl p-4 border border-slate-800">
          {parsed.errors.map((e, i) => <div key={i}>{e.message} </div>)}
        </div>
      )}
      {parsed.errors.length === 0 && (parsed.equalX || parsed.equalY ? (
        <div className="mt-2 text-xs text-amber-300 bg-slate-900/60 rounded-xl p-4 border border-slate-800">
          {parsed.equalY && <div>⚡ Есть <b>равные y</b> у кого-то <b>раньше</b> в списке</div> (сортировка стабильная)
          {parsed.equalX && <div>⚡ Есть <b>равные x</b> у кого-то <b>раньше</b> в списке</div> (сортировка стабильная)
          {!parsed.equalX && !parsed.equalY && <div>⚡ Нет <b>равных</b> в списке</div>}
        </div>
      ) : null}
    </div>
    {/* Координатная плоскость – НАВЕРХУ */}
    <div className="bg-emerald-950/20 rounded-xl p-4 border border-emerald-800">

```

```

        disabled={n < 2}
        className={`px-3 py-1.5 rounded-lg text-
go-600 text-white" : "bg-indigo-500/80 text
    >
        соединить сам
    </button>
  </>
  })
</div>
</div>
<Plane
  points={userPairs}
  drawnEdges={phase === "game" ? gameEdges : ph
  pending={null}
  interactive={phase === "game"}
  selected={sel}
  cursor={cursor}
  badEdge={badEdge}
  onPointDown={pointDown}
  onPointUp={pointUp}
  onMove={planeMove}
/>
<div className="mt-2 text-xs text-slate-300 bg-
  {phase === "sort" ? (
    algo === "none" ? (
      <>Ввод приходит <b>как попало</b> ({n} па
      ть</b>.</>
    ) : sortIdx < 3 ? (
      algo === "quicksort" ? (
        <>⚡ quicksort: счётчик сравнений: <b cl
        {comparisons}</b> (n·log2n).</>
      ) : (
        <> counting: точки в корзины по значени
        , потому что у – маленькие целые.</>
      )
    ) : (
      <>Отсортировано ({sortCost ?? "способ не
      <b>Игру </b></>

```



```

    })
  </ul>
  {phase === "game" && {
    <div className="flex flex-wrap gap-2 mt-3">
      <button onClick={undoEdge} disabled={!gameE
      white disabled:opacity-30">← Убрать линию</
      <button onClick={resetGame} disabled={!gameE
      ate-700 text-white disabled:opacity-30"> С
      <button onClick={showSolution} disabled={ga
      te disabled:opacity-30"> Показать решение<
    </div>
  })
</div>

{/* Победное дерево в игре */}
{phase === "game" && gameTree && {
  <div className="bg-emerald-950/20 rounded-xl p-
  <h4 className="text-sm font-bold text-emerald
  <TreeView root={gameTree} subtitle="Слева-нап
  ществует." />
  </div>
}}

{/* Дерево */}
{order === "y-desc" && userTree && {
  <div className="bg-slate-900/40 rounded-xl p-4
  <h4 className="text-sm font-bold text-slate-3
  <TreeView root={userTree} subtitle="Слева-нап
  </div>
  })
</div>
});
};

/* ===== РЕЖИМ SPLIT ===== */

const SPLIT_KEYS = [2, 3, 6, 7, 9, 12, 45];

```

```

<button onClick={() => setStepIdx(Math.max(0, i
  bg-slate-700 text-white disabled:opacity-30">
<button onClick={() => setStepIdx(Math.min(step
  font-bold bg-indigo-600 text-white disabled:op
<span className="text-xs text-slate-400">шаг {i
</div>

<div className="bg-slate-900/40 rounded-xl p-4 bo
  <h4 className="text-sm font-bold text-slate-300
    , <span className="text-rose-400">розовые → R
  <TreeView
    root={fullTree}
    nodeColor={(id) => (inL.has(id) ? "#1e3a8a" :
    nodeStroke={(id) => (inL.has(id) ? "#60a5fa"
    subtitle={done ? "Каждый узел покрашен ровно
  />
</div>

<div className="text-xs text-slate-300 bg-slate-9
  {steps[idx]?.note}
</div>

{done && (
  <div className="grid grid-cols-1 md:grid-cols-2
    <div className="bg-sky-950/20 rounded-xl p-3 l
      <h5 className="text-sm font-bold text-sky-3
      <TreeView root={lTree} width={480} height={
    </div>
    <div className="bg-rose-950/20 rounded-xl p-3
      <h5 className="text-sm font-bold text-rose-
      <TreeView root={rTree} width={480} height={
    </div>
  </div>
)}

<div className="text-xs text-slate-400 bg-slate-9
  <b>Как это работает на самом деле:</b> идём с
  деревом), отдаётся <b>вместе с веткой</b> – д

```

```

    </div>
    <div className="bg-rose-950/20 rounded-xl p-3 b
      <h5 className="text-sm font-bold text-rose-300
        <TreeView root={r} width={480} height={220} n
      </div>
    </div>

    <div className="text-xs text-slate-300 bg-slate-900
      {cur?.chosenId ? (
        <>⊗ {cur.note}</>
      ) : (
        <>Готово: {done ? "деревья склеены обратно в
          />
        </>
      )}
    </div>

    {done && (
      <div className="bg-emerald-950/20 rounded-xl p-3
        <h5 className="text-sm font-bold text-emerald-300
          <TreeView root={merged.root} subtitle="merge(
        </div>
      )}

    <div className="text-xs text-slate-400 bg-slate-900
      <b>Грамотно про «слияние»:</b> у корней сравни
      в (то, что сохраняет порядок x) отправляется
    </div>
  </div>
);
};

/* ===== РЕЖИМ ERASE ===== */

const EraseMode: React.FC = () => {
  const [key, setKey] = useState(9);
  const [stepIdx, setStepIdx] = useState(0);
  const live = useTreapLive();
  const fullTree = useMemo(() => buildTree(POINTS), [])

```

```

    </div>

    {!done ? (
      <div className="bg-slate-900/40 rounded-xl p-4" >
        <TreeView
          root={fullTree}
          nodeColor={(id) => (pathSet.has(id) ? "#783"
          nodeStroke={(id) => (pathSet.has(id) ? "#f5
          subtitle="Куча сама умеет удалять только вер
        />
      </div>
    ) : (
      <div className="bg-slate-900/40 rounded-xl p-4" >
        <h4 className="text-sm font-bold text-slate-300" >
        <TreeView root={res.root} subtitle="Валидность" >
        <ul className="mt-2 space-y-1 text-sm" >
          <li> обход слева-направо по-прежнему сортир
          <li> каждое ребро вниз по у (куча целая) –
        </ul>
      </div>
    )}

    <div className="text-xs text-slate-300 bg-slate-900/40" >
      {steps[idx]?.note}
    </div>

    <div className="text-xs text-slate-400 bg-slate-900/40" >
      <b>Ответ на «убрать можно только самый верхний»</b>
      ягивает merge двух детей. Удаляется <b>любая</b>
    </div>
  </div>
);
};

/* ===== РЕЖИМ LAYERS (2k/2k+1) =====

const LayersMode: React.FC = () => {
  const [pick, setPick] = useState<"heap" | "treap">("h

```

```

    </div>
    <p className="text-xs text-slate-400">Полное
      →  $2i/2i+1$  попадает в узел.</p>
  </div>
) : (
  <div className="bg-slate-900/40 rounded-xl p-4" >
    <h4 className="text-sm font-bold text-rose-300">
      >
    <TreeView root={full} subtitle="Попробуй «пос
      " />
    <div className="flex gap-2 overflow-x-auto mt-2">
      {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]}
      const id = slots[i];
      return (
        <div key={i} className={`px-2.5 py-2 rounded-
          der-slate-700` : "bg-rose-950/40 border-rose-
            <div className="text-slate-500 text-[0.8em]
              {id ? <div className="text-white font-
                </div>
              </div>
            </div>
          </div>
        </div>
      <p className="text-xs text-rose-300 mt-2">Смо
        то левая ветка ушла на 4 уровня и заняла да
        казателями (left/right), а не арифметикой.<
      </div>
    </div>
  </div>
  <div className="text-xs text-slate-400 bg-slate-900/40 rounded-xl p-4">
    <b>Грабля «дети в массиве по  $2k/2k+1$ »:</b> фор
    ивым: слои дырявые, и «адрес» ребёнка посчита
  </div>
</div>
);
};

/* ===== РЕЖИМ SEARCH (поиск ≠ сортировка) =====

```

```

<div className="space-y-4">
  <div className="flex gap-2">
    {stage < 2 && (
      <button onClick={() => { setStage(stage === 0
        digo-600 text-white">
        {stage === 0 ? "Найти у для x=7 бинарным по
      </button>
    )}
    {stage === 2 && (
      <button onClick={() => setBidx(Math.min(bsSte
        -1.5 rounded-lg text-xs font-bold bg-indigo
        Шаг поиска →
      </button>
    )}
    <button onClick={() => { setStage(0); setBidx(0
      C6poc</button>
  </div>

```

```

<div className="flex gap-1.5 overflow-x-auto">
  {(stage === 2 ? sortedByKey : shuffled).map((p,
    const highlight = stage === 1 && i === wrongM
    const inBs = stage === 2 && (() => { const s
    const isMid = stage === 2 && (() => { const s
    const isFound = stage === 2 && bidx >= bsStep
    return (
      <div key={` ${p.key}:${p.grade}-${i}`} class=
        nsition-all ${highlight ? "border-rose-500 l
        r-amber-500 bg-amber-950/30" : inBs ? "bord
        <div className="text-white font-bold">{p.
        <div className="text-slate-400 text-[10px
      </div>
    );
  )}}
</div>

```

```

<div className="text-xs text-slate-300 bg-slate-9
  {stage === 0 && <>Массив пар <b>не отсортирован
    а беспорядок.</>}

```

```

        return;
    }
    const res = tryConnect(edges, sel, p);
    if (res.ok) {
        const parent = sel.grade > p.grade ? sel : p;
        const child = sel.grade > p.grade ? p : sel;
        setEdges([...edges, { a: parent, b: child }]);
        setMsg(` Линия (${parent.key}; ${parent.grade})`);
        setSel(null);
    } else {
        setMsg("x " + res.reason);
        setSel(null);
    }
};

const win = v.win;

return (
    <div className="space-y-4">
        <div className="flex flex-wrap items-center gap-2">
            <button onClick={() => { setEdges(edges.slice(0,
                rounded-lg text-xs font-bold bg-slate-700 tex
            <button onClick={() => { setEdges([]); setSel(n
                3 py-1.5 rounded-lg text-xs font-bold bg-slate
            <button onClick={() => { setEdges(CANON_GAME.ma
                ); }} className="px-3 py-1.5 rounded-lg text-
            <span className="text-xs text-slate-400">линий:
        </div>

        <div className="bg-emerald-950/20 rounded-xl p-4 l
            <svg viewBox={`0 0 ${PL_W} ${PL_H + 20}`} class
                <defs>
                    <pattern id="gm-grid" width="14" height="14"
                        <path d="M 14 0 L 0 0 0 14" fill="none" s
                    </path>
                </defs>
                <rect x="34" y="16" width={PL_W - 46} height=
                <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={P

```

```

        <li>{v.hear ? " " : " "} каждое ребро – вниз
        <li>{v.oneParent ? " " : " "} у каждой точки
        <li>{v.acyclic ? " " : " "} циклов нет</li>
        <li>{v.bst ? " " : " "} обход слева-направо –
        <li>{v.complete ? " " : " "} линий  $n-1 = \{POI$ 
    </ul>
    {win && {
        <p className="mt-3 text-sm font-bold text-eme
            ения просто не существует.</p>
        })
    </div>

    <div className="text-xs text-slate-300 bg-slate-9
</div>
    );
};

/* ===== СВЯЗЬ С КОМПИЛЯТОРОМ =====

/**
 * Живые переменные Python-кода страницы treap:
 * points / pairs – набор пар (x; y): по нему ПЕРЕСТРА
 * mid             – разрез для вкладки Split;
 * key             – удаляемая точка для вкладки Erase;
 * step / i        – номер шага (листаёт кадры Split /
 * height, n       – высота дерева и число узлов (показ
 */
const useTreapLive = () => {
    const runtime = useVizRuntime();
    const vars = runtime?.variables;

    const pairsText = useMemo(() => {
        if (!vars) return null;
        const rawPts = vizArray(vars.points) ?? vizArray(va
        if (!rawPts || rawPts.length === 0) return null;
        const parts: string[] = [];
        rawPts.forEach((it) => {
            if (Array.isArray(it) && it.length >= 2) {

```



```

// вкладка = подрежим страницы: панель Python подстав.
useEffect(() => {
  if (chapterId) emitVizDemo(chapterId, tab);
}, [chapterId, tab]);

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border">
    <div className="flex flex-wrap items-center justify">
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-emerald-600 text-white">
          <span className="text-2xl"> </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white">
            <p className="text-sm text-emerald-300">Пар.
          </p>
        </div>
      </div>
    </div>

    <div className="flex flex-wrap gap-2 mb-5">
      {TABS.map((t) => (
        <button
          key={t.id}
          onClick={() => setTab(t.id)}
          className={`px-4 py-2 rounded-lg text-xs font-
            900/60 text-slate-400 hover:text-white border
          >
            {t.label}
          </button>
        )))}
    </div>

    {/* что сейчас сообщил компилятор */}
    <div className="mb-4 flex flex-wrap items-center">
      <span className="text-[10px] uppercase tracking-
        <Terminal className="w-3.5 h-3.5" /> из компи.
      </span>
      {live.vars ? (

```

```
import { liveMatrix, pickLiveVar, useVizRuntime } from
import { VizSourceBadge } from "../VizSourceBadge";
```

```
interface VizLiveStatusProps {
  /** Название демонстрации – подписывается в режиме вс
  demoTitle?: string;
  className?: string;
}
```

```
/** Имена, по которым визуализации обычно читают матрицу
const MATRIX_NAMES = ["d", "dist", "dp", "D", "S", "P",
```

```
/**
 * Строка состояния связи «компилятор → демонстрация» д
 *
 * Раньше компонент молча проигрывал собственный встроен
 * редакторе Python был пустой код, код с другими именам
 * размерности, – выглядело так, будто компилятор исполн
 * картинка живёт отдельной жизнью. Теперь источник данн
 * «данные из компилятора (i, j, dist 5x5)», «компилятор
 * пока нет» (отладчик стоит до первой строки) или «встр
 */
```

```
export function VizLiveStatus({ demoTitle, className =
  const runtime = useVizRuntime();
  const vars = runtime?.variables;
  const names = vars ? Object.keys(vars).filter((name) :
  const linked = names.length > 0;
```

```
const matrixSource = pickLiveVar(vars, MATRIX_NAMES);
const matrix = liveMatrix(matrixSource?.value);
```

```
const detail = matrix
  ? `${matrixSource?.name} ${matrix.sizeLabel}`
  : names.length
    ? names.slice(0, 6).join(", ") + (names.length >
    : undefined;
```

```
-----
import { SalmonAutomatonWidget } from "../SalmonAutomaton";
import { SegmentTreeVisualizer } from "../SegmentTreeVis";
import SplayPlayground from "../SplayPlayground";
import { StackViz } from "../StackViz";
import { StringAlgorithmsViz } from "../StringAlgorithms";
import { TopologicalSortViz } from "../TopologicalSortVi";
import { WaterfallAnimationWidget } from "../WaterfallAn";
import { Simulator } from "../Simulator";
import { PrefixSumViz } from "../visualizers/PrefixSumVi";
import { SparseTableViz } from "../visualizers/SparseTab";
import { TreapBuildViz } from "../TreapBuildViz";
```

```
export interface VizEntry {
  /** Заголовок панели/модалки. */
  title: string;
  /** Короткое пояснение, что здесь можно потыкать. */
  hint?: string;
  Component: React.FC;
}
```

```
/**
```

```
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающ
 * и для модального окна «Открыть симулятор».
 */
```

```
export const VIZ_REGISTRY: Record<string, VizEntry> = {
  "stack-dfs": {
    title: "Стек и обход в глубину",
    hint: "Кладите и снимайте тарелки — это ровно то, ч
    Component: StackViz,
  },
  "queue-bfs": {
    title: "Очередь и обход в ширину",
    hint: "Очередь слева, живой BFS по графу справа.",
    Component: () => (
      <div className="space-y-10">
        <QueueViz />
        <BfsViz />
      </div>
    )
  },
}
```

```

    Component: SplayPlayground,
  },
  "graph-dfs-bfs": { title: "DFS и BFS на графе", Component: SplayPlayground,
  "top-sort": { title: "Топологическая сортировка", Component: SplayPlayground,
  "scc-kosaraju": { title: "Компоненты сильной связности", Component: SplayPlayground,
  "graph-articulation": { title: "Точки сочленения", Component: SplayPlayground,
  "bridges-code": { title: "Мосты: DFS + tin/low", Component: SplayPlayground,
  "euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: SplayPlayground,
  "planarity-euler-formula": {
    title: "Планарность и покраска:  $K_5/K_{3,3}$ ,  $\chi \leq \Delta+1$ , дельта-краска,
    hint: "Четыре вкладки: непланарные  $K_5$  и  $K_{3,3}$ , жадная покраска,
    ашего кода красит вершины напрямую.",
    Component: PlanarColoringViz,
  },
  dijkstra: {
    title: "Дейкстра",
    hint: "Жадно забираем ближайшую вершину и релаксируем соседей.",
    Component: DijkstraViz,
  },
  "bellman-ford": {
    title: "Форд–Беллман",
    Component: BellmanFordViz,
  },
  floyd: {
    title: "Флойд–Уоршелл",
    Component: FloydViz,
  },
  "johnson-algo": { title: "Алгоритм Джонсона", Component: JohnsonViz,
  "mst": {
    title: "Остовное дерево: Краскал, Прима, Борувка",
    hint: "Три алгоритма (билеты 18–20) на одном графе. Переменные parent/mst/taken/u/v/w перекрашивают граф.",
    Component: KruskalSimulator,
  },
  "graph-components": {
    title: "Компоненты связности: обход и DSU",
    hint: "Две вкладки: покраска обходом (каждый новый компонент из вашего кода красят вершины.",
  },

```

```
import { Cpu, Film } from "lucide-react";
import { Tooltip } from "../Tooltip";

interface VizSourceBadgeProps {
  /**
   * true – картинку рисуют переменные из выполненного Python-кода
   * false – компонент показывает собственную встроенную картинку
   */
  linked: boolean;
  /**
   * Компилятор уже выполнил код и прислал шаг, но переключатель не нажат
   * (отладчик стоит ДО первой строки или в коде другие комментарии)
   * подпись должна объяснять, почему картинка ещё не инициализирована
   */
  awaiting?: boolean;
  /** Строка пользовательского кода, на которой стоит отладчик */
  line?: number;
  /** Короткая подпись данных компилятора: «матрица 5×5» */
  detail?: string;
  /** Как называется встроенная демонстрация (для подписи) */
  demoTitle?: string;
  /** Размеры демо-сетки и данных пользователя не совпадают */
  mismatch?: string | null;
  className?: string;
}

/**
 * Явный индикатор источника данных визуализации.
 *
 * Без него демонстрация молча проигрывала собственный Python-код
 * когда редактор Python пуст или в нём другие имена/переменные
 * так, будто компилятор «исполняет какой-то свой код».
 * «из компилятора» или «встроенное демо».
 */
export function VizSourceBadge({ linked, awaiting = false, line, detail, demoTitle, mismatch, className }) {
  const tip = linked
    ? `Значения берутся из выполненного Python-кода${detail ? `: ${detail}` : ''}`
    : `Встроенная демонстрация: ${demoTitle}`;
  return (
    

{awaiting ?  : }
      

{detail}
        {demoTitle}


```

```
        0"
      >
        размерность не совпадает
      </span>
    </Tooltip>
  ) : null}
</div>
);
}
```

---

ФАЙЛ 82/111: src/components/WaterfallAnimationWidget.ts

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useContext, useEffect, useMemo, useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from 'viz';
```

```
// Структура заранее подготовленного красивого бора для
// Словарь: ["HE", "SHE", "HIS", "HERS"]
```

```
interface WNode {
  id: number;
  label: string; // путь от корня
  char: string;
  x: number; // Координаты для SVG водопада
  y: number;
  depth: number;
  fail: number;
  term_link: number;
  is_terminal: boolean;
  words: string[];
}
```

```
const WATERFALL_NODES: { [key: number]: WNode } = {
  0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60,
    // Ветка H -> E -> R -> S
```

```

useEffect(() => {
  if (chapterId) emitVizDemo(chapterId, "aho");
}, [chapterId]);

const liveText = vars ? vizString(vars.text) ?? vizSt
const liveState = vars ? vizNumber(vars.state) ?? viz
const liveI = vars ? vizNumber(vars.i) ?? vizNumber(v
const liveMode = vars ? vizString(vars.mode) : null;
const liveMatchesRaw = vars ? vizArray(vars.matches)
const liveMatchesSig = useMemo(() => (liveMatchesRaw
const liveFail = vars ? vars.fail : undefined;

useEffect(() => {
  if (!vars) return;
  if (liveMode && /search|поиск/i.test(liveMode)) set
  else if (liveMode && /train|обуч|bfs/i.test(liveMod
  if (liveText) {
    const clean = liveText.toUpperCase().replace(/[^A
    if (clean && clean !== textToScan) {
      setTextToScan(clean);
      setCurrentIndex(0);
      setCurrentNode(0);
      setFoundMatches([]);
      setIsSearchDone(false);
      setJumpPath(null);
      setActiveMode("search");
    }
  }
  if (liveState !== null && liveState >= 0 && WATERFA
  if (liveI !== null && liveI >= 0) setCurrentIndex(M
    ToScan).length));
  if (liveMatchesSig) {
    const parsed = (JSON.parse(liveMatchesSig) as unk
      .map((it) => {
        if (Array.isArray(it) && it.length >= 2) {
          const idx = vizNumber(it[0]);
          const word = vizString(it[1]);

```

```

const resetSearchSimulation = () => {
  setCurrentIndex(0);
  setCurrentNode(0);
  setIsSearchDone(false);
  setSearchLog('Симуляция сброшена. Лосось вернулся на старт');
  setJumpPath(null);
  setFoundMatches([]);
  setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
  if (currentIndex >= textToScan.length) {
    setIsSearchDone(true);
    setSearchLog('Мы проплыли весь текст! Лосось доволен');
    setActiveSearchCodeLine(4);
    return;
  }

  const char = textToScan[currentIndex];
  let curr = currentNode;
  let logMsg = `[Символ '${char}']`;

  let jumpType: 'normal' | 'fail' | null = null;
  void jumpType;
  let originalCurr = curr;

  if (TRIE_TRANSITIONS[curr][char] !== undefined) {
    const nextNode = TRIE_TRANSITIONS[curr][char];
    logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label})`;
    setJumpPath({ from: curr, to: nextNode, type: 'normal' });
    curr = nextNode;
    setActiveSearchCodeLine(3);
  } else {
    setActiveSearchCodeLine(2);
    let jumps = 0;
    while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
      curr = WATERFALL_NODES[curr].fail;
    }
  }
};

```



```

        setFoundMatches((prev) => [...prev, ...currentMat
    }
};

```

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В

```

const TRAINING_STEPS_INFO = [
  {
    targetNodeId: 0,
    title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
    desc: 'Начало алгоритма BFS. Корень (ROOT) не обра
      рние вершины сразу инициализируются с fail = RO
    codeLine: 1,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'ø',
    checkingBranchesFrom: null,
  },
  {
    targetNodeId: 1,
    title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
    desc: 'Мы обрабатываем первую вершину из очереди -
      просто не существует. fail[H] автоматически нап
    codeLine: 5,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'H',
    checkingBranchesFrom: null,
  },
  {
    targetNodeId: 7,
    title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
    desc: 'Обрабатываем дочернюю вершину #7 (S) на De
      ,
    codeLine: 5,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'S',
    checkingBranchesFrom: null,
  },

```

```

desc: 'Переходим на Depth 3 к #3 (HER). Родитель
    е подходят. fail[HER] = ROOT.',
codeLine: 3,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'R',
checkingBranchesFrom: 0,
},
{
targetNodeId: 6,
title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
    но ветка "S" → #7 (S) совпадает! fail[HIS] = #7',
codeLine: 4,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'S',
checkingBranchesFrom: 0,
},
{
targetNodeId: 9,
title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С
desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
    ба плывет в #1 (H) и ищет ветку "E". У вершины
codeLine: 4,
scoutPos: 1,
inspectedParentFail: 1,
checkTargetChar: 'E',
checkingBranchesFrom: 1,
},
{
targetNodeId: 4,
title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро
    #7 (S). fail[HERS] = #7 (S). Автомат полностью
codeLine: 4,
scoutPos: 0,
inspectedParentFail: 0,

```

```

        <button
          onClick={() => setActiveMode('search')}
          className={`flex items-center space-x-1.5 p-2
            ${activeMode === 'search'
              ? 'bg-blue-600 text-white shadow-lg shadow-blue-500'
              : 'text-slate-400 hover:text-white'}
          `}
        >
        <Play className="w-4 h-4" />
        <span>Режим 2: Поиск в тексте</span>
      </button>
    </div>
  </div>

  { /* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */ }
  { activeMode === 'training' ? (
    /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
    <div className="grid grid-cols-1 lg:grid-cols-3 gap-4" >
      { /* Управление шагами обучения */ }
      <div className="bg-slate-900/90 p-5 rounded-xl" >
        <div>
          <h5 className="text-indigo-300 font-bold" >
            <span> Полный обход в ширину (BFS) </span>
          </h5>
          <p className="text-xs text-slate-300 mb-3" >
            Показаны все шаги очереди BFS (начиная с
            аргумента 8 (SHE → HE)</strong>, где наглядно видно
          </p>
          <div className="space-y-1.5 mb-6 overflow-hidden" >
            { TRAINING_STEPS_INFO.map((step, idx) => (
              <button
                key={idx}
                onClick={() => setTrainStep(idx)}
                className={`w-full text-left px-3 py-2
                  ${
                    idx === trainStep
                      ? 'bg-indigo-600 text-white font-medium'
                      : 'bg-slate-800 text-slate-400'
                  }
                `}
              >

```

```

Вершина: #{targetTrainingNode.id} ({t
</span>
</h5>
<div className="bg-slate-950 p-4 rounded-
  <div className={`p-1.5 rounded flex ite
-l-4 border-indigo-500 text-indigo-300 font
    <span>1. let u = trie[r][char]; // Те
    {currentTrainInfo.codeLine === 1 && <
тут</span>}
  </div>
  <div className={`p-1.5 rounded flex ite
-l-4 border-indigo-400 text-indigo-200 font
    <span>2. let v = fail[r]; // Подъем к
ainInfo.inspectedParentFail].label}}</span>
    {currentTrainInfo.codeLine === 2 && <
к родителю</span>}
  </div>
  <div className={`p-1.5 rounded flex ite
l-4 border-amber-500 text-amber-300 font-bo
    <span>3. while (v !== root && trie[v]
    {currentTrainInfo.codeLine === 3 && <
т, возврат в корень</span>}
  </div>
  <div className={`p-1.5 rounded flex ite
r-l-4 border-emerald-500 text-emerald-300 f
    <span>4. if (trie[v][char] !== undefin
/span>
    {currentTrainInfo.codeLine === 4 && <
ан IF!</span>}
  </div>
  <div className={`p-1.5 rounded flex ite
-l-4 border-blue-500 text-blue-300 font-bold'
    <span>5. else fail[u] = root; // Bero
    {currentTrainInfo.codeLine === 5 && <
root</span>}
  </div>
</div>
</div>
</div>

```

```

        <span
          key={idx}
          className={`w-7 h-8 flex items-center
            idx === currentIndex
              ? 'bg-blue-600 text-white shadow
              : idx < currentIndex
              ? 'bg-emerald-950/60 text-emera
              : 'bg-slate-800 text-slate-500'
            }`
        >
          {char}
        </span>
      )}
    {currentIndex >= textToScan.length && (
      <span className="bg-emerald-600 text-
        ГОТОВО ✓
      </span>
    )}
  </div>

  <button
    onClick={stepSearchSimulation}
    disabled={isSearchDone || textToScan.len
    className={`w-full py-2.5 rounded-lg fo
      isSearchDone || textToScan.length ===
        ? 'bg-slate-700 text-slate-500 curs
        : 'bg-gradient-to-r from-blue-600 t
ue-900/40 active:scale-[0.98]`
      }`
    >
      <span>{currentIndex === 0 ? 'Начать пла
      <ArrowRight className="w-4 h-4" />
    </button>
  </div>
</div>

<div className="lg:col-span-2 bg-slate-900/60
  <div>

```



```

        return (
            <g key={`edge-${fromNode.id}-${toNode.id}` /* Линия течения воды */>
                <line
                    x1={fromNode.x}
                    y1={fromNode.y}
                    x2={toNode.x}
                    y2={toNode.y}
                    stroke={isHighlighted ? '#3b82f6' : '#ccc'}
                    strokeWidth={isHighlighted ? 2 : 1}
                    className={isHighlighted ? 'highlighted' : 'normal'}
                />
                /* Символ перехода */
                <circle cx={{fromNode.x + toNode.x} / 2} cy={{fromNode.y + toNode.y} / 2}
                    r={10} fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5f') : '#ccc'}
                    stroke={isHighlighted ? '#3b82f6' : '#ccc'} strokeWidth={isHighlighted ? 2 : 1}
                />
                <text
                    x={{fromNode.x + toNode.x} / 2}
                    y={{fromNode.y + toNode.y} / 2 + 15}
                    fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5f') : '#ccc'}
                    fontSize="12"
                    fontWeight="bold"
                    textAnchor="middle"
                />
                {char}
            </text>

            /* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В ПРАВИЛЬН. ОТВЕТЕ */
            {isTrainingExamined && (
                <g transform={`translate(${(fromNode.x + toNode.x) / 2}, ${
                    (fromNode.y + toNode.y) / 2})`}
                    <rect x="-15" y="-12" width="30" height="24"
                        stroke={isHighlighted ? '#3b82f6' : '#ccc'}
                        strokeWidth="2" />
                    <text x="0" y="5" fontSize="14"
                        fill={isMatchBranch ? '#10b981' : '#f43f5f'}
                    />
                </g>
            )}
        )
    )
}

```

```

        <text x={cx} y={cy - 10} fill="#10b"
    >
        fail[{node.label}] = {targetNode.
    </text>
    })
</g>
);
}}}
```

```

{/* Рендер вершин (бассейнов водопада) */}
{Object.values(WATERFALL_NODES).map((node) => {
    const isCurrent = activeFishPosNode.id === node.id;
    const isTargetChild = activeMode === 'transition';
    const hasWords = node.words.length > 0;
```

```

    return (
        <g key={`node-${node.id}`} className="node"
        {/* Внешнее свечение для активной вершины */}
        {isCurrent && {
            <circle cx={node.x} cy={node.y} r={node.radius}
            fill="#fff" stroke="#000" stroke-width={2}
            opacity={0.5}
        }}
        {isTargetChild && {
            <circle cx={node.x} cy={node.y} r={node.radius}
            fill="#fff" stroke="#000" stroke-width={2}
            opacity={0.5}
        }}
    )
```

```

    {/* Основной круг вершины */}
    <circle
        cx={node.x}
        cy={node.y}
        r={isCurrent || isTargetChild ? '24px' : node.radius}
        fill={isCurrent ? '#2563eb' : isTargetChild ? '#f8d7da' : node.fill}
        stroke={isCurrent ? '#ffffff' : isTargetChild ? '#000000' : node.stroke}
        strokeWidth={isCurrent || isTargetChild ? 2 : node.strokeWidth}
        className="transition-all duration-300"
    />
```

```

    {/* Текст внутри вершины */}
    <text
        x={node.x}
        y={node.y - 10}
        fill={node.fill}
        font-size={node.fontSize}
        font-weight={node.fontWeight}
    />
```



```

        cy={activeFishPosNode.y - 25}
        r="18"
        fill={activeMode === 'training' ? '#8b5'
        stroke="#ffffff"
        strokeWidth="2"
        style={{ transition: 'cx 0.8s cubic-bez
        className="shadow-lg"
    />
    {/* Сама рыба */}
    <text
        x={activeFishPosNode.x + 15}
        y={activeFishPosNode.y - 19}
        fontSize="18"
        textAnchor="middle"
        style={{ transition: 'x 0.8s cubic-bezi
        className="animate-float select-none"
    >
        {activeMode === 'training' ? ' ♂ ' :
    </text>
    </g>

    </svg>
    </div>
    </div>

    {/* Список найденных совпадений (только в режиме
    {activeMode === 'search' && (
        <div className="mt-6 bg-slate-900/50 p-5 rounded
            <h5 className="text-white font-bold mb-3 text
                <span> </span> Улов лосося (Найденные слова
            </h5>
            {foundMatches.length === 0 ? (
                <p className="text-sm text-slate-400 italic
                    Пока ничего не поймали. Запустите шаги си
                </p>
            ) : (
                <div className="flex flex-wrap gap-2">
                    {foundMatches.map((match, idx) => (

```

```
        ) : (  
            <span className="text-[11px] text-slate-500">  
                открой панель Python и запусти Ахо–Корасик:  
                "font-mono text-cyan-300">state</code>, <code  
                an-300">matches</code> поведут лосося по эт  
            </span>  
        )}  
    </div>  
</div>  
);  
};
```

## РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 83/111: src/components/visualizers/PrefixSumViz.tsx  
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 466 | РАЗМЕР: 10.5 КБ

```
import React, { useContext, useEffect, useMemo, useState } from 'react';  
import {  
    emitVizDemo,  
    liveMatrix,  
    pickLiveVar,  
    useVizRuntime,  
    vizArray,  
    vizNumber,  
    VizChapterContext,  
} from '../../../data/vizStepBus';
```

```
/**
```

```
 * Префиксные суммы: 1D и 2D.
```

```
 *
```

```
 * Главная фишка – наведение на число:
```

```
 *   * в 1D наводим на P[i] – подсвечивается кусок массива
```

```

        onClick={() => setMode(key)}
        className={`px-3 py-2 rounded-lg text-xs sm
            mode === key ? "bg-indigo-600 text-white"
        }`}
    >
        {label}
    </button>
    )))
</div>
{mode === "1d" ? <Prefix1D /> : <Prefix2D />}
</div>
);
};

/* ----- 1D -----
const Prefix1D: React.FC = () => {
    const pref = useMemo(
        () => A1.reduce<number[]>((acc, v) => [...acc, acc[
        []
    );

    /** Что сейчас под курсором: префикс P[i] или элемент
    const [hover, setHover] = useState<{ kind: "pref" | "
    const [range, setRange] = useState<{ l: number; r: num
    /** Снимок i/P из автоматически выполненного Python-к
    const runtime = useVizRuntime();
    const vars = runtime?.variables;
    const liveI = vizNumber(vars?.i);
    const liveP = vizArray(vars?.P);
    const hasLiveP = !!vars && Object.prototype.hasOwnProperty
    const compilerLinked = liveI !== null || hasLiveP;
    // i/P управляют ячейками напрямую; индекс Python-стр
    const shownDriven = compilerLinked
        ? liveI !== null
            ? Math.max(0, Math.min(A1.length, Math.trunc(live
            : Math.max(0, Math.min(A1.length, (liveP?.length
        : null;

```

```

        : inCover
        ? "bg-indigo-500 text-white ring-indigo-500"
        : inRange
        ? "bg-indigo-900/70 text-indigo-100 ring-indigo-500"
        : "bg-slate-800 text-slate-300 ring-slate-800"
      }`}
      title={`a[${i}] = ${v}`}
    >
      {v}
    </div>
  );
}}
</div>

```

```

{/* массив P */}
<div className="flex gap-1.5 items-center">
  <div className="w-[46px] shrink-0 text-right">
    {pref.map((v, i) => {
      const active = hover?.kind === "pref" && i === range.l;
      const isL = i === range.l;
      const isR = i === range.r + 1;
      const liveValue = liveP?.[i];
      const blank = hasLiveP && (liveValue === null || liveValue === 0);
      const masked = !hasLiveP && shownDriven !== null;
      const isDrivenCur = shownDriven !== null && i === shownDriven;
      const shownValue = hasLiveP ? liveValue : null;
      return (
        <div
          key={i}
          onMouseEnter={() => setHover({ kind: "pref", i })}
          onMouseLeave={() => setHover(null)}
          className={`${cellBase} cursor-help ${
            isDrivenCur
              ? "bg-emerald-600 text-white ring-emerald-600"
              : blank
              ? "bg-slate-950 text-slate-600 ring-slate-950"
              : masked
              ? "bg-slate-800/50 text-slate-700 ring-slate-800/50"
            }`}
        >

```

```

        </b>{ " " }
        – это сумма всего, что набежало от начала
        <span className="font-mono text-indigo-300">
            a[0..{hover.i - 1}] = {A1.slice(0, hover.i)}
        </span>
    </p>
    )
    ) : hover?.kind === "a" ? (
        <p className="text-slate-300">
            <b className="text-indigo-400">
                a[{hover.i}] = {A1[hover.i]}
            </b>{ " " }
            входит во все префиксы, начиная с{ " " }
            <span className="font-mono text-emerald-300">
                подвинуть границу запроса.
            </span>
        </p>
    ) : (
        <p className="text-slate-500">
            Наведите курсор на любое число: у <b className="text-indigo-400">у</b>
            у <b className="text-indigo-400">a</b> – ку,
        </p>
    )
</div>

{/* Запрос */}
<div className="bg-slate-900 border border-indigo-400">
    <div className="text-xs text-slate-400 mb-2">
        Запрос суммы на отрезке (кликайте по массиву)
    </div>
    <p className="font-mono text-sm sm:text-base text-slate-300">
        sum(a[{range.l}..{range.r}]) = P[{range.r + 1}..{range.l}]
        <span className="text-emerald-400">{pref[range.r + 1]}
        <span className="text-rose-400">{pref[range.l]}
        <span className="text-indigo-300 font-bold">{
    </p>
    <p className="text-[11px] text-slate-500 mt-1">
        Один вычитание вместо цикла: запрос за O(1) по префиксам
    </p>

```

```

const liveArrSource = pickLiveVar(vars, ["a", "A", "g"]);
const liveA = liveMatrix(liveArrSource?.value);

const prefRows = liveS?.rows ?? P.length;
const prefCols = liveS?.cols ?? (P[0]?.length ?? 0);
const grid: (number | null)[][] = liveS
  ? Array.from({ length: prefRows }, (_, i) =>
    Array.from({ length: prefCols }, (_, j) => liveS[i][j]
  )
  : P;

const srcRows = liveA?.rows ?? n;
const srcCols = liveA?.cols ?? m;
const srcGrid: (number | null)[][] = liveA
  ? Array.from({ length: srcRows }, (_, i) => Array.from({ length: srcCols }, (_, j) => liveA[i][j]
  )
  : A2;

const cellAt = (i: number, j: number): number | null {
  if (i >= 0 && j >= 0 && i < prefRows && j < prefCols) return grid[i][j];
  return null;
}

const sizeMismatch =
  hasLiveS && liveS && (liveS.rows !== P.length || liveS.cols !== P[0].length)
  ? `Демо построено на матрице ${n}x${m} (сетка преобразована из ${liveS.sizeLabel}). Ниже нарисована таблица`
  : null;

useEffect(() => {
  const r1 = vizNumber(vars?.r1);
  const c1 = vizNumber(vars?.c1);
  const r2 = vizNumber(vars?.r2);
  const c2 = vizNumber(vars?.c2);
  if (r1 === null || c1 === null || r2 === null || c2 === null) return;
  setRect({
    r1: Math.max(0, Math.min(srcRows - 1, Math.trunc(r1))),
    c1: Math.max(0, Math.min(srcCols - 1, Math.trunc(c1))),
    r2: Math.max(0, Math.min(srcRows - 1, Math.trunc(r2))),
    c2: Math.max(0, Math.min(srcCols - 1, Math.trunc(c2))),
  });
});

```

```

{srcGrid.map((row, i) => {
  <div key={i} className="flex gap-1.5 mb-1"
    {row.map((v, j) => {
      const inRect = i >= r1 && i <= r2 &&
      return {
        <div
          key={j}
          onClick={() =>
            setRect((r) =>
              i < r.r1 || j < r.c1
                ? { r1: i, c1: j, r2: Math.max(r.r2, j), c2: Math.max(r.c2, i) }
                : { ...r, r2: i, c2: j }
            )
          }
          className={` ${cellBase} cursor-pointer ${
            inRect ? "bg-indigo-500 text-white" : ""
          }`}
        >
          {v === null ? "." : v}
        </div>
      }
    )}
  </div>
});
}}
</div>
)}}
</div>
</div>

```

```

{/* Матрица префиксных сумм */}
<div className="overflow-x-auto">
  <div className="text-xs text-slate-400 mb-2">
    Префиксные суммы {liveS ? `${livePrefSource}` : ""}
  </div>
  <div className="inline-block">
    {grid.map((row, i) => {
      <div key={i} className="flex gap-1.5 mb-1"
        {row.map((v, j) => {
          const blank = v === null || v === undefined;
          const shownValue = v;

```

```

        }}}
      </div>
    </div>
  </div>

  <div className="min-h-[54px] bg-slate-900 border border-indigo-500"
    {hover ? (
      <p className="text-slate-300">
        <b className="text-amber-400">
          {liveS ? livePrefSource?.name ?? "S" : "P"}
        </b>{" "}
        – сумма всего прямоугольника от левого верхнего угла
      </p>
    ) : (
      <p className="text-slate-500">Наведите курсор на элемент</p>
    )}
  </div>

  <div className="bg-slate-900 border border-indigo-500"
    <p className="font-mono text-sm sm:text-base text-slate-300">
      сумма = <span className="text-emerald-400">A</span> *
      <span className="text-rose-400">C</span> + <span className="text-emerald-400">B</span> *
      <span className="text-rose-400">D</span>
      = {" "}
      <span className="text-indigo-300 font-bold">{
        <math>({r1}, {c1}) \dots ({r2}, {c2}) \cdot c</math>
      </span>
    </p>
    <p className="font-mono text-[11px] text-slate-300">
      прямоугольник ({r1}, {c1}) ... ({r2}, {c2}) · c
    </p>
    <p className="text-[11px] text-slate-500 mt-1">
      Вычли два «хвоста» и вернули дважды вычитенный
    </p>
  </div>
</div>
);
};

```



```

const val2D = [
  [45, 12, 88, 34, 11, 76, 23, 90],
  [67, 19, 44, 55, 33, 21, 65, 87],
  [14, 51, 99, 13, 22, 64, 43, 76],
  [89, 32, 54, 71, 15, 88, 29, 60],
  [25, 41, 16, 92, 9, 17, 56, 31],
  [59, 18, 77, 24, 61, 82, 35, 12],
  [73, 8, 38, 85, 47, 95, 19, 58],
  [39, 81, 62, 28, 51, 42, 85, 14]
];

const ST2D: number[][][] = Array.from({length: 8}, () =>
  Array.from({length: 8}, () =>
    Array.from({length: 4}, () =>
      Array(4).fill(0)
    )
  )
);

for (let r = 0; r < 8; r++) {
  for (let c = 0; c < 8; c++) {
    ST2D[r][c][0][0] = val2D[r][c];
  }
}

for (let kx = 0; kx <= 3; kx++) {
  for (let ky = 0; ky <= 3; ky++) {
    if (kx === 0 && ky === 0) continue;
    for (let r = 0; r + (1 << kx) <= 8; r++) {
      for (let c = 0; c + (1 << ky) <= 8; c++) {
        if (kx > 0) {
          ST2D[r][c][kx][ky] = Math.min(
            ST2D[r][c][kx-1][ky],
            ST2D[r + (1 << (kx-1))][c][kx-1][ky]
          );
        } else {
          ST2D[r][c][kx][ky] = Math.min(
            ST2D[r][c][kx][ky-1],

```

```

        className={`px-3 sm:px-4 py-2 rounded-lg text-
          se-600 text-white" : "bg-slate-800 text-sla
        >
        3. Запрос 2D (Формула)
      </button>
    </div>

    <AnimatePresence mode="wait">
      {tab === "1d" ? (
        <Viz1D key="1d" />
      ) : tab === "2d_build" ? (
        <Viz2DBuild key="2d_build" />
      ) : (
        <Viz2DQuery key="2d_query" />
      )}
    </AnimatePresence>
  </div>
);
};

```

```

const Viz1D: React.FC = () => {
  const [step, setStep] = useState(0);
  const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
  const runtime = useVizRuntime();
  const runtimeVars = runtime?.variables;
  const liveI = vizNumber(runtimeVars?.i);
  const liveJ = vizNumber(runtimeVars?.j);
  const liveSource = vizArray(runtimeVars?.a);
  const liveRows = vizArray(runtimeVars?.st);
  const hasLiveSource = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'a');
  const hasLiveTable = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'st');
  const compilerLinked = liveI !== null || liveJ !== null;
  const shownSource = hasLiveSource
    ? liveSource?.length
    ? liveSource
    : Array.from({ length: N }, () => null)
    : arr1D;
  const liveTarget =

```

```

    <p className="text-sm text-slate-400">
      Формула:  $ST[i][j] = \min(ST[i][j-1], ST[i + 1][j])$ 
    </p>
    {dimMismatch ? {
      <p className="mt-2 max-w-[560px] rounded-lg">
        {dimMismatch}
      </p>
    } : null}
  </div>
  <div className="flex gap-2">
    <button
      onClick={() => setStep(Math.max(0, step - 1))}
      className="p-2 bg-slate-800 hover:bg-slate-700 disabled:opacity-50"
      disabled={step === 0}
    >
      <ChevronLeft size={20} />
    </button>
    <button
      onClick={() => setStep(Math.min(maxStep, step + 1))}
      className="p-2 bg-indigo-600 hover:bg-indigo-500 disabled:opacity-50"
      disabled={step === maxStep}
    >
      <ChevronRight size={20} />
    </button>
    <button
      onClick={() => setStep(0)}
      className="p-2 bg-slate-800 hover:bg-slate-700 disabled:opacity-50"
    >
      <RotateCcw size={20} />
    </button>
  </div>
</div>

<div className="bg-slate-900 rounded-xl border border-slate-800 p-6">
  <div className="text-[11px] uppercase tracking-wider">
    <div className="flex gap-1 sm:gap-1.5 overflow-x-auto">
      {shownSource.map((v, idx) => {
        const inCover = !!covered && idx >= covered
      })}
    </div>
  </div>
</div>

```

```

        Наведите курсор (или тапните) на любое чи
    </p>
  })
</div>
</div>

<div className="overflow-x-auto pb-2">
  <div className="inline-block min-w-full bg-slate-500">
    <div className="grid grid-cols-[auto_repeat(4,1fr)]">
      { /* Headers */ }
      <div className="text-slate-500 font-mono text-sm">
        i \ j
      </div>
      <div className="text-center font-bold text-slate-500">
        2^0 (L=1)
      </div>
      <div className="text-center font-bold text-slate-500">
        2^1 (L=2)
      </div>
      <div className="text-center font-bold text-slate-500">
        2^2 (L=4)
      </div>
      <div className="text-center font-bold text-slate-500">
        2^3 (L=8)
      </div>

      { /* Matrix */ }
      {Array.from({ length: N }).map((_, i) => {
        <React.Fragment key={i}>
          <div className="text-slate-500 font-mono text-sm">
            {i}
          </div>
          {Array.from({ length: LOG }).map((_, j) => {
            const isValid = i + (1 << j) <= N;

```

```

return (
  <div key={j} className="relative flex flex-col items-center justify-center"
    {!isValid ? (
      <div className="w-[clamp(30px,9vw,170px)] h-[clamp(11px,3vw,17px)] font-bold transition-colors duration-300"
    ) : (
      <motion.div
        onMouseEnter={() => (compilerLinked ? setHover(1) : setActive())}
        onMouseLeave={() => setHover(0)}
        onClick={() => (compilerLinked ? setSrc1() : setSrc2())}
      >
        {displayValue}
      </motion.div>
    )}
  >
    <div className="relative flex flex-col items-center justify-center"
      initial={{ opacity: 0, scale: 1 }}
      animate={{
        // При st = [] каркас уже с
        opacity: compilerLinked ? 1 : 0,
        scale: compilerLinked || isSrc1 || isSrc2 || isVisible ? 1 : 0,
      }}
      className={`w-[clamp(30px,9vw,170px)] h-[clamp(11px,3vw,17px)] font-bold transition-colors duration-300
        hover && hover.i === i && h
        ? "bg-sky-500 text-white"
        : isActive
        ? "bg-indigo-500 text-white"
        : isSrc1
        ? "bg-emerald-500 text-white"
        : isSrc2
        ? "bg-amber-500 text-white"
        : isVisible
        ? "bg-slate-800 text-white"
        : compilerLinked
        ? "bg-slate-950 text-white"
        : "bg-transparent"
      }`
    >
      <div className="relative flex flex-col items-center justify-center"
        title={compilerLinked ? `st[${j}]` : undefined}
      >
        {displayValue}
      </div>
    </div>
  </div>
)

```

```

        {(hasLiveTable || hasLiveSource) && <> = <source>
        ?.liveTarget.j ?? (liveTarget.j === 0 ? liveTarget.i : liveTarget.j)}
    </p>
  ) : step > 0 && step <= maxStep ? (
    (() => {
      const c = computeSteps[step - 1];
      const half = 1 << (c.j - 1);
      return (
        <p className="text-lg text-slate-300">
          <span className="text-white font-bold">
            ST[{c.i}][{c.j}]
          </span>{" "}
          = min(
            <span className="text-emerald-400 font-bold">
              ST[{c.i}][{c.j - 1}]
            </span>
            ,
            <span className="text-amber-400 font-bold">
              ST[{c.i + half}][{c.j - 1}]
            </span>
          ) &rarr; min(
            <span className="text-emerald-400">{st1D[{c.i}][{c.j}]}
            <span className="text-amber-400">
              {st1D[{c.i + half}][{c.j - 1}]}
            </span>
          ) = {" "}
            <span className="text-indigo-400 font-bold">
              {st1D[{c.i}][{c.j}]}
            </span>
          </p>
        );
      })();
    ) : (
      <p className="text-slate-500 italic">
        Нажмите далее, чтобы увидеть, как блоки сво
      </p>
    )
  )
</div>

```

```

      : k;

/** 2D-демонстрация нарисована на фиксированной сетке
const liveSource2D = liveMatrix(vars?.A);
const dimMismatch2D = liveSource2D
  ? liveSource2D.rows !== 8 || liveSource2D.cols !== 8
  ? `Демо 2D построено на матрице 8x8, а в вашем ко
    не её не отрисовываются, значения вашей матрицы
    : null
  : null;

// Уровень и клетка берутся прямо из k/r/c; номер стро
useEffect(() => {
  setHover(null);
  setLocked(null);
}, [k]);

const L = 1 << shownK;
const activeCell = codeCell || locked || hover;

return (
  <div className="flex flex-col xl:flex-row gap-6">
    <div className="flex-1 min-w-0 bg-slate-900 p-3 s
      <h3 className="text-xl font-bold text-indigo-400
      <p className="text-sm text-slate-400 mb-6 text-
        Для наглядности показываем только <b classNam
        <br/>
        <span className="text-rose-300 animate-pulse
          иц!</span>
      </p>
      {dimMismatch2D ? (
        <p className="mb-4 max-w-[520px] rounded-lg b
          {dimMismatch2D}
        </p>
      ) : null}

    <div className="flex bg-slate-950 p-1 rounded-l
      {[0, 1, 2, 3].map(step => (

```

```

{Array.from({length: stepSize * st
  const r = Math.floor(idx / step
  const c = idx % stepSize;

  const liveValue = liveTable?.[`
  // Базовый слой (блоки 1x1) – э
  // одного элемента равен ему са
  // показываем из A сразу, как A
  // цикла построения st2. Уровни
  const sourceValue = step === 0
  const shownValue = compilerLink
  const isBlank = shownValue ===
  let highlightClass = isBlank
    ? 'bg-slate-950 text-slate-600'
    : isCurr
    ? 'bg-slate-800/80 text-slate-600'
    : 'bg-slate-800/50 text-slate-600';
  let zIndex = 'z-0';

  const isActive = !!activeCell &&
ll.c + L) && ((r - activeCell.r) % (1 << st

  if (isActive) {
    if (isCurr) {
      // Значения в ячейке ещё не
      // контур «сюда смотрит код»
      // залитая пустая клетка вы
      highlightClass = isBlank
        ? `bg-indigo-500/10 text-white`
        : `bg-indigo-500 text-white`;
      == c ? 'ring-[3px] ring-white ring-offset-2'
        : `bg-indigo-500 text-white`;
      && locked.c === c ? 'ring-[3px] ring-white'
        : `bg-indigo-500 text-white`;
      zIndex = 'z-10';
    } else {
      const prevL = 1 << (shownK
      const isTop = r < activeCel
      const isLeft = c < activeCe

```





```

        <div className="space-y-2 pt-3 pl-2"
          {{{() => {
            const prevL = 1 << (shownK - 1)
            return (
              <>
                <div className="flex items-center"
                  <span className="flex-grow-1" style={{background-color: 'hsl(220, 10%, 40%)', padding: '0 8px #f43f5e'}}></div> ST[{activeCell.r}, {activeCell.c}, {shownK - 1}]
                  <span className="text-gray-500"></div>
                <div className="flex items-center"
                  <span className="flex-grow-1" style={{background-color: 'hsl(220, 10%, 40%)', padding: '0 8px #3b82f6'}}></div> ST[{activeCell.r}, {activeCell.c + prevL}, {shownK - 1}]
                  <span className="text-gray-500"></div>
                <div className="flex items-center"
                  <span className="flex-grow-1" style={{background-color: 'hsl(220, 10%, 40%)', padding: '0 8px #10b981'}}></div> ST[{activeCell.r + prevL}, {activeCell.c}, {shownK}]
                  <span className="text-gray-500"></div>
                <div className="flex items-center"
                  <span className="flex-grow-1" style={{background-color: 'hsl(220, 10%, 40%)', padding: '0 8px #f59e0b'}}></div> ST[{activeCell.r + prevL}, {activeCell.c + prevL}, {shownK}]
                  <span className="text-gray-500"></div>
              </>
            )
          }}}()}}
        </div>

        <div className="pt-3 mt-3 border-top border-gray-200"
          > <span className="text-white">
            0">{ST2D[activeCell.r][activeCell.c][shownK]}

```

```

const queryVars = runtime?.variables;
const liveCoords = ["r1", "c1", "r2", "c2"].map((name, i) => {
const queryMismatch = liveCoords.some((v) => v !== queryVars?.[name]);
  ? `Координаты запроса в вашем коде выходят за пределы
    [ ${liveCoords[0]}=${liveCoords[1]} ${liveCoords[2]}=${liveCoords[3]} ] ?? "-"`, r2=${liveCoords[2]} ?? "-", c2=${liveCoords[3]} ?? "-"
  : null;

const H = Math.max(1, r2 - r1 + 1);
const W = Math.max(1, c2 - c1 + 1);
const kx = Math.floor(Math.log2(H));
const ky = Math.floor(Math.log2(W));
const Lx = 1 << kx;
const Ly = 1 << ky;

const matRows = 8 - Lx + 1;
const matCols = 8 - Ly + 1;

return (
  <div className="flex flex-col xl:flex-row gap-6">
    <div className="flex-1 min-w-0 bg-slate-900">
      <h3 className="text-xl font-bold text-rose-500">Запрос
      <p className="text-slate-400 text-[13px] font-mono">
        ${liveCoords[0]}=${liveCoords[1]} ${liveCoords[2]}=${liveCoords[3]} (левый верх - левый
        еpx, r2, c2 - правый низ).</p>
      {queryMismatch ? (
        <p className="mb-3 rounded-lg border border-rose-500 p-2 text-rose-500">
          {queryMismatch}
        </p>
      ) : null}

      <div className="flex flex-col items-center">
        <div className="grid grid-cols-[repeat(8, 1fr)] gap-1">
          <div className="active shadow-inner w-max max-w-full">
            {Array.from({length: 64}).map((_, idx) => {
              const r = Math.floor(idx / 8);
              const c = idx % 8;
              const isInQuery = r >= r1 && r <= r2 && c >= c1 && c <= c2;
              return (

```

```

        <label className="flex items-center"
          <span className="text-slate-700"
            <div className="flex gap-2">
              <input type="number" min={0} max={100} value={x} />
14 bg-slate-900 border border-slate-700 py-2 px-2
              <input type="number" min={0} max={100} value={y} />
14 bg-slate-900 border border-slate-700 py-2 px-2
            </div>
          </label>
        <label className="flex items-center"
          <span className="text-slate-700"
            <div className="flex gap-2">
              <input type="number" min={0} max={100} value={x} />
14 bg-slate-900 border border-slate-700 py-2 px-2
              <input type="number" min={0} max={100} value={y} />
14 bg-slate-900 border border-slate-700 py-2 px-2
            </div>
          </label>
        </div>
      </div>
    </div>

    <div className="flex-1 bg-slate-900 p-6 rounded"
      <h3 className="text-xl font-bold text-emerald-400">Задача 1</h3>
      <p className="text-sm font-sans text-slate-300">
        Мы знаем, что максимальная степень дв
        rounded text-white">{Lx}</span>. Аналогично
        te">{Ly}</span>.
      </p>
      <p className="text-sm font-sans text-slate-300">
        Чтобы покрыть весь прямоугольник, мы
        Name="font-mono bg-[#0a0f1e] px-1 rounded border
        роса. (Цветные прямоугольники слева).
      </p>

      <div className="bg-[#0a0f1e] rounded-xl p-4"
        text-slate-300 shadow-inner overflow-x-auto
        <div className="absolute right-3 top-3"

```

```

border-slate-700 shadow-sm">ST[][][kx]][ky]
</h4>
<div className="grid gap-[2px] p-2.5
Columns: `repeat(${matCols}, 1fr)`}}>
  {Array.from({length: matRows * matCols}, (v, idx) => {
    const r = Math.floor(idx / matCols);
    const c = idx % matCols;
    let isTL = r === r1 && c === c1;
    let isTR = r === r1 && c === c2;
    let isBL = r === r2 - 1 && c === c1;
    let isBR = r === r2 - 1 && c === c2;

    let color = "bg-slate-800 text-white";
    let txt = ST2D[r][c][kx][ky];

    let badge = null;
    if (isTL) { color = "bg-rose-500 text-white";
    border-2"; badge = "TL"; }
    else if (isTR) { color = "bg-lime-500 text-white";
    ue-300 border-2"; badge = "TR"; }
    else if (isBL) { color = "bg-emerald-500 text-white";
    -emerald-300 border-2"; badge = "BL"; }
    else if (isBR) { color = "bg-amber-500 text-white";
    mber-300 border-2"; badge = "BR"; }

    return (
      <div key={idx} className={color}
      tion-all duration-300 relative shrink-0 ${c === c2 ? 'text-right' : ''}
        {txt}
        {badge && <div className="bg-slate-700 text-white px-1 font-bold rounded
      </div>
    )
  })}
</div>
</div>
</div>

```

```

function setMode(mode) {
    document.getElementById('btn-mode-normal').className =
        ? 'px-3 py-1 rounded text-sm font-bold'
        : 'px-3 py-1 rounded text-sm font-bold'

    document.getElementById('btn-mode-wtf').className =
        ? 'px-3 py-1 rounded text-sm font-bold'
        : 'px-3 py-1 rounded text-sm font-bold'

    const aside = document.querySelector('aside')
    const headerMain = document.querySelector('h1')
    const questionsContainer = document.getElementById('questions')
    const wtfContainer = document.getElementById('wtf')

    if (mode === 'wtf') {
        document.body.classList.add('wtf-mode')
        if (aside) aside.style.display = 'none'
        if (headerMain) headerMain.style.display = 'none'
        if (questionsContainer) questionsContainer.style.display = 'none'
        if (wtfContainer) wtfContainer.classList.remove('hidden')

        // Remove the URL hash to avoid scrolling
        history.pushState("", document.title, window.location.pathname + window.location.search)
        window.scrollTo(0, 0);
    } else {
        document.body.classList.remove('wtf-mode')
        if (aside) aside.style.display = '';
        if (headerMain) headerMain.style.display = '';
        if (questionsContainer) questionsContainer.style.display = '';
        if (wtfContainer) wtfContainer.classList.add('hidden')
    }
}

function setTheme(theme) {
    const body = document.body;
    body.classList.remove('theme-dark', 'theme-light');
    body.classList.add('theme-' + theme);
}

```

```

        setTimeout(() => drawerOverlay.classList.add("open"))
    }
    document.body.style.overflow = "hidden"
} else {
    mobileDrawer.classList.add("-translate-x-100%")
    if (drawerOverlay) {
        drawerOverlay.classList.add("open")
        setTimeout(() => drawerOverlay.classList.add("open"))
    }
    document.body.style.overflow = ""
}
}

if (mobileMenuBtn && mobileDrawer) {
    mobileMenuBtn.addEventListener("click", () => {
        toggleDrawer()
    })
}
if (closeDrawerBtn && mobileDrawer) {
    closeDrawerBtn.addEventListener("click", () => {
        toggleDrawer()
    })
}
if (drawerOverlay) {
    drawerOverlay.addEventListener("click", () => {
        toggleDrawer()
    })
}

// Close on link click
document.querySelectorAll("#mobile-drawer a").forEach(link => {
    link.addEventListener("click", () => {
        if (mobileDrawer && !mobileDrawer.classList.contains("open")) {
            toggleDrawer()
        }
    })
})

const observerOptions = {
    root: null,
    rootMargin: '-10% 0px -70% 0px',
    threshold: 0
};

```

```

<script id="MathJax-script" async src="https://cdn.
<script>
    window.MathJax = {
        tex: {
            inlineMath: [['$', '$'], ['\\(', '\\)']]
            displayMath: [['$$', '$$'], ['\\[', '\\
        }
    };
</script>
<style>
    @import url('https://fonts.googleapis.com/css2?
    @reference "./src/index.css";
    .math-font { font-family: 'Fira Code', monospac
    .uno-card {
        width: 70px; height: 105px; border-radius:
        box-shadow: 0 4px 8px rgba(0,0,0,0.5); posi
        align-items: center; justify-content: cente
        color: white; overflow: hidden; flex-shrink
        text-shadow: 2px 2px 0 #000, -1px -1px 0 #0
        font-family: 'Arial Black', Impact, sans-se
    }
    .uno-card::before, .uno-card::after {
        content: attr(data-val); position: absolute
        text-shadow: 1px 1px 0 #000, -1px -1px 0 #0
    }
    .uno-card::before { top: 2px; left: 4px; }
    .uno-card::after { bottom: 2px; right: 4px; tra
    .uno-card.mini { width: 45px !important; height
    .uno-card.mini::before, .uno-card.mini::after {

/* Global formula scroll settings */
.formula-scroll {
    overflow-x: auto;
    overflow-y: hidden;
    -webkit-overflow-scrolling: touch;
    scrollbar-width: none; /* Firefox */
    -ms-overflow-style: none; /* IE and Edge */
}

```



```

.uno-inner {
  position: absolute; width: 110%; height: 75%;
  border-radius: 50%; transform: rotate(-30deg);
  box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
}
.uno-val { position: relative; z-index: 2; }
.card-red { background-color: #e52521; } .card-
.card-blue { background-color: #004dcf; } .card-
.card-green { background-color: #339e35; } .card-
.card-yellow { background-color: #ffc400; } .ca
.card-black { background-color: #222; } .card-b
.card-black::before, .card-black::after { text-
.info-block { @apply bg-slate-800/80 border-l-4
.info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
  ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
  king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
  r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 fo

/* THEME OVERRIDES */
body.theme-light {
  background-color: #f8fafc !important;
  color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-light
body.theme-light .bg-slate-800 { background-col
body.theme-light .bg-slate-700, body.theme-light
  important; }
body.theme-light .text-slate-200, body.theme-li
body.theme-light .text-slate-400 { color: #47556
body.theme-light .border-slate-600, body.theme-
body.theme-light .border-slate-500 { border-col
body.theme-light #top-controls { background-col

```

```

    }
    .creepy-emoji {
      display: inline-block;
      transform-origin: center;
      animation: creepy-blink 3.8s infinite;
    }
  </style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

  <div class="sticky top-0 z-50 w-full bg-slate-950/90
    center items-center gap-4 sm:gap-8 transition-colors">
    <div class="flex items-center gap-3">
      <span class="text-sm font-bold uppercase tracking-wider">ТЕМА
      <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2
        <button onclick="setMode('normal')" id="btn-normal" class="text-white
          transition-colors">Список тем</button>
        <button onclick="setMode('wtf')" id="btn-wtf" class="text-white
          transition-colors">WTF</button>
      </div>
    </div>
    <div class="flex items-center gap-3">
      <span class="text-sm font-bold uppercase tracking-wider">ТЕМА
      <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2
        <button onclick="setTheme('dark')" id="btn-dark" class="text-white
          transition-colors">Темная</button>
        <button onclick="setTheme('light')" id="btn-light" class="text-white
          transition-colors">Светлая</button>
        <button onclick="setTheme('notebook')" id="btn-notebook" class="text-white
          transition-colors">Заметки</button>
      </div>
    </div>
  </div>

  <!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
  <svg width="0" height="0" class="absolute">
    <defs>
      <marker id="arrow-yellow" markerWidth="6" markerHeight="6" color="yellow"

```

```

        <line x1="2" y1="2" x2="38" y2="2" stro
    </g>
    <!-- МЯСОРУБКА -->
    <g id="meat-grinder">
        <path d="М 10 40 L 15 20 Q 25 15 35 20
        <rect x="5" y="40" width="40" height="2
        <!-- Ручка -->
        <path d="М 45 50 Q 60 50 60 30 L 75 30"
        <circle cx="75" cy="30" r="4" fill="#33
        <!-- Выход -->
        <path d="М 5 45 L -5 45 L -5 55 L 5 55
        <circle cx="25" cy="50" r="15" fill="#f
    </g>
    <g id="lego-green">
        <rect x="0" y="10" width="20" height="1
        <rect x="3" y="5" width="14" height="5"
    </g>
    <g id="lego-white">
        <rect x="0" y="0" width="40" height="20
        <rect x="6" y="-5" width="8" height="6"
        <rect x="26" y="-5" width="8" height="6
        <line x1="0" y1="2" x2="40" y2="2" stro
        <text x="20" y="14" fill="#475569" font
    </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed
    0/50 z-50 hover:bg-blue-500 transition-transform l
        <svg xmlns="http://www.w3.org/2000/svg" class="l
            <path stroke-linecap="round" stroke-linejoin
        </svg>
    </button>

<!-- Затемнение фона на мобилках при открытом меню
<div id="drawer-overlay" class="lg:hidden fixed ins

```



```
        <details class="group">
          <summary class="list-none cursor-pointer text-pink-500 pl-3 font-bold text-pink-300">
            7. Взаимно простые числа
          </a>
          </summary>
          <div class="pl-6 space-y-2 text-pink-300">
            <a href="#q7-1" class="block">
            <a href="#q7-2" class="block">
            <a href="#q7-3" class="block">
            <a href="#q7-4" class="block">
            <a href="#q7-5" class="block">
          </div>
        </details>
```

```
        <details class="group">
          <summary class="list-none cursor-pointer text-violet-500 pl-3 font-bold text-violet-300">
            8. Сравнимость и Поля Вейля
          </a>
          </summary>
          <div class="pl-6 space-y-2 text-violet-300">
            <a href="#q8-1" class="block">
            <a href="#q8-2" class="block">
            <a href="#q8-3" class="block">
            <a href="#q8-4" class="block">
            <a href="#q8-5" class="block">
            <a href="#q8-6" class="block">
            <a href="#q8-7" class="block">
          </div>
        </details>
```

```
        <details class="group">
          <summary class="list-none cursor-pointer text-green-500 pl-3 font-bold text-green-400">
```

```
нственности НОД</a>
      <a href="#q11-3" class="block">
ПИРАЛЬ)</a>
      <a href="#q11-4" class="block">
вращается!)</a>
    </div>
  </details>

  <details class="group">
    <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-400">
      12. Взаимно простые мно
    </a>
    </summary>
    <div class="pl-6 space-y-2 text-blue-500">
      <a href="#q12-1" class="block">
      <a href="#q12-2" class="block">
      <a href="#q12-3" class="block">
деление)</a>
    </div>
  </details>

  <details class="group">
    <summary class="list-none cursor-pointer text-emerald-500 pl-3 font-bold text-emerald-400">
      13. Корни многочленов.
    </a>
    </summary>
    <div class="pl-6 space-y-2 text-emerald-500">
      <a href="#q13-1" class="block">
      <a href="#q13-2" class="block">
      <a href="#q13-3" class="block">
    </div>
  </details>

  <details class="group">
```

```

<div class="pl-6 mt-2 space-y-2" data-bbox="474 96 1000 191">
  <a href="#q16-1" class="block" data-bbox="541 121 1000 141">
  <a href="#q16-2" class="block" data-bbox="541 146 1000 166">
  <a href="#q16-3" class="block" data-bbox="541 171 1000 191">
</div>
</details>

<div class="mt-4 mb-2 text-xs font-l" data-bbox="407 266 1000 286">
  <a href="#question-17" class="block" data-bbox="407 291 1000 311">
-500 pl-3 text-slate-400">
    <span class="font-bold text-tea" data-bbox="474 336 1000 356">
  </a>
  <a href="#question-18" class="block" data-bbox="407 381 1000 401">
-500 pl-3 text-slate-400">
    <span class="font-bold text-blue" data-bbox="474 426 1000 446">
  </a>
  <a href="#question-19" class="block" data-bbox="407 471 1000 491">
ald-500 pl-3 text-slate-400">
    <span class="font-bold text-eme" data-bbox="474 516 1000 536">
  </a>
  <a href="#question-20" class="block" data-bbox="407 561 1000 581">
-500 pl-3 text-slate-400">
    <span class="font-bold text-ros" data-bbox="474 606 1000 626">
  </a>
  <a href="#question-21" class="block" data-bbox="407 651 1000 671">
go-500 pl-3 text-slate-400">
    <span class="font-bold text-ind" data-bbox="474 696 1000 716">
  </a>
  <a href="#question-22" class="block" data-bbox="407 741 1000 761">
sia-500 pl-3 text-slate-400">
    <span class="font-bold text-fuc" data-bbox="474 786 1000 806">
  </a>

  <a href="#proof-algorithms" class="block" data-bbox="407 871 1000 891">
-red-500 pl-3 mt-4 text-red-500 font-bold ml-4" data-bbox="274 906 1000 926">
    ▲ Ликбез: Алгоритмы выживания
  </a>

```

```

        a.style.fontWeight = 'normal';
        if (a.href.endsWith('#')) {
            a.style.fontSize = '1.2em';
        }
    });

    // Open the parent modal if the link is a details link
    const correspondingLink = detailsLink.parentNode.querySelector('a');
    if (correspondingLink) {
        const details = detailsLink.parentNode;
        if (details && details.open) {
            document.querySelector(detailsLink.getAttribute('data-target'))
                .if(OTHER_MODAL_ID)
                .open();
        }
    }
    });
    });
    }, {
        root: null,
        rootMargin: '-10% 0px -70% 0px',
        threshold: [0, 0.5]
    });

    sections.forEach(section => {
        observer.observe(section);
    });
});

function fallbackModal(contentRaw, ...args) {
    const modal = document.createElement('div');
    modal.style.position = 'fixed';
    modal.style.top = '0';
    modal.style.left = '0';
    modal.style.width = '100vw';

```



```

btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop =

```

```

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = '#333';
copyBtn.style.color = 'white';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
    if (navigator.clipboard) {
        navigator.clipboard.writeText(text);
        copyBtn.innerText = 'Скопировано';
    } else {
        textarea.select();
        document.execCommand('copy');
        copyBtn.innerText = 'Скопировано';
    }
    setTimeout(() => copyBtn.innerText = 'Копировать', 2000);
};

```

```

const closeBtn = document.createElement('button');
closeBtn.innerText = 'Закрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = '#333';
closeBtn.style.color = 'white';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '5px';
closeBtn.style.cursor = 'pointer';
closeBtn.onclick = () => modal.close();

```

```

box.appendChild(header);
box.appendChild(subHeader);
if (title !== 'PDF') {
    box.appendChild(textarea);
}

```

```
        link.download = 'vyisshaya_algebra.pdf';
        document.body.appendChild(link);
        link.click();
        document.body.removeChild(link);
    }

    function setTheme(theme) {

        const body = document.body;
        body.classList.remove('bg-slate-900');

        document.getElementById('theme-dark').checked = theme === 'dark';
        document.getElementById('theme-light').checked = theme === 'light';
        document.getElementById('theme-auto').checked = theme === 'auto';

        let oldStyle = document.getElementById('theme-style');
        if (oldStyle) oldStyle.remove();
        const style = document.createElement('style');
        style.id = 'theme-style';

        if (theme === 'dark') {
            body.classList.add('bg-slate-900');
            document.getElementById('theme-dark').checked = true;
        } else if (theme === 'light') {
            body.classList.add('bg-white');
            document.getElementById('theme-light').checked = true;
            style.innerHTML = `
                body.bg-white section h1 { color: #000000; }
                body.bg-white main { color: #000000; }
                body.bg-white main p { color: #000000; }
                /* Universal text color */
                body.bg-white .text-slate-900 { color: #000000; }
                body.bg-white .text-slate-900 { color: #000000; }
                body.bg-white .text-slate-900 { color: #000000; }
                body.bg-white [class*="text-slate-900"] { color: #000000; }
            `;
        }
    }
}
```

```

body.bg-white .text-fuc
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre
body.bg-white .text-ind

`;
} else if (theme === 'terminal')
body.classList.add('bg-black')
document.getElementById('theme')
style.innerHTML = `
body.bg-black .bg-slate-950 {
background-color: transparent !important;
}
body.bg-black section header {
, body.bg-black button {
color: #4ade80 !important;
border-color: #22c55e !important;
font-family: monospace;
}
body.bg-black svg path,
stroke: #22c55e !important;
fill: transparent !important;
}
`;
}

document.head.appendChild(style)
}
</script>

```

```

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 border" style="border: 1px solid #ccc; padding: 10px;">
  <!-- Кнопки Конвертации / Экспорта
  <div class="flex flex-col gap-2">

```

```

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16"
  -base min-w-0">
  <header class="text-center mb-12 border-b border-slate-700">
    <h1 class="text-4xl md:text-5xl font-bl
4">
      ВЫСШАЯ АЛГЕБРА
    </h1>
    <p class="text-slate-400 italic text-lg">
      Высшая алгебра: теория и практика

    <!-- Print Table of Contents -->
    <div class="hidden print:block mt-8 text-center">
      <h2 class="font-bold text-xl mb-4">Содержание
      <ul class="text-base space-y-2">
        <li><a href="#question-1">1. Алгебра
        <li><a href="#question-2">2. Коммутативная алгебра
        <li><a href="#question-3">3. Тренировки
        <li><a href="#question-4">4. Структуры
        <li><a href="#question-5">5. Группы
        <li><a href="#question-6">6. Декартовы произведения
        <li><a href="#question-7">7. Алгебраические расширения
        <li><a href="#question-8">8. Клеймента
        <li><a href="#question-9">9. Коммутативная алгебра
        <li><a href="#question-10">10. Алгебраические структуры
        <li><a href="#question-11">11. Алгебраические структуры
        <li><a href="#question-12">12. Алгебраические структуры
        <li><a href="#question-13">13. Алгебраические структуры
        <li><a href="#question-14">14. Алгебраические структуры
        <li><a href="#question-15">15. Алгебраические структуры
        <li><a href="#question-17">17. Алгебраические структуры
        <li><a href="#question-18">18. Алгебраические структуры
        <li><a href="#question-19">19. Алгебраические структуры
        <li><a href="#question-20">20. Алгебраические структуры
        <li><a href="#question-21">21. Алгебраические структуры
        <li><a href="#question-22">22. Алгебраические структуры
      </ul>

```

```

                <div class="font-bold text-slate-700">
            </a>
            <a href="#question-10" onclick="setMainQuestion(10)"
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
                <div class="text-xs text-orange-500">
                <div class="font-bold text-slate-700">
            </a>
            <a href="#question-3" onclick="setMainQuestion(3)"
- 4 border-cyan-500 hover:bg-slate-700 transition-colors
                <div class="text-xs text-cyan-500">
                <div class="font-bold text-slate-700">
            </a>

            <!-- Row 3: НОД и НОК / Алгоритм Евклида -->
            <a href="#question-7" onclick="setMainQuestion(7)"
- 4 border-yellow-500 hover:bg-slate-700 transition-colors
                <div class="text-xs text-yellow-500">
                <div class="font-bold text-slate-700">
            </a>
            <a href="#question-12" onclick="setMainQuestion(12)"
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
                <div class="text-xs text-orange-500">
                <div class="font-bold text-slate-700">
            </a>
            <!-- Empty 3rd row cell -->
            <div class="hidden md:block"></div>

            <!-- Row 4: Факторизация / Структурная теория -->
            <a href="#question-8" onclick="setMainQuestion(8)"
- 4 border-yellow-500 hover:bg-slate-700 transition-colors
                <div class="text-xs text-yellow-500">
                <div class="font-bold text-slate-700">
            </a>
            <a href="#question-11" onclick="setMainQuestion(11)"
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
                <div class="text-xs text-orange-500">
                <div class="font-bold text-slate-700">
            </a>

```

```

<div class="col-span-1 md:col-span-3" style="border: 2px solid #008000; padding: 10px; margin-bottom: 10px;">
  <a href="#question-17" onclick="setQuestion(17)" style="display: block; padding: 5px 0;">
    <div class="text-xs text-teal-500" style="margin-bottom: 5px;">Вопрос 17: Какое из следующих утверждений верно?
```

```

                                <li><span class="text-w
                                <li><span class="text-w
ово.</li>
                                </ul>
</div>

<!-- От противного -->
<div class="bg-slate-800/80 p-6
  <h3 class="text-xl text-fuc
  <p class="text-sm text-slat
  <ul class="list-disc pl-5 s
    <li><span class="text-w
i>
    <li><span class="text-w
т.д.).</li>
    <li><span class="text-w
что так и задумано: "Ой, противоречие, теор
  </ul>
</div>

<!-- Единственность -->
<div class="bg-slate-800/80 p-6
  <h3 class="text-xl text-eme
  <p class="text-sm text-slat
  <ul class="list-disc pl-5 s
    <li><span class="text-w
, r_2$).</li>
    <li><span class="text-w
    <li><span class="text-w
счет ограничений (размер остатка) докажи, ч
  </ul>
</div>

<!-- Индукция -->
<div class="bg-slate-800/80 p-6
  <h3 class="text-xl text-yel
  <p class="text-sm text-slat
  <ul class="list-disc pl-5 s

```

```
        <p class="text-xs font-mono">
        $$$. Пока получается 0 – корень жив. Как то.
        корня минус один.</p>
```

```
    </div>
```

```
    <div class="bg-slate-800 p-4">
      <h4 class="text-red-400">
      <p class="text-sm text-slate-400">
      <p class="text-xs font-mono">
      ициентов (они делятся на $$), кроме старшего
      зложить.</p>
```

```
    </div>
```

```
    <div class="bg-slate-800 p-4">
      <h4 class="text-red-400">
      <p class="text-sm text-slate-400">
      <p class="text-xs font-mono">
      . Чаще всего после этого Фейс-контроль $$.
```

```
    </div>
```

```
</div>
```

```
    <h2 class="text-2xl font-bold bg-white">
    (Правила экстрасенса)</h2>
```

```
    <p class="text-sm text-slate-400">
    тношения), а ты их не помнишь. Юзай базовые
    нейность) для делимости.</p>
```

```
    <ul class="space-y-4 pb-10">
```

```
      <li class="bg-slate-800/50 p-4">
        <span class="text-2xl">
        <div>
```

```
          <h4 class="text-cyan-400">
          <p class="text-sm text-slate-400">
          <code class="text-xs font-mono">
```

```
        </div>
```

```
      </li>
```



---

## РАЗДЕЛ: УТИЛИТЫ

---

---

ФАЙЛ 87/111: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

---

ФАЙЛ 88/111: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 341 | РАЗМЕР: 15.5 KB

---

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";
import { annotateTicketLinks } from "../ticketLinks";

/**
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
 * Файл получается автономным: стили защиты внутрь, интерактив
 * его можно открыть с флешки, отправить в мессенджер и т.д.
 * Живые React-визуализации в статику не переносятся — вместо них
 * подставляется заметка со ссылкой на онлайн-версию, а комментарии
 * которые в приложении всплывают по наведению, собираюся в блок
 * в конце документа.
 */
```

```
-----
.export-term .cx { color: #6ee7b7; font-size: .78rem; }
.export-foot { margin-top: 3rem; border-top: 1px solid #ccc; }
body { font-size: 15px; line-height: 1.6; }
.export-shell p, .export-shell li { line-height: 1.65; }
.export-shell pre { font-size: 12px; line-height: 1.5; }
.export-figs { margin: 0 0 1.6rem; }
.export-figs-title { color: #e2e8f0; font-size: .95rem; }
.export-fig { margin: 0 0 .9rem; }
.export-fig img { width: 100%; border: 1px solid #1e293b; }
.export-fig figcaption { color: #94a3b8; font-size: .75rem; }
@media print {
  body { background: #fff; color: #0f172a; }
  .export-note, details { break-inside: avoid; }
  details { display: block; }
  details > div, details > p { display: block !important; }
  pre { white-space: pre-wrap; }
};
```

```
const escapeHtml = (s: string) =>
  s.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;");
```

```
/**
```

```
 * Размечает термины и превращает их в якорные ссылки на
```

```
 * Возвращает готовый HTML главы и список найденных терминов
```

```
 */
```

```
function prepareBody(html: string, opts: { selfId?: string; }) {
```

```
  body: string;
```

```
  termIds: string[];
```

```
} {
```

```
  const host = document.createElement("div");
```

```
  host.innerHTML = html;
```

```
  try {
```

```
    annotateTerms(host);
```

```
  } catch {
```

```
    /* если браузер не осилил регулярку – выгружаем текст как есть */
```

```
  }
```

```

    if (!termIds.length) return "";
    const items = termIds
      .map((id) => {
        const e = GLOSSARY_BY_ID.get(id);
        if (!e) return "";
        return `<div class="export-term" id="term-${escapeHtml(
          <h3>${escapeHtml(e.title)}</h3>
          <p>${escapeHtml(e.short)}</p>
          ${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>` : ""}
          ${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
        </div>`;
      })
      .join("\n");

    return `<section class="export-gloss">
    <h2>${book ? "Словарик терминов пособия" : "Словарик терминов"}
    <p style="font-size:.8rem;color:#94a3b8;margin:0 0 10px 0">
      ны сюда.</p>
    ${items}
  </section>`;
}

```

```

function wrap(opts: { title: string; subtitle: string; css: string; }) {
  const stamp = new Date().toLocaleString("ru-RU");
  return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>
</head>
<body>
<div class="export-shell">
  <header class="export-head">
    <div style="font-size:.7rem;text-transform:uppercase">
      <h1 style="color:#fff;font-size:1.6rem;font-weight:bold">

```

```

        const data = await new Promise<string>((resolve, reject) => {
            const fr = new FileReader();
            fr.onload = () => resolve(String(fr.result));
            fr.onerror = () => reject(fr.error);
            fr.readAsDataURL(blob);
        });
        img.setAttribute("src", data);
    } catch {
        /* картинка недоступна — глава останется со ссылкой */
    }
}

});
return doc.body.innerHTML;
}

/**
 * Кадры визуализаций: книга встраивает снимки демонстраций,
 * файл показывал, что крутилось в приложении. Снимки делятся на
 * (capture-viz-shots.mjs) и публикует в /export/viz-shots. Если
 * без них (офлайн-сборка, локальный дев) книга остаётся
 * нормально и молча пропускается.
 */
async function vizFigures(chapterId: string): Promise<string> {
    try {
        const base = import.meta.env.BASE_URL || "/";
        const manifestRes = await fetch(`${base}export/viz-shots/manifest.json`);
        if (!manifestRes.ok) return "";
        const manifest = (await manifestRes.json()) as { files: string[] };
        const mine = manifest.filter((m) => m.file === `chapter-${chapterId}.mjs`);
        const figures: string[] = [];
        for (const m of mine.slice(0, 4)) {
            const res = await fetch(`${base}export/viz-shots/${m.file}`);
            if (!res.ok) continue;
            const blob = await res.blob();
            if (blob.size > 3_000_000) continue;
            const data = await new Promise<string>((resolve, reject) => {
                const fr = new FileReader();
                fr.onload = () => resolve(String(fr.result));
            });
            figures.push(data);
        }
        return figures.join("\n");
    } catch {
        return "";
    }
}

```

```

-----
const { body: rawBody, termIds } = prepareBody(chapter)
const body = (await vizFigures(chapter.id)) + (await inlineFigures(chapter.id))
return wrap({
  title: chapter.title,
  subtitle: chapter.category || "Универсальное пособие",
  css,
  body,
  gloss: glossarySection(termIds),
  origin: window.location.origin,
});
}

/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter) {
  triggerDownload(await buildChapterHtml(chapter), safeDownloadUrl(chapter.id))
}

/** Собрать книгу: всё пособие одним файлом со сквозным индексом. */
export async function buildBookHtml(chapters: Chapter[]) {
  const css = await collectCss();
  const allTerms: string[] = [];
  const bodies: string[] = [];

  const toc = chapters
    .map((c) => `<li style="margin:.2rem 0"><a href="#${c.id}">${c.title}</a>`
    ).join("\n");

  for (const c of chapters) {
    const { body: rawBody, termIds } = prepareBody(c)
    // картинки глав встраиваются сразу: книга одна, а не по главам
    // снимки демонстраций подтягиваются из /export/viz
    const body = (await vizFigures(c.id)) + (await inlineFigures(c.id))
    termIds.forEach((t) => {
      if (!allTerms.includes(t)) allTerms.push(t);
    });
    bodies.push(`<div id="ch-${escapeHtml(c.id)}" style="border: 1px solid #ccc; padding: 10px; margin-bottom: 10px;">
      <p style="text-align:right;font-size:.72rem;margin:0 0 10px 0">Глава ${c.title}</p>
      ${body}
    `);
  }
  return {
    toc,
    css,
    allTerms,
    bodies
  };
}

```

```

* номер, но попасть на него не мог — ссылка была «битой»
* в никуда. Здесь такие упоминания превращаются в настоящее
*
*   * в приложении — `

```

```

const ticketToChapter = new Map<number, string>();
for (const [id, tickets] of Object.entries(chapterTopics))
  for (const n of tickets) if (!ticketToChapter.has(n))
    ticketToChapter.set(n, id);
}

```

```

/** id страницы по номеру билета (undefined, если такого нет)
export const chapterForTicket = (ticket: number): string | undefined

```

```

/** «билет», «билеты», «билетов», «билете», «билетам»...
const TICKET_RE = /билет(?:ы|ов|ам|ах|ом|у|а|е)?\s*(\d{1,2})/i;

```

```

/**
* Константы TreeWalker числами: глобальный `NodeFilter`
* во всех окружениях (в jsdom его нет) — с числами разницы нет
*/

```

```

const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

```

```

/** Внутри чего ссылки не ставим: код, уже существующие
const SKIP_SELECTOR = "code, pre, script, style, a, button,

```

```

/** Одна страница на диапазон? «билеты 18–20» → mst, а не
function chapterForRange(from: number, to: number): string | undefined

```

```

        return part.replace(TICKET_RE, (whole, a: string,
            const from = Number(a);
            const to = b ? Number(b) : from;
            const id = chapterForRange(from, to);
            if (!id || id === opts.selfId) return whole;
            return `<a class="ticket-link" data-goto="${id}
        }));
    });
    .join("");
}

/**
 * DOM-версия – для интерактивной страницы. Проходит то.
 * (как `annotateTerms`), поэтому атрибуты и разметка не
 * Возвращает число созданных ссылок – его проверяют DOM
 */
export function annotateTicketLinks(
    root: HTMLElement,
    opts: { selfId?: string; lite?: boolean } = {}
): number {
    if (typeof document === "undefined") return 0;
    const walker = document.createTreeWalker(root, SHOW_TEXT,
        acceptNode(node) {
            const parent = (node as Text).parentElement;
            if (!parent) return FILTER_REJECT;
            if (parent.closest(SKIP_SELECTOR)) return FILTER_REJECT;
            const text = node.nodeValue ?? "";
            if (!/билет/i.test(text)) return FILTER_REJECT;
            return FILTER_ACCEPT;
        },
    );
    let count = 0;
    while (current = walker.nextNode()) {
        targets.push(current as Text);
        current = walker.nextNode();
    }
}

```

```
-----  
    if (el) annotateTicketLinks(el, opts);  
    // eslint-disable-next-line react-hooks/exhaustive-deps  
  }, deps);  
}
```

## =====

## РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

## =====

-----

ФАЙЛ 90/111: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМЕР: 1.5 КБ

-----

```
/**  
 * Аудит покрытия глав пособия: у каких тем нет «объяснения»  
 * (блок «Аналогия») и/или 2D-визуализации (интерактивной)  
 *  
 * node scripts/audit-coverage.mjs          # отчёт  
 * node scripts/audit-coverage.mjs --md     # markdown  
 */  
import fs from "node:fs";  
import path from "node:path";  
import { fileURLToPath } from "node:url";  
import { build } from "esbuild";  
  
const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)),  
  "scripts/audit-coverage.mjs");  
const TMP = path.join(ROOT, "tmp", "audit");  
  
fs.mkdirSync(TMP, { recursive: true });  
const bundle = path.join(TMP, "content.bundle.mjs");  
  
await build({  
  entryPoints: [path.join(ROOT, "src/data/content.ts")],  
  bundle: true,  
  format: "esm",  
  platform: "node",  
});
```



```

-----
if (process.argv.includes("--md")) {
  console.log("| # | Глава | Аналогия | 2D-виз. | SVG |");
  console.log("| --- | --- | --- | --- | --- | --- |");
  for (const r of rows) {
    console.log(
      `| ${r.num} || "-" | ${r.title} | ${r.hasAnalogy}
      ge)) |`
    );
  }
} else {
  console.log(`Глав в пособии: ${rows.length}\n`);
  const dump = (title, list) => {
    console.log(`${title} (${list.length}):`);
    list.forEach((r) => console.log(`  * ${r.title} [$
    console.log());
  };
  dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
  dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
  dump("НЕТ 2D (аналогия есть)", gapViz);
  console.log(`Пропущенные номера билетов 1-24: ${missi
  console.log(`Визуализаторы без главы: ${orphanViz.join
}

```

ФАЙЛ 91/111: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```

/**

```

```

 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *

```

```

 *

```

```

 *   node scripts/audit-terms.mjs

```

```

 */

```

```

import { build } from "esbuild";

```

```

import path from "node:path";

```

```

    if (!id) continue;
    const ut = perTerm.get(id) ?? 0;
    const us = perSurface.get(surface) ?? 0;
    if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) continue;
    perTerm.set(id, ut + 1);
    perSurface.set(surface, us + 1);
    highlights += 1;
    ids.add(id);
    used.add(id);
  }

  if (ids.size === 0) bad.push(c.title);
  console.log(
    String(ids.size).padStart(3), "|", String(highlights).padStart(3),
    c.title.slice(0, 46).padEnd(47), [...ids].slice(0, 47).join(", ")
  );
}
console.log("\nГлав без единой подсказки:", bad.length, "шт.");
console.log("Терминов в глоссарии:", GLOSSARY.length, "шт.");
console.log("Не встретились:", GLOSSARY.filter((g) => !bad.includes(g)).length, "шт.");

```

ФАЙЛ 92/111: `scripts/export/build-guide-pdf.mjs`

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРӨК: 882 | РАЗМЕР: 10.5 КБ

```

/**
 * Компактный PDF пособия: public/export/guide_lite.pdf
 *
 * - текст всех тем (без служебной разметки)
 * + снимки визуализаций, если они отрендерены (tmp/viz/
 *   capture-viz-shots.mjs в CI через headless-браузер)
 * + иллюстрации страниц (jpg из src/assets)
 * + текстовое описание каждой демонстрации и переменные
 * Python-компилятор (работает и без браузера – локально)
 * - БЕЗ кода панели компилятора и без навигации приложения
 *
 */

```

```

    { u: "A", v: "B", w: 4, highlighted: true },
    { u: "A", v: "C", w: 2, highlighted: true },
    { u: "C", v: "B", w: 1, highlighted: true },
    { u: "B", v: "D", w: 5 },
    { u: "C", v: "G", w: 3, highlighted: true },
    { u: "C", v: "E", w: 8 },
    { u: "G", v: "D", w: 1, highlighted: true },
    { u: "G", v: "E", w: 2, highlighted: true },
    { u: "D", v: "F", w: 2 },
    { u: "E", v: "F", w: 1, highlighted: true },
  ],
  table: [
    ["Вершина", "Старт", "Кратчайшее dist", "Предшественник"],
    ["Пиццерия A", "s = A", "0", "-", "зафиксирована"],
    ["Парк C", "из A", "2", "A", "зафиксирована"],
    ["Дом 1 B", "из C", "3", "C", "зафиксирована"],
    ["Метро G", "из C", "5", "C", "зафиксирована"],
    ["Офис D", "из G", "6", "G", "зафиксирована"],
    ["Дом 2 E", "из G", "7", "G", "зафиксирована"],
    ["ТЦ F", "из E", "8", "E", "зафиксирована"],
  ],
},
"bellman-ford": {
  title: "Векторный граф релаксации (Форд–Беллман)",
  nodes: [
    { id: "S", label: "S", x: 40, y: 60, dist: "0", s: true },
    { id: "A", label: "A", x: 130, y: 30, dist: "4", s: false },
    { id: "B", label: "B", x: 130, y: 90, dist: "5", s: false },
    { id: "C", label: "C", x: 220, y: 60, dist: "3", s: false },
    { id: "D", label: "D", x: 310, y: 30, dist: "6", s: false },
    { id: "E", label: "E", x: 310, y: 90, dist: "7", s: false },
    { id: "F", label: "F", x: 400, y: 60, dist: "8", s: false },
  ],
  edges: [
    { u: "S", v: "A", w: 4, highlighted: true },
    { u: "S", v: "B", w: 5, highlighted: true },
    { u: "B", v: "C", w: -2, highlighted: true },
    { u: "A", v: "C", w: 1 },
  ],
}

```

```

    edges: [
      { u: "1", v: "2", w: 2, highlighted: true },
      { u: "2", v: "3", w: 3, highlighted: true },
      { u: "1", v: "4", w: 1, highlighted: true },
      { u: "4", v: "5", w: 4, highlighted: true },
      { u: "2", v: "5", w: 5 },
      { u: "3", v: "6", w: 2, highlighted: true },
      { u: "5", v: "6", w: 6 },
      { u: "6", v: "7", w: 3, highlighted: true },
    ],
    table: [
      ["Ребро", "Вес w", "Краскал (DSU)", "Прим (Куча)"],
      ["(1, 4)", "1", "добавлено (comp 1-4)", "добавлен"],
      ["(1, 2)", "2", "добавлено (comp 1-2-4)", "добавл"],
      ["(3, 6)", "2", "добавлено (comp 3-6)", "добавлен"],
      ["(2, 3)", "3", "добавлено (объединение)", "добав"],
      ["(6, 7)", "3", "добавлено (финиш)", "добавлено в"],
    ],
  },
};

/** A4 в пунктах. */
const PAGE = { width: 595.28, height: 841.89, margin: 40 };
const CONTENT_W = PAGE.width - PAGE.margin * 2;
const TOP = PAGE.margin;
const BOTTOM = PAGE.height - PAGE.margin - 16;

/* _____ шрифты _____

const CANDIDATES = {
  regular: [
    "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
    "/usr/share/fonts/dejavu/DejaVuSans.ttf",
    "/usr/local/share/fonts/DejaVuSans.ttf",
    "/System/Library/Fonts/Supplemental/Arial Unicode.ttf",
    "C:/Windows/Fonts/arial.ttf",
  ],
  bold: [

```

```

        .replace(/\s{2,}/g, " ")
        .replace(/\u0000/g, "")
        .trim();

/* ----- данные приложения (через esbuild) -----

async function loadAppData() {
  ensureDir(path.dirname(ENTRY));
  fs.writeFileSync(
    ENTRY,
    [
      'export { chapters, chapterTopics } from "../src/'
      'export { PAGE_SYNC } from "../src/data/vizSync";
      'export { VIZ_REGISTRY } from "../src/components/'
      'export { quizzes } from "../src/data/quizzes";',
    ].join("\n")
  );
  const res = await build({
    entryPoints: [ENTRY],
    bundle: true,
    write: false,
    format: "esm",
    platform: "node",
    jsx: "automatic",
    logLevel: "silent",
    loader: { ".ts": "ts", ".tsx": "tsx", ".jpg": "data:
      ss": "empty" },
  });
  return import("data:text/javascript;base64," + Buffer
}

/* ----- снимки визуализаций -----

```

```

function shotsFor(chapterId) {
  const dirs = [SHOTS_DIR, path.join(ROOT, "public", "e
  for (const dir of dirs) {
    if (fs.existsSync(dir)) {
      const files = fs

```

```

        this.doc.text("Универсальное пособие • алгоритмы и
            width: CONTENT_W * 0.62,
            align: "left",
            lineBreak: false,
            ellipsis: true,
        ));
        this.doc.text(`${this.footerTitle} • ${this.pages}`
            width: CONTENT_W * 0.38,
            align: "right",
            lineBreak: false,
            ellipsis: true,
        ));
        this.doc.page.margins.bottom = prevBottom;
        this.doc.y = this.cy;
    }

    newPage(footerTitle) {
        this.stampFooter();
        if (footerTitle !== undefined) this.footerTitle = footerTitle;
        this.doc.addPage({ size: [PAGE.width, PAGE.height],
            this.pages += 1;
            this.cy = TOP;
        }

    firstPage(footerTitle) {
        if (footerTitle !== undefined) this.footerTitle = footerTitle;
        this.doc.addPage({ size: [PAGE.width, PAGE.height],
            this.pages += 1;
            this.cy = TOP;
        }

    /** Сколько места нужно, чтобы элемент не оторвался от
    ensure(h, extra = 0) {
        if (this.cy + h + extra > BOTTOM) this.newPage();
    }

    height(text, { font, size, width, lineGap = 0 }) {
        return this.doc.font(font).fontSize(size).heightOfS

```

```

    }
  }
  flush();
  return chunks.length ? chunks : [source];
}

rule(color = "#c7d2fe", width = 1) {
  const y = this.cy;
  this.doc.moveTo(PAGE.margin, y).lineTo(PAGE.margin, y);
  this.cy = y + 8;
}

h1(rawText, { size = 20, color = "#111827", rule = true } = {}) {
  const text = plain(rawText);
  if (!text) return;
  const h = this.height(text, { font: this.fonts.bold });
  this.ensure(h + (rule ? 14 : 6) + gapBefore, 22);
  this.cy += gapBefore;
  this.doc.font(this.fonts.bold).fontSize(size).fillColor(color);
  this.cy = Math.max(this.cy, this.doc.y) + 2;
  if (rule) this.rule();
}

h2(rawText, { size = 13.5, color = "#1e293b" } = {}) {
  const text = plain(rawText);
  if (!text) return;
  const h = this.height(text, { font: this.fonts.bold });
  this.ensure(h + 6, 20);
  this.cy += 6;
  this.doc.font(this.fonts.bold).fontSize(size).fillColor(color);
  this.cy = Math.max(this.cy, this.doc.y) + 3;
}

h3(rawText, { size = 11.2, color = "#334155" } = {}) {
  const text = plain(rawText);
  if (!text) return;
  const h = this.height(text, { font: this.fonts.bold });
  this.ensure(h + 4, 16);
}

```

```

        gn: "right", lineGap: 1 }));
    }
    this.doc.font(this.fonts.regular).fontSize(size);
    this.cy = Math.max(y0 + h, this.doc.y) + 2.5;
    if (this.cy > BOTTOM) this.newPage();
  });
}

/** Код: режим на фрагменты по высоте страницы, каждый
code(rawText, { size = 8.0 } = {}) {
  const font = this.fonts.mono;
  const inner = CONTENT_W - 22;
  const lines = String(rawText ?? "").replace(UNSUPPORTED, " ");
  // собираем фрагменты, которые влезают в одну рамку
  let i = 0;
  while (i < lines.length) {
    // Если у низа страницы осталось меньше минимальной высоты
    // СРАЗУ: иначе рамка уедет за BOTTOM, и pdfkit сработает
    // в обход рендерера (без колонтитула и нумерации)
    if (BOTTOM - this.cy - 16 < 60) this.newPage();
    const avail = BOTTOM - this.cy - 16;
    const maxH = avail;
    let chunk = [];
    let h = 0;
    while (i < lines.length) {
      const candidate = chunk.concat(lines[i]).join(" ");
      const ch = this.height(candidate, { font, size, lineGap });
      if (chunk.length && ch > maxH) break;
      chunk.push(lines[i]);
      h = ch;
      i += 1;
    }
    const body = chunk.join("\n");
    const boxH = h + 14;
    this.ensure(boxH);
    const y0 = this.cy;
    this.doc.roundedRect(PAGE.margin, y0, CONTENT_W, boxH);
    this.doc.font(font).fontSize(size).fillColor("#1f1f1f").text(

```



```

        .lineTo(x2, y2)
        .lineWidth(strokeWidth)
        .strokeColor(strokeColor)
        .stroke();

    if (e.w !== undefined) {
        const mx = (x1 + x2) / 2;
        const my = (y1 + y2) / 2;
        this.doc
            .circle(mx, my, 7)
            .fillAndStroke("#ffffff", strokeColor);
        this.doc
            .font(this.fonts.bold)
            .fontSize(7)
            .fillColor("#0f172a")
            .text(String(e.w), mx - 7, my - 3.5, { width:
    }
}

// Вершины графа
for (const n of nodes) {
    const nx = PAGE.margin + n.x;
    const ny = startY + n.y;
    const r = 12;

    const fillColor = n.status === "done" ? "#dcfce7"
    const strokeColor = n.status === "done" ? "#16a34a"

    this.doc
        .circle(nx, ny, r)
        .lineWidth(1.5)
        .fillAndStroke(fillColor, strokeColor);

    this.doc
        .font(this.fonts.bold)
        .fontSize(8.5)
        .fillColor("#0f172a")
        .text(String(n.label ?? n.id), nx - r, ny - 4,

```

```

        .fontSize(size)
        .fillColor("#1f2937")
        .text(String(cells[i] ?? ""), x, y + pad, { w
    }
    this.doc.rect(PAGE.margin, y, CONTENT_W, h).lineW
    return h;
};

this.ensure(rowHeight(header) + 8, 14);
let y = this.cy;
rows.forEach((cells, idx) => {
    const row = [];
    for (let i = 0; i < cols; i++) row.push(cells[i]
    const h = rowHeight(row);
    if (y + h > BOTTOM) {
        this.cy = y;
        this.newPage();
        y = this.cy;
        if (idx > 0) y += drawRow(header, y, true, false
    }
    y += drawRow(row, y, idx === 0, idx % 2 === 1);
});
this.cy = y + 8;
}

```

```

quote(rawText) {
    const text = plain(rawText);
    if (!text) return;
    const font = this.fonts.oblique ?? this.fonts.regular;
    const width = CONTENT_W - 22;
    const opts = { font, size: 9.5, width, lineGap: 1 };
    // Длинная цитата режется по страницам заранее: рис
    // текст уйдёт за BOTTOM и pdfkit добавит страницу
    const chunks = this.splitToFit(text, opts, BOTTOM -
    chunks.forEach((chunk) => {
        const h = this.height(chunk, opts);
        this.ensure(h + 10);
        const y0 = this.cy;

```

```

    let rest = lines.slice();
    let first = true;
    while (rest.length || first) {
        first = false;
        const headH = this.height(title, { font: this.fonts.bold });
        this.ensure(headH + 24, 16);
        const y0 = this.cy;
        // сколько строк влезет на эту страницу
        const avail = BOTTOM - y0 - headH - 16;
        const taken = [];
        let h = 0;
        while (rest.length) {
            const ch = this.height(rest[0], { font: this.fonts.regular });
            if (taken.length && h + ch > avail) break;
            taken.push(rest.shift());
            h += ch;
        }
        const boxH = headH + h + 12;
        this.doc.roundedRect(PAGE.margin, y0, CONTENT_W, boxH, 10);
        this.doc.font(this.fonts.bold).fontSize(9.2).fill('black').text(
            title, PAGE.margin, y0 + 7 + headH);
        for (const l of taken) {
            this.doc.font(this.fonts.regular).fontSize(size).fill('black').text(
                l, PAGE.margin, y0 + 7 + headH + h);
            y += this.height(l, { font: this.fonts.regular });
        }
        this.cy = y0 + boxH + 7;
        if (rest.length) this.newPage();
    }
}
}
}

```

/\* ----- сборка ----- \*/

```

async function main() {
    const fonts = {
        regular: pick(CANDIDATES.regular),
        bold: pick(CANDIDATES.bold),
    };
}

```

```

    });
    R.para(
      "Автоматический компактный сборник: весь текст тем
        уализации. Навигация приложения и код панели Python
        b Pages.",
      { size: 9.6, color: "#374151" }
    );
    R.para(`Дата сборки: ${today}`, { size: 9, color: "#6B8E23" });

    const missingViz = chapters.filter((c) => !VIZ_REGISTERED[c.id]);
    R.box("Как устроено пособие", [
      "• Страница = один или несколько билетов: текст, анимация,
      "• У каждой страницы есть интерактивная демонстрация.
        совывают её напрямую.",
      "• В этом PDF демонстрации показаны снимками (когда
        управляют.",
      missingViz.length
        ? `• Страницы без демонстрации: ${missingViz.map(c => c.id).join(", ")}`
        : "• Демонстрация и синхронизация с компилятором Python",
    ]));

    /* — содержание — */
    R.newPage("Содержание");
    R.h1("Содержание", { size: 18 });
    chapters.forEach((c, i) => {
      const topics = chapterTopics[c.id] ?? [];
      R.para(
        `${String(i + 1).padStart(2, "0")}.  ${c.title}${
          topics.length > 0 ? " — " : ""
        }`,
        { size: 9.3, gapAfter: 2 }
      );
    });

    /* — темы — */
    let shotsTotal = 0;
    for (const chapter of chapters) {
      const topics = chapterTopics[chapter.id] ?? [];
      const shortTitle = chapter.title.replace(/^\\d+\\.--/);

```

```

    const captions = {
      dijkstra: "✂ Декстер Дуглас у CRT-монитора и ви
      "bellman-ford": " Бэтмен патрулирует Готэм в м
      floyd: " Все вершины связаны со всеми",
    };
    R.image(path.join(ROOT, img), { maxHeight: 220, c
  }

/* демонстрация: интерактивная схема (векторные гра
const viz = VIZ_REGISTRY[chapter.id];
const vecViz = VECTOR_VISUALIZERS[chapter.id];
const syncKeys = Object.keys(PAGE_SYNC).filter((k) :

if (viz || syncKeys.length || vecViz) {
  R.h2("Демонстрация и интерактивная схема");
  if (viz) {
    R.para(viz.title, { size: 10.2, color: "#1e293b
    if (viz.hint) R.para(viz.hint, { size: 9.1, col
  }

  if (vecViz) {
    if (vecViz.nodes && vecViz.edges) {
      R.vectorGraph(vecViz.nodes, vecViz.edges, { h
    }
    if (vecViz.table) {
      R.table(vecViz.table);
    }
  }
}

for (const key of syncKeys) {
  const sync = PAGE_SYNC[key];
  const demo = key.includes("#") ? key.split("#")
  const title = demo ? `Вкладка «${demo}»${sync.v
  const lines = [];
  if (sync.stepNote) lines.push(`Шаг демонстрации
  sync.variables.forEach((v) => lines.push(`• ${v
  lines.push("• Референсный код открывается в инт
  if (lines.length) R.box(title, lines);

```

```
        process.exitCode = 1;
    }
}

main().catch((err) => {
    console.error("Ошибка сборки PDF пособия:", err.message);
    process.exit(1);
});
```

---

ФАЙЛ 93/111: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМЕР: 1.5 КБ

---

```
/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/ в
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir, } from "../utils";

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
    if (!fs.existsSync(PAGES_DIR)) {
        throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
    }

    const pages = fs
        .readdirSync(PAGES_DIR)
        .filter((f) => f.endsWith(".png"))
        .sort();
```

```
process.exit(1);
});
```

ФАЙЛ 94/111: scripts/export/capture-viz-shots.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 122 | РАЗМ: 1.5 КБ

```
/**
 * Снимки визуализаций для PDF-конспекта: tmp/viz-shots.
 *
 * Headless-браузер открывает каждую страницу с демонстрацией
 * блок #chapter-viz. У страниц с вкладками (sparse-table)
 * снимается каждая вкладка – файл получает имя `chapter-viz-  
и PDF подписывает его как «режим «вкладка»».
 *
 * npm run export:shots # сервер уже поднят
 * SHOTS_BASE_URL=http://127.0.0.1:4173 npm run export:shots
 *
 * В CI это делает rebuild-pdf.yml: сборка → vite preview →
 * npm run export:guide. Локально без браузера скрипт просто
 * честно пишет, что снимки добавляются при сборке в CI.
 */
```

```
import fs from "node:fs";
import path from "node:path";
import { build } from "esbuild";
import puppeteer from "puppeteer";
import { ROOT } from "../common.mjs";
```

```
const OUT_DIR = path.join(ROOT, "tmp", "viz-shots");
const BASE_URL = process.env.SHOTS_BASE_URL || "http://127.0.0.1:4173";
```

```
/** Вкладки, которые стоит снять отдельно: глава → подглава
const TABS = {
  "sparse-table": ["1D", "2D: построение", "2D: запрос"],
  mst: ["Краскал", "Прим", "Борувка"],
  treap: ["Собрать", "Split", "Merge"],
};
```

```

let done = 0;
let failed = 0;
for (const id of ids) {
  try {
    await page.goto(`${BASE_URL}/?topic=${id}`, { waitU
    await page.waitForSelector("#chapter-viz", { timeout
    // демо-анимации: даём кадру собраться до осмысленн
    await new Promise((r) => setTimeout(r, 1600));
    const viz = await page.$("#chapter-viz");
    if (!viz) throw new Error("нет #chapter-viz");
    await viz.screenshot({ path: path.join(OUT_DIR, `${
    done += 1;

    for (const tab of TABS[id] ?? []) {
      try {
        const clicked = await page.evaluate((label) =>
          const root = document.querySelector("#chapter
          if (!root) return false;
          const btn = Array.from(root.querySelectorAll(
            (b.textContent ?? "").trim().startsWith(label
          ));
          if (!btn) return false;
          btn.dispatchEvent(new MouseEvent("click", { b
          return true;
        }, tab);
        if (!clicked) continue;
        await new Promise((r) => setTimeout(r, 1200));
        const el = await page.$("#chapter-viz");
        await el.screenshot({ path: path.join(OUT_DIR,
        done += 1;
      } catch (error) {
        console.log(`[export:shots]    вкладка «${tab}»
      }
    }
    console.log(`[export:shots] ok ${id}`);
  } catch (error) {
    failed += 1;
  }
}

```



```
const CODE_EXT = new Set([
    ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
    ".css", ".scss", ".html", ".json", ".md", ".yml", ".yaml"
]);
```

```
/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node modules", "dist", "build",
```

```
/** Человекочитаемые категории для оглавления. */
const CATEGORIES = [
  [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
  [/^src\/data\/$/, "Контент и данные пособия"],
  [/^src\/components\/visualizers\/$/, "Визуализаторы (визуализация)"],
  [/^src\/components\/$/, "Интерактивные компоненты и симуляторы"],
  [/^src\/layout\/$/, "HTML-разметка (layout)"],
  [/^src\/utils\/$/, "Утилиты"],
  [/^src\/$/, "Ядро приложения"],
  [/^scripts\/$/, "Скрипты сборки и экспорта"],
  [/^\.github\/$/, "CI / автоматизация"],
  [/^[^\/]+$/, "Конфигурация проекта"],
];
```

```
const categoryOf = (rel) => CATEGORIES.find(([re]) => r
```

**/\*\* Порядок разделов в документе. \*/**

```
const ORDER = [
    "Конфигурация проекта",
    "Ядро приложения",
    "Контент и данные пособия",
    "Билеты (учебный контент)",
    "Интерактивные компоненты и симуляторы",
    "Визуализаторы (вложенные)",
    "HTML-разметка (layout)",
    "Утилиты",
    "Скрипты сборки и экспорта",
    "CI / автоматизация",
    "Прочее",
]
```

```

function extractChapters(files) {
  const chapters = [];
  const seen = new Set();
  for (const rel of files.filter((f) => f.startsWith("s
    const src = fs.readFileSync(path.join(ROOT, rel), "
    const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"
    let m;
    while ((m = re.exec(src)) !== null) {
      const [, id, title] = m;
      if (seen.has(id)) continue;
      seen.add(id);
      chapters.push({ id, title, file: rel });
    }
  }
  return chapters.sort((a, b) => {
    const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
    const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
    return na - nb;
  }));
}

function main() {
  const files = listFiles();
  const chapters = extractChapters(files);
  ensureDir(OUT_DIR);

  const out = [];
  const push = (s = "") => out.push(s);

  const stamp = new Date().toLocaleString("ru-RU", { ti
  const totalBytes = files.reduce((s, f) => s + fs.stat

  push(HR);
  push("          УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТИ
  push("          ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФА
  push(HR);
  push();
  push(`Дата генерации:          ${stamp}`);

```

```

    }
    const content = fs.readFileSync(path.join(ROOT, rel), 'utf8');
    const lines = content.split("\n");
    push(SUBHR);
    push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
    push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} | РАЗМ: ${size}`);
    push(SUBHR);
    push();
    push(content.replace(/\n+$/, ""));
    push();
    push();
  });

  push(HR);
  push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано ${new Date().toLocaleDateString()}`);
  push(HR);

  const txt = out.join("\n") + "\n";
  fs.writeFileSync(TXT_PATH, txt, "utf8");

  console.log("[1/3] код -> txt");
  console.log(`    файлов:  ${files.length}`);
  console.log(`    глав:    ${chapters.length}`);
  console.log(`    строк:   ${txt.split("\n").length}`);
  console.log(`    выход:   ${path.relative(ROOT, TXT_PATH)}`);
}

main();

```

ФАЙЛ 96/111: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМ: 1.1 КБ

/\*\*

\* Общие настройки и утилиты пайплайна экспорта.

\*

```
export function ensureDir(dir) {
  fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
  return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
  for (const cmd of ["magick", "convert"]) {
    try {
      execFileSync(cmd, ["-version"], { stdio: "ignore" });
      return cmd;
    } catch {
      /* пробуем следующий */
    }
  }
  throw new Error(
    "ImageMagick не найден. Установите его: sudo apt-get install imagemagick"
  );
}
```

---

ФАЙЛ 97/111: scripts/export/html-to-blocks.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 266 | РАЗМЕР: 1.1 КБ

---

```
/**
 * HTML страницы пособия → плоский список блоков для переноса
 *
 * Задача: вытащить из Tailwind-разметки САМ ТЕКСТ (как в ридере), сохранив структуру и ПОРЯДОК слов: заголовки,
 * таблицы, код, спойлеры <details>. Служебная разметка (классы,
 * svg, кнопки) отбрасывается, содержимое спойлеров — нет.
```

```

    if (closeTag) {
        const tag = closeTag.toLowerCase();
        // закрываем до совпадения (прощает незакрытые <p
        for (let i = stack.length - 1; i >= 1; i--) {
            if (stack[i].tag === tag) {
                stack.length = i;
                break;
            }
        }
        continue;
    }

    if (openTag) {
        const tag = openTag.toLowerCase();
        const attrs = {};
        const attrRe = /([a-zA-Z-_:]+)\s*=\s*"([^"]*)"|'
        let a;
        while ((a = attrRe.exec(attrsStr || "")) !== null)
            const node = { tag, attrs, children: [] };
            stack[stack.length - 1].children.push(node);
            if (!VOID.has(tag) && !selfClose) stack.push(node)
            continue;
    }

    if (textChunk !== undefined && textChunk.length) {
        stack[stack.length - 1].children.push({ tag: "#text",
    }
}
return root;
}

const decodeEntities = (s) =>
    s
        .replace(/&lt;/g, "<")
        .replace(/&gt;/g, ">")
        .replace(/&quot;/g, '"')
        .replace(/&#39;/g, "'")
        .replace(/&nbsp;/g, "\u00a0")

```

```

    }
    return clean(out);
}

/** Весь текст поддерева (для ячеек таблиц и подписей).
function collectText(node) {
  if (node.tag === "#text") return clean(node.text);
  if (SKIP.has(node.tag)) return "";
  if (node.tag === "table") return tableRows(node).map(
    return clean(node.children.map(collectText).join(" "))
  )
}

/** Сырой текст внутри <pre> – переносы строк важны. */
function collectRawPre(node) {
  if (node.tag === "#text") return node.text;
  if (node.tag === "br") return "\n";
  if (SKIP.has(node.tag) && node.tag !== "img") return
  return node.children.map(collectRawPre).join("");
}

/** Таблица → массив строк из массивов ячеек. */
function tableRows(node) {
  const rows = [];
  const visit = (n) => {
    if (n.tag === "tr") {
      const cells = n.children.filter((c) => c.tag ===
      if (cells.some((c) => c.length)) rows.push(cells)
      return;
    }
    n.children.forEach(visit);
  };
  node.children.forEach(visit);
  return rows;
}

/**
 * Обход дерева: собираем блоки в порядке документа.
 * @returns {Array<{type:string,text?:string,rows?:string}>}

```

```

    const text = decodeEntities(collectRawPre(node)).
    if (text.trim()) push({ type: "code", text: text.
    return;
}

// есть блочные дети: идём по порядку, инлайн-пробе
if (hasBlockChild(node)) {
    let buf = "";
    const flush = () => {
        const t = clean(buf);
        if (t) push({ type: "p", text: t });
        buf = "";
    };
    for (const c of node.children) {
        if (c.tag === "#text") {
            buf += c.text;
            continue;
        }
        if (SKIP.has(c.tag)) continue;
        if (!BLOCK.has(c.tag)) {
            buf += inlineText(c) + " ";
            continue;
        }
        if (c.tag === "ul" || c.tag === "ol") {
            flush();
            emitList(c, listDepth);
            continue;
        }
        flush();
        walk(c, listDepth);
    }
    flush();
    return;
}

// листовой блок: один блок по типу тега
const text = inlineText(node);
if (!text) return;

```

```
* ШАГ 2: TXT -> PNG
*
* Разбивает full_code_all_pages.txt на страницы формата
* каждую страницу в PNG через ImageMagick (шрифт DejaVu)
*
* Результат: tmp/export-pages/page-0001.png, page-0002
*/
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
  ROOT, PAGES_DIR, TXT_PATH, PAGE, ensureDir, humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
  const expanded = line.replace(/\t/g, " ").replace(
    if (expanded.length <= PAGE.cols) return [expanded];
    const parts = [];
    const indent = " ".repeat(Math.min((expanded.match(/^
    let rest = expanded;
    let first = true;
    while (rest.length > 0) {
      const width = first ? PAGE.cols : PAGE.cols - inden
      parts.push((first ? "" : indent) + rest.slice(0, wi
      rest = rest.slice(width);
      first = false;
    }
    return parts;
  }

function paginate(txt) {
  const source = txt.replace(/\r\n/g, "\n").split("\n")
  const pages = [];
```



```

    "-colorspace", "Gray",
    ...(PAGE.monochrome ? ["-monochrome"] : []),
    "-strip",
    "-define", "png:compression-level=9",
    job.pngFile,
  ];

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

async function worker() {
  while (cursor < jobs.length) {
    const job = jobs[cursor++];
    await execFileAsync(im, args(job));
    fs.rmSync(job.txtFile, { force: true });
    done += 1;
    if (done % 25 === 0 || done === total) {
      process.stdout.write(`      отрендерено ${done} страниц\n`);
    }
  }
}

await Promise.all(Array.from({ length: concurrency }, () => worker()));
process.stdout.write("\n");

const missing = jobs.filter((j) => !fs.existsSync(j.pngFile));
if (missing.length) throw new Error(`Не отрендерены ${missing.length} страниц`);

const bytes = jobs.reduce((s, j) => s + fs.statSync(j.pngFile).size, 0);
console.log("[2/3] txt -> png");
console.log(`      страниц: ${total}`);
console.log(`      выход:   ${path.relative(ROOT, PAGE.pngFile)}`);
}

main().catch((err) => {
  console.error("Ошибка рендера страниц:", err.message);
  process.exit(1);
});

```

```
-----  
const realConsoleError = console.error;
```

```
/* -----  
/*  утилиты  
/* -----
```

```
const sleep = (ms: number) => new Promise((r) => setTim
```

```
async function settle(ms = 40) {  
  await act(async () => {  
    await sleep(ms);  
  });  
}
```

```
function bodyText(): string {  
  return (document.body.textContent ?? "").replace(/\s+  
}
```

```
function allButtons(): string[] {  
  return Array.from(document.querySelectorAll("button")  
    (b) => (b.getAttribute("aria-label") || b.getAttrib  
  );  
}
```

```
/** Кнопки без доступного имени: ни aria-label, ни title  
function namelessButtons(scope: ParentNode = document):  
  return Array.from(scope.querySelectorAll("button"))  
    .filter((b) => {  
      const name = [b.getAttribute("aria-label"), b.get  
        .filter(Boolean)  
        .join(" ")  
        .replace(/\s+/g, " ")  
        .trim();  
      return name.length === 0;  
    })  
    .map((b) => `<button class="${(b.className || "").s  
}
```

```
-----  
async function key(k: string, init: KeyboardEventInit =  
  await act(async () => {  
    document.dispatchEvent(new window.KeyboardEvent("ke  
    await sleep(20);  
  }));  
}
```

```
/** Ждём появления признака в DOM (текст/предикат), пер  
async function waitFor(pred: () => boolean, timeoutMs =  
  const deadline = Date.now() + timeoutMs;  
  while (Date.now() < deadline) {  
    if (pred()) return true;  
    await settle(step);  
  }  
  return pred();  
}
```

```
const RUN_DONE = [  
  "Трасса готова автоматически",  
  "Нет шагов трассы",  
  "Пустой код",  
  "Код не выполнялся",  
  "Выполнение остановлено после",  
  "Не удалось выполнить код",  
  "Ошибка:",  
];
```

```
let root: Root | null = null;  
let host: HTMLElement | null = null;
```

```
function freshHost(): HTMLElement {  
  const div = document.createElement("div");  
  document.body.appendChild(div);  
  return div;  
}
```

```
async function mountApp(topic: string) {  
  if (root) {
```

```

    });
  });
  await settle(60);
  return host;
}

const BAD_DOM = [
  { re: /\bNaN\b/, why: "NaN в разметке" },
  { re: /\bundefined\b/, why: "undefined в разметке" },
  { re: /\[object Object\]/, why: "[object Object] в ра
];

/**
 * Битые подстановки видны в КОРОТКИХ текстовых узлах (
 * вершины): «undefined», «NaN», «[object Object]». В д
 * псевдокода те же слова законны — их не трогаем.
 */
function domDefects(scope: HTMLElement | null): string[] {
  if (!scope) return [];
  const found: string[] = [];
  const walker = document.createTreeWalker(scope, 4 /*
  let node = walker.nextNode() as Text | null;
  while (node) {
    const text = (node.nodeValue ?? "").replace(/\s+/g,
    if (text.length > 0 && text.length < 60) {
      for (const bad of BAD_DOM) {
        if (bad.re.test(text)) found.push(`${bad.why}:
      }
    }
    node = walker.nextNode() as Text | null;
  }
  return [...new Set(found)];
}

async function test(group: string, name: string, fn: ()
  if (skipGroups.has(group[0])) return true;
  consoleErrors = [];
  const t0 = Date.now();

```

```
consoleErrors.push(args.map((a) => (a instanceof Er
});
```

```
/* — 1. Панель Python: реальный запуск на каждой глав
for (const chapter of chapterList) {
  const group = "A. страницы";
  await test(group, `${chapter.id}: монтируется, бейдж
    await mountApp(chapter.id);
    const txt = bodyText();
    if (!txt.includes(chapter.title.slice(0, 18))) th
    if (!VIZ_REGISTRY[chapter.id]) return "визуализац
    if (!/встроенное демо|данные из компилятора/.test
    const defects = domDefects(host);
    if (defects.length) throw new Error(`артефакты в
    const nameless = namelessButtons(host!);
    if (nameless.length) throw new Error(`кнопок без
    return `бейдж «встроенное демо» на месте, кнопок
  });
```

```
await test(group, `${chapter.id}: панель открывается
  await mountApp(chapter.id);
  const toggle = findButton("Боковая панель Python"
  await click(toggle, "кнопка панели Python в навба
  const ta = textarea();
  if (!ta) throw new Error("редактор (textarea) не
  if (!ta.value.includes("# «")) return `редактор е
  const eye = findButton(/эта?лонный код/i);
  if (!eye) throw new Error("нет кнопки-«глаза» (вс
  const run = findButton(/Запустить код|автопроход/
  if (!run) throw new Error("нет кнопки запуска ►")
  return `кнопок в панели: ${allButtons().length}`;
});
```

```
await test(group, `${chapter.id}: автозапуск инициа
  await mountApp(chapter.id);
  await click(findButton("Боковая панель Python"),
  const done = await waitFor(() => RUN_DONE.some((m
  if (!done) throw new Error(`запуск не завершился
```

```

        await sleep(30);
    });
    const txt = (host?.textContent ?? "").replace(/\\s/g, ' ');
    if (!/данные из компилятора/.test(txt)) throw new Error(`ожидали данные из компилятора`);
    if (!txt.includes(c.expect)) throw new Error(`в тесте не хватает ${c.expect}`);
    const defects = domDefects(host);
    if (defects.length) throw new Error(`артефакты в тесте: ${defects.join(' ')} `);
    return `бейдж: ${txt.match(/данные из компилятора/)?.length || 0}`;
}

/* — 3. Префиксные суммы 2D и разреженная таблица — */
const gridCases: { viz: string; chapter: string; name: string; data: any; expect: string } = [
    { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2d", name: "2D Prefix Sums", data: {
        9, 10, 11, 12]], S: [[1, 3, 6, 10], [6, 14, 24, 35]] },
    { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2d", name: "2D Prefix Sums", data: {
        pref: [[1, 3], [4, 10]], r1: 0, c1: 0, r2: 1, c2: 1 } },
    { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2d", name: "2D Prefix Sums", data: {
        9]], S: [[1, 3, 6], [5, 12, 21], [12, 27, 45]] },
    { viz: "sparse-table", chapter: "sparse-table", name: "Sparse Table", data: {
        [3, 3, 1, 1, 0], [1, 1, 0, 0, 0]] }, expect: /не совпадает/ },
    { viz: "sparse-table", chapter: "sparse-table#2d-bu", name: "Sparse Table", data: {
        0, i: 2, j: 3, st: Array.from({ length: 4 }, () => new Array(5)), expect: /ожидали данные из компилятора/ },
    { viz: "sparse-table", chapter: "sparse-table#2d-qu", name: "Sparse Table", data: {
        30, j2: 40, res: 2 }, expect: /не совпадает|данные из компилятора/ },
];

for (const c of gridCases) {
    await test("С. матричные страницы", `${c.viz}: ${c.name}`);
    await mountViz(c.viz);
    await act(async () => {
        emitVizState(c.viz, { line: 2, func: "<module>" });
        await sleep(30);
    });
    const txt = (host?.textContent ?? "").replace(/\\s/g, ' ');
    if (!c.expect.test(txt)) throw new Error(`ожидали ${c.expect}, но получили: ${txt}`);
    const defects = domDefects(host);

```

```

] as [string, Record<string, unknown>][[]] {
  await test("D. графы путей", `${viz}: вершины 0...4 в
    await mountViz(viz);
    const before = allButtons().length;
    const playEnabled = (findButton(/^Пуск|Пуск$/)) as
    await act(async () => {
      emitVizState(viz, { line: 5, func: "<module>",
      await sleep(30);
    });
    const txt = (host?.textContent ?? "").replace(/\\s
    if (!/данные из компилятора/.test(txt)) throw new
    if (!/ваши расстояния|dist/i.test(txt)) throw new
    const step = findButton(/^Шаг$|Шаг /) as HTMLButt
    if (step && !step.disabled) throw new Error("кноп
    if (domDefects(host).length) throw new Error(`арт
    await act(async () => {
      clearVizState(viz);
      await sleep(30);
    });
    const step2 = findButton(/^Шаг$|Шаг /) as HTMLBut
    if (step2 && step2.disabled) throw new Error("кноп
    return `кнопок до/после: ${before}/${allButtons()}
  });
}

```

```

/* — 5. Все референсные коды: настоящий Python → нас
const syncKeys = Object.keys(PAGE_SYNC).filter((k) =>
for (const key of syncKeys) {
  const sync = PAGE_SYNC[key];
  const chapterId = key.split("#")[0];
  await test("E. эталонный код", `${key}: выполняется
    const raw = await py.runPythonAsync(buildDebugRun
    const result = JSON.parse(String(raw)) as DebugRe
    if (result.error) throw new Error(`Python-ошибка:
    if (result.steps.length === 0) throw new Error("н
    // объявленные переменные синхронизации реально п
    const seen = new Set<string>();
    for (const s of result.steps) for (const n of Obj

```

```

typeInto(ta, "# просто комментарий\n# и ещё один\n");
const done = await waitFor(() => bodyText().include
if (!done) throw new Error(`не дождались сообщения`);
if (!textarea()) throw new Error("редактор исчез по
if (!/встроенное демо/.test(bodyText())) throw new Error
return "редактор на месте, бейдж «встроенное демо»";
});

```

```

await test("F. редактор", "синтаксическая ошибка: редактор", {
  await mountApp(ch);
  await click(findButton("Боковая панель Python"), "поиск");
  const ta = textarea();
  if (!ta) throw new Error("нет редактора");
  typeInto(ta, "def broken(:\n    pass\n");
  const done = await waitFor(() => /Код не выполнен/);
  if (!done) throw new Error(`ошибка синтаксиса не показана`);
  if (!textarea()) throw new Error("редактор исчез на панели");
  if (!/invalid syntax|SyntaxError/i.test(bodyText())) throw new Error
  return "ошибка показана, редактор остался";
});

```

```

await test("F. редактор", "гигантский print обрезается", {
  await mountApp(ch);
  await click(findButton("Боковая панель Python"), "поиск");
  const ta = textarea();
  if (!ta) throw new Error("нет редактора");
  typeInto(ta, 'dist = {0: 0}\nprint("x" * 200000)\nprint("x" * 200000)');
  const ready = await waitFor(() => bodyText().include("x" * 200000));
  if (!ready) throw new Error(`программа с огромным print не завершилась`);
  const block = document.querySelector('[data-testid="Результат"]');
  if (!block) throw new Error("блок «Результат» не отображается");
  const out = block.textContent ?? "";
  if (!/вывод обрезан/.test(out)) throw new Error(`вывод не обрезан`);
  if (out.includes("ХВОСТ_ВЫВОДА")) throw new Error("хвост выводится");
  if (out.length > 40000) throw new Error(`в панели вывод больше 40000 знаков`);
  if (!textarea()) throw new Error("редактор исчез по панели");
  if (domDefects(host).length) throw new Error(`артефакты в DOM`);
  return `вывод обрезан до ${out.length} знаков, хвост выводится`;
});

```



```

    0, 100)}}`);
}
const mustBe = [
  { what: "пуск/автопроход", ge: /Запустить код|Про/ },
  { what: "копирование кода", ge: /Скопировать код|/ },
  { what: "заккрыть панель", ge: /Заккрыть компилятор/ },
  { what: "шаг вперёд", ge: /Шаг вперёд|вперёд/i },
  { what: "шаг назад", ge: /Шаг назад|назад/i },
  { what: "свернуть листинг", ge: /Свернуть листинг/ },
];
const btns = allButtons().join(" | ");
for (const m of mustBe) {
  if (!m.re.test(btns)) throw new Error(`в панели не`);
}
// На первом шаге переменных ещё нет (отладчик встал)
// поэтому связь с демонстрацией проверяем, дойдя до
await click(findButton("В конец трассы"), "в конец");
const linked = await waitFor(() => /данные из компи/);
if (!linked) throw new Error(`эталон выполнен до кода`);

// — глаз ВЫКЛ: кода нет вовсе, ничего не выполняе
await click(findButton("Убрать эталонный код"), "глаз");
const cleared = await waitFor(() => bodyText().includes(""));
if (!cleared) throw new Error(`после выключения глаза`);
const ta3 = textarea();
if (!ta3) throw new Error("редактор пропал после выключения");
if (ta3.value.trim() !== "") throw new Error(`глаз не выключен`);
if (!/встроенное демо/.test(bodyText())) throw new Error("демо не включено");
if (/данные из компилятора/.test(bodyText())) throw new Error("демо не включено");
const afterButtons = allButtons();
const lost = editorButtons.filter((b) => b && !afterButtons.includes(b));
if (lost.length > 2) throw new Error(`после снятия`);

// ѓ возвращает инициализацию страницы
await click(findButton("Сбросить к шаблону страницы"), "сбросить");
const back = await waitFor(() => textarea()?.value === "");
if (!back) throw new Error(`ѓ не вернул инициализацию`);
return "эталон вставляется в редактор, ведёт демонстрацию";

```

```

    if (codeLines() !== 0) throw new Error(`листинг не
    if (!findButton("В конец трассы")) throw new Error(
    await click(findButton("Показать листинг трассы"),
    await settle(80);
    if (codeLines() === 0) throw new Error("листинг не
    // Esc: прогон остановлен, трасса убрана, редактор
    await key("Escape");
    await settle(150);
    if (!textarea()) throw new Error("Esc убрал редактор
    if (findButton(/Остановить прогон трассы/)) throw n
    if (!/встроенное демо/.test(bodyText())) throw new
    return "редактор всегда на месте: трасса рядом, сво
  ));

```

```

await test("F. редактор", "закрытие панели во время пр
  await mountApp("floyd"); // 869 шагов: прогон долги
  await click(findButton("Боковая панель Python"), "п
  await waitFor(() => panel() !== null, 4000);
  await click(findButton("Закреть компилятор"), "крес
  await settle(150);
  if (panel()) throw new Error("панель не закрылась")
  const backToDemo = await waitFor(() => /встроенное
  if (!backToDemo) throw new Error(`после закрытия па
  await settle(3000); // ждём, пока догоняющий прогон
  if (/данные из компилятора/.test(bodyText())) {
    throw new Error("закрытая панель всё-таки опублик
  }
  if (!/встроенное демо/.test(bodyText())) throw new
  return "панель закрыта во время выполнения – демонс
  ));

```

```

await test("H. доступность", "панель: все кнопки подп
  await mountApp(ch);
  await click(findButton("Боковая панель Python"), "п
  const namelessEditor = namelessButtons(panel() ?? d
  const editorBtns = allButtons().length;
  const listing = await waitFor(() => !!findButton(/О
  if (!listing) throw new Error("панель не показала т

```

```

    if (/переменных пока нет/.test(bodyText())) throw new Error("нет переменных пока нет");
    return "демо → выполнено/переменных нет → данные из";
  });

```

/\* — 7. Текстовый режим для PDF/парсеров —————

/\* — 9. Ссылки между билетами, разделы содержания, б.

```

await test("Н. доступность", "содержание сжимается в");
await mountApp("bridges-code");
const root = () => document.querySelector<HTMLElement>("#content");
const toggle = () => findButton(/Сжать содержание|С
if (!root()) throw new Error("нет сайдбара");
if (!/lg:w-56/.test(root()!.className)) throw new Error("неверная ширина");
await click(toggle(), "сжать");
if (!/lg:w-44/.test(root()!.className)) throw new Error("неверная ширина");
const items = Array.from(root()!.querySelectorAll("li"));
if (items.length < chapters.length) throw new Error("неверная ширина");
if (root()!.querySelector("input")) throw new Error("неверная ширина");
await click(toggle(), "скрыть");
if (!/lg:w-0/.test(root()!.className)) throw new Error("неверная ширина");
await click(toggle(), "развернуть");
if (!/lg:w-56/.test(root()!.className)) throw new Error("неверная ширина");
return "широкое → узкое (темы на месте, поиска нет)";
});

```

```

await test("J. шапка", "кнопки загрузки не прячутся и");
await mountApp("segment-trees");
const menu = document.querySelector("#download-menu");
if (!menu) throw new Error("нет меню загрузки в шапке");
const want = [
  { id: "download-guide-pdf-btn", file: "export/guide.pdf" },
  { id: "download-pdf-btn", file: "export/full_code.pdf" },
  { id: "download-txt-btn", file: "export/full_code.txt" },
];
for (const w of want) {
  const el = menu.querySelector<HTMLAnchorElement>(`a[id="${w.id}"]`);
  if (!el) throw new Error(`в шапке нет кнопки «${w.id}»`);
  const href = el.getAttribute("href") ?? "";

```

```

-----
    if (headers.length !== SECTIONS.length) {
      throw new Error(`в содержании ${headers.length} р
    }
    const expected = SECTIONS.map((sec) => sec.title).join(" | ")
    if (headers.join(" | ") !== expected) throw new Error(`ожидалось:
    // 11 «Мосты» и 12 «Точки сочленения» читаются парой
    const items = Array.from(sidebar.querySelectorAll("b"));
    const iBridges = items.findIndex((t) => t.includes("Мосты"));
    const iArtic = items.findIndex((t) => t.includes("Точки сочленения"));
    if (iBridges < 0 || iArtic < 0) throw new Error(`в содержании не найдены
    if (Math.abs(iArtic - iBridges) !== 1) throw new Error(`ожидалось:
    // 2 «разреженная» и 5 «префиксные» – в одном разделе
    if (sectionOf("sparse-table").id !== sectionOf("prefixes").id)
      throw new Error("билеты 2 и 5 попали в разные разделы");
  }
  if (sectionOf("bridges-code").id !== sectionOf("graphs-code").id)
    throw new Error("билеты 11 и 12 попали в разные разделы");
  // порядок чтения: номера билетов сверху вниз не убывающий
  const nums = Array.from(sidebar.querySelectorAll("b"))
    .map((b) => (b.querySelector("span")?.textContent || ""))
    .filter((t) => /\d/.test(t))
    .map((t) => Number.parseInt(t, 10));
  const ticketed = chapters.filter((c) => (chapterTopic(c) !== ""));
  if (nums.length !== ticketed) {
    throw new Error(`номера билетов видны у ${nums.length} билетов`);
  }
  for (let i = 1; i < nums.length; i += 1) {
    if (nums[i] < nums[i - 1]) throw new Error(`порядок билетов не убывающий`);
  }
  if (!sidebar.textContent?.includes("Порядок чтения"))
    throw new Error("в содержании нет подсказки, в каком порядке читать");
  return `разделов ${headers.length}, 11+12 соседствуют`;
});

```

```

await test("I. ссылки", "введение: базовая база, обзор");
await mountApp("intro");

```

```

    }
    const nextBtn = ask("Следующий вопрос");
    if (!nextBtn) throw new Error("нет кнопки «Следующий»");
    await click(nextBtn, "дальше");
    await settle(80);
    if (!shown().includes(list[1].question.slice(0, 24)))
    // верный ответ
    await click(ask(list[1].options[list[1].correctIndex]));
    await settle(80);
    if (!/Верно/.test(shown())) throw new Error("на верном ответе");
    if (!/вопрос 2\\//.test(shown())) throw new Error(`список вопросов`);
    return `блиц из ${list.length} вопросов отвечает, правильно`;
  });
});

await test("I. факты", "память: разреженная жирная, префиксная", async () => {
  const memoryRow = async (topic: string) => {
    await mountApp(topic);
    const rows = Array.from(document.querySelectorAll("table tbody tr"));
    const row = rows.find((r) => /^\\s*Память/.test(r.textContent));
    if (!row) throw new Error(`на странице «${topic}» не найдено`);
    return Array.from(row.querySelectorAll("td")).map((td) => td.textContent);
  };
  const sparse = await memoryRow("sparse-table");
  const prefix = await memoryRow("prefix-sums-2d");
  // билет 2: у префиксной N × M, у разреженной – с логарифмами
  if (!/log/.test(sparse.join(" "))) throw new Error("билет 2: разреженная");
  const prefixCell = sparse.find((c) => !/log/.test(c));
  if (!prefixCell) throw new Error("в сравнении билетов");
  // билет 5: зеркально – у префиксной без логарифмов
  if (!/log/.test(prefix.join(" "))) throw new Error("билет 5: префиксная");
  if (/log/.test(prefix[1] ?? "")) throw new Error("билет 5: префиксная");
  const txt = bodyText();
  if (/префиксн[^.]\\{0,40\\}(жирн|больше памяти)/i.test(txt))
    throw new Error("на странице билета 5 префиксная");
  await mountApp("sparse-table");
});

```

```

typeInto(
  ta,
  [
    "n = 4",
    "INF = 999",
    "d = [[0, 3, INF, 7], [8, 0, 2, INF], [5, INF, 0, 6], [4, 1, 0, 0]]",
    "k = 0",
    "i = 0",
    "j = 0",
    "for k in range(n):",
    "    for i in range(n):",
    "        for j in range(n):",
    "            if d[i][k] + d[k][j] < d[i][j]:",
    "                d[i][j] = d[i][k] + d[k][j]",
    "print(d[0], d[1][0])",
  ].join("\n")
);
const ready = await waitFor(() => bodyText().includes("Готово"));
if (!ready) throw new Error(`своя программа не выполнена`);
if (!bodyText().includes("[0, 3, 5, 6]")) {
  throw new Error(`Флойд на своей матрице 4x4 посчитал`);
}
if (!textarea()) throw new Error("редактор исчез, попробуйте еще раз");
await click(findButton("В конец трассы"), "в конец трассы");
const linked = await waitFor(() => /данные из компилятора/);
if (!linked) throw new Error("демонстрация не принята");
const page = bodyText();
if (!/4x4/.test(page)) throw new Error(`демонстрация не принята`);
const defects = domDefects(document.querySelector("#defects"));
if (defects.length) throw new Error(`артефакты в демонстрации`);
return "набрал 4x4 с INF=999 → d[0] = [0, 3, 5, 6]";
});

await test("К. свой код", "своя разреженная таблица на Python");
await mountApp("sparse-table");
await click(findButton("Боковая панель Python"), "поиск");
const ta = textarea();
if (!ta) throw new Error("нет редактора");

```

```

    if (!/компонент: 2/.test(out)) throw new Error(`в г
    if (!textarea()) throw new Error("редактор исчез, п
    await click(findButton("В конец трассы"), "в конец
    const linked = await waitFor(() => /данные из компи
    if (!linked) throw new Error("демонстрация не приня
    const defects = domDefects(document.querySelector("
    if (defects.length) throw new Error(`артефакты в де
    return "эталон Косарайю: order считается DFS-ом, на
  });

await test("G. lite-режим", "?lite=1 отдаёт текст без
  if (root) {
    await act(async () => {
      root?.unmount();
    });
    root = null;
  }
  if (host) host.remove();
  host = freshHost();
  window.history.replaceState({}, "", "/?lite=1&topic
  root = createRoot(host);
  await act(async () => {
    root!.render(<App />);
  });
  await settle(120);
  const txt = bodyText();
  if (txt.length < 2000) throw new Error(`lite-режим
  if (/main\.py|Pyodide/.test(txt)) throw new Error("
  if (!/разреженн/i.test(txt)) throw new Error("в lite
  if (findButton("Боковая панель Python")) throw new
  // текстовая версия: ссылки на другие билеты ведут
  // а блиц напечатан списком с ответами
  const liteLinks = Array.from(document.querySelectorAll
  const bad = liteLinks.filter((a) => !(a.getAttribute
  if (liteLinks.length === 0) throw new Error("в lite
  if (bad.length) throw new Error(`в lite-версии ${ba
  const unknown = liteLinks.filter((a) => !chapters.s
  if (unknown.length) throw new Error(`в lite-версии

```

---

ФАЙЛ 100/111: scripts/verify/check-links.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 199 | РАЗМЕР: 10 000 байт

---

```
/**
 * Аудит внутренних ссылок пособия: ничего не должно ве
 *
 *   npm run verify:links
 *
 * Проверяет (по настоящим модулям приложения, собранным)
 *   1. билеты 1–24 закрыты страницами, id в chapterTopics
 *   2. разделы содержания: каждая страница в своём разд
 *   3. упоминания «билет N» в тексте ведут на существующ
 *   4. все data-goto / href="?topic=..." указывают на су
 *   5. якоря href="#..." есть в той же главе;
 *   6. glossary.simulator существует в реестре и (кроме
 *       имеет свою страницу — иначе модалка не получит
 *   7. glossary.демо — работающая мини-демо;
 *   8. у каждого ключа PAGE_SYNC есть запись в реестре
 *   9. у каждой страницы есть визуализация;
 *  10. страницы связаны перекрёстными ссылками (темы н
 */
import * as esbuild from "esbuild";
import { mkdirSync, writeFileSync } from "fs";

const OUT = "tmp/verify/links-entry.mjs";
const FAILS = [];
const NOTES = [];
const ok = (cond, message) => (cond ? true : (FAILS.push(message), false));

mkdirSync("tmp/verify", { recursive: true });
writeFileSync(
  "tmp/verify/links-entry.ts",
  [
    'export { chapters, chapterTopics, SECTIONS, section
    'export { GLOSSARY } from "../../src/data/glossary"
```



```

    tickets.forEach((n) => covered.add(n));
  }
  for (let n = 1; n <= 24; n += 1) {
    ok(covered.has(n), `билет ${n} не закрыт ни одной стр
  }

/* 2. разделы содержания */
for (const chapter of chapters) {
  const sec = sectionOf(chapter.id);
  ok(!!sec, `страница «${chapter.id}» не попала ни в од
  const tickets = chapterTopics[chapter.id] ?? [];
  ok(
    tickets.every((n) => n >= sec.from && n <= sec.to),
    `билеты страницы «${chapter.id}» (${tickets.join(",
  });
}
{
  const bySection = SECTIONS.map((s) => ({
    s,
    nums: chapters.flatMap((c) => (sectionOf(c.id).id =
  }));
  for (let i = 1; i < bySection.length; i += 1) {
    const prevMax = Math.max(...bySection[i - 1].nums);
    const curMin = Math.min(...bySection[i].nums);
    ok(curMin > prevMax, `разделы пересекаются: «${bySe
  }
  const all = bySection.flatMap((x) => x.nums);
  for (let n = all[0]; n <= all[all.length - 1]; n += 1
    ok(all.includes(n), `в содержании дыра: билет ${n}
  }
}

/* 3–5. ссылки внутри текста страниц */
const TICKET_RE = /билет(?:ы|ов|ам|ах|ом|у|а|е)?\s*(\d{
let ticketMentions = 0;
let explicitLinks = 0;
const withoutLinks = [];
for (const chapter of chapters) {

```

```

-----
for (const term of GLOSSARY) {
  if (term.simulator) {
    ok(registryIds.includes(term.simulator), `глоссарий
    ok(
      chapterIds.has(term.simulator) || EXTRA_DEMOS.has
      `глоссарий «${term.id}»: simulator "${term.simula
      ые компилятора`
    );
  }
  if (term.demo) ok(demoKinds.has(term.demo), `глоссарий
}

/* 8–9. реестр и синхронизация */
for (const key of Object.keys(PAGE_SYNC)) {
  const base = key.split("#")[0];
  ok(registryIds.includes(base), `PAGE_SYNC["${key}"]:
}
for (const id of registryIds) {
  if (!chapterIds.has(id) && !EXTRA_DEMOS.has(id)) {
    FAILS.push(`реестр: визуализация "${id}" не является
  }
}
for (const chapter of chapters) {
  ok(registryIds.includes(chapter.id), `страница «${cha
}

/* 96. файлы, на которые ведут кнопки шапки, действител
{
  const { existsSync, statSync } = await import("fs");
  for (const file of ["public/export/guide_lite.pdf", "
    ok(existsSync(file), `шапка ссылается на ${file}, н
    if (existsSync(file) && statSync(file).size < 1000)
  }
}

/* 10. связанные темы + блиц */
for (const id of withoutLinks) NOTES.push(`нет перекрёс
const noQuiz = chapters.filter((c) => !(quizzes[c.id] ?

```

```
import { writeFileSync, mkdirSync } from "fs";
import { loadPyodide } from "pyodide";

const OUT_DIR = "tmp/verify";
const t0 = Date.now();
mkdirSync(OUT_DIR, { recursive: true });

writeFileSync(
  `${OUT_DIR}/contract-entry.ts`,
  `import { buildDebugRunner } from "../../src/data/debug";
import { PAGE_SYNC } from "../../src/data/vizSync";
import { chapters } from "../../src/data/content";
export const runnerOf = (src: string) => buildDebugRunner(src);
export const sync = Object.fromEntries(Object.entries(PAGE_SYNC));
export const declared = Object.fromEntries(Object.entries(DECLARED));
export const chapterIds = chapters.map((c) => c.id);
`
);

const built = await esbuild.build({
  entryPoints: [`${OUT_DIR}/contract-entry.ts`],
  bundle: true,
  outfile: `${OUT_DIR}/contract-bundle.mjs`,
  format: "esm",
  platform: "node",
  jsx: "automatic",
  sourcemap: false,
  loader: { ".css": "empty", ".png": "dataurl", ".jpg": "dataurl" },
  logLevel: "warning",
});
void built;
const mod = await import(`${process.cwd()}/${OUT_DIR}/contract-bundle.mjs`);

const py = await loadPyodide({ indexURL: "node_modules/pyodide/" });
const printed = [];
py.setStdout({ batched: (m) => printed.push(m) });
py.setStderr({ batched: (m) => printed.push("[stderr] " + m) });
```

```

const isEmpty = (v) =>
  Array.isArray(v) ? v.length === 0
  : typeof v === "string" ? v.length === 0
  : typeof v === "object" && v !== null ? Object.keys
  : false;
const neverFilled = (mod.declared[key] ?? [])
  .map(alts)
  .filter((a) => a.some(ident))
  .map((a) => a.find((x) => ident(x) && seen.has(x)))
  .filter(Boolean)
  .filter((name) => {
    let sawValue = false;
    let sawNonEmpty = false;
    for (const step of [...result.steps, { values: re
      const v = step.values?.[name];
      if (v === undefined || v === null) continue;
      sawValue = true;
      if (!isEmpty(v)) sawNonEmpty = true;
    })
    return sawValue && !sawNonEmpty;
  });
if (neverFilled.length) issues.push(`объявлены, но ни

const stderr = printed.filter((p) => p.startsWith("[s
if (stderr.length) issues.push(`предупреждения: ${std

if (issues.length) problems++;
rows.push({ key, steps: result.steps.length, vars: se
}

rows.sort((a, b) => (a.status === "ок") - (b.status ===
console.log("ЭТАЛОННЫЕ КОДЫ СТРАНИЦ (настоящий CPython
console.log("=".repeat(96));
for (const r of rows) {
  const mark = r.status === "ок" ? "  ok  " : "  x  ";
  console.log(`${mark}${r.key.padEnd(34)} шагов ${String
    === "ок" ? "" : r.status}`);
}

```

```
const dom = new JSDOM(
  `<!doctype html><html lang="ru"><head><meta charset="utf-8"><body><div id="root"></div></body></html>`,
  { url, pretendToBeVisual: true, runScripts: "outside-only" }
);
const { window } = dom;
```

```
window.ResizeObserver = window.ResizeObserver ?? OBSERVER_API.ResizeObserver;
window.IntersectionObserver = window.IntersectionObserver ?? OBSERVER_API.IntersectionObserver;
window.matchMedia =
  window.matchMedia ??
  ((media) => ({
    matches: false,
    media,
    onchange: null,
    addListener() {},
    removeListener() {},
    addEventListener() {},
    removeEventListener() {},
    dispatchEvent: () => false,
  }));
window.scrollTo = () => {};
window.HTMLElement.prototype.scrollIntoView = () => {};
window.HTMLElement.prototype.focus = window.HTMLElement.prototype.blur = () => {};
window.URL.createObjectURL = window.URL.createObjectURL || () => "";
window.URL.revokeObjectURL = window.URL.revokeObjectURL || () => "";
Object.defineProperty(window, "innerWidth", { value: 400 });
Object.defineProperty(window, "innerHeight", { value: 400 });
Object.defineProperty(window, "devicePixelRatio", { value: 1 });
window.Element.prototype.getBoundingClientRect = function() {
  return { x: 0, y: 0, top: 0, left: 0, right: 400, bottom: 400 };
};
```

```
const copy = [
  "window",
  "document",
  "navigator",
  "HTMLElement",
];
```

```
    return { dom, window, document: window.document, cons
```

```
}
```

---

ФАЙЛ 103/111: scripts/verify/live-matrix.test.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 103 | РАЗМЕР: 1000

---

```
/**
 * Модульные проверки разбора «живых» данных компилятор
 *
 * npm run verify:unit
 *
 * liveMatrix() – то, благодаря чему визуализации матриц
 * суммы 2D, разреженная таблица) принимают данные ЛЮБОГО
 * пользователя вместо захардкоженной демо-сетки.
 */
```

```
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";
```

```
const OUT_DIR = "tmp/verify";
mkdirSync(OUT_DIR, { recursive: true });
writeFileSync(
  `${OUT_DIR}/unit-entry.ts`,
  `export { liveMatrix, pickLiveVar, clampIndex } from
`);
```

```
await esbuild.build({
  entryPoints: [`${OUT_DIR}/unit-entry.ts`],
  bundle: true,
  outfile: `${OUT_DIR}/unit-bundle.mjs`,
  format: "esm",
  platform: "node",
  jsx: "automatic",
  loader: { ".css": "empty" },
  logLevel: "warning",
});
```

```

-----
check("пустой список → null", m([]), null);
check("пустой словарь → null", m({}), null);
check("undefined → null", m(undefined), null);
check("null → null", m(null), null);
check("число → null", m(42), null);
check("строка → null", m("abc"), null);
check("массив строк-мусора → null", m(["a", "b"], ["c"

/* — liveMatrix: особые значения -----
check('"inf" → +Infinity', cells(["inf", 1], [2, "-inf
check('числа-строки "42" → 42', cells(["1", "2"], ["3"
check("булевы значения ячейкой не считаются", m([[true,
check("обрывок до 64×64", shape(Array.from({ length: 80
    4", numbers: true }));
check("hasNumbers=false невозможна для живой матрицы",

/* — pickLiveVar -----
check("первое найденное имя по приоритету", pickLiveVar
check("берёт следующее имя, если первого нет", pickLive
check("ничего не найдено → null", pickLiveVar({ x: 1 },
check("нет снимка → null", pickLiveVar(undefined, ["d"]
check("ключ со значением undefined всё равно считается"

/* — clampIndex -----
check("null остаётся null", clampIndex(null, 5), null);
check("размер 0 → null", clampIndex(2, 0), null);
check("обрезка сверху", clampIndex(99, 5), 4);
check("обрезка снизу", clampIndex(-7, 5), 0);
check("дробный индекс усекается", clampIndex(2.9, 5), 2
check("NaN → null", clampIndex(Number.NaN, 5), null);
check("Infinity → null", clampIndex(Number.POSITIVE_INF

console.log("=".repeat(72));
console.log(`liveMatrix / pickLiveVar / clampIndex: пр
for (const f of failures) console.log(`  × ${f}`);
console.log("=".repeat(72));
process.exit(failures.length ? 1 : 0);

```

```

        step ? step[0] : "-" } | футер: ${footer()}\n`
    );
    process.stdout.write(`                бейдж страницы: ${
        /встроенное демо/.test(document.body.textContent ??
    }
function typeInto(ta: HTMLTextAreaElement, value: string) {
    const setter = Object.getOwnPropertyDescriptor(window
    act(() => {
        setter.call(ta, value);
        ta.dispatchEvent(new window.Event("input", { bubble
    });
}
async function click(el: Element | null | undefined, label: string) {
    if (!el) throw new Error(`нет кнопки: ${label}`);
    await act(async () => {
        el.dispatchEvent(new window.MouseEvent("click", { bubble
        await sleep(20);
    });
}
function findButton(match: RegExp): HTMLButtonElement | null {
    return Array.from(document.querySelectorAll("button"))
        .filter(b => match.test([b.getAttribute("aria-label"), b.getAttribute("value")]))
        .first() as HTMLButtonElement | undefined;
}

export async function probe() {
    const host = document.createElement("div");
    document.body.appendChild(host);
    window.history.replaceState({}, "", `/?topic=dijkstra`);
    const root = createRoot(host);
    await act(async () => {
        root.render(<App />);
    });
    await settle(80);

    process.stdout.write("\n=== 1. открыли панель, ждём а
    await click(findButton(/Боковая панель Python/), "панель Python");
    state("t=0");

```



```

    state("очистили");
    for (const t of [200, 800, 2000]) {
      await settle(t);
      state(`t+${t}`);
    }
  } else {
    process.stdout.write("  редактора нет (листинг) – ждём редактора\n");
    await act(async () => {
      document.dispatchEvent(new window.KeyboardEvent("keydown", {
        key: "Esc",
      }));
      await sleep(30);
    });
    state("после Esc");
  }
}

```

ФАЙЛ 105/111: scripts/verify/README.md

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМЕР: 1.5 КБ

# Проверки приложения (`npm run verify`)

Три уровня проверок, которые можно гонять локально и в CI. Для запуска нужен: DOM даёт `jsdom`, Python – `pyodide` из `node\_modules`. Версия 3.12, что грузится в браузере пользователя, только без Node.js.

```

```bash
npm run verify          # всё сразу: unit → эталонные коды
npm run verify:unit     # 38 проверок разбора живых данных
npm run verify:code     # 46 эталонных кодов страниц по DOM
npm run verify:dom      # 139 сценариев: страницы, панели
npm run verify:probe    # диагностика панели Python во время
```

```

Отчёты кладутся в `tmp/verify/` (каталог в .gitignore): по умолчанию по DOM-сценариям, `reference-code.json` – по эталонным кодам.

## Универсальное пособие • полный исходный код

.....  
| лает эталон и возвращает инициализацию; код не перете  
| G. lite-режим (1) | `?lite=1` отдаёт текст без навига  
| H. доступность (2) | в панели подписаны все кнопки в  
| очника |

Фильтры: `HAR\_CHAPTERS=floyd,dijkstra`, `HAR\_SKIP=A,E`,

## Как это заведено в приложение

- \* `window.\_\_algv0PythonRuntime` – точка подмены интерпретатора  
`PythonCompiler.tsx`. В браузере пользователя поле не  
грузит Pyodide с CDN как раньше; харнес кладёт туда л
- \* Состояние визуализаций передаётся через `emitVizState`  
(`src/data/vizStepBus.ts`) – те же события, что шлёт  
идёт по боевому пути, а не через внутренние API компо

.....  
ФАЙЛ 106/111: scripts/verify/run-browser-checks.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 96 | РАЗМ

.....

/\*\*

\* DOM-проверки приложения: jsdom + настоящий Pyodide и

\*

\* npm run verify:dom

\*

\* Фильтры:

\* HAR\_CHAPTERS=floyd,dijkstra только эти страницы

\* HAR\_SKIP=A,E пропустить группы (A

\*

\* C матрицы, D графы, E

\*

\* F редактор, G lite, H

\*

\* HAR\_SYNC=floyd,mst#prim только эти записи эт

\*

\* Оркестратор: грузит CPython ДО создания jsdom (иначе

\* в браузере, и лезет за wasm на CDN), кладёт runtime и

\* window.\_\_algv0PythonRuntime и собирает реальные компо

\*/

```
    logLevel: "warning",
  });
const mod = await import(`${process.cwd()}/${OUT_DIR}/b

process.on("unhandledRejection", (reason) => {
  jsdomErrors.push(`unhandledRejection: ${reason instan
});

log("прогоняю сценарии...\n");
const results = await mod.runSuite({ py, window, document
const sum = mod.summary();

writeFileSync(`${OUT_DIR}/report.json`, JSON.stringify(

const failed = results.filter((r) => !r.ok);
console.log("\n" + "=".repeat(96));
console.log("СВОДКА");
console.log("=".repeat(96));
for (const [group, g] of Object.entries(sum.byGroup)) {
  console.log(`  ${group.padEnd(26)} ok: ${String(g.ok)
}
console.log(`\n  ВСЕГО ${sum.total}:  прошло ${sum.pass
if (failed.length) {
  console.log("\nПРОВАЛЫ:");
  for (const f of failed) console.log(`  x [${f.group}]
}
if (jsdomErrors.length) {
  console.log("\nОШИБКИ ОКРУЖЕНИЯ (jsdom/неперехваченны
  for (const e of [...new Set(jsdomErrors)].slice(0, 10
}
const stray = consoleErrors.filter((e) => !/not wrapped
if (stray.length) {
  console.log(`\nCONSOLE.ERROR вне тестов (${stray.leng
  for (const e of [...new Set(stray)].slice(0, 10)) con
}
console.log(`\nотчёт: ${OUT_DIR}/report.json  |  время:
process.exit(sum.failed > 0 || jsdomErrors.length > 0 ?
```

```
target: "es2022",
jsx: "automatic",
sourcemap: true,
define: {
  "process.env.NODE_ENV": '"development"',
  "import.meta.env.BASE_URL": '"/'',
  "import.meta.env.DEV": "true",
  "import.meta.env.PROD": "false",
},
loader: { ".css": "empty", ".png": "dataurl", ".jpg":
logLevel: "warning",
});
const mod = await import(`${process.cwd()}/${OUT_DIR}/p
log("зондирую панель...");
await mod.probe();
process.exit(0);
```

---

## РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

---

---

ФАЙЛ 108/111: `.github/workflows/deploy-pages.yml`  
КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9

---

name: Deploy app to GitHub Pages

```
# ДВЕ настройки репозитория нужно сделать один раз ВРУЧ
# (у бота-интеграции нет admin-прав на них – API отвеча
#
# 1. Settings → Pages → Build and deployment → Source
#    Если там «Deploy from a branch» (legacy) – шаг D
# 2. Settings → Environments → github-pages → Deploym
#    добавьте паттерн «arena/**» и ветку «main» (или
#    иначе деплой из новой рабочей ветки отклоняется .
```

- name: Make GitHub Pages fallback  
run: cp dist/index.html dist/404.html
- name: Upload Pages artifact  
uses: actions/upload-pages-artifact@v5  
with:  
path: ./dist

deploy:

name: Publish to GitHub Pages  
needs: build  
runs-on: ubuntu-latest  
environment:  
name: github-pages  
url: \${ steps.deployment.outputs.page\_url }

steps:

# Ранняя и понятная ошибка вместо невнятного «Fail»  
# когда в репозитории Pages не переключён на сбор

- name: Preflight – Pages собирается через GitHub

env:  
GH\_TOKEN: \${ secrets.GITHUB\_TOKEN }

run: |

bt=\$(gh api "repos/\${ github.repository }/pages/build\_type")

echo "Pages build\_type: \${bt}"

if [ "\${bt}" != "workflow" ]; then

echo "::error title=Pages не настроен::Settings не настроены. См. комментарий в начале деплоя"

exit 1

fi

- name: Deploy  
id: deployment  
uses: actions/deploy-pages@v5

jobs:

  rebuild-pdf:

    name: код → txt → png → pdf

    runs-on: ubuntu-latest

    # не реагируем на собственный коммит бота

    if: "!contains(github.event.head\_commit.message, '[

  steps:

    - name: Checkout

      uses: actions/checkout@v6

      with:

        fetch-depth: 0

        persist-credentials: true

    - name: Setup Node.js

      uses: actions/setup-node@v7

      with:

        node-version: "24"

        cache: npm

    - name: Install ImageMagick + DejaVu fonts

      run: |

        sudo apt-get update -qq

        sudo apt-get install -y --no-install-recommen

        # На Ubuntu политика ImageMagick иногда запре

        # разрешаем то, что нужно пайплайну (text -> p

        for policy in /etc/ImageMagick-6/policy.xml /

          if [ -f "\$policy" ]; then

            sudo sed -i 's/rights="none" pattern="\{P

          fi

        done

        convert -version

    - name: Install dependencies

      run: npm ci --no-audit --no-fund

    - name: Step 1 – код → txt

      run: npm run export:txt

- name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \`public/export/full\_code\_all\_pages`"  
 echo "| \`public/export/full\_code\_all\_pages`"  
 echo "| \`public/export/guide\_lite.pdf\` | "  
 echo "| Снимков визуализаций | \${ls tmp/viz\*} "  
 echo "| PNG-страниц | \${ls tmp/export-pages\*} "  
 } >> "\$GITHUB\_STEP\_SUMMARY"
- name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v7  
with:  
 name: full-code-export  
 path: |  
 public/export/full\_code\_all\_pages.pdf  
 public/export/full\_code\_all\_pages.txt  
 if-no-files-found: error  
 retention-days: 30
- name: Upload artifacts (PNG-страницы)  
uses: actions/upload-artifact@v7  
with:  
 name: full-code-pages-png  
 path: tmp/export-pages/\*.png  
 if-no-files-found: error  
 retention-days: 7
- name: Commit rebuilt export back to the branch  
run: |  
 git config user.name "github-actions[bot]"  
 git config user.email "41898282+github-action@users.noreply.github.com"  
 git add -- public/export/full\_code\_all\_pages.pdf  
 git add -- tmp/viz-shots

```
permissions:  
  contents: read
```

```
jobs:  
  verify:  
    name: unit + эталонный код + DOM  
    runs-on: ubuntu-latest
```

```
  steps:  
    - uses: actions/checkout@v4
```

```
    - uses: actions/setup-node@v4  
      with:  
        node-version: 22  
        cache: npm
```

```
# puppeteer здесь не нужен (он для растеризации Р  
# тянет Chromium – пропускаем, чтобы установка не
```

```
- name: Зависимости  
  run: npm ci  
  env:  
    PUPPETEER_SKIP_DOWNLOAD: 1  
    PUPPETEER_SKIP_CHROMIUM_DOWNLOAD: 1
```

```
- name: Типы и сборка  
  run: |  
    npx tsc --noEmit -p tsconfig.json  
    npm run build
```

```
- name: Модульные проверки (liveMatrix)  
  run: npm run verify:unit
```

```
- name: Эталонные коды страниц (настоящий CPython)  
  run: npm run verify:code
```

```
- name: DOM-проверки приложения (jsdom)  
  run: npm run verify:dom
```



```

    title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
    type: "html", // всегда "html": страница
    category: "Графы. Пути", // группа в навигации
    content: "...HTML ниже...",
},
];

```

**Правила:**

- ``id`` – kebab-case, уникальный, совпадает с ключом ``VI``.  
подставляется автоматически, вручную ``div#slot-*`` вставляются  
такие заглушки остались в старых главах и считаются мёртвыми.
- ``title`` начинается с номера билета: сортировка оглавления по  
цифре заголовка. Одна страница может закрывать несколько билетов.  
(``mst`` → 18, 19, 20) – тогда номер в заголовке один, билет  
объявляется в ``chapterTopics`` (``src/data/content.ts``)
- ``content`` – шаблонная строка. Внутри **никаких** ``` экранирования: это сломает сборку.

## ## 2. Каркас страницы

```
```html
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
  <!-- шапка: бейдж билета + заголовок -->
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>

  <div class="space-y-8">
    <!-- смысловой блок (один или несколько) -->
    <div class="bg-slate-700/50 p-6 rounded-xl border-l">
      <h3 class="text-xl font-bold text-emerald-400 mb-4">
      ...определение, абзацы, стат-сетка, аналогии, спомогательные
    </div>
  </div>
</section>
```
```

```
````html
<span class="bg-slate-950 px-2 py-0.5 rounded text-xs f
````
```

### ### 3.3 Сетка ключевых чисел (3 ячейки)

```
````html
<div class="grid grid-cols-1 sm:grid-cols-3 gap-3 my-6"
  <div class="bg-slate-900 border border-slate-700 rounded
    <p class="text-[10px] uppercase tracking-wider text
    <p class="font-mono text-emerald-300 text-sm">0{(V+H
  </div>
  <!-- 2-я и 3-я ячейки — тот же markup -->
</div>
````
```

### ### 3.4 Сетка аналогий — **\*\*обязательна\*\*** на каждой странице

Слева — неформальная аналогия, справа — техническая альбом

```
````html
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border border-slate-700"
    <div class="absolute -top-3 left-4 bg-slate-700 text-emerald-500 shadow-md">
      Аналогия 1: Позитивное планирование
    </div>
    <p class="text-slate-300 text-sm mb-4">...</p>
  </div>

  <div class="bg-slate-900 p-6 rounded-lg border-2 border-slate-800"
    <div class="absolute -top-3 left-4 bg-rose-900 text-emerald-500 shadow-md">
      Ограничения
    </div>
    <p class="text-slate-300 text-sm mb-4">...</p>
  </div>
</div>
````
```

```
...</code></pre>
```

```
    <p class="text-slate-300 text-sm">Комментарий после  
  </div>
```

```
</details>
```

```
```
```

- Вложенные списки внутри спойлера: `<ul class="list-disc">`
- Подзаголовок внутри блока: `<h4 class="text-lg font-bold">`  
или `<p class="text-lg font-bold text-white mt-8 mb-3">`

### ### 3.7 Ловушки / частые ошибки

```
```html
```

```
<div class="bg-rose-950/30 p-5 rounded-lg border border-rose-300">  
  <p class="text-sm font-bold text-rose-300 mb-3">▲ Где  
  <ul class="list-disc pl-5 space-y-2 text-sm text-slate-300">  
    <li><b>Отрицательные веса.</b> Дейкстра их не переварит.  
    <li>Переполнение: <span class="font-mono text-emerald-300">  
  </ul>
```

```
</div>
```

```
```
```

## ## 4. Порядок разделов на странице

1. Шапка: бейдж билета + `<h2>`.
2. «Одной фразой» / «Суть алгоритма» + центрированное описание.
3. 1–3 абзаца объяснения (3.2), затем сетка ключевых чисел.
4. Аналогии (3.4) – обязательный блок.
5. Механика: пошаговое описание, таблица состояний/сравнений.
6. Спойлеры (3.6): референсный код, доказательство, формулы.
7. Ловушки (3.7) + границы применимости.
8. Связь с другими билетами («тот же find/union – сердце»).

Демонстрацию вставлять не нужно: `<ChapterView>` монтирует по `<id>` главы после текста.

Универсальное пособие • полный исходный код

- картинки страниц берутся из `CHAPTER\_IMAGES` (`script` снимки демонстраций – из `tmp/viz-shots/\*.png` (их ре `npm run export:shots` в CI); локально без браузера в описание вкладок и переменных из `PAGE\_SYNC`.

Цель по объёму: 40–60 страниц, жёсткий предел – 100.

---

---

КОНЕЦ ДОКУМЕНТА • 111 файлов • сгенерировано 15.09.2026

---

---